

В книге представлены основные концепции байесовской статистики и ее практическая реализация на языке Python с использованием современной библиотеки вероятностного программирования PyMC3 и новой библиотеки исследовательского анализа байесовских моделей ArviZ.

Искусственно сформированные и реально применяемые наборы данных используются для представления разнообразных типов моделей, таких как обобщенные линейные модели для регрессии и классификации, смешанные модели, иерархические модели, а также гауссовы процессы и многие другие. Знания о вероятностном моделировании, почерпнутые из этой книги, позволят вам самостоятельно проектировать и реализовать байесовские модели для собственных задач научной обработки данных.

Представленный материал дает основательный уровень подготовки для углубленного изучения более сложных тем и освоения специализированного статистического моделирования.

Вы научитесь:

- создавать вероятностные модели с использованием библиотеки PyMC3, написанной на языке Python;
- анализировать вероятностные модели с помощью библиотеки ArviZ;
- применять навыки и умения, требуемые для проверки работоспособности моделей и их модификации (если таковая нужна);
- понимать преимущества и недостатки иерархических моделей;
- правильно определять возможности практического применения различных моделей для ответов на вопросы, возникающие в процессе анализа данных;
- сравнивать модели и выбирать наиболее подходящие для конкретной задачи;
- определять, насколько различные модели являются универсальными с вероятностной точки зрения;
- применять вероятностное мышление и получать преимущества, определяемые гибкостью и универсальностью байесовского статистического анализа.

Введение в статистическое моделирование и вероятностное программирование

Интернет-магазин:
www.dmkpress.com

Оптовая продажа:
КТК «Галактика»
books@aliens-kniga.ru

Packt

ДМК
ИЗДАТЕЛЬСТВО
www.dmk.ru

ISBN 978-5-97060-768-8



9 785970 607688 >

Байесовский анализ на Python



Освальдо Мартин

Байесовский анализ на Python



Oswaldo Martin

Bayesian Analysis with Python

*Introduction to statistical modeling
and probabilistic programming
using PyMC3 and ArviZ*



Освальдо Мартин

Байесовский анализ на Python

*Введение в статистическое моделирование
и вероятностное программирование
с использованием PyMC3 и ArviZ*



Москва, 2020

УДК 004.021
ББК 32.973.3
М29

Мартин О.

М29 Байесовский анализ на Python / пер. с англ. А. В. Снастина. – М.: ДМК Пресс, 2020. – 340 с.: ил.

ISBN 978-5-97060-768-8

В книге представлены основные концепции байесовской статистики и ее практическая реализация на языке Python с использованием современной библиотеки вероятностного программирования PyMC3 и новой библиотеки исследовательского анализа байесовских моделей ArviZ.

Полученные знания о вероятностном моделировании позволят вам самостоятельно проектировать и реализовать байесовские модели для собственных задач научной обработки данных.

Издание будет полезно всем специалистам по анализу данных, использующих в своей работе байесовское моделирование.

УДК 004.021
ББК 32.973.3

Authorized Russian translation of the English edition of Bayesian Analysis with Python ISBN 9781789341652 © 2018 Packt Publishing.

This translation is published and sold by permission of Packt Publishing, which owns or controls all rights to publish and sell the same.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-78934-165-2 (анг.)
ISBN 978-5-97060-768-8 (рус.)

© 2018 Packt Publishing
© Оформление, издание, перевод, ДМК Пресс, 2020

Я посвящаю эту книгу Эйбрил

Содержание

Вступительное слово	10
Об авторе	12
О рецензентах	13
Предисловие	14
Глава 1. Вероятностное мышление	19
Статистика, модели и подход, принятый в этой книге.....	19
Работа с данными	21
Байесовское моделирование	22
Теория вероятностей.....	23
Объяснение смысла вероятностей	23
Определение вероятности	25
Байесовский вывод с одним параметром.....	34
Задача о подбрасывании монеты.....	35
Взаимодействие с байесовским анализом.....	46
Нотация и визуализация модели	46
Обобщение апостериорного распределения.....	47
Проверки апостериорного прогнозируемого распределения.....	49
Резюме	50
Упражнения.....	52
Глава 2. Вероятностное программирование	54
Вероятностное программирование.....	55
Основы использования библиотеки PyMC3	56
Решение задачи о подбрасывании монет с использованием библиотеки PyMC3.....	57
Обобщение апостериорного распределения.....	59
Решения на основе апостериорного распределения	61
Гауссова модель в подробном изложении	67
Гауссовы статистические выводы	68
Надежные статистические выводы	73
Сравнение групп.....	79
d-мера Коэна.....	81
Вероятность превосходства	82
Набор данных tips.....	82
Иерархические модели	86

Редуцирование.....	91
Еще один пример.....	94
Резюме.....	96
Упражнения.....	99
Глава 3. Моделирование с использованием линейной регрессии.....	101
Простая линейная регрессия	102
Связь с машинным обучением	102
Сущность моделей линейной регрессии.....	103
Линейные модели и сильная автокорреляция	108
Интерпретация и визуальное представление апостериорного распределения	111
Коэффициент корреляции Пирсона	114
Робастная линейная регрессия	118
Иерархическая линейная регрессия.....	122
Корреляция, причинно-следственная связь и беспорядочность жизни ...	128
Полиномиальная регрессия	130
Интерпретация параметров полиномиальной регрессии.....	131
Является ли полиномиальная регрессия конечной моделью.....	132
Множественная линейная регрессия	133
Спутывающие переменные и избыточные переменные	137
Мультиколлинеарность или слишком сильная корреляция.....	140
Маскировочный эффект переменных.....	144
Добавление взаимодействий.....	146
Дисперсия переменной.....	147
Резюме.....	150
Упражнения.....	151
Глава 4. Обобщение линейных моделей	154
Обобщенные линейные модели	154
Логистическая регрессия	156
Логистическая модель.....	157
Набор данных iris.....	157
Множественная логистическая регрессия	163
Граница решения.....	163
Реализация модели.....	164
Интерпретация коэффициентов логистической регрессии.....	165
Обработка коррелирующих переменных	167
Работа с несбалансированными классами	169
Регрессия с использованием функции softmax.....	171
Дискриминативные и порождающие модели	173
Регрессия Пуассона.....	176
Распределение Пуассона.....	176
Модель Пуассона с дополнением нулевыми значениями	178

Регрессия Пуассона и модель Пуассона с дополнением нулевыми значениями	179
Робастная логистическая регрессия	181
Модуль GLM	183
Резюме	184
Упражнения	185
Глава 5. Сравнение моделей	188
Проверки прогнозируемого апостериорного распределения	188
Лезвие Оккама – простота и точность	194
Лишние параметры приводят к перепогонке	196
Недостаточное количество параметров приводит к недопогонке	197
Баланс между простотой и точностью	197
Измерения прогнозируемой точности	198
Информационные критерии	200
Логарифмическая функция правдоподобия и отклонение	201
Информационный критерий Акаике	202
Часто применяемый информационный критерий	202
Парето-сглаженная выборка по значимости для перекрестной проверки LOOCV	203
Другие информационные критерии	203
Сравнение моделей с помощью библиотеки PyMC3	204
Усреднение моделей	207
Коэффициенты Байеса	210
Некоторые дополнительные замечания	212
Коэффициенты Байеса и информационные критерии	216
Регуляризация априорных распределений	220
Более подробно об информационном критерии WAIC	222
Энтропия	222
Расхождение Кульбака–Лейблера	224
Резюме	227
Упражнения	228
Глава 6. Смешанные модели	230
Смешанные модели	231
Конечные смешанные модели	232
Категориальное распределение	234
Распределение Дирихле	235
Неидентифицируемость смешанных моделей	238
Как правильно выбрать число K	241
Смешанные модели и кластеризация	245
Смешанные модели с бесконечной размерностью	246
Процесс Дирихле	246
Непрерывные смешанные модели	253

Биномиальное бета-распределение и отрицательное биномиальное распределение	254
t-распределение Стьюдента.....	255
Резюме.....	255
Упражнения.....	257
Глава 7. Гауссовы процессы	258
Линейные модели и нелинейные данные	258
Функции моделирования	259
Многомерные гауссовы распределения и функции.....	261
Ковариационные функции и ядра.....	261
Гауссовы процессы	264
Регрессия на основе гауссовых процессов	265
Регрессия с пространственной автокорреляцией	270
Классификация с использованием гауссова процесса.....	277
Процессы Кокса.....	283
Модель катастроф в угледобывающей промышленности	284
Набор данных redwood	286
Резюме.....	289
Упражнения.....	289
Глава 8. Механизмы статистического вывода	291
Механизмы статистического вывода	292
Немарковские методы.....	293
Грид-вычисления.....	293
Метод квадратической аппроксимации	296
Вариационные методы	298
Марковские методы.....	301
Метод Монте-Карло.....	303
Цепи Маркова	305
Алгоритм Метрополиса–Гастингса	305
Метод Монте-Карло с использованием механики Гамильтона	310
Последовательный метод Монте-Карло	312
Диагностирование выборок.....	314
Сходимость.....	316
Ошибка метода Монте-Карло	319
Автокорреляция.....	320
Эффективный размер выборки	321
Расхождения	322
Резюме.....	326
Упражнения.....	326
Глава 9. Что дальше?	328
Предметный указатель	332

Вступительное слово

Вероятностное программирование – это программная среда, которая позволяет создавать гибкие байесовские статистические модели в программном коде. После создания такой модели для обработки в ней данных могут быть использованы мощные алгоритмы логического вывода, работающие независимо. Такое сочетание гибкого определения модели и механизма автоматического логического вывода предоставляет исследователю мощный инструмент для быстрого создания, анализа и постепенного усовершенствования новых статистических моделей. Подобный итеративный подход абсолютно противоположен ранее применявшемуся способу подгонки байесовских моделей к данным: ранее используемые алгоритмы логического вывода обычно работали только с одной конкретной моделью. При этом требовались глубокие и прочные математические знания и навыки для формирования модели и разработки схемы логического вывода, что существенно замедляло итеративный цикл: изменение модели, модификация процесса логического вывода. Таким образом, вероятностное программирование делает статистическое моделирование доступным практически для всех, значительно снижая требования к уровню математической подготовки и сокращая время, требуемое для успешного создания новых моделей и нового, ранее недоступного, глубокого понимания исследуемых данных.

Сама идея вероятностного программирования не нова: BUGS, самый первый инструмент такого типа, появился в 1989 году. Количество моделей, для которых этот инструмент успешно применялся, было крайне ограниченным, а логический вывод выполнялся медленно, поэтому первое поколение языков этого типа не получило широкого распространения на практике. В наши дни существует множество специализированных языков вероятностного программирования, которые широко используются как для академических научных исследований, так и в компаниях Google, Microsoft, Amazon, Facebook и Uber для решения крупномасштабных и сложных задач. Что же изменилось? Главным фактором роста значимости вероятностного программирования и эволюции от состояния занимательной игрушки до мощного механизма, способного решать сложнейшие крупномасштабные задачи, стало появление алгоритма выборки на основе гамильтонова метода Монте-Карло, на несколько порядков более мощного, чем предыдущие алгоритмы выборки. Несмотря на то что этот алгоритм был разработан в 1987 году, только в последнее время системы вероятностного программирования Stan и PyMC3 сделали эту методику выборки широко доступной и удобной в практическом применении.

Предлагаемая книга представляет собой практический вводный курс по использованию этого чрезвычайно мощного и гибкого инструментального средства. Она, несомненно, окажет большое воздействие на ваш образ мыш-

ления и на понимание путей решения сложных аналитических задач. Лишь немногие люди подошли бы для написания такой книги лучше, чем один из основных разработчиков системы PyMC3 Освальдо Мартин (Osvaldo Martin). Освальдо обладает редким талантом подробного постепенного объяснения сложных тем, упрощая их понимание. Его глубокое знание и понимание этих тем, основанное на солидном практическом опыте, позволяет вести читателя по наиболее эффективному пути освоения этой области, которая иначе могла бы показаться недоступной. Наглядные иллюстрации и схемы, примеры программного кода делают эту книгу в высшей степени полезным практическим ресурсом, с помощью которого вы сможете в полной мере овладеть всеми необходимыми теоретическими основами.

Читатели, которые приобрели данную книгу, сделали правильный выбор. Это не простой и не быстрый путь. В наше время, когда широко рекламируется глубокое обучение, как методика решения всех текущих и будущих аналитических задач, более осмотрительный и взвешенный подход к созданию специализированных моделей для конкретной цели, возможно, не выглядит столь привлекательным. Но вы сможете решать задачи, которые трудно решаются любыми другими способами.

Это не говорит о том, что глубокое обучение не является весьма перспективной методикой. В действительности само по себе вероятностное программирование не ограничено классическими статистическими моделями. Изучая современную литературу по машинному обучению, вы наверняка обнаружите, что байесовская статистика определяется как мощный инструментальный комплекс для формирования и исследования следующего поколения глубоких нейронных сетей. Таким образом, эта книга вооружит читателя не только знаниями и навыками решения трудных аналитических задач, но также позволит создать более широкомасштабную основу для одного из самых великих достижений человечества: разработки искусственного интеллекта. Желаю успеха.

Томас Виецки (Thomas Wiecki),
д-р философии (PhD),
руководитель отдела исследований
в компании Quantopian (Бостон, США)

Об авторе

Освальдо Мартин (Osvaldo Martin) – ученый-исследователь агентства The National Scientific and Technical Research Council (CONICET) (Аргентина), занимающийся разработками в области структурной биоинформатики протеина, полисахаридов и молекул РНК. Обладает большим опытом использования цепей Маркова с применением метода Монте-Карло для имитации молекулярных систем, предпочитает пользоваться языком программирования Python для решения задач анализа данных.

Освальдо являлся преподавателем курсов по структурной биоинформатике, науке о данных и байесовскому анализу данных. Также возглавлял организационный комитет PyData в Сан-Луисе (Аргентина) в 2017 году. Освальдо – один из основных разработчиков программных систем PyMC3 и ArviZ.

«Я благодарен Ромине за ее постоянную поддержку. Также хочу поблагодарить Уолтера Лападула (Walter Lapadula), Билла Энгелса (Bill Engels), Эрика Х Ма (Eric J Ma) и Остина Рошфора (Austin Rochford) за бесценные замечания, поправки, комментарии и предложения к черновому варианту книги. Особая благодарность основным разработчикам и всем участникам проектов PyMC3 и ArviZ. Создание этой книги стало возможным благодаря поддержке, позитивному отношению и упорному труду, который все они вложили и вкладывают в эти библиотеки, а также в формирование великолепного сообщества пользователей».

О рецензентах

Эрик Х Ма (Eric J Ma) – ученый в области обработки данных в институте биомедицинских исследований корпорации Novartis. Занимается исследованиями биомедицинских данных с применением в основном байесовских статистических методов с целью улучшения качества медицинского обслуживания пациентов. До работы в корпорации Novartis был действительным членом научного общества Insight Health Data летом 2017 года, защитил докторскую диссертацию весной 2017 года.

Кроме того, Эрик является разработчиком ПО с открытым исходным кодом, ранее возглавлял разработку `nxviz`, пакета визуализации для `NetworkX`, и `rujanitor`, открытого API для очистки данных на языке Python. Также участвовал в разработке ряда инструментальных средств с открытым исходным кодом, включая `PyMC3`, `Matplotlib`, `bokeh` и `CuPy`.

Основной жизненный принцип (девиз) Эрика можно найти в Евангелии от Луки 12:48.

Остин Рошфор (Austin Rochford) – главный научный сотрудник в Monetate Labs, где он занимается разработкой продуктов, позволяющих розничным продавцам персонализировать свой рынок с учетом миллиардов событий, происходящих ежегодно. По образованию Остин математик, являющийся активным пропагандистом байесовских методов.

Предисловие

Методы байесовской статистики разрабатываются уже более 250 лет. Не только признание и одобрение, но не меньшее пренебрежение и даже неприятие постоянно сопровождали эту ветвь математики. На протяжении нескольких последних десятилетий байесовская статистика стала привлекать все большее внимание специалистов, занимающихся статистикой, и почти всех других ученых, инженеров и даже людей, не принадлежащих к миру науки. Подобный рост интереса стал возможным благодаря теоретическим и вычислительным разработкам, выполненным в основном во второй половине XX века. Разумеется, современная байесовская статистика является главным образом вычислительной статистикой. Необходимость в создании гибких и прозрачных моделей и более глубокая и подробная интерпретация статистических моделей и методов анализа также внесли свой вклад в тенденцию роста.

В предлагаемой книге применяется прагматический подход к изучению байесовской статистики, здесь не уделяется слишком много внимания другим статистическим парадигмам и их взаимосвязям с байесовской статистикой. Главная цель книги – научить практическому выполнению байесовского анализа данных. Философские теоретические дискуссии интересны, но на страницах этой книги вы их не обнаружите, попробуйте поискать в других, более подходящих местах.

В книге излагается методический подход моделирования в статистике, она поможет научиться мыслить в терминах вероятностных моделей и применять теорему Байеса для вывода логических следствий из используемых моделей и данных. Такой подход также является вычислительным, модели кодируются с использованием PyMC3, библиотеки поддержки байесовской статистики, которая скрывает от конечного пользователя большинство математических подробностей и вспомогательных вычислений, и ArviZ, пакета языка Python для исследовательского анализа байесовских моделей.

Байесовские методы с теоретической точки зрения основаны на теории вероятностей, поэтому неудивительно, что многие книги по байесовской статистике содержат множество математических формул, требующих определенного уровня математической подготовки. Изучение математических основ статистики может оказать немалую помощь при создании более качественных моделей и улучшить интуитивное понимание задач, моделей и результатов. Тем не менее такие библиотеки, как PyMC3, позволяют изучать и применять методы байесовской статистики даже при скромном объеме математических знаний, в чем вы сможете убедиться сами, читая эту книгу.

Для кого предназначена эта книга

Если вы студент, специалист по обработке данных, исследователь в области естественных или общественных наук или разработчик, начинающий изучать байесовский анализ данных и вероятностное программирование, то эта книга предназначена для вас. Книга представляет собой введение в байесовский анализ, поэтому не требует предварительных знаний в области статистики, хотя некоторый практический опыт использования языка Python и библиотеки NumPy был бы полезен.

Краткое содержание книги

В главе 1 «Вероятностное мышление» рассматриваются основные концепции байесовской статистики и ее практическое применение для анализа данных. Здесь содержится большинство базовых понятий и положений, которые используются в следующих главах книги.

Глава 2 «Вероятностное программирование» дает обзор концепций из предыдущей главы с точки зрения вычислительной практики. Здесь представлена ознакомительная информация о библиотеке вероятностного программирования PyMC3, а также о библиотеке ArviZ (пакет Python), предназначенной для исследовательского анализа байесовских моделей. На нескольких конкретных примерах рассматриваются иерархические модели.

В главе 3 «Моделирование с использованием линейной регрессии» рассматриваются основные элементы линейной регрессии, весьма широко применяемой модели и структурного компонента более сложных моделей.

В главе 4 «Обобщение линейных моделей» описывается, как расширить область применения линейных моделей для распределений, отличающихся от распределения Гаусса, открывая путь к решению многих задач анализа данных.

В главе 5 «Сравнение моделей» обсуждаются методы сравнения, выбора и усреднения моделей с использованием факторов Байеса, WAIC, LOO, а также основные характерные особенности и детали применения этих методов.

В главе 6 «Смешанные модели» рассматриваются способы повышения гибкости моделей с помощью объединения простых распределений для создания более сложных. Здесь представлена первая непараметрическая модель, рассматриваемая в книге: процесс Дирихле.

В главе 7 «Гауссовы процессы» описывается теоретическая концепция, лежащая в основе гауссовых процессов, а также способы ее применения для создания непараметрических моделей для решения широкого спектра задач.

В главе 8 «Механизмы логического вывода» представлено введение в методы числовой аппроксимации апостериорного распределения (вероятности), а также весьма важная с практической точки зрения тема: как точно определить надежность аппроксимированного апостериорного распределения.

В главе 9 «Что дальше» предлагается список информационных ресурсов, которыми можно воспользоваться для более глубокого изучения байесовского анализа, а также очень короткое итоговое резюме автора.

МАКСИМАЛЬНО ЭФФЕКТИВНОЕ ИСПОЛЬЗОВАНИЕ КНИГИ

Код в этой книге написан на языке Python версии 3.6. Для установки программной среды Python и всех необходимых библиотек рекомендуется воспользоваться Anaconda, специализированным для научных вычислений дистрибутивом. Получить более подробную информацию о дистрибутиве Anaconda и скачать его можно на сайте <https://www.anaconda.com/download/>. При этом в вашей системе будет установлено множество полезных пакетов на языке Python. После этого потребуется установить еще два пакета. Для установки библиотеки PyMC3 используйте утилиту conda:

```
conda install -c conda-forge pymc3
```

Чтобы установить пакет ArviZ, можно выполнить следующую команду:

```
pip install arviz
```

Другой способ установки необходимых пакетов, после того как дистрибутив Anaconda установлен в системе, – перейти по адресу <https://github.com/alocetavodia/BAP> и загрузить файл описания среды *bap.yml*. С помощью этого файла можно установить все необходимые пакеты следующей командой:

```
conda env create -f bap.yml
```

Все пакеты языка Python, использованные при написании этой книги, перечислены ниже:

- IPython 7.0;
- Jupyter 1.0 (или Jupyter-lab 0.35);
- NumPy 1.14.2;
- SciPy 1.1;
- pandas 0.23.4;
- Matplotlib 3.0.2;
- Seaborn 0.9.0;
- ArviZ 0.3.1;
- PyMC3 3.6.

При выполнении кода, приведенного в каждой главе, предполагается, что вы установили и импортировали, по крайней мере, некоторые из перечисленных выше пакетов. Вместо копирования и вставки кода из книги рекомендуется скачивать файлы исходного кода из репозитория <https://github.com/alocetavodia/BAP> и запускать их с помощью Jupyter Notebook или Jupyter Lab. Автор регулярно обновляет этот репозиторий после выхода новых версий PyMC3 и ArviZ.

Если при выполнении кода из книги возникает техническая проблема или обнаружена опечатка либо какая-либо другая ошибка, то передайте информацию об этом в указанный репозиторий, и автор попытается устранить проблему как можно быстрее.

Большинство иллюстраций в книге сгенерировано при выполнении программного кода. Основная схема такова: сначала приводится блок кода, за которым сразу же следует соответствующая иллюстрация (сгенерированная при выполнении приведенного выше кода). Автор надеется, что такая схема окажется привычной для тех, кто использует Jupyter Notebook/Lab, и не вызовет никаких затруднений у других читателей.

СКАЧИВАНИЕ ИСХОДНОГО КОДА ПРИМЕРОВ

Скачать файлы с дополнительной информацией для книг издательства «ДМК Пресс» можно на сайте www.dmkpress.com или www.дмк.рф на странице с описанием соответствующей книги.

ЗАГРУЗКА ЦВЕТНЫХ ИЛЛЮСТРАЦИЙ

Мы также предоставляем файл в формате PDF, содержащий цветные изображения снимков экрана и схем из данной книги. Этот файл доступен по адресу https://www.packtpub.com/sites/default/files/downloads/9781789341652_Color-Images.pdf.

ТИПОГРАФСКИЕ СОГЛАШЕНИЯ, ПРИНЯТЫЕ В КНИГЕ

В этой книге используется несколько стилей выделения некоторых элементов текста.

Фрагмент кода в тексте – ключевые слова, операторы, имена переменных и функций непосредственно в тексте. Пример: «Большая часть приведенного выше кода предназначена для построения и вывода графической схемы, вероятностные вычисления выполняются в строке `y = stats.norm(mu, sd).pdf(x)`».

Блок кода отображается в следующем формате:

```
μ = 0.
σ = 1.
X = stats.norm(μ, σ)
x = X.rvs(3)
```

Курсив – имена файлов, каталогов и прочих объектов.

Полужирный шрифт – важные (ключевые) слова, элементы пользовательского интерфейса или слова, которые выводятся на экран.

ОТЗЫВЫ И ПОЖЕЛАНИЯ

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

СПИСОК ОПЕЧАТОК

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг – возможно, ошибку в основном тексте или программном коде, – мы будем очень благодарны, если вы сообщите нам о ней. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Если вы найдете какие-либо ошибки в коде, пожалуйста, сообщите о них главному редактору по адресу dmkpress@gmail.com, и мы исправим это в следующих тиражах.

НАРУШЕНИЕ АВТОРСКИХ ПРАВ

Пиратство в интернете по-прежнему остается насущной проблемой. Издательства «ДМК Пресс» и Springer очень серьезно относятся к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Глава 1

Вероятностное мышление

«Теория вероятностей – это не что иное, как здравый смысл, сведенный к вычислениям».

– *Пьер Симон Лаплас*

В этой главе мы начнем изучать основные концепции байесовской статистики и познакомимся с некоторыми инструментами из байесовского арсенала. Здесь приводятся небольшие фрагменты кода на языке Python, но глава в основном теоретическая, поэтому почти все концепции, описываемые в ней, будут многократно использоваться на протяжении всей книги. Из-за обилия теоретического материала глава может показаться немного усложненной для программистов-кодеров, но я полагаю, что такой подход способствует упрощению эффективного применения байесовской статистики при решении практических задач.

В этой главе рассматриваются следующие темы:

- статистическое моделирование;
- вероятность и неопределенность;
- теорема Байеса и статистический вывод;
- статистический вывод с одним параметром и классическая задача о подбрасывании монеты;
- выбор априорного распределения вероятностей и почему людям часто не нравится эта процедура, хотя она обязательна;
- взаимодействие с байесовским анализом (представление результатов).

СТАТИСТИКА, МОДЕЛИ И ПОДХОД, ПРИНЯТЫЙ В ЭТОЙ КНИГЕ

Статистика занимается сбором, организацией (упорядочением), анализом и интерпретацией данных, следовательно, знание статистики чрезвычайно важно для анализа данных. При анализе данных используются два основных статистических метода:

- разведочный анализ данных (РАД)¹ (Exploratory Data Analysis – EDA) – используются числовые обобщающие характеристики, такие как среднее

¹ Термин взят из русской версии Википедии, хотя более подходящим кажется вариант «исследовательский анализ данных». – *Прим. перев.*

значение, мода, стандартное отклонение и вероятные отклонения (этот раздел разведочного анализа данных также называют описательной статистикой). Кроме того, при разведочном анализе данные исследуются визуально с применением широко известных инструментальных средств, таких как гистограммы и диаграммы рассеяния;

- статистический вывод (inferential statistics) – вывод утверждений на основе текущих данных. Это может быть необходимо для понимания некоторых конкретных явлений, или для прогнозирования в будущем точек данных (которые ранее не наблюдались), либо для выбора одного из нескольких альтернативных объяснений результатов наблюдений. Статистический вывод – это набор методов и инструментов, которые помогают ответить на типы вопросов, перечисленные выше.

! В этой книге главное внимание сосредоточено на выполнении байесовского статистического вывода и последующем применении метода разведочного анализа данных для обобщения, интерпретации, проверки и предъявления результатов байесовского статистического вывода.

В большинстве вводных курсов по статистике, по крайней мере для людей, незнакомых со статистикой, материал излагается как набор готовых рецептов, более или менее похожих на следующий: войти в статистическую кладовую, выбрать одну из банок и открыть ее, добавить данные по вкусу, перемешивать до получения твердого Р-значения (Р-критерия), предпочтительно меньшего 0.05. Главная цель курсов с таким подходом – научить выбирать правильную банку в статистической кладовой. Мне никогда не нравился подобный подход, главным образом потому, что наиболее частым результатом такого обучения становится группа сбитых с толку людей, неспособных хорошо понимать, даже на концептуальном уровне, совокупность различных изучаемых методов. Мы будем придерживаться другой методики: также будем изучать некоторые конкретные рецепты, но при этом предпочитая домашнее приготовление, а не консервы из банок. То есть мы будем учиться смешивать свежие ингредиенты, наиболее подходящие для разнообразных блюд, и, что более важно, такой подход позволит применять изученные концепции в последующих примерах, содержащихся в книге.

Применение подобного подхода возможно по двум причинам:

- онтологическая причина – статистика является формой моделирования, унифицированной с помощью математического аппарата, основанного на теории вероятностей. Использование вероятностного подхода обеспечивает единую универсальную точку зрения на все, что может показаться весьма несопоставимыми методами, – статистические методы и методы машинного обучения выглядят намного более похожими друг на друга при взгляде с вероятностной точки зрения;
- техническая причина – современное программное обеспечение, например библиотека PyMC3, позволяет специалистам-практикам, таким как

вы и я, определять и создавать модели решений относительно простым способом. Многие из этих моделей оставались неразрешимыми (следовательно, неприменимыми на практике) буквально несколько лет назад или требовали слишком высокого уровня математической и технической подготовки.

Работа с данными

Данные – это важнейший ингредиент в статистике и науке о данных (даталогии). Данные поступают из различных источников, таких как эксперименты, компьютерные имитации, опросы и полевые наблюдения. Если мы являемся ответственными за генерацию или сбор данных, то всегда в первую очередь необходимо тщательно продумать и сформулировать вопросы, на которые нужно получить ответы, и определить используемые для этого методы, и только после этого приступить к обработке данных. В действительности существует целая область статистики, занимающаяся сбором данных, – планирование эксперимента (experimental design). В наше время, когда поток данных достиг невероятных размеров, мы иногда можем забыть о том, что сбор данных не всегда является простым и дешевым делом. Например, всем известно, что Большой адронный коллайдер генерирует сотни терабайтов данных в день, но не все помнят о том, что для его создания потребовались годы ручного и умственного труда.

В качестве обобщенного правила можно интерпретировать процесс генерации данных как случайный (стохастический), поскольку в этом процессе существует онтологическая, техническая и/или эпистемологическая неопределенность, то есть система по своей внутренней сущности является случайной, также существуют технические проблемы, добавляющие шум или ограничивающие наши возможности измерения с произвольной точностью, а кроме того, некоторые концептуальные теоретические ограничения, скрывающие от нас подробности. Из-за всех вышеперечисленных причин всегда необходимо интерпретировать данные в контексте используемых моделей, включая ментальные и формальные. Данные без моделей ни о чем не говорят.

В книге предполагается, что все необходимые данные уже собраны. Кроме того, данные считаются предварительно подготовленными и очищенными, что чрезвычайно редко встречается в реальной практике. Эти исходные предположения сделаны для того, чтобы полностью сосредоточиться на основной теме книги. Очень существенное замечание, особенно для начинающих изучать анализ данных: даже несмотря на то, что подготовка и очистка данных не рассматривается в нашей книге, это весьма важные практические навыки, которыми вы должны овладеть и развивать их для успешной работы с данными.

Очень полезной способностью при анализе данных является умение написать код на каком-либо языке программирования, например на Python. Обработка данных является неизбежной необходимостью с учетом того, что мы живем в беспорядочном мире с еще более беспорядочными данными, поэтому

умение писать программный код помогает решать эти задачи. Даже если вы настолько удачливы, что находящиеся в вашем распоряжении данные предварительно обработаны и очищены, умение писать код все равно останется полезным навыком, потому что современная байесовская статистика реализована в основном с помощью языков программирования, таких как Python или R.

Если вы хотите более подробно узнать, как использовать Python для очистки и обработки данных, рекомендуется изучить превосходную книгу «Python Data Science Handbook» Джейка Ван-дер-Пласа (Jake VanderPlas).

Байесовское моделирование

Модели – это упрощенные описания конкретной системы или процесса, которые, по тем или иным причинам, нас интересуют. Эти описания преднамеренно формулируются так, чтобы отобразить самые важные аспекты системы, но не уделять внимание каждой малозначимой подробности. Это одна из причин, по которой более сложная модель не всегда является лучшим вариантом.

Существует множество различных типов моделей, но в этой книге мы ограничимся байесовскими моделями. В общем случае процесс байесовского моделирования включает три основных этапа.

1. Выбираются некоторые данные и делаются предварительные предположения о том, как эти данные могли быть сгенерированы (извлечены), затем формируется модель с помощью объединения структурных компонентов, известных под названием распределения вероятностей. В основном эти модели представляют собой достаточно грубые приближения (аппроксимации), но в большинстве случаев это именно то, что нам нужно.
2. Используется теорема Байеса для добавления данных в сформированные модели, и выполняются логические выводы по совокупности данных и наших предварительных предположений. Обычно это называют формированием или уточнением условий работы модели по имеющимся данным.
3. Модель критически оценивается посредством проверки степени осмысленности результатов по различным критериям, включая данные, уровень наших экспертных знаний в области исследований, а иногда путем сравнения нескольких моделей.

Вообще говоря, три перечисленных выше этапа в большинстве случаев выполняются итеративно и без соблюдения строгого порядка. Мы будем возвращаться для повторения любого из этих этапов в произвольные моменты времени: возможно, была совершена ошибка при написании кода, или был найден способ изменить и усовершенствовать модель, или возникла необходимость добавления новых данных или сбора данных другого типа.

Байесовские модели также называют вероятностными моделями, потому что они создаются с использованием вероятностей. Почему сделан именно такой выбор? Потому что вероятности являются самым правильным мате-

математическим инструментом для моделирования неопределенности. Поэтому необходимо поближе познакомиться с этой «новой землей», покрытой сетью разветвляющихся дорог.

ТЕОРИЯ ВЕРОЯТНОСТЕЙ

Название этого раздела может показаться излишне претенциозным, ведь мы не собираемся изучить теорию вероятностей всего на нескольких страницах, да это и не было моим намерением. Я хотел лишь представить несколько общих и наиболее важных концепций, необходимых для лучшего понимания байесовских методов, достаточных для освоения материала данной книги. При необходимости мы будем более подробно рассматривать или вводить новые концепции, относящиеся к теории вероятности. Для более глубокого изучения теории вероятности настоятельно рекомендую книгу «Introduction to Probability» Джозефа К Блитцштайна (Joseph K Blitzstein) и Джессики Хван (Jessica Hwang). Другой весьма полезной книгой может оказаться «Mathematical Theory of Bayesian Statistics» Сумио Ватанабе (Sumio Watanabe), поскольку по названию понятно, что эта книга в большей степени ориентирована на байесовские методы, чем первая, но она более сложна с математической точки зрения.

Объяснение смысла вероятностей

Несмотря на то что теория вероятностей является вполне сформировавшейся и прочно обоснованной математической дисциплиной, существуют и другие интерпретации смысла термина «вероятность». С байесовской точки зрения вероятность – это мера, которая определяет в числовом выражении уровень неопределенности высказывания. С учетом этого определения вероятности абсолютно допустимо и даже естественно задать вопрос о вероятности существования жизни на Марсе, о вероятности того, что масса электрона составляет 9.1×10^{-31} кг, или о вероятности того, что 9 июля 1816 года в Буэнос-Айресе был солнечный день. Но при этом следует отметить, например, что жизнь на Марсе либо существует, либо не существует, то есть итоговый результат бинарный, это тип вопроса с ответом да-нет. Но, учитывая то, что мы не уверены в самом факте существования жизни на Марсе, разумным образом действий является попытка определить, насколько вероятна жизнь на Марсе. Поскольку приведенное выше определение вероятности связано с эпистемологическими, то есть познавательными, функциями нашего мышления, его часто называют субъективным определением вероятности. Однако отметим, что любой человек с научным складом ума не будет использовать свои личные верования или «информацию, полученную от ангела», для ответа на вопросы такого рода, а воспользуется всеми доступными геофизическими данными о Марсе, обратится к своим знаниям в области биомеханики, чтобы определить необходимые условия для жизни, и т. д. Таким образом, байесовские вероятности, следовательно, и вся байесовская статистика, так же субъективны (или объективны),

как и любой другой прочно обоснованный научный метод, имеющийся в нашем распоряжении.

Если у нас нет информации о задаче, то вполне разумно утверждать, что любое возможное событие одинаково вероятно (правдоподобно), а с формальной точки зрения это равносильно предположению об одинаковой вероятности наступления любого возможного события. При отсутствии информации неопределенность максимальна. Но если нам известно, что наступление каких-либо событий более правдоподобно, то это можно формально представить предположением о более высокой вероятности наступления этих событий и более низкой вероятности для прочих событий. Отметим, что когда мы говорим о событиях в контексте статистики, то не ограничиваемся «событиями, которые могут произойти», такими как падение астероида на Землю или вечеринка по поводу 60-летия моей тетушки. Событие – это любое из возможных значений (или набора значений), которые может принимать некоторая переменная, например событие (event), соответствующее утверждению, что вам больше 30 лет, или цена Sachertorte, или количество велосипедов, проданных в прошлом году по всему миру. Концепция вероятности также связана с понятиями логики. Пользуясь аристотелевой или классической логикой, мы можем формулировать утверждения, значениями которых могут быть только истина или ложь. С учетом байесовского определения вероятности определенность является всего лишь особым случаем: истинному утверждению соответствует вероятность 1, а ложному – вероятность 0. Присвоить значение вероятности 1 утверждению «жизнь на Марсе существует» можно было бы только после получения убедительных данных о том, что какие-то объекты развиваются и воспроизводятся, а также выполняют другие действия, которые мы считаем присущими живым организмам. Но следует также отметить, что присваивание значения вероятности 0 сложнее, потому что мы всегда можем считать, что некоторые области Марса остаются неисследованными, или что мы совершили ошибки в некотором эксперименте, либо какие-то другие причины могли привести к ложному выводу об отсутствии жизни на Марсе, хотя этот факт не доказан. Здесь уместно применение правила Кромвеля, гласящего, что необходимо избегать применения априорных вероятностей 0 или 1 к логически истинным или ложным утверждениям. Весьма любопытно, что Ричард Кокс (Richard Cox) математически доказал, что при необходимости расширения логики с включением неопределенности мы непременно должны использовать значения вероятности и теорию вероятностей. Теорема Байеса представляет собой логический вывод на основе правил вычисления вероятности, и мы вскоре убедимся в этом сами. Таким образом, другим способом интерпретации байесовской статистики является расширение логики с учетом неопределенности, то есть нечто, явно не влияющее на субъективное обоснование в уничижительном смысле, именно в этом смысле люди часто употребляют термин «субъективный».

Если обобщить все вышесказанное, то использование вероятности для описания неопределенности модели не обязательно связано с дискуссиями о том,

является ли природа детерминированной или случайной на самом базовом, фундаментальном уровне, кроме того, не связано и с субъективными личными верованиями. Это чисто методологический подход к моделированию неопределенности. Мы считаем большинство явлений трудными для глубокого понимания, потому что в основном имеем дело с неполными и/или зашумленными (загрязненными) данными. Кроме того, существует внутреннее ограничение, зависящее от сформированного в процессе эволюции мозга приматов, в частности человеческого мозга, или от любой другой важной причины, которую можно найти. Вследствие этого мы используем методику моделирования, которая явно принимает во внимание неизбежную неопределенность.



С практической точки зрения большинство важных элементов информации в этом разделе подтверждает использование байесовских вероятностей как инструментального средства для числового выражения неопределенности.

После обсуждения байесовской интерпретации вероятности рассмотрим подробнее несколько математических свойств вероятностей.

Определение вероятности

Вероятности – это числа в интервале $[0,1]$, то есть числа между 0 и 1, включая оба граничных значения. Вероятности подчиняются нескольким правилам. Одно из них – правило умножения:

$$p(A,B) = p(A|B)p(b). \quad (1.1)$$

Это читается следующим образом: вероятность A и B равна вероятности A при условии B , умноженной на вероятность B . Выражение $p(A,B)$ представляет совместную вероятность A и B . Выражение $p(A|B)$ используется для обозначения условной вероятности. Это название указывает на то, что вероятность A обусловлена знанием B и зависит от него. Например, вероятность того, что автодорога мокрая, отличается от вероятности влажности автодороги, если мы знаем (или предполагаем), что идет дождь. Условная вероятность может быть больше, меньше или равна безусловной вероятности. Если знание B не дает нам информацию об A , то $p(A|B) = p(A)$. Это выражение будет истинным, только если A и B независимы друг от друга. С другой стороны, если знание B дает нам полезную информацию об A , то условная вероятность может быть больше или меньше безусловной вероятности в зависимости от того, делает ли знание B более или менее возможным A . Рассмотрим простой пример с использованием обычного игрального кубика (кости) с шестью гранями. Какова вероятность выпадения числа 3 при броске кости $p(\text{die}=3)$? Она равна $1/6$, так как каждое из шести чисел имеет одинаковый шанс для обычной шестигранной кости. А чему равна вероятность выпадения числа 3, при условии что мы получаем нечетное число, то есть $p(\text{die}=3|\text{die}=\text{odd})$? Вероятность равна $1/3$, поскольку если мы знаем, что получаем нечетное число, то возможно выпадение только чисел $\{1, 3, 5\}$, и каждое из этих чисел имеет равные шансы. Наконец,

какова вероятность $p(\text{die}=3|\text{die}=\text{even})$? Она равна 0, так как если известно, что выпадает четное число, то возможны лишь варианты {2, 4, 6}, следовательно, появление числа 3 невозможно.

Из этого простого примера можно видеть, что при назначении условий для исследуемых данных мы действительно изменяем вероятность наступления событий, а вместе с тем и присущую им неопределенность. Условные вероятности можно назвать сердцем статистики независимо от того, какая задача поставлена перед нами: бросание игральные костей или создание самоуправляемых автомобилей.

Распределения вероятностей

Распределение вероятностей – это математический объект, который описывает, насколько возможными (вероятными) являются различные события. Вообще говоря, эти события ограничены каким-либо образом, то есть представляют собой набор возможных событий, например набор возможных чисел {1, 2, 3, 4, 5, 6} для игральной кости (исключая неопределенные случаи). Общепринятым и полезным представлением концепции в статистике является интерпретация данных как генерируемых из некоторого истинного распределения вероятностей с неизвестными параметрами. Далее логический вывод – это процесс поиска значений этих параметров с использованием только частичной выборки (также называемой набором данных) из истинного распределения вероятностей. В обобщенном случае у нас нет доступа к такому истинному распределению вероятностей, поэтому необходимо примириться с действительностью и создать модель, чтобы попытаться как-то аппроксимировать это распределение. Вероятностные модели создаются при помощи правильного объединения распределений вероятностей.



Следует отметить, что в обобщенном случае мы не можем быть уверены в том, является ли наша модель корректной, следовательно, необходимо критически оценивать создаваемые модели, чтобы повысить степень уверенности и убедить других людей в том, что предлагаемые модели соответствуют задаче, которую нужно исследовать или решить.

Если переменная X может быть описана с помощью распределения вероятностей, то ее называют случайной величиной (переменной). Существует общепринятое правило, согласно которому для обозначения объектов случайных величин используются заглавные буквы, например X , а для обозначения экземпляра случайной величины принято использовать строчные буквы, например x . Экземпляр x может быть вектором и содержать множество элементов или отдельных значений $x = (x_1, x_2, \dots, x_n)$. Рассмотрим пример с применением языка Python. Для этого примера в качестве истинного распределения вероятностей принято нормальное, или гауссово, распределение со средними значениями $\mu = 0$ и $\sigma = 1$ – эти два параметра полностью и недвусмысленно определяют нормальное распределение. Используя библиотеку SciPy, можно определить случайную переменную X , записав выражение `stats.norm( $\mu$ ,  $\sigma$ )`, после чего мы можем получить экземпляр x этой переменной с помощью метода `rvs` (сокращенно

щение от random variates). В приведенном ниже примере запрашиваются три значения:

```
μ = 0.
σ = 1.
X = stats.norm(μ, σ)
x = X.rvs(3)
```

Обратите внимание на то, что при каждом выполнении этого кода (в терминах статистики: при каждом испытании) вы получаете различные случайные результаты. Отметим, что поскольку значения параметров распределений известны, то вероятность каждого значения x также известна. Случайность состоит в тех самых значениях x , которые мы получаем при каждом испытании. Весьма часто возникает неправильная интерпретация случайности в том смысле, что можно получать любые возможные значения из случайной переменной или что все значения равновозможны. Допустимые значения случайной переменной и вероятности их появления строго управляются распределением вероятностей, а случайность возникает только из того факта, что мы не можем предсказать точные значения, которые будут выдаваться при каждом испытании. При каждом выполнении приведенного выше кода мы будем получать три различных числа, но если повторить выполнение этого кода несколько тысяч раз, то получим возможность опытным путем проверить тот факт, что среднее значение выбираемых экземпляров чисел приближается к нулю, а также что в 95 % выборок содержатся значения в интервале $[-1.96, +1.96]$. Не следует принимать мое высказывание на веру, проверьте его на практике в среде программирования Python. К точно такому же выводу можно прийти, изучая математические свойства нормального распределения.

Ниже приведена общепринятая форма записи, используемая для обозначения того факта, что переменная имеет нормальное распределение с параметрами μ и σ :

$$x \sim N(\mu, \sigma). \quad (1.2)$$

В этом контексте символ $\sim$ (тильда) читается как «имеет распределение» или «описывается распределением».



Достаточно часто в публикациях можно встретить обозначение нормального распределения, выраженное в терминах дисперсии, а не стандартного отклонения. То есть записывается выражение $N(\mu, \sigma^2)$. В этой книге параметризация нормального распределения будет выполняться с использованием стандартного отклонения, во-первых, потому что оно проще для интерпретации, во-вторых, потому что так работает библиотека PyMC3.

В этой книге будут встречаться и некоторые другие распределения вероятностей, в таких случаях будет приводиться краткое описание соответствующего распределения. Мы начинаем с нормального распределения, потому что оно является своеобразным родоначальником всех распределений вероятностей. Переменная X описывается гауссовым распределением, если его значения определяются следующим выражением:

$$p(x|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}. \quad (1.3)$$

Это функция плотности распределения вероятностей для нормального распределения. Нет необходимости запоминать эту формулу, она приведена здесь только для того, чтобы показать, откуда берутся числовые значения. Как уже было сказано ранее, здесь μ и σ являются параметрами распределения, поэтому, задавая их значения, мы полностью определяем конкретное распределение. Это можно понять из формулы 1.3, так как все прочие члены выражения являются константами. Параметр μ может принимать любое действительное значение, то есть $\mu \in \mathbb{R}$, и определяет среднее значение распределения (а также срединное значение или медиану и моду, которые равны). Параметр σ – это стандартное отклонение, которое может быть только положительным числом и определяет размах распределения. Чем больше значение σ , тем больше размах распределения. Поскольку существует бесконечное число возможных сочетаний значений μ и σ , возможно бесконечное количество экземпляров гауссова распределения, но все они принадлежат к одному семейству распределения Гаусса.

Математические формулы лаконичны и недвусмысленны, некоторые люди даже называют их красивыми, но необходимо признать, что при первой встрече они могут показаться непонятными и даже пугающими, особенно для людей, которые не очень любят математику. Неплохим способом преодоления этого препятствия является использование языка программирования Python для исследования и реализации математических формул на практике:

```
mu_params = [-1, 0, 1]
sd_params = [0.5, 1, 1.5]
x = np.linspace(-7, 7, 200)
_, ax = plt.subplots(len(mu_params), len(sd_params), sharex=True,
                    sharey=True,
                    figsize=(9, 7), constrained_layout=True)
for i in range(3):
    for j in range(3):
        mu = mu_params[i]
        sd = sd_params[j]
        y = stats.norm(mu, sd).pdf(x)
        ax[i,j].plot(x, y)
        ax[i,j].plot([], label=f"μ = {3.2f} \n σ = {3.2f}".format(mu,
            sd), alpha=0)
        ax[i,j].legend(loc=1)
ax[2,1].set_xlabel('x')
ax[1,0].set_ylabel('p(x)', rotation=0, labelpad=20)
ax[1,0].set_yticks([])
```

Большая часть приведенного выше кода предназначена для построения и вывода графических схем, а вероятностные вычисления выполняются в строке

`y = stats.norm(mu, sd).pdf(x)`. В этой строке вычисляется значение функции плотности вероятности для нормального распределения с передачей параметров `mu` и `sd` для набора значений `x`. Приведенный выше код генерирует схемы, показанные на рис. 1.1. На каждом отдельном графике изображена темно-серая кривая, представляющая гауссово распределение с заданными параметрами μ и σ , указанными в описании (легенде) каждого графика:

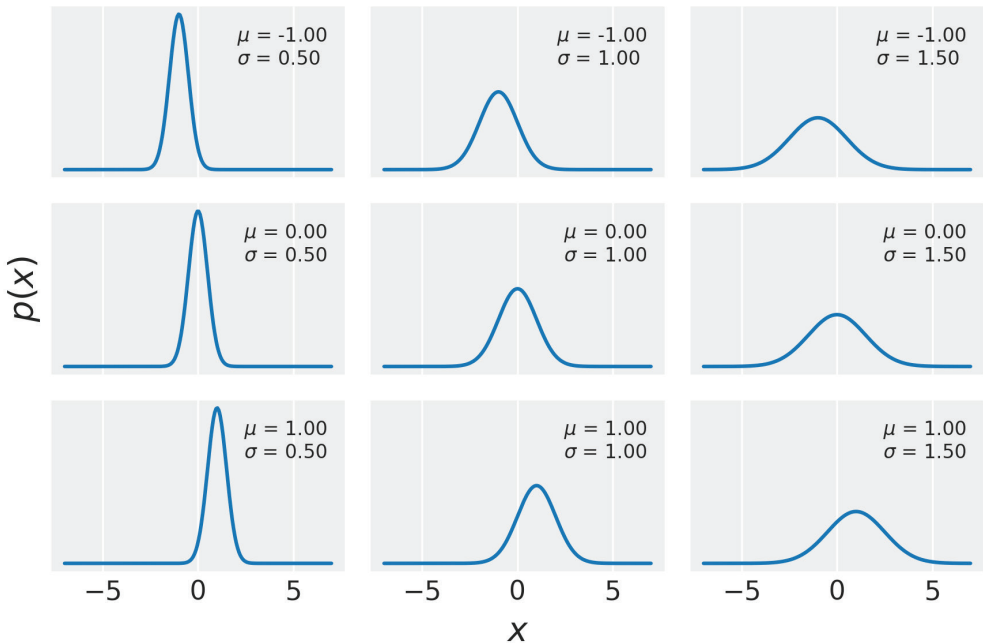


Рис. 1.1

! Большинство иллюстраций в этой книге сгенерировано непосредственно из приведенного перед каждой иллюстрацией программного кода, во многих случаях даже без предварительного описания, связывающего код с иллюстрацией. Такой стиль должен быть хорошо знаком тем, кто работает с Jupyter Notebooks или Jupyter Lab.

Существуют два типа случайных переменных: непрерывные и дискретные. Непрерывные переменные могут принимать любое значение из некоторого интервала (для их представления можно воспользоваться числами с плавающей точкой (float) языка Python). Дискретные переменные могут иметь только определенные значения (их можно представить целыми числами (integer) языка Python). Нормальное распределение является непрерывным распределением.

Отметим, что на схемах рис. 1.1 не указаны значения `yticks`, но это сделано преднамеренно. На основе своего опыта могу сказать, что эти значения не

добавляют какой-либо важной информации и даже могут запутать некоторых людей. Более подробное объяснение: числа, указанные по оси y , в действительности не имеют особого смысла – важны лишь их относительные значения. Если взять два значения из x , например x_i и x_j , и обнаружить, что $p(x_i) = 2p(x_j)$ (то есть на графике второе значение выше в два раза), то можно с уверенностью сказать, что вероятность значения x_i в два раза больше вероятности значения x_j . Большинство людей понимает это интуитивно, и, к счастью, это правильная интерпретация. Сложность возникает, когда мы имеем дело с непрерывными распределениями, а значения по оси y являются не вероятностями, а плотностями вероятности. Для получения правильного значения вероятности необходимо выполнить интегрирование по заданному интервалу, то есть требуется вычислить площадь под кривой распределения в заданном интервале. Вероятность не может быть больше единицы, но плотности вероятности могут превышать это значение, поэтому общая площадь под кривой плотности вероятности ограничена значением 1. Понимание различия между вероятностями и плотностями вероятностей чрезвычайно важно с математической точки зрения. При практическом подходе, принятом в этой книге, можно допустить небольшую неточность, поскольку различие между этими понятиями не столь важно, если вы правильно поняли, как интерпретировать схемы на рис. 1.1 с учетом относительности значений.

Независимые одинаково распределенные случайные величины

Во многих моделях предполагается, что все последовательные значения случайных переменных выбраны из одного и того же распределения и независимы друг от друга. В этом случае их называют независимыми одинаково распределенными случайными величинами (переменными) (иногда для краткости используют аббревиатуру *нор* или *iid* – *independently and identically distributed*). Используя математическую нотацию, можно показать, что две переменные являются независимыми, если $p(x,y) = P(x)p(y)$ для любых значений x и y .

В качестве обобщенного примера случайных величин, которые не являются независимыми одинаково распределенными (не-*нор*), можно привести временные ряды (*temporal series*), в которых временная зависимость, существующая в случайной переменной, представляет собой главную характеристику, которую необходимо принимать во внимание. Например, возьмем следующие данные с сайта <http://cdiac.esd.ornl.gov>. Это данные измерений содержания CO_2 в атмосфере с 1959 по 1997 год. Загрузим эти данные (и прилагаемый к ним исходный код) и построим по ним график:

```
data = np.genfromtxt('../data/mauna_loa_CO2.csv', delimiter=',')
plt.plot(data[:,0], data[:,1])
plt.xlabel('year')
plt.ylabel('$CO_2$ (ppmv)')
plt.savefig('B11197_01_02.png', dpi=300)
```

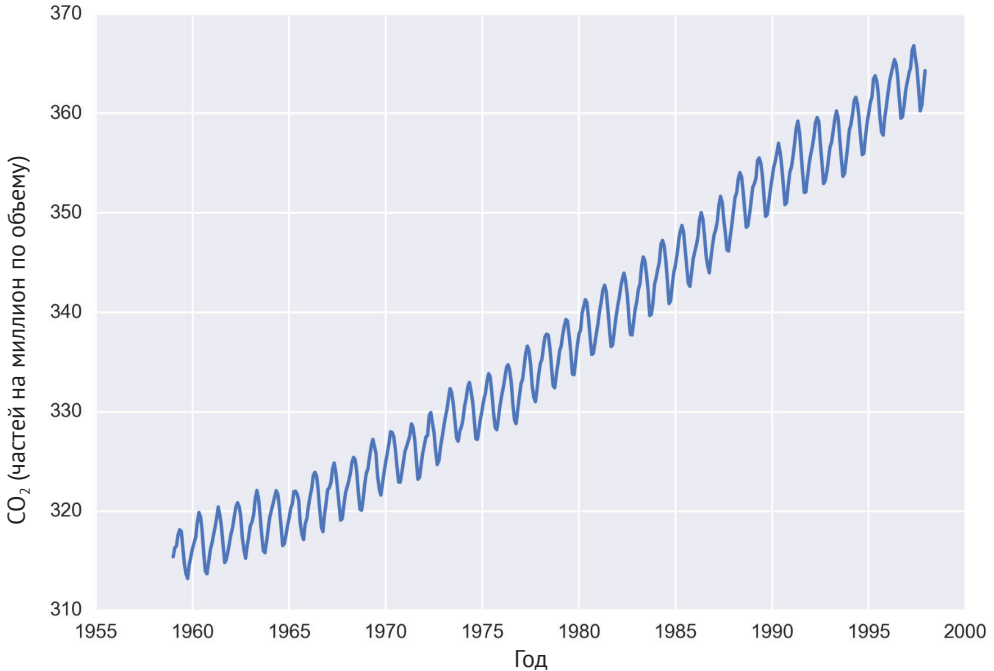


Рис. 1.2

Каждая точка данных соответствует измеренному уровню содержания CO_2 в атмосфере за месяц. Временную зависимость точек данных легко видеть на этом графике. В действительности здесь наблюдаются два тренда: сезонный (связанный с циклами роста и снижения) и глобальный, отображающий постоянный рост концентрации CO_2 в атмосфере.

Теорема Байеса

После ознакомления с некоторыми основными концепциями и терминами теории вероятностей можно перейти к главному предмету наших ожиданий. Без лишних церемоний позвольте представить теорему Байеса во всем ее величии:

$$p(\theta|y) = \frac{p(y|\theta)p(\theta)}{p(y)}. \quad (1.4)$$

Возможно, выглядит не слишком впечатляюще. Больше похоже на простую формулу из курса средней школы, тем не менее, перефразируя Ричарда Фейнмана (Richard Feynman), это все, что вам необходимо знать о байесовской статистике.

Происхождение теоремы Байеса и логические ходы, приводящие к ее формулировке, помогут нам понять ее смысл.

В соответствии с правилом произведения получаем:

$$p(\theta, y) = p(\theta|y)p(y). \quad (1.5)$$

Это выражение можно также записать в следующем виде:

$$p(\theta, y) = p(y|\theta)p(\theta). \quad (1.6)$$

С учетом того, что члены в левых частях выражений 1.5 и 1.6 равны, можно объединить эти выражения и записать:

$$p(\theta|y)p(y) = p(y|\theta)p(\theta). \quad (1.7)$$

После простого преобразования выражения 1.7 (перенос члена $p(y)$ в правую часть) получим формулу 1.4, то есть теорему Байеса.

Теперь рассмотрим, что означает формула 1.4 и почему она так важна. Во-первых, необходимо отметить, что $p(\theta|y)$ не обязательно является тем же самым, что $p(y|\theta)$. Это чрезвычайно важный факт, который легко упустить из вида в повседневных ситуациях даже людям, вполне сведущим в статистике и теории вероятностей. Рассмотрим простой пример для разъяснения того факта, что эти числовые характеристики не обязательно могут быть одинаковыми. Вероятность того, что человек является римским папой, при условии что этот человек аргентинец, не равна вероятности того, что человек является аргентинцем, принимая во внимание, что этот человек является римским папой. Поскольку в настоящее время в Аргентине проживает около 44 млн человек и один из них является действующим римским папой, получаем $p(\text{Pope}|\text{Argentinian}) \cong 1/44\,000\,000$, но при этом $p(\text{Argentinian}|\text{Pope}) = 1$.

Если заменить элемент θ на «предположение» (гипотезу), а элемент y на «данные», то теорема Байеса показывает, как вычислить вероятность предположения θ при наличии данных y . Именно такое объяснение смысла практического применения теоремы Байеса вы найдете во многих местах. Но как превратить предположение в некий объект, который можно поместить в формулу теоремы Байеса? Это делается с использованием распределений вероятностей. Вообще говоря, наше предположение (или гипотеза) представляет собой предположение в чрезвычайно узком смысле, если говорить более точно, то мы ищем наиболее подходящее значение для параметров выбранных нами моделей, то есть для параметров распределений вероятностей. Кстати, не следует пытаться в качестве предположения θ подставить выражения типа «единороги существуют», если только вы не намерены создать реалистичную вероятностную модель существования единорогов.

Теорема Байеса занимает центральное место в байесовской статистике, и как мы увидим в главе 2 «Вероятностное программирование», использование таких инструментов, как PyMC3, освобождает от необходимости всякий раз явно записывать теорему Байеса при создании байесовской модели. Тем не менее важно знать название каждого элемента теоремы, поскольку мы постоянно будем ссылаться на эти элементы, и весьма важно понимать, что означает

каждый элемент, так как это помогает теоретическому обоснованию (концептуализации) моделей. Ниже приведены обозначения и соответствующие названия элементов теоремы Байеса:

- $p(\theta)$ – априорная вероятность;
- $p(y|\theta)$ – правдоподобие;
- $p(\theta|y)$ – апостериорная вероятность;
- $p(y)$ – предельное правдоподобие.

Априорная вероятность должна соответствовать тому, что нам известно о значении параметра θ перед рассмотрением данных y . Если нам ничего неизвестно, как Джону Сноу¹, то можно использовать постоянные фиксированные априорные вероятности, которые не содержат сколько-нибудь значимого объема информации. Вообще говоря, можно выбрать более удачный вариант, чем фиксированные априорные вероятности, как мы увидим далее в этой книге. Использование априорных вероятностей – это основная причина, по которой некоторые люди продолжают называть байесовскую статистику субъективной, даже если априорные вероятности представляют собой всего лишь другой способ предположений, формулируемых при моделировании, следовательно, являются субъективными (или объективными) в той же мере, что и любые другие предположения, такие как правдоподобия.

Правдоподобие определяет, как будут представлены данные в дальнейшем анализе. Это выражение правдоподобности данных с учетом принятых параметров. В некоторых публикациях используются термины «сэмплинг-модель», «статистическая модель» или просто «модель». Мы продолжим использовать термин «правдоподобие» и будем обозначать им комбинацию априорных вероятностей и модели правдоподобия.

Апостериорная вероятность – это результат байесовского анализа, который отображает все, что известно о задаче (проблеме) с учетом имеющихся данных и используемой модели. Апостериорная вероятность – это распределение вероятностей для θ параметров в используемой модели, и это не единственное значение. Такое распределение представляет собой баланс между априорной вероятностью и правдоподобием. Существует широко известная шутка: «Байесовский аналитик – это тот, кто смутно надеется увидеть лошадь, но, заметив быстро промелькнувшего осла, твердо верит, что видел мула». Превосходным способом ликвидации впечатления от этой шутки является объяснение того, что если правдоподобие и априорные вероятности неясны и смутны, то вы получите апостериорную вероятность, отражающую «смутную веру» в то, что видели мула, а не твердую уверенность. В любом случае мне по душе эта шутка и мне нравится, как точно она выражает мысль о том, что апостериорная вероятность является в некоторой степени компромиссом между априорной вероятностью и правдоподобием. С теоретической концептуальной точки зрения

¹ Джон Сноу (Jon Snow) – один из главных персонажей цикла романов Джорджа Р. Р. Мартина «Песнь льда и огня» и телесериала «Игра престолов». – *Прим. перев.*

можно воспринимать апостериорную вероятность как обновленную априорную вероятность в свете (новых) данных. В действительности апостериорная вероятность, полученная в результате одного процесса анализа, может использоваться как априорная вероятность для нового процесса анализа. Это свойство делает байесовский анализ особенно подходящим для анализа данных, которые становятся доступными в определенном последовательном порядке. Примерами могут служить системы раннего оповещения о природных катастрофах, которые обрабатывают в режиме онлайн данные, поступающие с метеорологических станций и спутников. Более подробно об этом можно узнать в публикациях о методах онлайн-машинного обучения.

Последний элемент и термин – предельное правдоподобие, также называемое обоснованностью. Формально предельное правдоподобие – это вероятность исследуемых данных, усредненная по всем возможным значениям, которые могут принимать параметры (в соответствии с предварительно описанной априорной вероятностью). В любом случае практически во всей этой книге мы не уделяем особого внимания предельному правдоподобию и будем считать его простым фактором нормализации. Такой подход принят потому, что при анализе распределения апостериорной вероятности нас будут интересовать только относительные, а не абсолютные значения параметров. Возможно, вы помните, что мы упоминали это обстоятельство при обсуждении методики интерпретации графиков распределений вероятности в предыдущем разделе. Если не принимать во внимание предельное правдоподобие, то можно записать теорему Байеса как прямое пропорциональное отношение:

$$p(\theta, y) \propto p(y|\theta)p(\theta). \quad (1.8)$$

Точное понимание роли каждого элемента (и смысл соответствующего термина) теоремы Байеса занимает некоторое время и требует определенной практики. Также потребуются работа с рядом примеров в следующих главах книги.

БАЙЕСОВСКИЙ ВЫВОД С ОДНИМ ПАРАМЕТРОМ

В двух предыдущих разделах рассматривались некоторые важные концепции, но две из них представляют собой самые важные компоненты ядра байесовской статистики, поэтому необходимо кратко напомнить их смысл.

! Вероятности используются для измерения неопределенности, присущей параметрам, а теорема Байеса – это механизм правильного обновления этих вероятностей при поступлении новых данных в расчете на уменьшение существующей неопределенности.

Теперь, когда мы знаем, что такое байесовская статистика, рассмотрим, как ее применять на практике, с помощью простого примера. Начнем с байесовского вывода с одним неизвестным параметром.

Задача о подбрасывании монеты

Задача о подбрасывании монеты, или модель бета-биномиального распределения (если нужно вставить солидно звучащий термин в беседу), – это классическая задача статистики, которая выполняется следующим образом: монета подбрасывается некоторое количество раз, и фиксируется, сколько выпало орлов и решек. На основе полученных в результате данных мы пытаемся ответить на такие вопросы, как: является ли монета настоящей («правильной»)? Или в более общем смысле: насколько несимметричной («неправильной») является монета? Хотя эта задача может казаться чрезвычайно скучной, не следует ее недооценивать. Задача о подбрасывании монеты представляет собой великолепный пример для обучения основам байесовской статистики, поскольку это простая модель, которую можно с легкостью решить и вычислить. Кроме того, многие реальные практические задачи определяются бинарными, взаимоисключающими итоговыми результатами, такими как 0 или 1, положительный или отрицательный ответ, четные или нечетные числа, спам или не спам, удача или неудача, кот или собака, безопасно или небезопасно, здоровый или больной. Таким образом, даже если мы говорим о монетах, эта модель применима к любой из вышеперечисленных задач.

Для того чтобы определить отклонение монеты от нормы и в общем смысле ответить на любые вопросы при байесовском подходе, потребуются данные и вероятностная модель. В рассматриваемом здесь примере предполагается, что монета уже подброшена некоторое количество раз и у нас есть записи о количестве выпавших орлов и решек, так что этап сбора данных уже выполнен. Для выбора модели потребуется немного больше усилий. Поскольку это самая первая наша модель, запишем теорему Байеса в ее основной форме и выполним необходимые математические операции (не пугайтесь, обещаю, что все математические выкладки будут простыми и понятными). Такой способ будет работать очень медленно. Но в главе 2 «Вероятностное программирование» мы воспользуемся библиотекой PyMC3, и компьютер произведет все необходимые математические вычисления.

Общая модель

В первую очередь необходимо обобщить концепцию отклонения (от нормы). Будем считать, что монета с отклонением 1 всегда падает орлом вверх, а монета с отклонением 0 всегда падает вверх решкой. При отклонении 0.5 в половине случаев выпадает орел, в половине случаев – решка. Для представления отклонения будет использоваться параметр θ , а при общем количестве подбрасываний монеты N число выпавших орлов обозначается переменной Y . В соответствии с теоремой Байеса (формула 1.4) необходимо определить априорную вероятность $p(\theta)$ и правдоподобие $p(Y|\theta)$, которые будут использоваться в примере. Начнем с определения правдоподобия.

Выбор правдоподобия

Предположим, что возможны только два итоговых результата – выпадение орла или решки. Также предположим, что любое подбрасывание монеты никак не влияет на другие броски, то есть предполагается, что все подбрасывания монеты абсолютно независимы друг от друга. Далее предположим, что все подбрасывания монеты подчиняются одному и тому же распределению. Таким образом, случайная переменная «подбрасывание монеты» представляет собой пример независимой одинаково распределенной случайной переменной, или нор-переменной. Надеюсь, что читатели согласны с этими достаточно разумными предположениями для определения нашей задачи. С учетом сделанных предположений подходящим кандидатом для выражения правдоподобия является биномиальное распределение:

$$p(y|\theta, N) = \frac{N!}{y!(N-y)!} \theta^y (1-\theta)^{N-y}. \quad (1.9)$$

Это дискретное распределение, возвращающее вероятность выпадения Y орлов (или в обобщенном смысле успешных результатов) при N подбрасываниях монеты (или в обобщенном смысле испытаний либо экспериментов) с учетом постоянного значения θ :

```
n_params = [1, 2, 4] # Количество испытаний
p_params = [0.25, 0.5, 0.75] # Вероятность успешного результата
x = np.arange(0, max(n_params)+1)
f, ax = plt.subplots(len(n_params), len(p_params), sharex=True,
                    sharey=True,
                    figsize=(8, 7), constrained_layout=True)
for i in range(len(n_params)):
    for j in range(len(p_params)):
        n = n_params[i]
        p = p_params[j]
        y = stats.binom(n=n, p=p).pmf(x)
        ax[i,j].vlines(x, 0, y, colors='C0', lw=5)
        ax[i,j].set_ylim(0, 1)
        ax[i,j].plot(0, 0, label="N = {:.2f}\nθ =
                        {:.2f}".format(n,p), alpha=0)
        ax[i,j].legend()
ax[2,1].set_xlabel('y')
ax[1,0].set_ylabel('p(y|θ, N)')
ax[0,0].set_xticks(x)
```

На рис. 1.3 показано девять биномиальных распределений, каждая схема снабжена собственным описанием (легендой), определяющим значения параметров. Отметим, что на этих графиках указаны значения по оси y . Это сделано для того, чтобы вы могли сами проверить результаты: если сложить высоты всех столбцов на отдельной схеме, то получится 1, то есть для дискретных распределений высоты столбцов на схеме представляют действительные вероятности.

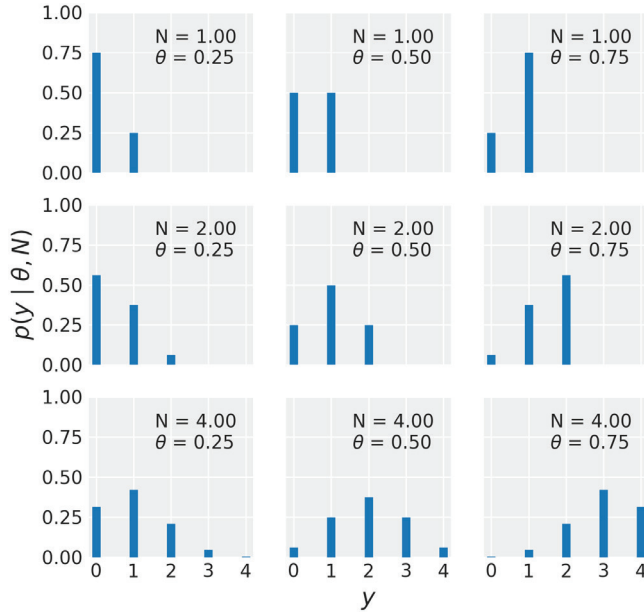


Рис. 1.3

Биномиальное распределение является обоснованным выбором для представления правдоподобия. Можно видеть, что θ обозначает, насколько вероятным является выпадение орла при подбрасывании монеты (это проще всего наблюдать при $N = 1$, но остается справедливым для любого значения N) – нужно просто сравнить значение θ с высотой столбца для $y = 1$ (орлы).

Итак, если нам известно значение θ , то биномиальное распределение покажет ожидаемое распределение выпадений орлов. Единственная проблема состоит в том, что нам неизвестно значение θ . Но не следует отчаиваться, потому что в байесовской статистике при неизвестном значении параметра для него принимается априорная вероятность. Двинемся дальше и выберем априорную вероятность для θ .

Выбор априорной вероятности

В качестве априорной вероятности будет использоваться бета-распределение – наиболее часто применяемое распределение в байесовской статистике, которое описывается следующей формулой:

$$p(\theta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1} (1-\theta)^{\beta-1}. \quad (1.10)$$

Если внимательно взглянуть на эту формулу, то можно заметить, что бета-распределение похоже на биномиальное распределение, за исключением первого множителя, в котором используется только функция Γ . Первый мно-

житель – это константа нормализации, которая обеспечивает равенство 1 при суммировании элементов распределения. Отметим, что здесь Γ – это греческая заглавная (прописная) буква гамма, которая обозначает гамма-функцию. В формуле 1.10 также можно видеть, что бета-распределение имеет два параметра, α и β . С помощью приведенного ниже кода исследуем третье используемое нами распределение:

```
params = [0.5, 1, 2, 3]
x = np.linspace(0, 1, 100)
f, ax = plt.subplots(len(params), len(params), sharex=True,
                    sharey=True,
                    figsize=(8, 7), constrained_layout=True)
for i in range(4):
    for j in range(4):
        a = params[i]
        b = params[j]
        y = stats.beta(a, b).pdf(x)
        ax[i,j].plot(x, y)
        ax[i,j].plot(0, 0, label="a = {:.21f}\nb = {:.21f}".format(a,
            b), alpha=0)
        ax[i,j].legend()
ax[1,0].set_yticks([])
ax[1,0].set_xticks([0, 0.5, 1])
f.text(0.5, 0.05, '\theta', ha='center')
f.text(0.07, 0.5, 'p(\theta)', va='center', rotation=0)
```

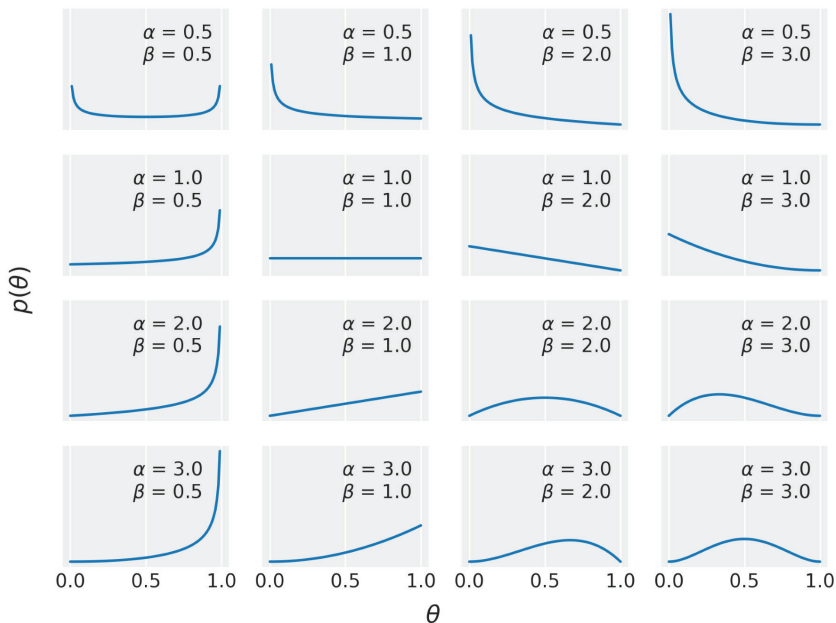


Рис. 1.4

Мне действительно нравится бета-распределение и все формы, которые можно получить из него, но почему мы воспользовались именно этим распределением для своей модели? Существует множество обоснований использования бета-распределения для этой и других задач. Одно из обоснований состоит в том, что бета-распределение ограничено интервалом от 0 до 1, точно так же, как параметр θ . В общем случае бета-распределение используется, когда необходимо создать модель, пропорциональную биномиальной переменной. Другим обоснованием может служить универсальность бета-распределения. На рис. 1.4 можно видеть, что бета-распределение принимает несколько разнообразных форм (всегда ограниченных интервалом $[0, 1]$), в том числе равномерное распределение, гаусс-подобные распределения и U-образные распределения. Третье обоснование: бета-распределение – это сопряженная априорная вероятность биномиального распределения (которое мы используем как представление правдоподобия). Сопряженное априорное распределение правдоподобия – это априорная вероятность, которая при использовании в сочетании с заданным правдоподобием возвращает апостериорную вероятность в той же самой функциональной форме, в которой была представлена априорная вероятность. Проще говоря, каждый раз при использовании бета-распределения как априорной вероятности и биномиального распределения как правдоподобия мы получаем бета как апостериорное распределение. Существуют и другие пары сопряженных априорных распределений, например нормальное распределение является сопряженным априорным распределением с самим собой. В течение долгого времени байесовский анализ был ограничен применением сопряженных априорных распределений. Сопряженность обеспечивала удобство математического манипулирования апостериорной вероятностью, а это весьма важно с учетом того, что общей проблемой байесовской статистики является получение конечного результата в виде апостериорной вероятности, которая не может быть разрешена аналитически. Это было камнем преткновения, пока не были разработаны эффективные вычислительные методы для решения вероятностных задач. В главе 2 мы научимся применять современные вычислительные методы для решения байесовских задач вне зависимости от выбора сопряженных априорных распределений или отказа от них.

Получение апостериорной вероятности

Вспомним, что теорема Байеса (формула 1.4) определяет апостериорную вероятность как пропорциональную произведению правдоподобия на априорную вероятность. Следовательно, в нашей задаче необходимо перемножить биномиальное распределение и бета-распределение:

$$p(\theta|y) \propto \frac{N!}{y!(N-y)!} \theta^y (1-\theta)^{N-y} \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1} (1-\theta)^{\beta-1}. \quad (1.11)$$

Это выражение можно упростить. Из конкретных практических соображений можно отбросить все множители, которые не зависят от θ , при этом ре-

зультат останется корректным. Поэтому выражение можно записать следующим образом:

$$p(\theta|y) \propto \theta^y (1 - \theta)^{N-y} \theta^{\alpha-1} (1 - \theta)^{\beta-1}. \quad (1.12)$$

После очередного преобразования получаем:

$$p(\theta|y) \propto \theta^{y+\alpha-1} (1 - \theta)^{N-y+\beta-1}. \quad (1.13)$$

При внимательном изучении полученной формулы можно заметить, что она имеет ту же функциональную форму, что и формула бета-распределения (за исключением нормализующего множителя) с учетом того, что $\alpha_{\text{posterior}} = \alpha_{\text{prior}} + y$ и $\beta_{\text{posterior}} = \beta_{\text{prior}} + N - y$. В действительности апостериорное распределение для нашей задачи является бета-распределением:

$$p(\theta|y) \propto \text{Beta}(\alpha_{\text{prior}} + y, \beta_{\text{prior}} + N - y). \quad (1.14)$$

Вычисление и графическое отображение апостериорной вероятности

Теперь воспользуемся средствами языка Python для вычисления и графического отображения апостериорного распределения на основе аналитических формул, выведенных в предыдущем разделе. В следующем коде можно видеть, что в действительности только одна строка вычисляет результаты, тогда как все прочие предназначены для построения наглядных графических схем:

```
plt.figure(figsize=(10, 8))

n_trials = [0, 1, 2, 3, 4, 8, 16, 32, 50, 150]
data = [0, 1, 1, 1, 1, 1, 4, 6, 9, 13, 48]
theta_real = 0.35

beta_params = [(1, 1), (20, 20), (1, 4)]
dist = stats.beta
x = np.linspace(0, 1, 200)

for idx, N in enumerate(n_trials):
    if idx == 0:
        plt.subplot(4, 3, 2)
        plt.xlabel('θ')
    else:
        plt.subplot(4, 3, idx+3)
        plt.xticks([])
    y = data[idx]
    for (a_prior, b_prior) in beta_params:
        p_theta_given_y = dist.pdf(x, a_prior + y, b_prior + N - y)
        plt.fill_between(x, 0, p_theta_given_y, alpha=0.7)
    plt.axvline(theta_real, ymax=0.3, color='k')
    plt.plot(0, 0, label=f'{N:4d} trials\n{y:4d} heads', alpha=0)
    plt.xlim(0, 1)
    plt.ylim(0, 12)
    plt.legend()
    plt.yticks([])
plt.tight_layout()
```

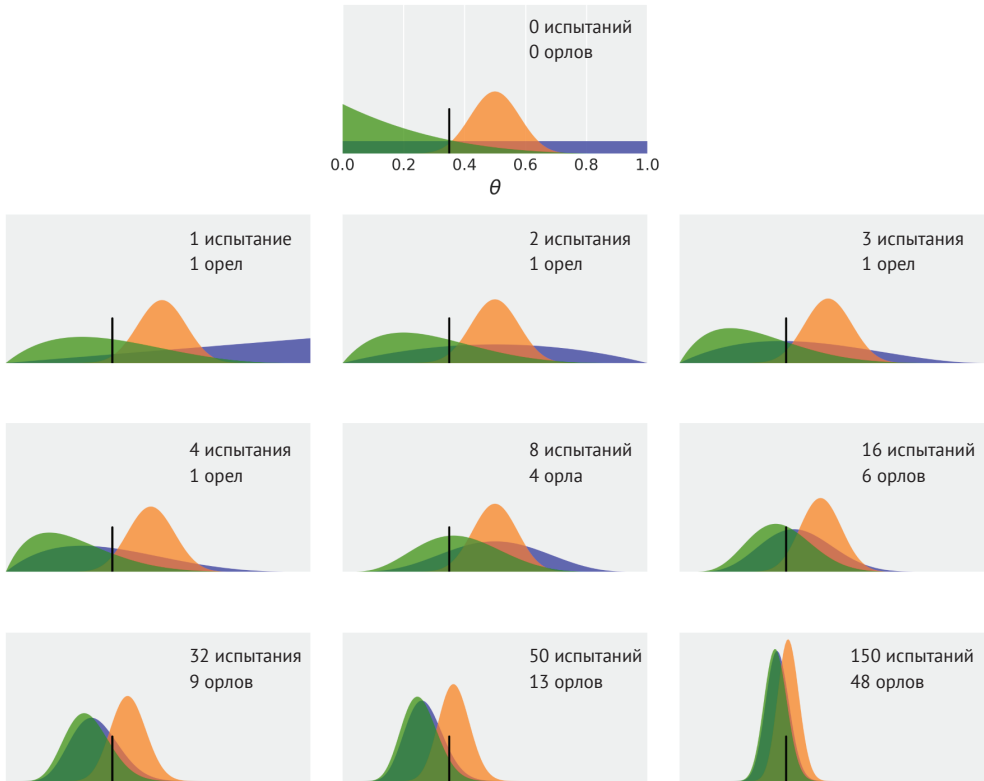


Рис. 1.5

На самом первом (верхнем) графике рис. 1.5 показано состояние при нуле испытаний, следовательно, три кривые представляют априорные распределения вероятностей:

- универсальное (синий цвет) априорное распределение, которое представляет все возможные значения для отклонения, вероятно равного априори (то есть до испытания);
- гаусс-подобное (оранжевый цвет) априорное распределение сконцентрировано в центральной области около значения 0.5, таким образом, это априорное распределение совместимо с информацией, сообщающей, что монета имеет более или менее равные шансы упасть вверх орлом или решкой при подбрасывании. Также можно сказать, что это априорное распределение совместимо со степенью уверенности в том, что монета является «правильной». Несмотря на то что понятие степени уверенности широко используется при обсуждении байесовского анализа, здесь мы предпочитаем говорить о моделях и параметрах, для которых информацию предоставляют данные;

- асимметричное (зеленый цвет) априорное распределение придает наибольший вес результатам с отклонением в сторону выпадения решки.

Все остальные графики на рис. 1.5 отображают апостериорные распределения для последовательно проводимых испытаний. Количество испытаний (подбрасываний монеты) и количество выпавших орлов показано в легенде каждого графика. Кроме того, на всех графиках изображена черная вертикальная линия на абсциссе 0.35, представляющая истинное значение для θ . Разумеется, в реальных задачах это значение неизвестно, но здесь оно приведено только в учебных целях. Графики на рис. 1.5 могут помочь узнать очень много о байесовском анализе, так что приготовьте чашку своего любимого напитка и приступайте к внимательному изучению:

- результатом байесовского анализа является апостериорное распределение, а не единственное значение, но распределение правдоподобных значений предоставляет данные и выбранную нами модель;
- наиболее вероятное значение выбирается по моде апостериорного распределения (то есть по пиковому значению этого распределения);
- размах (разброс) апостериорного распределения пропорционален неопределенности, присущей значению параметра: чем больше размах распределения, тем меньше степень уверенности;
- интуитивно мы более уверены в результате, если наблюдаем больше данных, подтверждающих этот результат. Таким образом, даже если в числовом выражении $1/2 = 4/8 = 0.5$, наблюдение четырех выпавших орлов в восьми испытаниях дает большую степень уверенности в том, что отклонение равно 0.5, нежели наблюдение одного выпавшего орла в двух испытаниях. Эта интуитивная уверенность отображается в апостериорном распределении, так как можно проверить себя, если обратить внимание на схему апостериорного распределения (синий цвет) на третьем и шестом графиках. Несмотря на то что мода одинакова, размах (неопределенность) больше на третьем графике, чем на шестом;
- при наличии достаточно большого объема данных две и более байесовские модели с различными априорными распределениями будут демонстрировать сходимость к одному и тому же результату. В экстремальном случае при бесконечных данных не имеет значения, какое априорное распределение используется – при любом распределении результатом является одно и то же апостериорное распределение. Напомним, что бесконечность – это предельный случай, который не имеет числового выражения, поэтому с практической точки зрения существует возможность получения по существу не различающихся между собой апостериорных распределений при конечном и относительно небольшом количестве точек данных;
- скорость сходимости апостериорных распределений к одной форме распределения зависит от данных и от модели. На рис. 1.5 можно видеть, что апостериорные распределения, происходящие от универ-

сального априорного распределения (синий цвет) и от априорного распределения с отклонением в сторону выпадения решек (зеленый цвет), быстрее сходятся к почти одинаковой форме распределения при большей длительности испытаний по сравнению с апостериорным распределением, показанным оранжевым цветом (происходящим от концентрированного априорного распределения). В действительности даже после 150 испытаний можно достаточно легко отличить апостериорное распределение, обозначенное оранжевым цветом, от двух других распределений;

- по рис 1.5 не очевидно, что можно получить тот же результат, если последовательно обновлять апостериорное распределение после каждого испытания вместо однократного итогового определения. Можно вычислять апостериорное распределение 150 раз, добавляя по одному наблюдению испытания и используя полученный результат как новое априорное распределение, или можно один раз вычислить апостериорное распределение после 150 бросков монеты. Результат будет абсолютно одинаковым. Это свойство не только имеет особый (положительный) смысл, но также естественным образом приводит к обновлению наших прогнозов при получении новых данных. Подобная ситуация является вполне обычной при решении многих задач анализа данных.

Влияние априорной вероятности и методика ее выбора

Из примера, приведенного в предыдущем разделе, понятно, что априорные вероятности могут влиять на логические выводы. В целом это положительное влияние. Предполагается, что априорные вероятности обладают этим свойством. Начиная изучать байесовский анализ (а также критики этой парадигмы) часто испытывают легкое беспокойство по поводу выбора априорных вероятностей, поскольку им кажется нежелательным, чтобы априорная вероятность действовала как цензор, не позволяющий данным говорить самим за себя. Это правильно, но следует помнить, что в действительности данные далеко не всегда говорят, в лучшем случае они шепчут. Данные имеют смысл только в контексте выбранных нами моделей, включая математические и мысленные модели. В истории науки можно найти множество примеров, когда одни и те же данные приводили людей к различному восприятию и образу мышления в одной области. Такое может происходить, даже если взгляды и выводы исследователя основаны на формальных моделях.

Некоторым людям нравится идея использования неинформативных априорных вероятностей (также называемых плоскими, неопределенными (нечеткими) или диффузными априорными вероятностями). Этот тип априорных вероятностей оказывает наименьшее возможное воздействие на анализ. Можно воспользоваться этой идеей, но, вообще говоря, есть возможность выбрать более эффективное решение. На протяжении этой книги мы будем следовать рекомендациям Гельмана (Andrew Gelman), МакЭлриша (McElreath), Крушке (John Kruschke) и многих других, то есть отдадим предпочтение сла-

бо информативным (или не вполне информативным) априорным вероятностям. При решении многих задач часто известно кое-что о значениях, которые могут принимать параметры, то есть можно знать, что диапазон параметра ограничен только положительными значениями, или известен приблизительный возможный интервал его значений, или можно предположить, что значение параметра близко к нулю или меньше/больше некоторого конкретного значения. В таких случаях можно использовать априорные вероятности для передачи некоторой нечеткой («слабой») информации в применяемые модели, не боясь оказаться излишне настойчивыми. Поскольку такие априорные вероятности действуют как факторы, позволяющие сохранять апостериорное распределение в определенных разумных границах, их также называют регуляризирующими априорными вероятностями. Использование информативных априорных вероятностей также является возможным вариантом, если мы располагаем высококачественной информацией для определения априорной вероятности. Информативные априорные вероятности – это весьма строгие априорные вероятности, содержащие и передающие большой объем информации. В зависимости от конкретной задачи нахождение этого типа априорной вероятности может оказаться простым или сложным делом. Например, в области науки, которой занимаюсь я (структурная биоинформатика), исследователи применяют как байесовские, так и отличающиеся от них методики. Здесь вся предварительная (априорная) информация, которую можно использовать для изучения и особенно прогнозирования, – это структура протеинов. Это прочное обоснование, так как нами собраны данные из тысяч тщательно запланированных и проведенных экспериментов в течение десятилетий, следовательно, в нашем распоряжении имеется огромный объем надежной и достоверной предварительной информации для дальнейших исследований. Не воспользоваться этой априорной информацией было бы абсурдом. Отсюда главный вывод: если имеется надежная предварительная информация, то нет никаких причин отказываться от ее использования. Не следует принимать во внимание довольно бессмысленное возражение по поводу того, что быть объективным – значит отбрасывать всю полезную значимую информацию. Представьте, что было бы, если бы инженер по проектированию автомобилей, каждый раз приступая к созданию новой машины, начинал бы с нуля и вновь изобретал двигатель внутреннего сгорания, колесо и вообще всю концепцию современного автомобиля. Это абсолютно неподходящий метод для эффективной работы.

Возможность классификации априорных вероятностей по категориям, соответствующим их относительной мощности (в смысле информативности), не делает более легким выбор одной из них. Возможно, лучше было бы совсем отказаться от априорных вероятностей – это упростило бы моделирование, не так ли? Вероятно, это так, но не всегда. Априорные вероятности могут улучшить поведение моделей, придать им более эффективные обобщающие свойства и помочь передать для нас полезную информацию. Кроме того, любая мо-

дель, байесовская или другая, обладает некоторой разновидностью априорной вероятности в той или иной форме, даже если априорная вероятность явно не определяется. В действительности многие результаты частотной статистики могут рассматриваться как особые случаи байесовской модели при определенных обстоятельствах, таких как плоские априорные вероятности. Широко применяемый метод частотной статистики для оценки параметров известен как метод максимального правдоподобия. Этот метод избегает определения априорной вероятности и действует только посредством поиска значения θ , которое дает максимальное правдоподобие. Это значение обычно записывают с добавлением небольшого символа (циркумфлекса) над буквой оцениваемого параметра, например $\hat{\theta}$ или иногда θ_{mle} (или даже оба варианта). $\hat{\theta}$ – это точка оценки (число), а не распределение. В задаче о подбрасывании монеты это значение можно вычислить аналитически:

$$\hat{\theta} = \frac{Y}{N}. \quad (1.15)$$

Если вернуться к рис. 1.5, то можно самостоятельно проверить, что мода обозначенного синим цветом апостериорного распределения (соответствующего равномерной/плоской априорной вероятности) согласовывается со значениями $\hat{\theta}$, вычисленными для каждого графика. Поэтому, по крайней мере в этом примере, можно видеть, что даже если метод максимального правдоподобия не обращается в явном виде к какой-либо априорной вероятности, его можно считать особым случаем байесовской модели, относящейся к типу с равномерной априорной вероятностью.

В действительности практически невозможно уйти от априорных вероятностей, но если они включаются в анализ, то мы получаем некоторые преимущества, в том числе распределение правдоподобных значений, а не единственное, наиболее вероятное значение. Другим преимуществом явного использования априорных значений является формирование более прозрачных моделей, то есть упрощается их критическая оценка, отладка (в самом широком смысле этого слова) и усовершенствование (как можно надеяться). Создание моделей – итеративный процесс, иногда итерации занимают несколько минут, иногда на это уходят годы. Иногда в этом процессе участвуете только вы, иногда к работе привлекаются люди, с которыми вы до этого не были знакомы. Добавляют сложность проблемы воспроизводимости (результатов) и очевидные предварительные предположения о модели. Кроме того, ничто не мешает использовать более одной априорной вероятности (или функции правдоподобия) для выполняемого анализа, если мы не уверены в каком-либо одном типе. Изучение эффектов от применения различных априорных вероятностей также может предоставлять ценную информацию исследователю. Неотъемлемой частью процесса моделирования являются предположения, требующие проверки, и использование априорных вероятностей (и функций правдоподобия). Различные предварительные предположения приводят к соз-

данию разнообразных моделей и, вероятнее всего, к различным результатам. Используя данные и собственную область знаний о конкретной решаемой задаче, мы получаем возможность сравнивать модели и при необходимости выбирать наилучший вариант. В главе 5 «Сравнение моделей» мы более подробно рассмотрим эту тему. Поскольку априорные вероятности играют главную роль в байесовской статистике, мы продолжим обсуждать их свойства и характеристики при рассмотрении решения каждой новой задачи. И даже если у вас все же остаются сомнения и небольшое недопонимание, наберитесь терпения и не беспокойтесь – многие люди десятилетиями сомневались и недопонимали эту тему, а активные дискуссии продолжаются и сейчас.

ВЗАИМОДЕЙСТВИЕ С БАЙЕСОВСКИМ АНАЛИЗОМ

Создание отчетов и обмен результатами – это центральный пункт в практическом применении статистики и науки о данных. В этом разделе кратко рассматриваются характерные особенности этой задачи при работе с байесовскими моделями. В следующих главах продолжится рассмотрение примеров, связанных с этой весьма важной темой.

Нотация и визуализация модели

Если необходимо обменяться результатами анализа (или опубликовать их), то вы должны также обеспечить обмен информацией с используемой моделью. общепринятая нотация (система записи) для компактного представления вероятностных моделей выглядит следующим образом:

$$\begin{aligned}\theta &\sim \text{Beta}(\alpha, \beta); \\ y &\sim \text{Bin}(n=1, p=\theta).\end{aligned}\tag{1.16}$$

Это простая модель, которая используется в примере с подбрасыванием монеты. Как вы помните, символ $\sim$ (тильда) означает, что переменная слева от него является случайной переменной, описанной формулой распределения, расположенной в правой части. В большинстве контекстов символ тильда используется для обозначения того факта, что переменная принимает некоторое значение, определяемое лишь приблизительно, но при рассмотрении вероятностных моделей необходимо интерпретировать этот символ как «переменная определяется распределением». Таким образом, можно говорить, что переменная θ определяется бета-распределением с параметрами α и β , а переменная y определяется биномиальным распределением с параметрами $n = 1$ и $p = \theta$. Эту же модель можно представить графически с помощью диаграмм Круске (рис. 1.6).

На первом уровне диаграммы показана априорная вероятность, которая генерирует значения для θ , ниже – функция правдоподобия, и на самом нижнем уровне – данные y . Стрелки показывают отношения между переменными, а символ $\sim$ отображает случайную (стохастическую) природу этих перемен-

ных. Все диаграммы Крушке в этой книге выполнены с использованием шаблонов, предоставленных Расмусом Боотом (Rasmus Bååth) (<http://www.sumsar.net/blog/2013/10/diy-kruschke-style-diagrams/>).

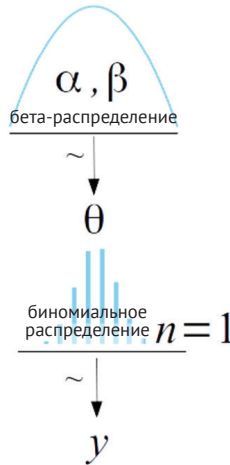


Рис. 1.6

Обобщение апостериорного распределения

Результатом байесовского анализа является апостериорное распределение, а вся информация о параметрах исследуемого набора данных и модели содержится в апостериорном распределении. Таким образом, обобщая апостериорное распределение, мы обобщаем логические выводы о модели и о данных. Общепринятым практическим методом является отчет по каждому параметру с указанием среднего значения (или моды, или срединного значения) для получения общего представления о размещении распределения и некоторой меры, например такой, как стандартное (среднеквадратическое) отклонение, а также для получения представления о вариации, следовательно, о неопределенности полученной оценки. Стандартное отклонение хорошо подходит для распределений нормального типа, но может оказаться ошибочным для других типов распределений, таких как асимметричные распределения.

Плотность апостериорного распределения

Широко применяемым приемом для обобщения размаха апостериорного распределения является использование интервала плотности апостериорного распределения (Highest-Posterior Density – HPD). Интервал плотности апостериорного распределения (ПАР) – это самый короткий интервал, содержащий определенную часть плотности распределения. Чаще всего применяется интервал ПАР 95 %, часто дополняемый 50%-ным интервалом ПАР. Если интервал ПАР 95 % для некоторого анализа [2–5], то для исследуемых данных и ис-

пользуемой модели можно считать, что рассматриваемый параметр имеет значение в интервале от 2 до 5 с вероятностью 0.95.

❗ Выбор 95 %, 50 % или любого другого значения не имеет какого-либо особого смысла. Это просто произвольно выбираемые часто используемые значения. При желании можно выбрать интервал ПАР 91.37 %. Если вы предпочитаете интервал ПАР 95 %, то это правильный выбор, следует помнить, что фактически это значение по умолчанию. В идеальном случае интервал ПАР не устанавливается автоматически, а должен выбираться в зависимости от текущего контекста.

ArviZ – это пакет языка Python для выполнения анализа данных с использованием байесовских моделей. ArviZ содержит множество функций, помогающих обобщить апостериорное распределение, например `az.plot_posterior` может использоваться для генерации графика со средним значением и интервалом ПАР. В приведенном ниже примере вместо апостериорного распределения по результатам реально выполненного анализа генерируется случайная выборка из бета-распределения:

```
np.random.seed(1)
az.plot_posterior({'θ': stats.beta.rvs(5, 11, size=1000)})
```

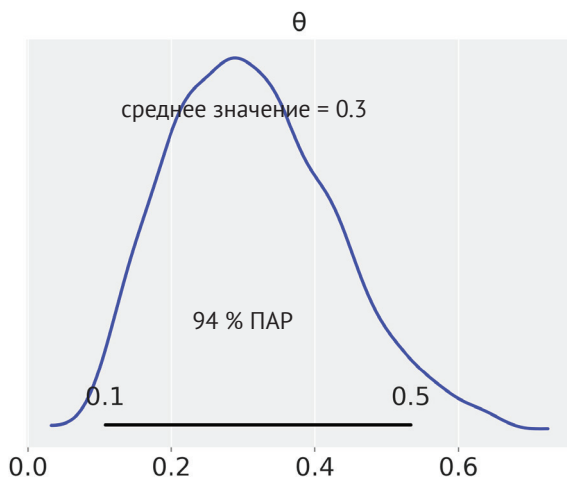


Рис. 1.7

Отметим, что на рис. 1.7 указан интервал ПАР 94 %. Это еще одно ненавязчивое напоминание о произвольности выбора значения 95 %. При каждом вычислении и указании в отчете ПАР ArviZ по умолчанию будет использовать значение 0.94 (соответствующее 94 %). Это значение можно изменить, передавая в аргументе `credible_interval` другое число.

! Если вы знакомы с парадигмой частотной вероятности, то обратите внимание на то, что интервалы ПАР – это не то же самое, что доверительные интервалы (confidence intervals). Интерпретация интервала ПАР весьма интуитивна до такой степени, что люди часто неправильно воспринимают частотные доверительные интервалы как байесовские доверительные интервалы (Bayesian credible intervals). Выполнение полноценного байесовского анализа позволяет говорить о вероятности того, что некоторый параметр имеет некоторое значение. Но это невозможно в среде частотной статистики, так как параметры имеют фиксированные значения по факту, таким образом, частотный доверительный интервал либо содержит, либо не содержит истинное значение параметра.

ПРОВЕРКИ АПОСТЕРИОРНОГО ПРОГНОЗИРУЕМОГО РАСПРЕДЕЛЕНИЯ

Одним из полезных элементов байесовского комплекта инструментальных средств является наличие апостериорного распределения вероятностей, что позволяет использовать апостериорное распределение $p(\theta|y)$ для генерации прогнозов $\hat{y}$ на основе данных y и предполагаемых оценочных параметров θ . Апостериорное прогнозируемое распределение выражается следующей формулой:

$$p(\hat{y}|y) = \int p(\hat{y}|\theta)p(\theta|y)d\theta. \quad (1.17)$$

Таким образом, апостериорное прогнозируемое распределение является усреднением условных прогнозов по апостериорному распределению параметра θ . С теоретической точки зрения (и с точки зрения выполнения вычислений) интеграл в формуле 1.17 можно приближенно представить в виде итеративного процесса, состоящего из следующих двух этапов:

- 1) выбор значения θ из апостериорного распределения $p(\theta|y)$;
- 2) передача выбранного значения θ в функцию правдоподобия (или, другими словами, выборочное распределение), позволяющую получить точку данных $\tilde{y}$.

! Следует особо отметить, как в этом процессе объединяются два источника неопределенности: неопределенность параметров в том виде, в котором она фиксируется апостериорным распределением, и неопределенность выборки в том виде, в котором она принимается функцией правдоподобия.

Генерируемые прогнозы $\tilde{y}$ могут использоваться в тех случаях, когда необходимо сделать, если можно так выразиться, собственно прогнозы. Но их также можно использовать для критического обзора моделей при сравнении наблюдаемых данных y и прогнозируемых данных $\tilde{y}$ для выявления различий между этими двумя наборами. Этот процесс называют проверками апостериорного прогнозируемого распределения (posterior predictive checks). Главная цель – проверка собственной согласованности и целостности. Сгенерированные и наблюдаемые данные должны выглядеть более или менее одинаково, в противном случае явно существует проблема, возникающая при моделирова-

нии или при передаче данных в модель. Но даже при отсутствии ошибок могут существовать различия. Попытка понять природу этого несоответствия может привести к усовершенствованию моделей или, по крайней мере, к пониманию их ограничений. Знание того, какие части задачи/данных точно соответствуют модели, а какие не вполне соответствуют, предоставляет полезную информацию, даже если неизвестно, как можно усовершенствовать модель. Применяемая модель может правильно сохранить усредненное поведение исследуемых данных, но совершать критические ошибки при прогнозировании редко встречающихся значений. Это может стать проблемой на практике, но если нас интересует только среднестатистический результат, то такая модель может быть признана вполне удовлетворительной. Главная цель проверок заключается не в том, чтобы объявить модель неправильной. Необходимо лишь узнать, какой части модели можно доверять, и попытаться проверить, достаточно ли хорошо подходит эта модель для достижения конкретной поставленной цели. Степень уверенности в выбранной модели может быть абсолютно различной в разных областях деятельности и науки. Физики могут изучать системы с полностью управляемыми условиями с использованием теорий высокого уровня абстракции, поэтому их модели часто выглядят как достаточно точные описания реальной действительности. Другие дисциплины, такие как социология и биология, изучают сложные, трудно поддающиеся изоляции системы, поэтому модели обычно имеют более слабый эпистемологический статус. Но в любом случае, независимо от любой области деятельности модели всегда должны проверяться, и одним из надежных способов являются проверки апостериорного прогнозируемого распределения в совокупности с выводами, полученными при исследовательском анализе данных.

РЕЗЮМЕ

Мы начали изучение байесовского анализа с очень краткого описания статистического моделирования, основ теории вероятности и представления теоремы Байеса. Затем использовали задачу о подбрасывании монеты как основу для первоначального ознакомления с базовыми аспектами байесовского моделирования и анализа данных. В этом классическом примере были продемонстрированы некоторые наиболее важные положения байесовской статистики, такие как использование распределений вероятности для создания моделей и для представления неопределенностей. Попытались как можно проще объяснить практическое использование априорных вероятностей и применение их на равных основаниях с другими элементами, являющимися компонентами процесса моделирования, такими как правдоподобие, или даже с постановкой более обобщенных основополагающих вопросов, как, например: почему мы пытаемся решить некоторую конкретную задачу в первую очередь. Глава завершилась описанием методов интерпретации и взаимодействия, то есть представления результатов байесовского анализа.

На рис. 1.8 показан рабочий процесс (поток) байесовского анализа именно так, как он описан в этой главе, в одной из форм, предложенных Сумио Ватанабе (Sumio Watanabe):



Рис. 1.8

Здесь мы полагаем, что истинное распределение (true distribution) в общем случае неизвестно (и кроме того, в принципе его узнать невозможно). Из этого распределения мы получаем конечную выборку (sample) как результат выполнения эксперимента, опроса, наблюдения или имитации некоторого явления/события. Для того чтобы обучиться чему-либо по истинному распределению с учетом того, что у нас есть возможность наблюдать только выборку, создается вероятностная модель. Двумя основными элементами вероятностной модели являются априорная вероятность и правдоподобие. Используя созданную модель и выборку данных, мы выполняем байесовский статистический вывод и получаем апостериорное распределение вероятностей (posterior distribution). В этом распределении содержится вся информация о решаемой задаче с учетом используемой модели и имеющихся данных. С точки зрения байесовского анализа апостериорное распределение является основным объектом нашего интереса, и вся прочая информация извлекается и выводится из апостериорного распределения, в том числе и прогнозы в форме апосте-

риорного прогнозируемого распределения (posterior predictive distribution). Поскольку апостериорное распределение (и любые другие выводимые из него величины) представляет собой последовательный результат использования модели и данных, полезность байесовских статистических выводов ограничена качеством моделей и данных. Одним из способов оценки применяемой модели является сравнение апостериорного прогнозируемого распределения с конечной выборкой данных, полученной на первом этапе. Следует отметить, что апостериорное распределение – это распределение (значений) параметров в модели (обусловленных наблюдаемыми выборками данных), тогда как апостериорное прогнозируемое распределение – это распределение прогнозируемых выборок данных (усредненных по апостериорному распределению). Процесс контрольной проверки (валидации) модели чрезвычайно важен не потому, что нам необходима уверенность в правильности используемой модели, а поскольку такие проверки достаточно полезны в определенных контекстах. Даже если это не так, подобные проверки позволяют получить представление о том, как можно усовершенствовать модели.

В этой главе приведен краткий обобщающий обзор основных аспектов выполнения байесовского анализа данных. Во всех последующих главах книги мы будем постоянно возвращаться к этим положениям и принципам, чтобы подтвердить их практикой и использовать как основу более развитых и расширенных концепций. В следующей главе будет рассматриваться PyMC3 – библиотека на языке Python для байесовского моделирования и вероятностного машинного обучения, а также ArviZ – библиотека на языке Python для исследовательского анализа байесовских моделей.

УПРАЖНЕНИЯ

Нам неизвестно, работает ли в действительности человеческий мозг по байесовской методике, или по принципам, приблизительно похожим на байесовскую методику, или, может быть, в соответствии с некоторыми эволюционно развивающимися оптимизированными (более или менее) эвристиками. В любом случае, мы знаем, что можем обучаться с помощью данных, примеров и упражнений. Хотя скептики могут возразить, что люди ничему не учатся, приводя в качестве конкретных примеров войны или экономические системы, в которых главным является получение прибыли, а человеческое благополучие вообще не принимается во внимание. Но как бы то ни было, я рекомендую выполнять все упражнения, предлагаемые в конце каждой главы.

1. Какое из приведенных ниже выражений соответствует высказыванию: «Вероятность того, что 9 июля 1816 года был солнечный день»?
 - $p(\text{солнечно})$
 - $p(\text{солнечно}|\text{июль})$
 - $p(\text{солнечно}|\text{9 июля 1816 года})$
 - $p(\text{9 июля 1816 года}|\text{солнечно})$
 - $p(\text{солнечно}, \text{9 июля 1816 года}) / p(\text{9 июля 1816 года})$

2. Докажите, что вероятность выбора человека при случайном и произвольном выборе папы римского не равна вероятности того, что папа является человеком. В мультипликационном сериале Футурама (Космический) папа является рептилией. Как это изменит ваши предыдущие вычисления?
3. В следующем определении вероятностной модели укажите априорную вероятность и функцию правдоподобия:

$$y_i \sim \text{Normal}(\mu, \sigma)$$

$$\mu \sim \text{Normal}(0, 10)$$

$$\sigma \sim \text{HalfNormal}(25)$$

4. В модели из предыдущего упражнения определите, сколько параметров будут иметь апостериорное распределение вероятностей. Сравните результат с моделью для задачи о подбрасывании монеты.
5. Запишите теорему Байеса для модели из упражнения 3.
6. Предположим, что имеется две монеты. При подбрасывании первой монеты в половине случаев выпадают решки, в половине случаев – орлы. Вторая монета несимметричная («мошенническая») – при ее подбрасывании всегда выпадают орлы. Если наугад взять одну из монет и при ее подбрасывании получить орел, то какова вероятность того, что эта монета является несимметричной?
7. Измените исходный код примера, в котором генерируются схемы на рис. 1.5, чтобы добавить пунктирную вертикальную линию, показывающую отношение орел/(количество бросков). Сравните расположение этой линии с модой апостериорного распределения на каждом графике рис. 1.5.
8. Попробуйте получить схемы, отличающиеся от показанных на рис. 1.5, используя другие априорные вероятности (`beta_params`) и другие данные (испытания и данные).
9. Исследуйте различные параметры для графиков гауссова, биномиального и бета-распределения (рис. 1.1, 1.3 и 1.4 соответственно). Другой вариант: возможно, потребуется генерация графика единого распределения вместо совмещения распределений в одной координатной сетке.
10. Прочитайте статью «Правило Кромвеля» в Википедии:
https://ru.wikipedia.org/wiki/Правило_Кромвеля.
11. Прочитайте статью о вероятности и «голландской книге» (Dutch book) в Википедии (англ.):
https://en.wikipedia.org/wiki/Dutch_book.

Глава 2

.....

Вероятностное программирование

«Наши големы редко имеют физическую форму, чаще всего они сделаны из той глины, которая существует в кремниевых микросхемах в виде компьютерного кода».

– Ричард МакЭлрис

После того как мы получили общее представление о байесовской статистике, можно приступить к изучению процесса создания вероятностных моделей с использованием вычислительных инструментальных средств. Более конкретно, мы будем изучать вероятностное программирование с использованием библиотеки PyMC3. Основная идея состоит в том, чтобы определять модели с помощью программного кода, а затем использовать их для решения задачи более или менее автоматизированным способом. Такой подход применяется не потому, что мы слишком ленивы, чтобы изучать математические методы решения, и не потому, что мы «элитные крутые хакеры-программисты». Важная причина такого выбора заключается в том, что многие модели не сводятся к аналитической законченной форме, поэтому решение для таких моделей может быть найдено только с использованием численных методов.

Другим обоснованием необходимости изучения вероятностного программирования является тот факт, что современная байесовская статистика в основном выполняется посредством написания программного кода, а поскольку мы уже знаем Python, то почему бы нам не воспользоваться этим языком для достижения еще одной цели. Вероятностное программирование предлагает эффективный способ создания и решения сложных моделей и позволяет сосредоточиться главным образом на проектировании, усовершенствовании и интерпретации модели и меньше времени тратить на математические и вычислительные подробности. В этой и последующих главах мы будем использовать PyMC3, чрезвычайно универсальную и гибкую библиотеку языка Python для вероятностного программирования, а также ArviZ, новую библиотеку Python, которая поможет интерпретировать результаты, полученные от

вероятностных моделей. Кроме того, знание PyMC3 и ArviZ поможет изучать сложные современные концепции байесовского анализа с более практической точки зрения.

В этой главе рассматриваются следующие темы:

- вероятностное программирование;
- основы использования библиотеки PyMC3;
- другое решение задачи о подбрасывании монеты;
- обобщение апостериорного распределения вероятностей;
- модели Гаусса и t-распределения Стьюдента;
- сравнение групп и размер эффекта;
- иерархические модели и сокращение.

ВЕРОЯТНОСТНОЕ ПРОГРАММИРОВАНИЕ

С теоретической точки зрения байесовская статистика очень проста: у нас есть знания (известные факты) и незнания (неизвестные факты) и мы используем теорему Байеса для выяснения неизвестных фактов в зависимости от условий, определяемых известными фактами. Если повезет, то этот процесс позволит снизить степень неопределенности, присущую неизвестным фактам (то есть уменьшить наши незнания). В общем случае знания или известные факты называют данными и считают их постоянными (или условно постоянными), а незнания или неизвестные факты называют параметрами и интерпретируют их как распределения вероятностей. Если применить более строгую терминологию, то распределения вероятностей ставятся в соответствие неизвестным величинам. Затем теорема Байеса используется для преобразования априорного распределения вероятностей $p(\theta)$ в апостериорное распределение вероятностей $p(\theta|y)$. Несмотря на теоретически простую концепцию, полные вероятностные модели часто приводят к результатам, которые невозможно описать аналитическими выражениями. В течение многих лет это являлось действительно крупной проблемой и, возможно, одной из главных причин, препятствующих широкому применению байесовских методов.

С наступлением эпохи компьютеров и активной разработки вычислительных методов появилась возможность, – по крайней мере, теоретическая, – решения любой задачи статистического вывода, которая кардинальным образом изменила практическое применение байесовского анализа данных. Можно считать эти вычислительные методы универсальными механизмами статистического вывода или, подобно Томасу Виецки (Thomas Wiecki), одному из основных разработчиков библиотеки PyMC3, называть их «кнопкой статистического вывода». Возможность автоматизации процессов статистического вывода привела к разработке языков вероятностного программирования (probabilistic programming languages – PPL), позволяющих полностью разделить процессы создания модели и статистического вывода.

В рабочей среде языка вероятностного программирования пользователи определяют полную вероятностную модель, записывая несколько строк кода,

после чего процесс статистического вывода выполняется автоматически. Можно ожидать, что вероятностное программирование окажет огромное воздействие на науку о данных и другие научные дисциплины, позволяя исследователям-практикам создавать сложные вероятностные модели с меньшими затратами времени и с меньшей вероятностью ошибок. Возможно, удачной аналогией предполагаемого воздействия языков вероятностного программирования на научные вычисления может являться появление языка программирования Fortran более шестидесяти лет назад. В наши дни Fortran в определенной степени сдал свои позиции, но в свое время он считался весьма революционным достижением. В первую очередь ученые получили возможность меньше внимания уделять подробностям вычислений и сосредоточиться на создании вычислительных методов, моделей и имитаций более естественным образом. Ситуация очень похожа на недавнее появление языков вероятностного программирования, которые скрывают от пользователей подробности обработки вероятностей и выполнения статистических выводов, позволяя сосредоточиться на подробном описании модели и анализе полученных результатов.

В этой главе будет рассматриваться практическое использование библиотеки PyMC3 для определения и решения моделей. Мы будем считать «кнопку статистического вывода» черным ящиком, который выдает корректные выборки из апостериорного распределения вероятностей. Используемые здесь методы являются стохастическими (случайными, вероятностными), поэтому выборки будут различными при каждом конкретном обращении к тому или иному методу. Но если процесс статистического вывода работает как предполагалось, то выборки будут оставаться репрезентативными по отношению к апостериорному распределению, следовательно, мы будем приходить к одному и тому же заключительному выводу по любой из этих выборок. Подробности всего происходящего незаметно для нас при нажатии «кнопки статистического вывода», а также методы проверки достоверности и надежности получаемых выборок будут подробно рассматриваться в главе 8 «Механизмы статистического вывода».

ОСНОВЫ ИСПОЛЬЗОВАНИЯ БИБЛИОТЕКИ PyMC3

PyMC3 – это библиотека языка Python для вероятностного программирования. На момент написания этой книги самой последней была версия 3.6. Библиотека PyMC3 предлагает очень простой и интуитивно понятный синтаксис, который легко читать. Этот синтаксис почти совпадает с нотацией, используемой в статистической литературе для описания вероятностных моделей. Основной код библиотеки PyMC3 написан на языке Python, а в частях, зависящих от интенсивных вычислений, используются библиотеки NumPy и Theano.

Theano – это библиотека языка Python, которая изначально была разработана для поддержки глубокого обучения. Она позволяет эффективно определять, оптимизировать и оценивать математические выражения, содержащие

многомерные массивы. Главная причина использования Theano в библиотеке PyMC3 состоит в том, что для некоторых методов выборки, таких как NUTS, требуется вычисление градиентов, а библиотека Theano предоставляет методы вычисления градиентов с использованием автоматического дифференцирования. Кроме того, Theano компилирует код Python в код на языке C, таким образом обеспечивая действительно высокую скорость работы PyMC3. Это все, что необходимо знать о библиотеке Theano, чтобы использовать PyMC3. Более подробную информацию можно получить из официального руководства по библиотеке Theano: <http://deeplearning.net/software/theano/tutorial/index.html#tutorial>.



Возможно, вы слышали о том, что активная разработка библиотеки Theano в настоящее время не выполняется, но это не является причиной для беспокойства. Разработчики PyMC будут заниматься текущим сопровождением Theano, обеспечивая работу этой библиотеки в качестве вспомогательного инструментального средства для PyMC3 в течение нескольких следующих лет. В то же время разработчики PyMC оперативно занимаются созданием библиотеки-преемника PyMC3. Вероятнее всего, она будет основана на библиотеке TensorFlow, используемой в качестве ядра, хотя в настоящее время рассматриваются и другие варианты. Подробнее об этой перспективной разработке можно узнать в блоге: https://medium.com/@pymc_devs/theano-tensorflow-and-the-future-of-pymc-6c9987bb19d5.

Решение задачи о подбрасывании монет с использованием библиотеки PyMC3

Вернемся к задаче о подбрасывании монеты, которая рассматривалась в главе 1, но теперь используем для ее решения библиотеку PyMC3. Воспользуемся теми же синтезированными данными, которые использовались в главе 1. Поскольку данные генерируются, нам известно истинное значение параметра θ , который имеет имя `theta_real` в приведенном ниже коде. Разумеется, для реального набора данных значение этого параметра будет неизвестным.

```
np.random.seed(123)
trials = 4
theta_real = 0.35 # в реальном эксперименте это значение неизвестно
data = stats.bernoulli.rvs(p=theta_real, size=trials)
```

Подробное описание модели

У нас есть данные, теперь необходимо определить модель. Напомним, что этот процесс включает определение функции правдоподобия и априорной вероятности с использованием распределений вероятностей. Для функции правдоподобия воспользуемся биномиальным распределением с параметрами $n = 1$ и $p = \theta$, а для априорной вероятности применим бета-распределение с параметрами $\alpha = \beta = 1$. Бета-распределение с такими параметрами равнозначно равномерному распределению в интервале $[0,1]$. В математической нотации можно записать эту модель следующим образом:

$$\begin{aligned}\theta &\sim \text{Beta}(\alpha, \beta); \\ y &\sim \text{Bern}(n = 1, p = \theta).\end{aligned}\tag{2.1}$$

Эту статистическую модель практически без изменений можно преобразовать в код, использующий библиотеку PyMC3:

```
with pm.Model() as our_first_model:
    theta = pm.Beta('theta', alpha=1., beta=1.)
    y = pm.Bernoulli('y', p=theta, observed=data)
    trace = pm.sample(1000, random_seed=123)
```

В первой строке кода создается контейнер для используемой модели. Весь код, расположенный внутри блока `with`, автоматически добавляется в объект `our_first_model`. Это можно считать синтаксическим сахаром для упрощения определения модели, поскольку нет необходимости вручную определять переменные для этой модели. Во второй строке определяется априорная вероятность. Как видите, синтаксис очень похож на математическую нотацию.



Необходимо обратить особое внимание на двукратное использование имени `theta`: сначала как переменной языка Python, затем в качестве первого аргумента функции `Beta`. Использование одного и того же имени является нормальным практическим приемом для того, чтобы избежать путаницы в данном случае. Переменная `theta` – случайная переменная (величина), это не число, а объект, представляющий распределение вероятностей, по которому можно генерировать случайные числа и плотности вероятностей.

В третьей строке определяется функция правдоподобия. Синтаксис почти тот же, что при определении априорной вероятности, за исключением того, что данные передаются с использованием аргумента `observed`. Это способ, которым мы сообщаем библиотеке PyMC3 о том, что необходимо определить условие для неизвестного по известному (`data`). Наблюдаемые значения могут быть переданы как список языка Python, кортеж, массив NumPy или фрейм данных DataFrame в формате Pandas.

Итак, полное определение модели завершено. Согласитесь, это было коротко, ясно и совсем не сложно.

Нажимаем «кнопку статистического вывода»

Последний этап – та самая «кнопка статистического вывода». Мы запрашиваем 1000 элементов выборки из апостериорного распределения и сохраняем их в объекте `trace`. За одной простой строкой PyMC3 скрывает сотни умпа-лумпа¹, поющих и готовящих превосходный байесовский статистический вывод для пользователя. Это, конечно же, метафора, но PyMC3 действительно автоматизирует огромное количество операций и задач. После запуска кода примера, приведенного выше, вы получите сообщение, подобное следующему:

¹ Трудолюбивые маленькие человечки, персонажи книги Роалда Дала «Чарли и шоколадная фабрика» и фильма, снятого по этой книге. – *Прим. перев.*

```
Auto-assigning NUTS sampler...
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (2 chains in 2 jobs)
NUTS: [0]
100 %|██████████| 3000/3000 [00:00<00:00, 3695.42it/s]
```

В первой и второй строках сообщается, что PyMC3 автоматически выбирает для применения сэмплер NUTS (один из механизмов статистического вывода, который очень хорошо подходит для работы с непрерывными переменными) и использует указанный метод для инициализации этого сэмплера. Третья строка говорит о том, что PyMC3 будет параллельно обрабатывать две цепочки, следовательно, мы получим две независимые выборки из апостериорного распределения по цене одной.

Точное число обрабатываемых цепочек определяется с учетом количества процессоров на используемом компьютере. Это число можно изменить, воспользовавшись аргументом `chains`, передаваемым в функцию `sample`. В следующей строке указано, какие переменные включаются в выборку каждым сэмплером. В данном конкретном примере эта строка не добавляет какой-либо новой информации. Поскольку для выборки используется механизм NUTS, обрабатывается единственная переменная θ . Но это не самый типичный случай, так как PyMC3 может назначать различные сэмплеры для обработки различных переменных. Это делается автоматически на основе тех свойств переменных, которые определяют выбор наилучшего возможного сэмплера для каждой переменной. Пользователи могут вручную назначать сэмплеры с помощью аргумента `step`, передаваемого в функцию `sample`.

В последней строке располагается индикатор процесса обработки с несколькими метриками, показывающими скорость работы сэмплера, в том числе число итераций в секунду. Если вы выполните код этого примера, то заметите, что индикатор процесса обработки обновляется чрезвычайно быстро. Выше приведен самый последний этап, когда сэмплер уже завершил работу. Справа от индикатора расположены числа 3000/3000, где первое число – это количество циклов обработки сэмплера (начиная с 1), а второе – общее количество элементов выборки. Отметим, что изначально мы запросили 1000 элементов выборки, но PyMC3 выполнил обработку 3000 элементов. По 500 элементов в цепочке предназначено для автоматической точной настройки алгоритма выборки (в данном примере NUTS). Эта часть выборки отбрасывается по умолчанию. Далее в каждой цепочке обрабатывается 1000 элементов, дающих итоговый результат, таким образом всего генерируется 3000 элементов. Этап точной настройки помогает PyMC3 формировать надежную выборку из апостериорного распределения. Количество этапов точной настройки можно изменить с помощью аргумента `tune`, передаваемого в функцию `sample`.

ОБОБЩЕНИЕ АПОСТЕРИОРНОГО РАСПРЕДЕЛЕНИЯ

В общем случае первой задачей, выполняемой после выборки из апостериорного распределения, является проверка визуального представления получен-

ных результатов. Для решения этой задачи почти идеально подходит функция `plot_trace` из библиотеки ArviZ:

```
az.plot_trace(trace)
```

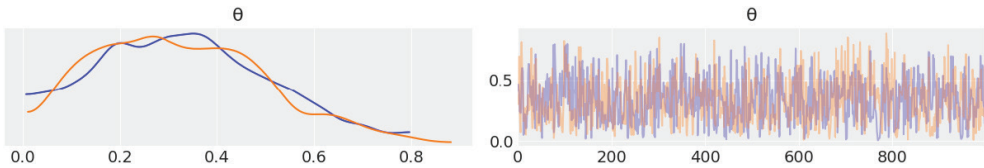


Рис. 2.1

Используя функцию `az.plot_trace`, мы получаем два отдельных графика для каждой ненаблюдаемой переменной. В нашей модели единственной такой переменной является θ . Отметим, что y – это наблюдаемая переменная, представленная данными, поэтому нет необходимости создавать по ней выборку, так как нам уже известны ее значения. На рис. 2.1 показаны два графика. Слева изображен график ядерной оценки плотности (ЯОП), представляющий собой сглаженную версию гистограммы. Справа приведены отдельные значения элементов выборки на каждом шаге процесса сэмплирования. По такому трассировочному графику можно визуальнo получить правдоподобные значения из апостериорного распределения. Вы должны сравнить этот результат применения библиотеки PyMC3 с результатом, полученным аналитическим методом в примере из предыдущей главы.

Библиотека ArviZ предоставляет несколько других типов графиков, помогающих интерпретировать процесс трассировки. Эти графики будут использоваться в следующих примерах. Кроме того, может потребоваться обобщение процесса трассировки в числовом выражении. Для этого можно воспользоваться функцией `az.summary`, которая возвращает объект `DataFrame` библиотеки `pandas`:

```
az.summary(trace)
```

Таблица 2.1

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
θ	0.33	0.18	0.0	0.02	0.64	847.0	1.0

Здесь мы получаем среднее значение (mean), стандартное отклонение (sd) и интервал 94 % плотности апостериорного распределения (ПАР – hpd 3 % и hpd 97 %). Как уже отмечалось в главе 1, эти числовые характеристики можно использовать для интерпретации и создания отчета о результатах байесовского статистического вывода. Две последние метрики относятся к диагностиро-

ванию выборок. Более подробно об этом вы узнаете в главе 8 «Механизмы статистического вывода».

Другим способом визуального обобщения апостериорного распределения является использование функции `plot_posterior` из библиотеки ArviZ. В предыдущей главе мы уже пользовались этим представлением распределения для имитации апостериорного распределения вероятностей. Теперь применим данный метод для реального апостериорного распределения. По умолчанию `plot_posterior` выводит гистограмму для дискретных переменных и ядерную оценку плотности (ЯОП) для непрерывных переменных. Также указывается среднее значение (можно запросить срединное значение или моду, используя для этого аргумент функции `point_estimate`) и ПАР 94 % в виде черной линии в нижней части графика. Для ПАР могут быть установлены различные интервалы значений с помощью аргумента `credible_interval`. Этот тип графика был представлен Джоном К. Крушке (John K. Kruschke) в его книге «Doing Bayesian Data Analysis»:

```
az.plot_posterior(trace)
```

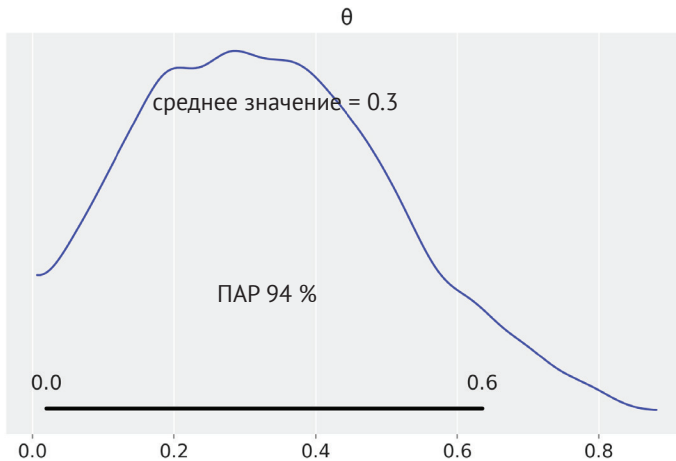


Рис. 2.2

Решения на основе апостериорного распределения

Иногда описания апостериорного распределения вероятностей недостаточно. Часто необходимо принять какие-либо решения на основе полученных статистических выводов. Мы должны свести непрерывную оценку к дихотомии, то есть к выбору одного из двух вариантов: да-нет, здоров-болен, загрязненный-чистый и т. д. Например, может потребоваться принятие решения о симметричности или несимметричности конкретной монеты. Симметричной монетой считается та, для которой значение θ в точности равно 0.5. Можно сравнить это значение 0.5 с интервалом ПАР. На рис. 2.2 можно видеть, что интервал ПАР размещен от ≈ 0.02 до ≈ 0.71 , таким образом, значение 0.5 включено в ПАР.

В соответствии с апостериорным распределением, полученным в нашем примере, монета выглядит несимметричной с отклонением в сторону решек, но мы не можем с полной определенностью сделать заключение о возможности того, что монета является симметричной. Если необходимо более четкое и ясное решение, то потребуются собрать больше данных, чтобы сократить интервал разброса апостериорного распределения, или, возможно, нужно будет найти способ определения более информативной априорной вероятности.

Пространство практической равнозначности (ППР)

Строго говоря, вероятность наблюдения точного значения 0.5 (то есть значения с бесконечной последовательностью хвостовых нулей) почти равна нулю. Кроме того, на практике обычно никто не заботится о получении абсолютно точных результатов, необходимы результаты в определенных границах допустимой погрешности. Поэтому в действительности можно ослабить определение подлинности (симметричности) монеты и сказать, что симметричной монете соответствует значение θ , приблизительно равное 0.5. Например, можно утверждать, что любое значение в интервале $[0.45, 0.55]$ будет практически равным 0.5 и представлять собой приемлемое решение нашей задачи. Этот интервал называют пространством практической равнозначности (ППР) (Region of Practical Equivalence – ROPE). После определения ППР мы сравниваем его с плотностью апостериорного распределения (ПАР). При таком сравнении возможны как минимум три варианта:

- ППР не пересекается с ПАР, поэтому можно утверждать, что монета несимметрична;
- ППР полностью включает в себя весь интервал ПАР, поэтому можно утверждать, что монета симметрична;
- ППР частично пересекается с ПАР, при этом невозможно точно определить симметричность или несимметричность монеты.

Если ППР выбирается в интервале $[0, 1]$, то всегда можно сказать, что монета симметрична. Отметим, что при этом нет необходимости собирать данные для выполнения любого типа статистического вывода. Разумеется, это примитивный, бессмысленный и необъективный вариант выбора, поэтому вряд ли кто-то согласится с таким вариантом определения ППР. Здесь он упомянут только для того, чтобы особо выделить тот факт, что определение ППР зависит от контекста. Поэтому не существует какого-либо суперуниверсального правила, соответствующего намерениям и предположениям каждого испытателя. Любые решения субъективны по своей природе, и наша цель – определить самые обоснованные (то есть принятые на основе наиболее полной информации) возможные решения, соответствующие нашим целям.



Пространство практической равнозначности (ППР) – это произвольный интервал, который выбирается на основе второстепенных, менее значимых знаний. При этом предполагается, что любые значения в этом интервале могут считаться практически равнозначными.

Можно воспользоваться функцией `plot_posterior` для построения графика апостериорного распределения с указанием интервала ПАР и ППР. На графике ППР изображается как полупрозрачная утолщенная (зеленая) линия:

```
az.plot_posterior(trace, rope=[0.45, .55])
```

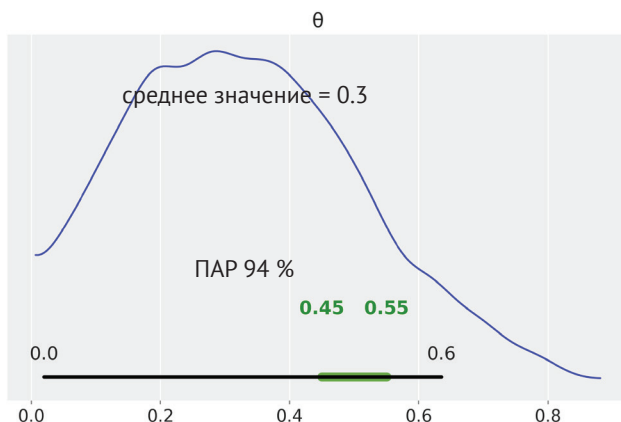


Рис. 2.3

Кроме того, можно использовать другой инструмент, помогающий принять решение, – сравнение апостериорного распределения с эталонным значением. Для этого также используется функция `plot_posterior`. На рис. 2.4 можно видеть вертикальную (оранжевую) линию и пропорциональное соотношение апостериорного распределения, расположенного соответственно выше и ниже заданного эталонного значения:

```
az.plot_posterior(trace, ref_val=0.5)
```

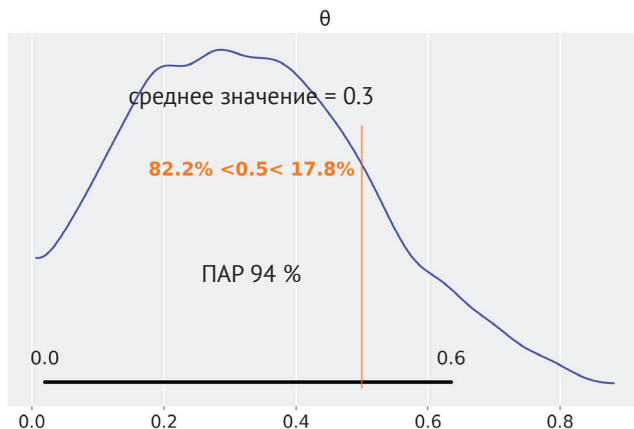


Рис. 2.4

Более подробно об использовании ППР можно прочитать в главе 12 книги «Doing Bayesian Data Analysis» Джона Крушке. В этой главе также рассматриваются методы выполнения проверки гипотез в рабочей среде байесовского анализа, а также тонкости процесса проверки гипотез независимо от того, выполняются ли они в байесовской или небайесовской среде.

Функции потерь

Если приведенные выше правила применения ППР показались вам не слишком убедительными и требуются более формализованные условия, то следует обратить внимание на функции потерь. Для принятия правильного решения важно иметь наивысший возможный уровень точности оцениваемого значения самых важных параметров, но не менее важно учитывать цену издержек при совершении ошибки. Компромисс между издержками и выгодами может быть математически формализован при помощи функций потерь. В различных областях деятельности функции потерь могут называться по-разному: функции стоимости, целевые функции, функции соответствия, функции полезности и т. п. Независимо от имени основная идея заключается в использовании функции, которая определяет различие между истинным и оцениваемым значением параметра. Чем больше значение функции потерь, тем хуже полученная оценка (в соответствии со смыслом функции потерь). Некоторые примеры широко используемых функций потерь приведены ниже:

- квадратичная функция потерь $(\theta - \hat{\theta})^2$;
- абсолютная функция потерь $|\theta - \hat{\theta}|$;
- 0-1-функция потерь $I(\theta \neq \hat{\theta})$, где I – индикаторная или характеристическая функция.

На практике обычно неизвестно истинное значение параметра θ . Вместо этого имеется его оценка в форме апостериорного распределения вероятностей. Поэтому можно только лишь найти такое значение $\hat{\theta}$, которое минимизирует ожидаемые потери. Говоря о функции ожидаемых потерь, мы подразумеваем функцию потерь, усредненную по всему апостериорному распределению. В приведенном ниже фрагменте кода используются две функции потерь: абсолютная функция потерь `lossf_a` и квадратичная функция потерь `lossf_b`. Значение $\hat{\theta}$ исследуется на решетке из 200 точек. Затем формируются графики соответствующих кривых, в которые включается значение $\hat{\theta}$, минимизирующее каждую функцию потерь:

```
grid = np.linspace(0, 1, 200)
theta_pos = trace['theta']
lossf_a = [np.mean(abs(i - theta_pos)) for i in grid]
lossf_b = [np.mean((i - theta_pos)**2) for i in grid]

for lossf, c in zip([lossf_a, lossf_b], ['C0', 'C1']):
    mini = np.argmin(lossf)
    plt.plot(grid, lossf, c)
    plt.plot(grid[mini], lossf[mini], 'o', color=c)
```

```
plt.annotate('{:.2f}'.format(grid[mini]),
             (grid[mini], lossf[mini] + 0.03), color=c)
plt.yticks([])
plt.xlabel(r'$\hat{\theta}$')
```

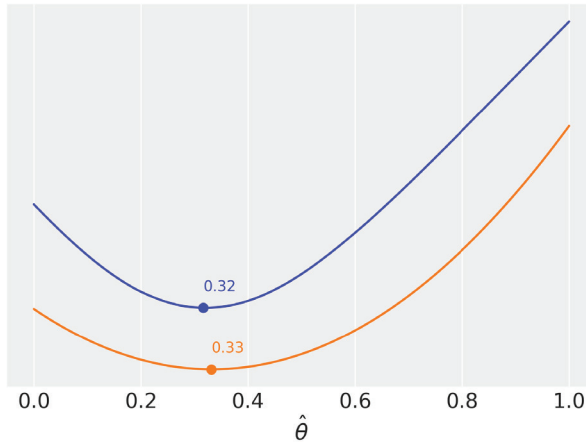


Рис. 2.5

На этом графике можно видеть определенную степень схожести результата для функции lossf_a $\hat{\theta} = 0.31$ и для функции lossf_b $\hat{\theta} = 0.33$. Этот результат интересен тем, что первое значение равно срединному значению (медиане) апостериорного распределения, а второе значение равно среднему значению апостериорного распределения. Вы можете проверить это самостоятельно, вычислив `pr.mean( $\theta\_pos$ )`, `pr.median( $\theta\_pos$ )`. Очевидно, что это не формальное доказательство, но здесь самым важным является то, что различные функции потерь связаны с различными точечными оценками.

Но если необходимо формальное доказательство и требуется вычисление единственной точечной оценки, то мы обязательно должны решить, какая именно функция потерь нам нужна, или, с другой стороны, если мы выбираем конкретную точечную оценку, то неявно (или, возможно, даже бессознательно) принимаем решение о выборе функции потерь. Преимущество явного выбора функции потерь состоит в том, что мы получаем возможность адаптировать эту функцию для нашей задачи, а не использовать какое-то предопределенное правило, которое может не вполне соответствовать нашему конкретному случаю. Например, при решении многих задач стоимость принятия решения асимметрична, то есть неодинаковы решения по безопасному или вызывающему опасения применению некоторой вакцины для детей младше пяти лет — здесь чрезвычайно важен выбор правильного решения. Неправильное решение может стоить тысяч жизней и привести к критической ситуации, которой можно было бы избежать, применяя доступную и безопасную вакцину. Таким образом, если задача требует, то можно сформировать асимметричную

функцию потерь. Кроме того, важно отметить, что поскольку апостериорное распределение представлено в форме числовых выборок, можно вычислять сложные функции потерь, для которых не требуются ограничения, связанные с математическими преимуществами (удобством применения) или с явной простотой.

Ниже приведен простейший пример применения такого подхода:

```
lossf = []
for i in grid:
    if i < 0.5:
        f = np.mean(np.pi *  $\theta_{\text{pos}}$  / np.abs(i -  $\theta_{\text{pos}}$ ))
    else:
        f = np.mean(1 / (i -  $\theta_{\text{pos}}$ ))
    lossf.append(f)

mini = np.argmin(lossf)
plt.plot(grid, lossf)
plt.plot(grid[mini], lossf[mini], 'o')
plt.annotate('{:.2f}'.format(grid[mini]),
             (grid[mini] + 0.01, lossf[mini] + 0.1))
plt.yticks([])
plt.xlabel(r'$\hat{\theta}$')
```

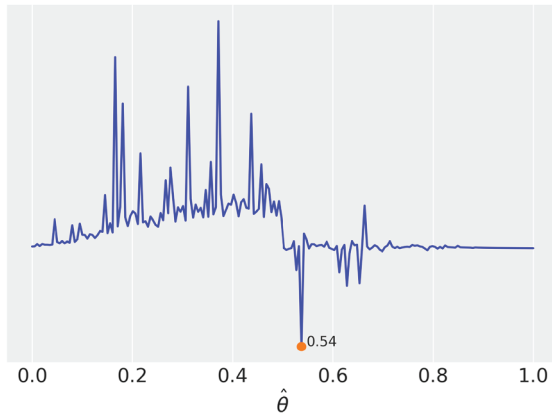


Рис. 2.6

Все сказанное выше предназначено для более правильного понимания сущности функций потерь. Существует ошибочное мнение о том, что при каждом случае использования точечной оценки люди мыслят исключительно в терминах функций потерь. В действительности функции потерь не имеют слишком широкого распространения во многих областях науки, с которыми я более или менее знаком. Исследователи часто выбирают срединное значение (медиану) только потому, что она более устойчива к промахам, чем среднее значение, или, наоборот, используют среднее значение только потому, что это более

простая и хорошо знакомая всем концепция, или просто потому, что считается очевидной истинность усредненных характеристик для некоторого процесса на некотором уровне, как, например, колебания молекул относительно друг друга или взаимодействие генов между собой и с окружающей средой.

В этом разделе было представлено чрезвычайно краткое и поверхностное введение в функции потерь. Для получения более подробной информации по этой теме рекомендуется более глубоко изучить теорию принятия решений, позволяющую формализовать процесс принятия решений.

ГАУССОВА МОДЕЛЬ В ПОДРОБНОМ ИЗЛОЖЕНИИ

В предыдущих разделах были представлены основные положения байесовского анализа при использовании бета-биномиальной модели, в основном из-за ее простоты. Другой весьма простой моделью является гауссова, или нормальная, модель. Гауссовы модели чрезвычайно привлекательны с математической точки зрения, потому что с ними легко работать. Например, известно, что сопряженное априорное распределение гауссова среднего значения представляет собой само гауссово распределение. Кроме того, существует множество явлений, которые можно успешно аппроксимировать с помощью гауссовой модели. В частности, почти каждый раз при измерении какой-либо средней величины с использованием выборки достаточно большого размера эта средняя величина будет распределяться в соответствии с гауссовой моделью. Подробности о том, когда это истинно, когда это не является истиной и когда это более или менее истинно, точно определены в центральной предельной теореме. Здесь вы можете прервать чтение и начать поиск информации об этой действительной центральной статистической концепции (неудачный каламбур, признаю).

Итак, мы говорили о том, что многие явления в действительности представляют собой средние значения (распределения). Если следовать банальным шаблонам, то рост (и почти все прочие подобные физические характеристики человека) является результатом воздействия множества факторов природной среды и множества генетических факторов, следовательно, мы получаем достаточно точное гауссово распределение для роста взрослых людей. Точнее говоря, мы получаем объединение двух гауссовых распределений как результат совмещения распределений роста мужчин и женщин, но основная идея должна быть понятна. В общем, с гауссовыми моделями легко работать, по своей природе они обладают множеством полезных характеристик, а многие статистические методы, с которыми вы, вероятно, уже знакомы, основаны на предположениях о нормальности. Поэтому важно хорошо знать, как создаются эти модели, но не менее важно знать, как сделать менее строгими предположения о нормальности. Иногда это удивительно просто сделать в рабочей среде байесовского анализа с применением современных вычислительных инструментов, таких как библиотека PyMC3.

Гауссовы статистические выводы

Ядерный магнитный резонанс (ЯМР) – это мощная методика, используемая для изучения молекул и всех материальных вещей и существ, например людей или ферментов (поскольку все мы представляем собой набор молекул). ЯМР позволяет измерять различные типы наблюдаемых количественных явлений, которые связаны с ненаблюдаемыми и весьма интересными молекулярными свойствами. Одно из таких наблюдаемых явлений носит название химический сдвиг. Химический сдвиг можно наблюдать только в ядрах атомов определенных типов.

Эти факты из области квантовой химии и подробности не существенны для темы нашего обсуждения, тем не менее название описанного выше явления необходимо объяснить. Слово «сдвиг» применено, потому что измеряется сдвиг (смещение) сигнала по отношению к исследуемому значению, а «химическим» этот сдвиг назван потому, что он некоторым образом связан с химической средой исследуемых ядер. В настоящий момент нас интересует, можно ли измерить рост группы людей, среднее время возвращения домой, вес упаковок с апельсинами или даже некоторую дискретную переменную, например такую, как количество половых партнеров геккона токи. Если потребуются аппроксимировать данное числовое значение как непрерывную переменную, то это тоже может иметь смысл для гекконов токи, слишком неразборчивых в связях. Я не этолог и не папарацци, поэтому не могу с уверенностью утверждать, что это удачная аппроксимация. Но всегда следует помнить о том, что нам не нужны данные, чтобы получить истинное гауссово распределение (или любое другое распределение в рассматриваемом здесь случае): необходимо только лишь, чтобы гауссово распределение являлось приемлемой аппроксимацией для исследуемых данных. В рассматриваемом ниже примере используется 48 значений химических сдвигов, которые можно загрузить в массив библиотеки NumPy и построить график, используя для этого следующий код:

```
data = np.loadtxt('../data/chemical_shifts.csv')
az.plot_kde(data, rug=True)
plt.yticks([0], alpha=0)
```

График ЯОП (ядерной оценки плотности) для этого набора данных демонстрирует распределение, похожее на гауссово, за исключением двух точек данных, располагающихся далеко от среднего значения (рис. 2.7).

На некоторое время забудем об этих двух точках и предположим, что гауссово распределение является правильным описанием исследуемых данных. Так как нам неизвестно среднее значение или стандартное отклонение, необходимо определить априорные распределения для обеих характеристик. Таким образом, можно сформировать следующую допустимую модель:

$$\begin{aligned}\mu &\sim U(l, h); \\ \sigma &\sim |\mathcal{N}(0, \sigma_o)|; \\ y &\sim \mathcal{N}(\mu, \sigma).\end{aligned}\tag{2.2}$$

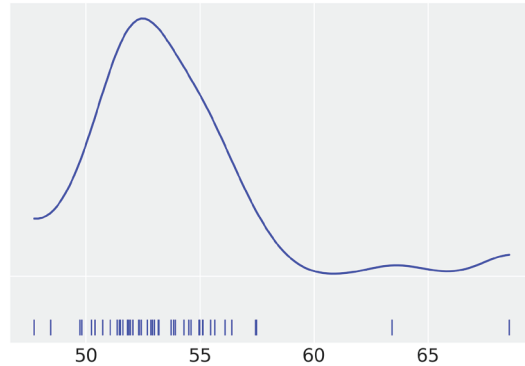


Рис. 2.7

Здесь μ вычисляется по равномерному распределению с нижней границей l и верхней границей h . Значение σ определяется как полунормальное распределение со стандартным отклонением σ_σ . Полунормальное распределение похоже на обычное нормальное распределение, но ограничено только положительными значениями (включая ноль). Можно получить выборки из полунормального распределения, выполняя сэмплинг из нормального распределения, а затем взять абсолютные величины от всех выбранных значений. Наконец, в этой модели данные y определяются нормальным распределением с параметрами μ и σ . На диаграмме Крашке это выглядит следующим образом:

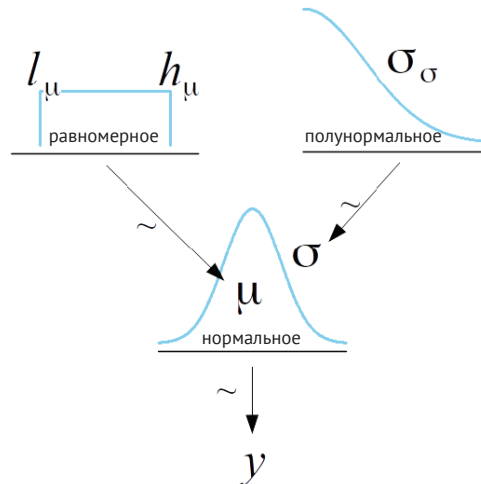


Рис. 2.8

Если неизвестны возможные значения параметров μ и σ , то можно назначить априорные распределения, отображающие наше незнание. Одним из ва-

риантов является установление границ равномерного распределения равными $l = 40$, $h = 75$, то есть диапазона, превышающего диапазон данных. Другой вариант – возможность выбора диапазона на основе ранее полученных знаний. Может быть известно, что физически невозможны значения меньше 0 и больше 100 для этого типа измерений. В этом случае можно назначить априорное распределение для среднего значения как равномерное распределение с параметрами $l = 0$, $h = 100$. Для полунормального распределения можно установить $\sigma_0 = 10$, достаточно большое значение для исследуемых данных. Применяя библиотеку PyMC3, мы можем записать эту модель в виде следующего кода:

```
with pm.Model() as model_g:
     $\mu$  = pm.Uniform('μ', lower=40, upper=70)
     $\sigma$  = pm.HalfNormal('σ', sd=10)
    y = pm.Normal('y', mu= $\mu$ , sd= $\sigma$ , observed=data)
    trace_g = pm.sample(1000)
az.plot_trace(trace_g)
```

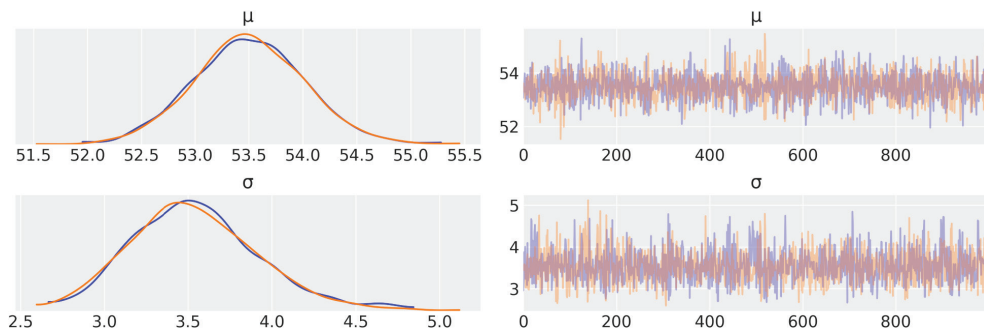


Рис. 2.9

Как вы могли заметить, на рис. 2.9, сгенерированном с помощью функции `plot_trace` из библиотеки ArviZ, для каждого параметра отведена отдельная строка. Для используемой модели апостериорное распределение двумерное, поэтому на рис. 2.9 показаны предельные распределения для каждого параметра. Можно воспользоваться функцией `plot_joint` из библиотеки ArviZ, чтобы наглядно показать внешний вид двумерного апостериорного распределения совместно с предельными распределениями для параметров μ и σ :

```
az.plot_joint(trace_g, kind='kde', fill_last=False)
```

Если необходим доступ к значениям любого параметра, сохраненным в объекте `trace`, то можно проиндексировать значения трассировки с помощью имени рассматриваемого параметра. Результатом этой операции является массив NumPy. Попробуйте самостоятельно получить результаты `trace_g['σ']` или `az.plot_kde(trace_g['σ'])`. Кстати, при использовании Jupyter Notebook/Lab такие символы, как σ , можно получить, записав `\sigma` в панели кода с последующим нажатием клавиши **Tab**.

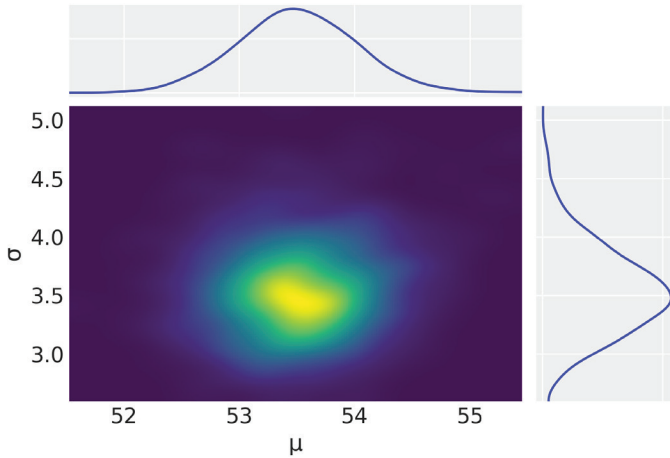


Рис. 2.10

Теперь выведем обобщенную сводку трассировки для дальнейшего использования:

```
az.summary(trace_g)
```

Таблица 2.2

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
μ	53.49	0.50	0.00	52.5	54.39	2081.0	1.0
σ	3.54	0.38	0.01	2.8	4.22	1823.0	1.0

После вычисления апостериорного распределения можно использовать эти результаты для имитационных данных и для проверки согласованности (связности) имитационных данных с наблюдаемыми данными. Как вы помните, в главе 1 данный тип сравнений был назван в обобщенном смысле проверками прогнозируемого апостериорного распределения, потому что мы используем апостериорное распределение для прогнозов, а эти прогнозы, в свою очередь, используются для проверки модели. С помощью библиотеки РумСЗ действительно легко получить выборки прогнозируемого апостериорного распределения, если воспользоваться функцией `sample_posterior_predictive`. Приведенный ниже фрагмент кода генерирует 100 прогнозов из апостериорного распределения, при этом каждая выборка имеет тот же размер, что и набор исследуемых данных. Отметим, что необходимо передать объект `trace` и модель в функцию `sample_posterior_predictive`, тогда как другие аргументы не являются обязательными:

```
y_pred_g = pm.sample_posterior_predictive(trace_g, 100, model_g)
```

Переменная `y_pred_g` – это словарь, в котором ключи `keys` представлены именем наблюдаемой переменной в нашей модели, а значения `values` – это массив вида `(samples, size)`, в нашем примере – `(100, len(data))`. Словарь используется потому, что исследуемые модели могут содержать более одной наблюдаемой переменной. Также можно воспользоваться функцией `plot_ppc` для визуального представления проверки прогнозируемого апостериорного распределения:

```
data_ppc = az.from_pymc3(trace=trace_g, posterior_predictive=y_pred_g)
ax = az.plot_ppc(data_ppc, figsize=(12, 6), mean=False)
ax[0].legend(fontsize=15)
```

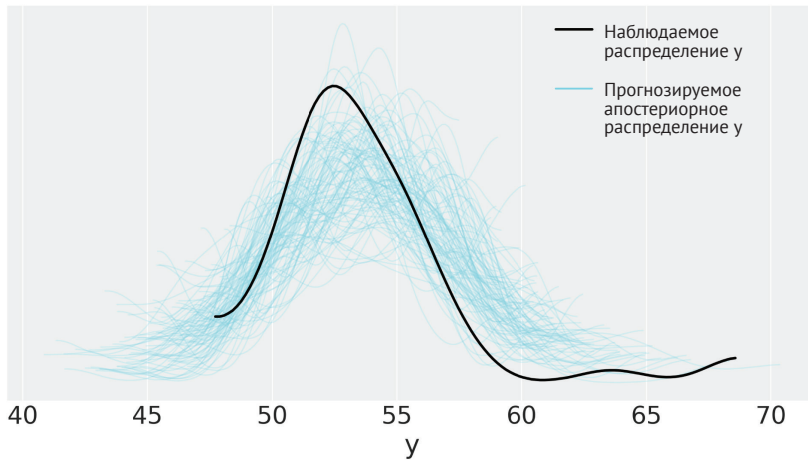


Рис. 2.11

На рис. 2.11 одна (черная) линия представляет ядерную оценку плотности (ЯОП), а множественные полупрозрачные (зеленовато-голубые) линии – это ЯОП, вычисленные для каждой из 100 выборок прогнозируемого апостериорного распределения. Эти полупрозрачные линии отображают неопределенность информации о статистически выведенном распределении прогнозируемых данных. Иногда при слишком малом количестве точек данных на подобном графике прогнозируемые кривые могут выглядеть как непричесанная копна волос или веревочная швабра. Причина в том, как реализовано вычисление ЯОП в библиотеке ArviZ. Плотность оценивается в реальном диапазоне данных, передаваемых в функцию `kde`, тогда как вне этого диапазона плотность полагается равной нулю. Существует мнение, что это ошибка, я же предпочитаю считать это полезным свойством, поскольку оно отображает реальную характеристику данных, а не сглаживает их сверх меры.

На рис. 2.11 можно видеть, что среднее значение имитационных данных слегка смещено вправо. Наблюдаемая дисперсия выглядит несколько большей для имитационных данных по сравнению с реальными данными. Это прямое следствие двух наблюдений, которые отделены от основной массы данных.

Можно ли воспользоваться этим графиком для того, чтобы с уверенностью сказать, что модель некорректна и требует изменения? Как всегда, интерпретация модели и ее оценка зависят от контекста. На основе личного опыта измерений подобного типа и способов обычного использования подобных данных я могу утверждать, что рассматриваемая здесь модель представляет собой достаточно разумное представление данных и пригодна для большинства вариантов выполняемого мною анализа. Тем не менее в следующем разделе мы рассмотрим, как можно усовершенствовать `model_g` и получить прогнозы, более точно совпадающие с реальными данными.

Надежные статистические выводы

Одним из недостатков, присущих модели `model_g`, представленной в предыдущем разделе, является предварительное предположение о нормальном распределении, хотя в конце распределения имеются две точки данных, из-за которых такое предположение выглядит слегка натянутым, не совсем корректным. Поскольку концевые части графика нормального распределения быстро снижаются по мере удаления от среднего значения, для нормального распределения (по крайней мере, с точки зрения человека) эти две точки выглядят неожиданным событием, а ответная реакция на их появление – увеличение стандартного отклонения. Можно считать, что эти точки имеют избыточный вес при определении параметров нормального распределения. Какие меры могут исправить ситуацию?

Первый вариант – объявить такие точки промахами и исключить их из набора данных. Для удаления этих точек имеется разумное обоснование – возможно, причиной является некорректная работа (или сбой) оборудования или ошибка человека при измерениях. Иногда можно даже исправить эти точки данных, например если точно известно, что это всего лишь результат написания неэффективного кода для очистки данных. Во многих случаях также может потребоваться автоматизация процесса исключения промахов с использованием одного из многочисленных правил обработки промахов. Ниже приведены два таких правила:

- любая точка данных, расположенная в 1.5 (и более) раза ниже самого нижнего квартиля в диапазоне интерквартилей или в 1.5 (и более) раза выше самого верхнего квартиля в диапазоне интерквартилей, считается промахом;
- любая точка данных, расположенная в два раза ниже или выше стандартного отклонения для исследуемых данных, считается промахом и исключается из набора данных.

Вместо использования одного из этих правил исключения промахов при работе с данными можно изменить модель, как показано в следующем разделе.

Распределение Стьюдента

В качестве обобщенного правила в байесовском анализе предпочтение отдается кодированию предварительных предположений напрямую в модель с ис-

пользованием априорных распределений вероятностей и правдоподобий, а не применению специализированных для конкретных ситуаций эвристик, таких как правила исключения промахов.

Одним весьма полезным вариантом для обработки промахов при использовании гауссовых распределений является замена гауссова правдоподобия на распределение Стьюдента или t -распределение. Распределение Стьюдента имеет три параметра: среднее значение, коэффициент масштаба (аналогичен стандартному отклонению) и число степеней свободы, которое обычно обозначается греческой буквой ν и может изменяться в интервале $[0, \infty]$. В соответствии с терминологией Крушке будем называть ν параметром нормальности (normality parameter), так как он действительно управляет степенью нормальности распределения Стьюдента. При значении $\nu = 1$ мы получаем распределение с медленно убывающими (или широкими) хвостами, также известными как распределение Коши или распределение Лоренца. Последнее название особенно широко распространено в среде физиков. Выражение «медленно убывающие (или широкие) хвосты» подразумевает, что вероятность обнаружения значений в более удаленных от среднего значения областях выше, чем в гауссовом распределении. Другими словами, значения не так плотно сконцентрированы в области, близкой к среднему значению, как в распределениях с быстро убывающими или узкими хвостами, подобных гауссову распределению. Например, 95 % значений из распределения Коши расположены между -12.7 и 12.7 . Для сравнения: в гауссовом распределении (со стандартным отклонением, равным единице) 95 % значений находятся в интервале от -1.96 до 1.96 . На другой границе пространства параметров, где ν приближается к бесконечности, мы снова возвращаемся к гауссову распределению (поскольку невозможно быть более нормальным, чем нормальное распределение). Чрезвычайно любопытной особенностью распределения Стьюдента является тот факт, что при $\nu \leq 1$ не определено среднее значение. Разумеется, на практике любая конечная выборка из распределения Стьюдента представляет собой обычный набор чисел, по которым всегда возможно вычислить эмпирическое среднее значение. Может ли в теоретическом распределении отсутствовать явно определенное среднее значение? Это можно понимать следующим образом: хвосты такого распределения настолько широки (убывают настолько медленно), что в любой момент может случиться так, что мы получим значение выборки практически из любой точки реальной линии, поэтому если сохранять получаемые числа, то никогда не удастся выполнить аппроксимацию по некоторому фиксированному значению. Вместо этого оценка будет непрерывно колебаться в окрестности данного значения. Попробуйте выполнить приведенный ниже код несколько раз (а затем изменить значение df на большее число, например 100):

```
np.mean(stats.t(loc=0, scale=1, df=1).rvs(100))
```

Аналогичным образом дисперсия этого распределения определяется только для значений $\nu > 2$. Поэтому внимательно следите за тем, чтобы коэффициент

масштаба распределения Стьюдента не был равен стандартному отклонению. Для $\nu > 2$ распределение не имеет явно определенной дисперсии, следовательно, не имеет определенного стандартного отклонения. Коэффициент масштаба и стандартное отклонение приближаются друг к другу, когда ν стремится к бесконечности:

```
plt.figure(figsize=(10, 6))
x_values = np.linspace(-10, 10, 500)
for df in [1, 2, 30]:
    distri = stats.t(df)
    x_pdf = distri.pdf(x_values)
    plt.plot(x_values, x_pdf, label=f'$\nu = {df}$', lw=3)
x_pdf = stats.norm.pdf(x_values)
plt.plot(x_values, x_pdf, 'k--', label=r'$\nu = \infty$')
plt.xlabel('x')
plt.yticks([])
plt.legend()
plt.xlim(-5, 5)
```

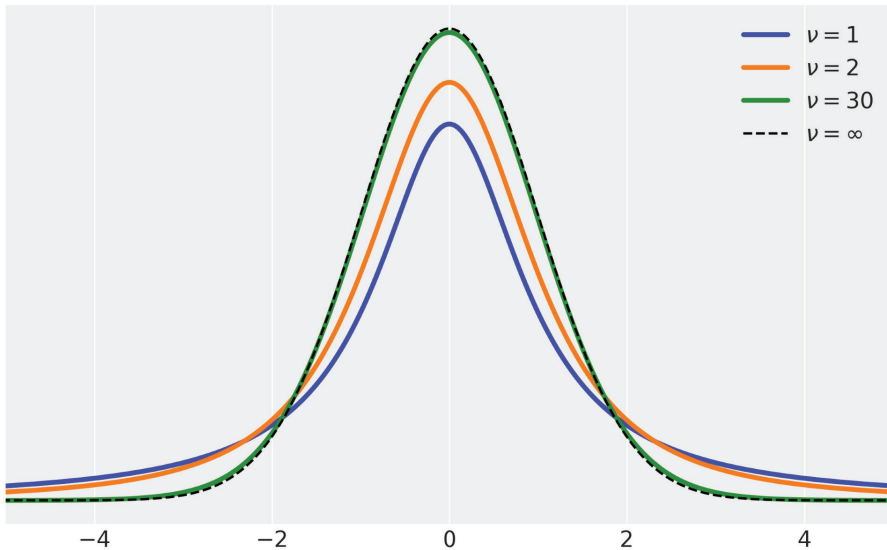


Рис. 2.12

Изменим модель, которая рассматривалась в предыдущих разделах, заменив гауссово распределение на распределение Стьюдента (t-распределение):

$$\begin{aligned}
 \mu &\sim \mathcal{U}(l, h); \\
 \sigma &\sim |\mathcal{N}(0, \sigma_o)|; \\
 \nu &\sim \text{Exp}(\lambda); \\
 y &\sim \mathcal{T}(\mu, \sigma, \nu).
 \end{aligned}
 \tag{2.3}$$

Так как распределение Стьюдента имеет на один параметр (v) больше, чем гауссово распределение, необходимо определить еще одно априорное распределение вероятностей. Определим v как экспоненциальное распределение со средним значением 30. На рис. 2.12 можно видеть, что t -распределение Стьюдента с $v = 30$ выглядит почти похожим на гауссово распределение (хотя не является таковым). В действительности на этой же схеме можно видеть, что большинство действий происходит при относительно малых значениях v . Следовательно, можно утверждать, что экспоненциальное априорное распределение со средним значением 30 является низкоинформативным априорным распределением, сообщаям модели, что мы более или менее уверены в том, что значение v должно приблизительно равняться 30, но при этом с легкостью может принимать немного меньшие или немного большие значения. Во многих задачах оценка v не является прямым объектом интереса. Созданную модель можно представить следующей графической схемой:

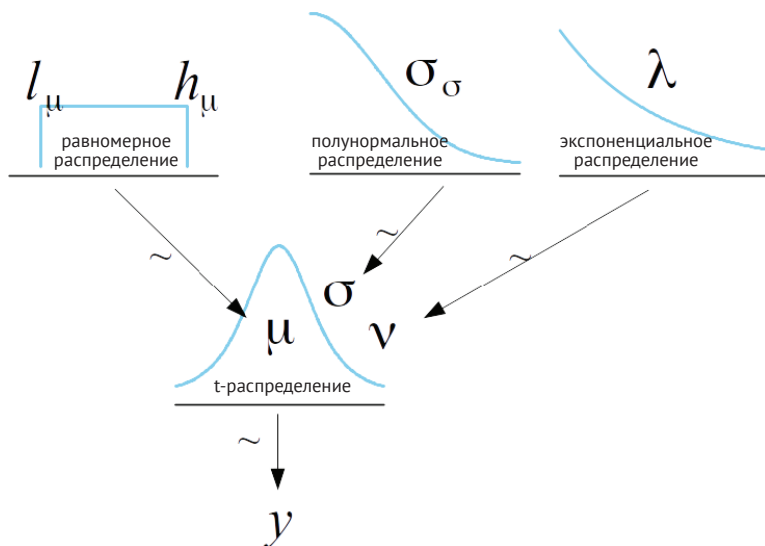


Рис. 2.13

Как обычно, библиотека РумСЗ позволяет записать (или переписать) модели всего лишь несколькими строками исходного кода. Здесь особого внимания требует тот факт, что экспоненциальная зависимость в библиотеке РумСЗ параметризуется как противоположность (обратная величина) среднему значению:

```
with pm.Model() as model_t:
    μ = pm.Uniform('μ', 40, 75)
    σ = pm.HalfNormal('σ', sd=10)
    ν = pm.Exponential('ν', 1/30)
```

```

y = pm.StudentT('y', mu=μ, sd=σ, nu=v, observed=data)
trace_t = pm.sample(1000)
az.plot_trace(trace_t)

```

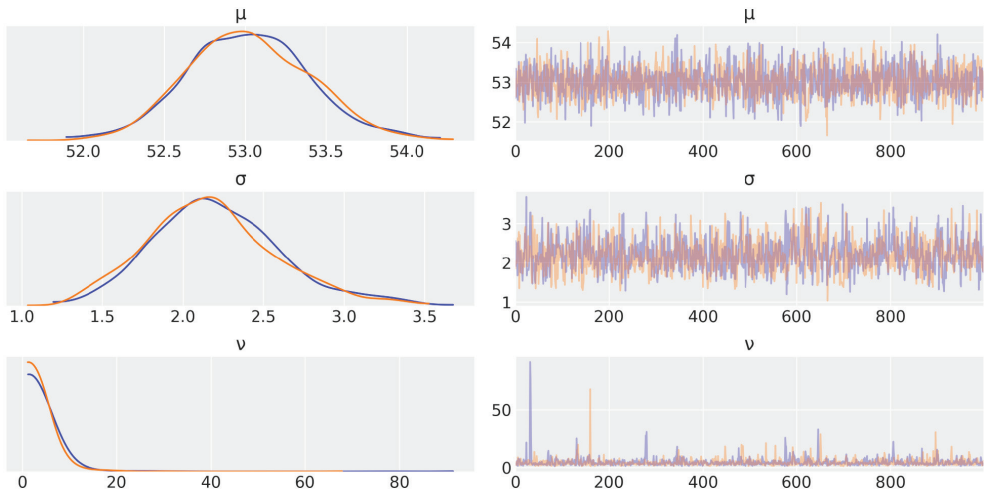


Рис. 2.14

Сравним трассировку для модели `model_g` (рис. 2.9) с трассировкой для модели `model_t` (рис. 2.14). Далее выведем итоговый отчет по модели `model_t` и сравним его с итоговым отчетом по модели `model_g`. Прежде чем продолжить чтение, задержитесь на минуту и попробуйте найти различия между этими результатами. Возможно, вы заметите кое-что интересное.

```
az.summary(trace_t)
```

Таблица 2.3

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
μ	53.00	0.39	0.01	52.28	53.76	1254.0	1.0
σ	2.20	0.39	0.01	1.47	2.94	1008.0	1.0
ν	4.51	3.35	0.11	1.10	9.27	898.0	1.0

Оценка μ в обеих моделях почти одинакова, значения различаются на ≈ 0.5 . Оценка σ изменяется от ≈ 3.5 до ≈ 2.1 . Это следствие того, что t-распределение Стьюдента присваивает меньший вес (то есть подвержено меньшему воздействию) значениям, удаленным от среднего значения. Также можно видеть, что $\nu \approx 4.5$, то есть мы получили не очень похожее на гауссово распределение, отличающееся от него более медленно убывающими (более широкими) хвостами.

Теперь выполним проверку прогнозируемого апостериорного распределения для модели Стьюдента и сравним ее с гауссовой моделью:

```
y_ppc_t = pm.sample_posterior_predictive(
    trace_t, 100, model_t, random_seed=123)
y_pred_t = az.from_pymc3(trace=trace_t, posterior_predictive=y_ppc_t)
az.plot_ppc(y_pred_t, figsize=(12, 6), mean=False)
ax[0].legend(fontsize=15)
plt.xlim(40, 70)
```

Использование t-распределения Стьюдента в применяемой модели приводит к получению прогнозируемых выборок, которые визуально лучше соответствуют реальным данным, если рассматривать пиковое значение распределения, а также его размах. Отметим, что выборки распространяются дальше, чем основной массив данных. Кроме того, обратите внимание на то, что некоторые прогнозируемые выборки выглядят слишком пологими (практически без пика). Это прямое следствие того, что при использовании t-распределения Стьюдента ожидается появление точек данных, удаленных от среднего значения или от основного массива данных. Это также является обоснованием установки значения `xlim` в интервале `[40, 70]`:

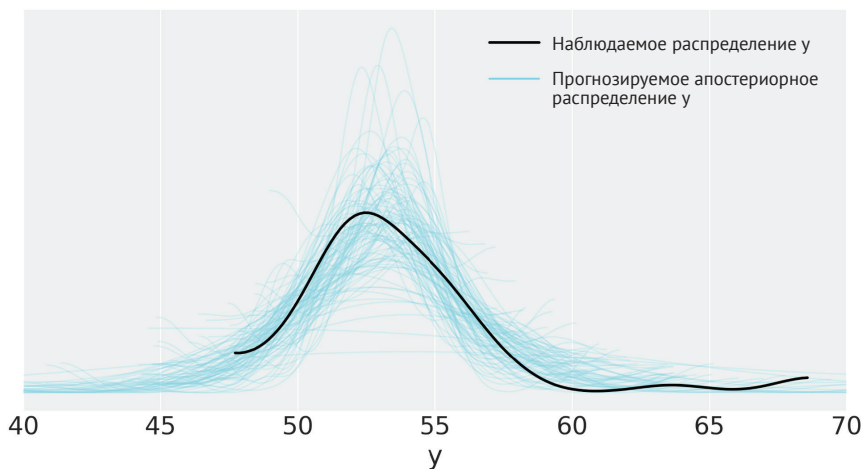


Рис. 2.15

Распределение Стьюдента позволяет получить более надежную оценку, так как воздействие промахов уменьшает значение ν , а не смещает его в направлении промахов, увеличивая при этом стандартное отклонение. Таким образом, среднее значение и коэффициент масштаба оцениваются с учетом более значимых весов точек данных из основного массива по сравнению с точками данных вне его. Еще раз подчеркнем, что важно помнить о том, что коэффициент масштаба не является стандартным отклонением. Тем не менее коэффициент

масштаба связан с размахом данных: чем меньше коэффициент масштаба, тем более компактным становится распределение. Кроме того, для значений $v \geq 2$ коэффициент масштаба становится весьма близким (по крайней мере, для большинства практических целей) к значениям, оцениваемым после удаления промахов. Поэтому в качестве практического правила для не слишком малых значений v с учетом того, что это не вполне правильно с теоретической точки зрения, можно считать коэффициент масштаба t -распределения Стьюдента разумным практическим представлением, заменяющим стандартное отклонение для данных после удаления промахов.

СРАВНЕНИЕ ГРУПП

При выполнении статистического анализа часто встречается задача сравнения групп. Предметом исследований может быть реакция пациентов на прием определенного лекарства, сокращение дорожно-транспортных происшествий после ввода в эксплуатацию новой системы регулирования движения, эффективность работы студентов при использовании различных методик обучения и т. д.

Иногда этот тип задач ограничен рамками сценария проверки предположений (гипотез) с целью объявления результата статистически значимым. Надежда только на статистическую значимость может оказаться проблематичной по многим причинам: с одной стороны, статистическая значимость не равноценна практической значимости, с другой – действительно небольшой эффект может быть объявлен значительным только при условии сбора достаточного объема данных. Здесь идея проверки гипотезы объединяется с концепцией p -значений (p -уровня значимости). Это объединение обосновывается не с научной точки зрения, а скорее с культурной – люди привыкли к такому образу мышления в основном благодаря тому, что они изучали в большинстве вводных курсов по статистике. Многочисленные учебные конспекты и статьи свидетельствуют о том, что гораздо чаще p -значения используются и интерпретируются неправильно, даже теми, кто ежедневно имеет дело с ними.

Вместо выполнения проверок гипотез мы выберем другой путь и сосредоточимся на оценке размера эффекта (effect size), то есть на количественной оценке различий между двумя группами. Преимуществом подхода с использованием размера эффекта является отказ от типа вопросов, требующих ответов (да/нет; действует ли это? возникает ли какой-либо эффект?), – и переход к более точным вопросам типа: насколько хорошо это действует? насколько велик/мал эффект?



Размер эффекта (effect size) – это всего лишь способ числовой оценки размера различий между двумя группами.

Иногда при сравнении групп говорят о контрольной группе и об экспериментальной группе (возможно существование нескольких контрольных и экс-

периментальных групп). Например, это имеет смысл, когда необходимо проверить действие нового лекарства: чтобы исключить эффект плацебо и другие факторы, необходимо сравнить процесс лечения с применением нового препарата с контрольной группой (которая не принимает этот препарат). В этом случае непременно нужно знать, насколько эффективно действует конкретное лекарство по сравнению с отсутствием действия каких-либо лекарств (или, если обобщить, по сравнению с эффектом плацебо). Здесь возникает важный альтернативный вопрос: насколько эффективно новое лекарство по сравнению с уже апробированным наиболее широко применяемым препаратом для лечения данной болезни. В этом случае контрольной группой не может быть плацебо, в ней должно тестироваться другое лекарство. Фиктивные контрольные группы – прямой путь ко лжи с использованием статистики.

Например, предположим, что вы работаете в компании, производящей молочные продукты, которая планирует начать продажи йогуртов с высоким содержанием сахара для детей, убеждая их родителей в том, что этот конкретный йогурт укрепляет иммунную систему или способствует быстрому росту. Одним из способов обмана и фальсификации для обоснования таких утверждений являются данные, полученные при использовании молока или даже воды в качестве контрольной группы вместо другого, более дешевого, менее «раскрученного» йогурта с низким содержанием сахара. В моем изложении такой подход может выглядеть весьма глупо, но огромное количество реальных исследований выполняется именно таким способом. На самом деле я фактически цитирую реальные документы, а не выдуманные предполагаемые ситуации. Когда кто-то заявляет о том, что некий объект или продукт прочнее, лучше, быстрее, долговечнее, не забудьте уточнить, на какой основе выполнялось сравнение.

Для сравнения групп необходимо выбрать признак или несколько признаков, по которым будет производиться сравнение. Наиболее часто выбираемым признаком является среднее значение каждой группы. Поскольку мы занимаемся байесовским анализом, будем пытаться получить апостериорное распределение различий в средних значениях между группами, а не только точечную оценку различий. Для визуального наблюдения и интерпретации такого апостериорного распределения воспользуемся тремя инструментами:

- график апостериорного распределения с эталонным (нормальным) значением;
- d-мера Коэна;
- вероятность превосходства.

В предыдущей главе уже рассматривался пример с использованием функции `az.plot_posterior` с эталонным (нормальным) значением, другой пример мы вскоре рассмотрим. Здесь применяются новые средства: d-мера Коэна (Cohen's d) и вероятность превосходства (probability of superiority) – два наиболее широко используемых способа выражения размера эффекта. Рассмотрим эти два способа подробнее.

d-мера Коэна

Общепринятым способом измерения размера эффекта является d-мера Коэна, определяемая следующей формулой:

$$\frac{\mu_2 - \mu_1}{\sqrt{\frac{\sigma_2^2 + \sigma_1^2}{2}}}. \quad (2.4)$$

В соответствии с этой формулой размер эффекта является разностью средних значений, разделенной на объединенное стандартное отклонение для обеих групп. Так как можно получить апостериорное распределение средних значений и стандартных отклонений, можно также вычислить апостериорное распределение значений d-меры Коэна. Разумеется, если необходимо всего лишь одно значение, то можно вычислить среднее значение этого апостериорного распределения и получить единственное значение d-меры Коэна. В общем случае при вычислении объединенного стандартного отклонения явно учитывается размер выборки каждой группы, но в формулу (2.4) размеры выборок для групп не включены. Причина в том, что значения стандартного отклонения выводятся из апостериорного распределения, следовательно, неопределенность стандартных отклонений для каждой группы уже учтена.

! d-мера Коэна – это способ измерения размера эффекта, при котором разность средних значений нормализуется с учетом объединенных стандартных отклонений для обеих групп.

d-мера Коэна вводит свойство вариации для каждой группы, используя стандартные отклонения этих групп. Это действительно важно, поскольку различия при стандартном отклонении 0.1 выглядят большими по сравнению с теми же различиями при стандартном отклонении 10. Кроме того, изменение на x единиц в одной группе по отношению к другой может быть объяснено как изменение в точности на x единиц каждой отдельной точки данных, или как изменение половины точек данных на $2x$ единиц, тогда как другая половина не изменилась, или множеством других возможных вариантов. Таким образом, включение внутренних вариаций групп – это способ размещения различий в контексте. Изменение масштаба (нормализация) различий помогает правильно оценить важность конкретных различий между группами, даже если точно неизвестен коэффициент масштаба, используемый при измерениях.

! d-меру Коэна можно интерпретировать как стандартизованную оценку (z-оценку). Стандартизованная оценка – это число (со знаком) стандартных отклонений, на которое некоторое значение отличается от среднего значения для наблюдаемого или измеряемого объекта или явления (то есть мера относительного разброса наблюдаемого или измеряемого значения). Таким образом, d-меру Коэна, равную 0.5, можно интерпретировать как различие в 0.5 стандартного отклонения одной группы по отношению к другой.

Даже если различия средних значений нормализованы, может сохраняться необходимость автокалибровки на основе контекста решаемой задачи, чтобы получить возможность определить, что полученное значение является большим,

малым, средним и т. п. Подобная процедура автокалибровки формируется как результат накапливаемого практического опыта. Можно привести следующий пример: если при выполнении нескольких процессов анализа для решения более или менее одинаковых типов задач вычислена d-мера Коэна, равная, скажем, 1, то при вычислении (для аналогичной задачи) d-меры Коэна, равной 2, становится понятно, что получен важный результат (или допущена ошибка в процессе анализа). Если ваш практический опыт не слишком велик, то можно обратиться к эксперту в исследуемой области, чтобы правильно оценить результат. Для подробного изучения внешнего вида различных значений d-меры Коэна можно обратиться к весьма информативной веб-странице <http://rpsychologist.com/d3/cohend>. Здесь также можно найти другие методы выражения размера эффекта. Некоторые из них могут оказаться более интуитивно понятными, как, например, вероятность превосходства, которая рассматривается в следующем разделе.

Вероятность превосходства

Это альтернативный метод описания размера эффекта, который определяет-ся как вероятность того, что точка данных, выбранная случайным образом из одной группы, имеет большее значение, чем точка данных, также случайным образом выбранная из другой группы. Если предположить, что распределение исследуемых данных является нормальным, то можно вычислить вероятность превосходства по d-мере Коэна, используя следующую формулу:

$$ps = \Phi\left(\frac{\delta}{\sqrt{2}}\right). \quad (2.5)$$

Здесь Φ – совокупное нормальное распределение, δ – d-мера Коэна. Можно также вычислить точечную оценку вероятности превосходства (которую обычно и указывают в отчетах) или полное апостериорное распределение значений. Если мы с уверенностью формулируем предположения о нормальности, то можем использовать эту формулу для получения вероятности превосходства по d-мере Коэна. Но поскольку мы располагаем выборками из апостериорного распределения, можно напрямую вычислять вероятность превосходства (см. раздел «Упражнения»). Это чрезвычайно значительное преимущество при использовании методов Монте-Карло с использованием цепей Маркова (Markov chain Monte Carlo – MCMC) – поскольку выборки получаются из апостериорного распределения, становится возможным вычисление многих величин.

Набор данных tips

Для практических исследований по теме текущего раздела мы будем использовать набор данных tips. Впервые этот набор данных упоминался П. Дж. Брайантом (P. G. Bryant) и М. Смитом (M. Smith) в 1995 году в публикации «Practical Data Analysis: Case Studies in Business Statistics».

Необходимо исследовать влияние дня недели на сумму чаевых, оставляемых посетителями ресторана. В этом примере различными группами являются дни

недели. Особо отметим отсутствие контрольной группы и экспериментальной группы. При необходимости можно установить один день, например четверг, как эталонную или контрольную группу. Начнем анализ с загрузки набора данных `tips` как фрейма данных `DataFrame` библиотеки `pandas`, используя для этого одну строку кода. Если вы незнакомы с библиотекой `pandas`, то отметим, что команда `tail` применяется для вывода нескольких последних строк `DataFrame` (также можно воспользоваться командой `head` для вывода нескольких первых строк фрейма):

```
tips = pd.read_csv('../data/tips.csv')
tips.tail()
```

Таблица 2.4

	сумма_счета (total_bill)	сумма чаевых (tip)	пол (sex)	курящий (smoker)	день (day)	время (time)	размер (size)
239	29.03	5.92	Мужской (Male)	Нет (No)	Сб (Sat)	Обед (Dinner)	3
240	27.18	2.00	Женский (Female)	Да (Yes)	Сб (Sat)	Обед (Dinner)	2
241	22.67	2.00	Мужской (Male)	Да (Yes)	Сб (Sat)	Обед (Dinner)	2
242	17.82	1.75	Мужской (Male)	Нет (No)	Сб (Sat)	Обед (Dinner)	2
243	18.78	3.00	Женский (Female)	Нет (No)	Чт (Thurs)	Обед (Dinner)	2

Из этого фрейма данных (объекта `DataFrame`) мы будем использовать только столбцы `day` и `tip`. Можно сформировать графики используемых данных с помощью функции `violinplot` из библиотеки `seaborn`:

```
sns.violinplot(x='day', y='tip', data=tips)
```

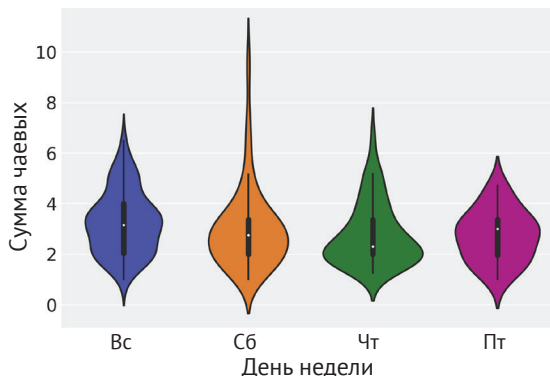


Рис. 2.16

Для упрощения создадим три переменные: `y`, представляющую набор данных `tips`, `idx` – категориальную фиктивную переменную для кодирования дней недели с помощью чисел, то есть `[0,1,2,3]` вместо [четверг, пятница, суббота, воскресенье], а также переменную `groups`, содержащую количество групп (4):

```
tip = tips['tip'].values
idx = pd.Categorical(tips['day'],
                    categories=['Thur', 'Fri', 'Sat', 'Sun']).codes
groups = len(np.unique(idx))
```

Модель для этой задачи почти та же, что и модель `model_g`. Единственное различие заключается в том, что здесь μ и σ должны быть векторами, а не скалярными переменными. При таких условиях синтаксис РунМСЗ чрезвычайно удобен: вместо циклов `for` можно записать используемую здесь модель в векторизованной форме. Это означает, что для априорных распределений передается аргумент `shape`, а для функции правдоподобия соответствующим образом индексируются переменные `means` и `sds` с использованием переменной `idx`:

```
with pm.Model() as comparing_groups:
    mu = pm.Normal('mu', mu=0, sd=10, shape=groups)
    sigma = pm.HalfNormal('sigma', sd=10, shape=groups)

    y = pm.Normal('y', mu=mu[idx], sd=sigma[idx], observed=tip)

    trace_cg = pm.sample(5000)
    az.plot_trace(trace_cg)
```

Приведенный ниже код представляет только один из способов графического отображения различий без повторения операции сравнения. Вместо вывода содержимого матрицы «все против всех» строится график только ее верхней треугольной части:

```
dist = stats.norm()

_, ax = plt.subplots(3, 2, figsize=(14, 8), constrained_layout=True)

comparisons = [(i, j) for i in range(4) for j in range(i+1, 4)]
pos = [(k, l) for k in range(3) for l in (0, 1)]

for (i, j), (k, l) in zip(comparisons, pos):
    means_diff = trace_cg['mu'][:, i] - trace_cg['mu'][:, j]
    d_cohen = (means_diff / np.sqrt((trace_cg['sigma'][:, i]**2 + trace_cg['sigma'][:, j]**2) /
                                     2)).mean()
    ps = dist.cdf(d_cohen/(2**0.5))
    az.plot_posterior(means_diff, ref_val=0, ax=ax[k, l])
    ax[k, l].set_title(f'$\mu_{i}$-$\mu_{j}$')
    ax[k, l].plot(0, label=f"Cohen's d = {d_cohen:.2f}\nProb sup = {ps:.2f}", alpha=0)
    ax[k, l].legend()
```

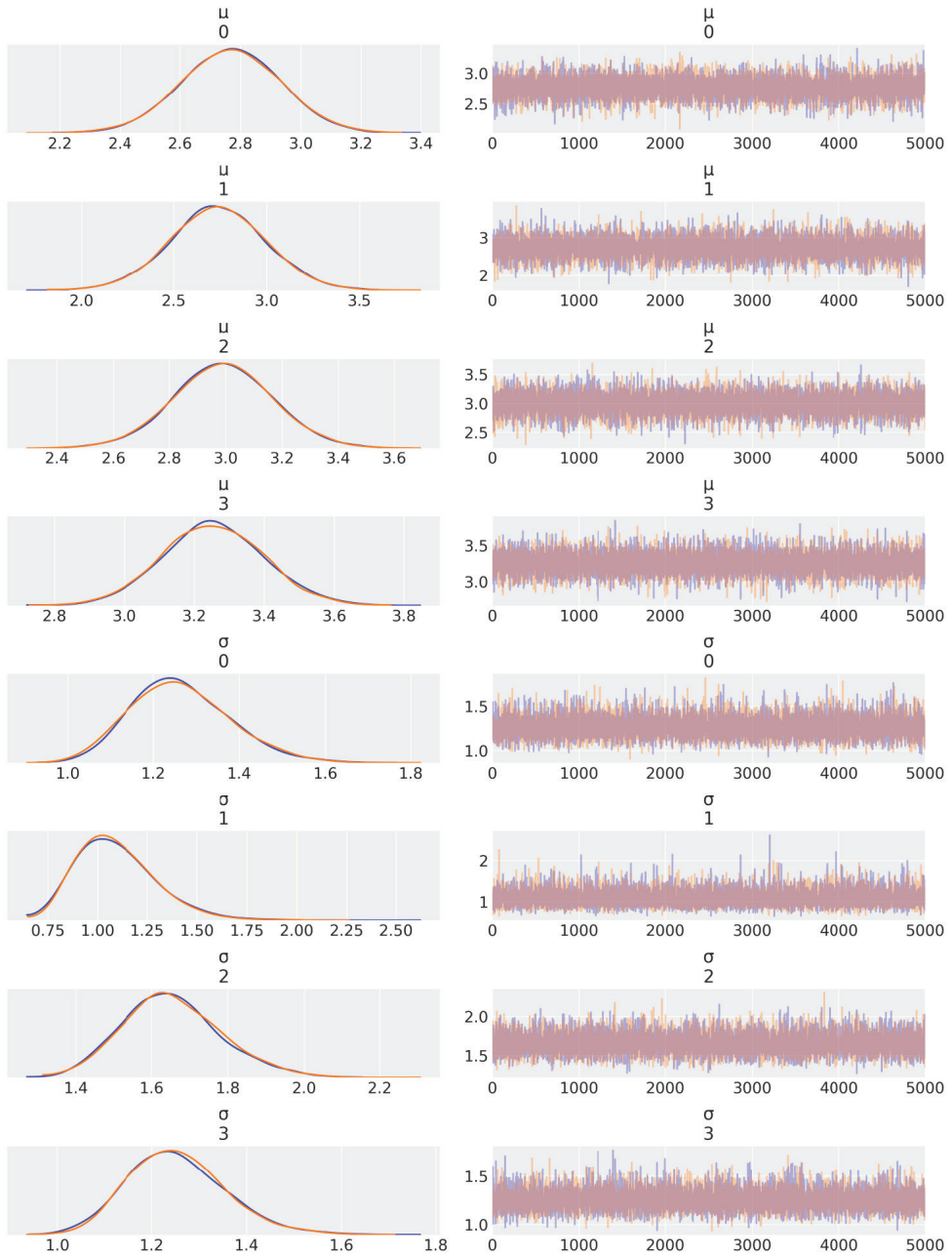


Рис. 2.17

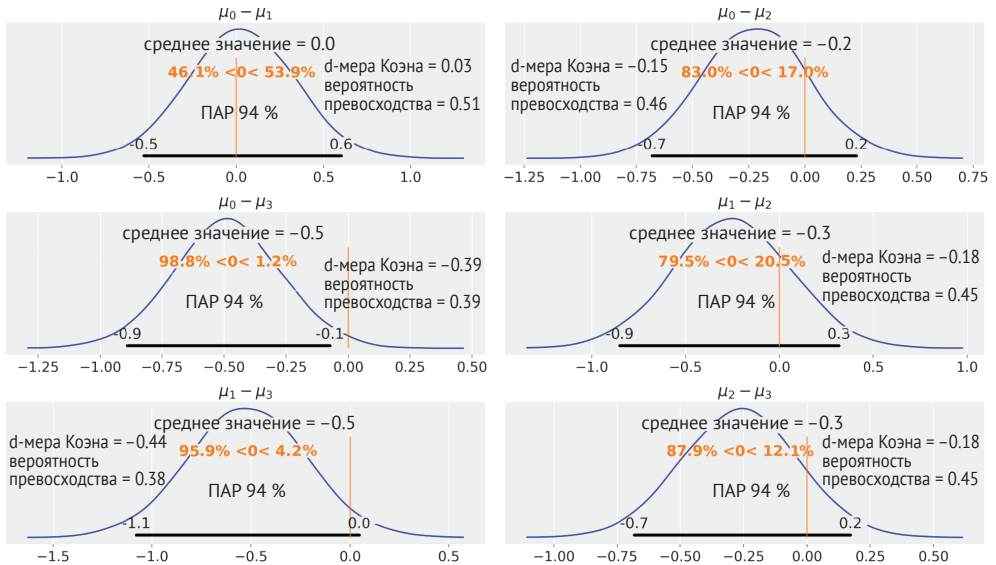


Рис. 2.18

Один из способов интерпретации полученных результатов – сравнение эталонного значения с интервалом плотности апостериорного распределения (ПАР). В соответствии с графиками на рис. 2.18 наблюдается только один случай, когда из интервала ПАР 94 % исключено эталонное значение, равное нулю, то есть существует различие в суммах чаевых между четвергом (Thursday) и воскресеньем (Sunday). Для всех остальных примеров невозможно исключить вероятность нулевого различия (по критерию перекрытия ПАР и эталонного значения). Но даже в этом случае можно ли считать различие приблизительно в 0.5 доллара довольно большим? Достаточно ли существования такого различия, чтобы работать в воскресенье, лишившись возможности провести время с семьей или с друзьями? Достаточно ли велико это различие для того, чтобы выровнять среднюю сумму чаевых по рассматриваемым четырем дням и оставлять каждому официанту одинаковую сумму чаевых? На вопросы такого рода невозможно ответить с помощью статистики, поскольку статистика всего лишь предоставляет определенную (числовую) информацию.

ИЕРАРХИЧЕСКИЕ МОДЕЛИ

Предположим, что необходимо проанализировать качество воды в некотором городе, поэтому мы используем выборки на основе разделения городской территории на смежные районы. Предполагается также, что имеются два возможных варианта анализа получаемых данных:

- исследование каждого района как отдельного объекта;

- объединение всех полученных данных и оценка качества воды в городе как в одной большой группе.

Оба варианта могут быть вполне разумными решениями в зависимости от того, что именно мы хотим узнать. Можно предпочесть первый вариант, обосновав его тем, что при этом мы получаем более подробную картину задачи, тогда как некоторые подробности могут остаться незамеченными или менее убедительными, если усреднить данные. Обоснование второго варианта может состоять в том, что при объединении данных получается выборка более крупного размера, следовательно, более точная оценка. В обоих случаях обоснования корректны, но есть возможность выбрать еще и некоторый промежуточный вариант. Можно создать модель для оценки качества воды в каждом районе и одновременно оценивать качество воды во всем городе. Такой тип модели называют иерархической или многоуровневой моделью, так как данные моделируются с использованием иерархической структуры, то есть структуры с несколькими уровнями.

Но как создать такую иерархическую модель? Если говорить кратко, то вместо жесткого определения параметров априорных распределений как некоторых постоянных чисел выполняется их оценка непосредственно по исследуемым данным посредством размещения совместно используемых априорных распределений для всех параметров. Подобные априорные распределения более высокого уровня часто называют априорными гиперраспределениями, а их параметры – гиперпараметрами (приставка «гипер» происходит от греч. *hyper* – над, сверх). Разумеется, также возможно размещение априорных распределений над априорными гиперраспределениями, то есть создание любого необходимого количества уровней. Но при этом модель быстро становится трудной для понимания, и если задача в действительности не требует более глубокой структуры, то добавление избыточных уровней не поможет улучшить качество статистических выводов. С другой стороны, можно окончательно запутаться в переплетении априорных гиперраспределений и гиперпараметров, лишившись возможности какой-либо осмысленной их интерпретации и в определенной степени дискредитируя преимущества статистики, основанной на моделях. Основным принципом создания моделей является их полное соответствие смыслу исследуемых данных, и модели прежде всего должны отображать структуру этих данных (и получать соответствующие преимущества).

Для демонстрации основных концепций использования иерархических моделей воспользуемся имитационной моделью из примера об оценке качества воды, который обсуждался в начале текущего раздела, где будем использовать искусственно сгенерированные данные. Предположим, что уже собраны образцы воды из трех различных районов города и измерено содержание свинца в этих образцах. Образцы с содержанием свинца, превышающим концентрацию, рекомендованную Всемирной организацией здравоохранения (ВОЗ), помечаются нулем, а образцы с нормальным содержанием свинца помечаются

единицей. Это учебный пример, в реальной практике потребуются постоянные измерения концентрации свинца и, возможно, гораздо большее количество групп. Но для текущих целей такого примера вполне достаточно, чтобы понять сущность и подробности применения иерархических моделей.

Данные для примера можно искусственно сгенерировать с помощью следующего кода:

```
N_samples = [30, 30, 30]
G_samples = [18, 18, 18]

group_idx = np.repeat(np.arange(len(N_samples)), N_samples)
data = []
for i in range(0, len(N_samples)):
    data.extend(np.repeat([1, 0], [G_samples[i], N_samples[i]-G_samples[i]]))
```

Здесь имитируется эксперимент, где выполнены измерения в трех группах, каждая из которых состоит из определенного количества образцов. Общее число образцов в каждой группе сохранено в списке `N_samples`. Используя этот список, мы сохраняем записи о количестве образцов с хорошим качеством воды в каждой группе. Остальной код предназначен для генерации списка данных с заполнением его нулями и единицами.

По существу здесь применяется та же модель, которая использовалась для задачи о подбрасывании монеты, за исключением двух важных характеристик:

- здесь определены два априорных гиперраспределения, на которые воздействует априорное бета-распределение;
- вместо применения априорных гиперраспределений к параметрам α и β мы определяем эти априорные гиперраспределения напрямую для μ – среднего значения бета-распределения и для κ – точности (концентрации) бета-распределения. Точность – это аналог значения, обратного стандартному отклонению: чем больше значение κ , тем более концентрированным будет бета-распределение:

$$\begin{aligned}
 \mu &\sim \text{Beta}(\alpha_\mu, \beta_\mu); \\
 \kappa &\sim |\text{Normal}(0, \sigma_\kappa)|; \\
 \alpha - \mu &\neq \kappa; \\
 \beta &= (1 - \mu) * \kappa; \\
 \theta_i &\sim \text{Beta}(\alpha_i, \beta_i); \\
 y_i &\sim \text{Bern}(\theta_i).
 \end{aligned} \tag{2.6}$$

Отметим использование дополнительного индекса i для обозначения факта существования в модели групп с различными значениями для некоторых параметров. Таким образом, не все параметры совместно используются в различных группах. На диаграмме Крушке отчетливо видно, что новая модель содержит один дополнительный уровень по сравнению со всеми ранее используемыми моделями:

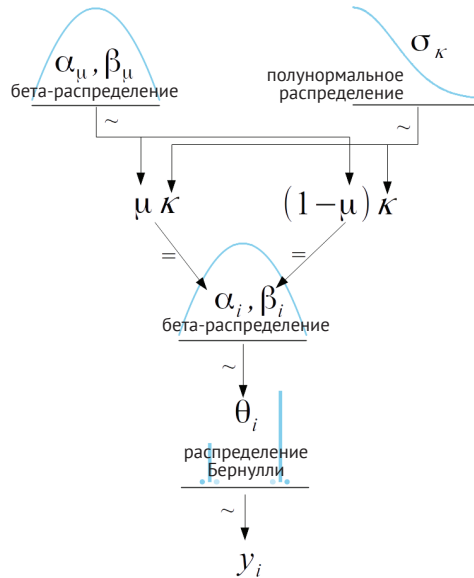


Рис. 2.19

Обратите внимание на использование на рис. 2.19 символа $=$ вместо символа $\sim$ при определении α_i и β_i , поскольку при известных μ и κ значения α_i и β_i становятся полностью определенными. В соответствии с этим условием мы называем такой тип переменной детерминированным в противоположность стохастическим (случайным) переменным, таким как μ , κ или θ .

Теперь необходимо сказать несколько слов о параметризации. Использование среднего значения и точности является математически равнозначным использованию параметризации α и β , подразумевая при этом, что мы должны получить те же результаты. Но почему выбирается этот обходной маневр вместо более прямого пути? Для такого выбора существуют две причины:

- во-первых, параметризация среднего значения и точности, являясь математически равнозначной, в числовом отношении лучше подходит для сэмплирования, следовательно, мы можем быть более уверенными в правильности результатов, возвращаемых методами библиотеки РуМСЗ. Более подробное обоснование и объяснение этого утверждения будет приведено в главе 8 «Механизмы статистического вывода»;
- вторая причина является учебно-педагогической. Это конкретный пример, демонстрирующий потенциальные возможности применения более одного метода выражения модели. Математически равнозначные реализации могут никоим образом не различаться на практике, а единственной подробностью, заслуживающей внимания, является эффектив-

ность механизма сэмплирования. Все прочие детали могут представлять лишь вопросы интерпретируемости модели. Для некоторых специфических задач или для особого состава аудитории, возможно, более удачным выбором будет отчет о среднем значении бета-распределения, нежели сведения о параметрах α и β .

Выполним реализацию и решим модель средствами библиотеки PyMC3, чтобы продолжить рассмотрение иерархических моделей:

```
with pm.Model() as model_h:
     $\mu$  = pm.Beta('μ', 1., 1.)
     $\kappa$  = pm.HalfNormal('κ', 10)

     $\theta$  = pm.Beta('θ', alpha= $\mu * \kappa$ , beta=(1.0- $\mu$ )* $\kappa$ , shape=len(N_samples))
    y = pm.Bernoulli('y', p= $\theta$ [group_idx], observed=data)

    trace_h = pm.sample(2000)
az.plot_trace(trace_h)
```

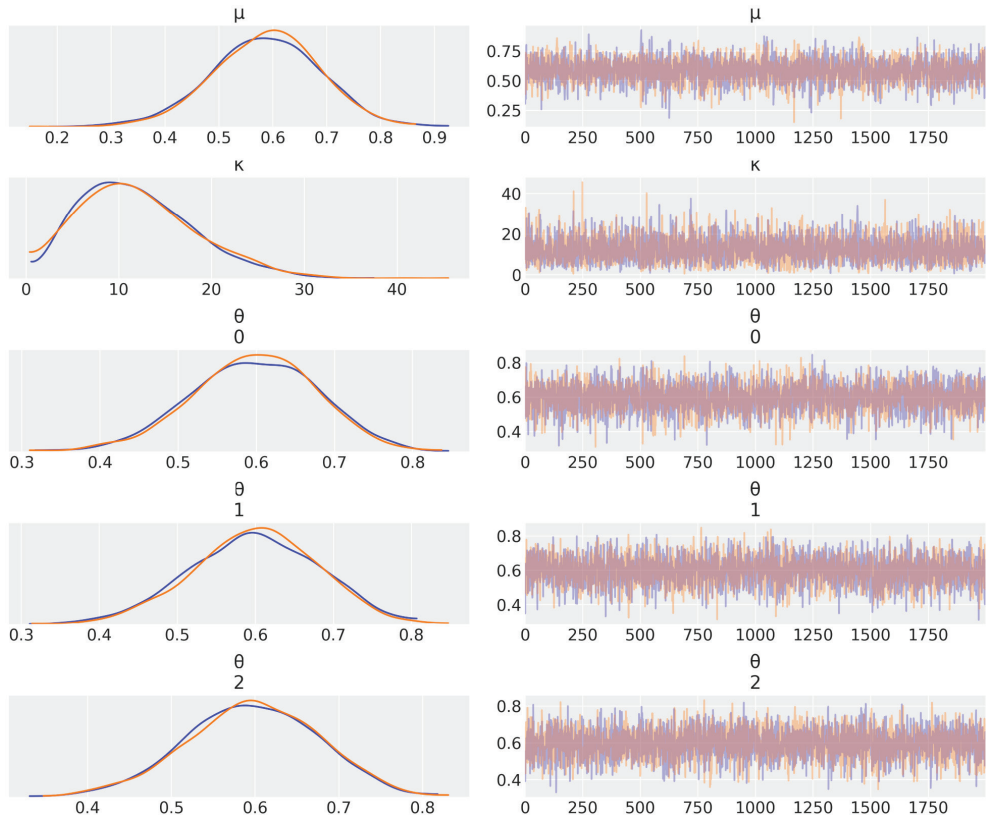


Рис. 2.20

Редуцирование

Чтобы продемонстрировать одно из основных последствий использования иерархических моделей, потребуется ваше участие, поэтому предлагаю присоединиться к небольшому эксперименту. Вам необходимо вывести и сохранить обобщенные полученные результаты с помощью функции `az.summary(trace_h)`. Затем предлагается повторно выполнить запуск модели еще два раза после внесения небольших изменений в искусственно сгенерированные данные и постоянно хранить записи обобщенных результатов. В общей сложности будет выполнено три запуска:

- с установкой для всех элементов списка `G_samples` значения 18;
- с установкой для всех элементов списка `G_samples` значения 3;
- в последнем случае для одного элемента списка `G_samples` устанавливается значение 18, а для прочих двух – значение 3.

Прежде чем продолжить, задержитесь на мгновение, чтобы мысленно представить себе итоговый результат этого эксперимента. Сосредоточьтесь на оценке среднего значения θ_i на каждом этапе эксперимента. Если взять за основу результаты двух первых запусков модели, то можно ли предсказать конечный результат в третьем случае?

Если разместить полученные результаты в таблице, то получится нечто более или менее похожее на табл. 2.5. Следует помнить, что небольшие вариации могут возникать из-за случайной природы процесса выборки (сэмплирования).

Таблица 2.5

G_samples	θ (среднее значение)
18, 18, 18	0.6, 0.6, 0.6
3, 3, 3	0.11, 0.11, 0.11
18, 3, 3	0.55, 0.13, 0.13

В первой строке можно видеть, что для набора данных с 18 «хорошими» образцами из 30 получено среднее значение 0.6 для θ . Напомним, что теперь среднее значение θ – это вектор из трех элементов, по одному для каждой группы. Во второй строке только 3 «хороших» образца из 30, а среднее значение θ равно 0.11. В последней строке наблюдается кое-что интересное и, возможно, неожиданное. Вместо получения комбинации оценок среднего значения θ из предыдущих строк, а именно 0.6, 0.11, 0.11, получены другие значения: 0.55, 0.13, 0.13. Что произошло? Может быть, это проблема сходимости или какая-то ошибка в определении модели? Ничего подобного – здесь мы наблюдаем редуцирование оценок к обобщенному среднему значению. В общем смысле это положительное явление, которое, несомненно, представляет собой следствие применения выбранной нами модели: используя априорные гиперраспределения, мы оцениваем параметры априорного бета-распределения исследуемых данных, и каждая группа информирует остальные, которые, в свою

очередь, получают информацию об оценке других групп. Короче говоря, группы эффективно обмениваются информацией через априорное гиперраспределение. В результате мы наблюдаем явление, называемое редуцированием (shrinkage). Этот эффект является следствием частичного объединения данных в общий пул. Мы моделируем группы ни как абсолютно независимые друг от друга объекты, ни как единую большую группу. Вместо этого создается некий усредненный вариант.

Чем это может быть полезно? Эффект редуцирования вносит определенный вклад в получение более стабильных статистических выводов. Это во многом похоже на то, что наблюдалось при использовании t -распределения Стьюдента для обработки промахов, где применялись результаты распределения с медленно убывающими (широкими) хвостами в модели, более устойчивой или менее чувствительной к наличию точек данных, расположенных далеко от среднего значения. Ввод априорных гиперраспределений и последующие статистические выводы результатов на высоком уровне в более консервативной модели (возможно, я впервые использовал слово «консервативный» в положительном смысле, даже с некоторым оттенком лести) позволяют сделать ее менее чувствительной к экстремальным значениям в отдельных группах. Для наглядности предположим, что размеры выборок отличаются друг от друга – некоторые меньше, некоторые больше. Чем меньше размер выборки, тем проще получить некорректные результаты. В самом крайнем случае если вы взяли только одну пробу воды в конкретном районе, то, возможно, просто случайно наткнулись на единственный старый свинцовый трубопровод, сохранившийся в этом районе, или наоборот – на единственный трубопровод, сделанный из поливинилхлорида. В первом случае имеет место переоценка плохого качества воды, во втором – недооценка. При использовании иерархической модели некорректная оценка по одной группе будет улучшена с помощью информации, предоставленной другими группами. Разумеется, больший размер выборки также оказывает положительное влияние, но зачастую этот вариант недоступен.

Конечно, размер эффекта редуцирования зависит от данных: группа с большим объемом данных будет сильнее влиять на оценку других групп, чем группа с меньшим количеством точек данных. Если несколько групп одинаковы, а одна группа отличается от них, то одинаковые группы будут информировать друг друга о своей одинаковости и подкреплять общую оценку. В то же время одинаковые группы будут подтягивать оценку менее похожей на них группы, это как раз тот случай, о котором говорилось в предыдущем примере.

Априорные гиперраспределения также участвуют в модулировании размера редуцирования. Можно эффективно использовать информативное априорное гиперраспределение для редуцирования оценки до некоторого разумного значения, если имеется заслуживающая доверия информация о распределении на уровне групп. Ничто не мешает создать иерархическую модель всего лишь с двумя группами, но предпочтительнее иметь несколько групп. Можно рас-

смаатривать редуцирование следующим образом: предположим, что каждая группа является точкой данных и выполняется оценка стандартного отклонения на уровне групп. В общем случае нельзя доверять оценке по слишком малому количеству точек данных, если только не существует строго определенного априорного распределения для информирования оценки. Аналогичное правило справедливо и для иерархической модели.

Также может вызывать интерес наблюдение за визуальной формой оцениваемого априорного распределения. Один из способов показан ниже:

```
x = np.linspace(0, 1, 100)
for i in np.random.randint(0, len(trace_h), size=100):
    u = trace_h['μ'][i]
    k = trace_h['κ'][i]
    pdf = stats.beta(u*k, (1.0-u)*k).pdf(x)
    plt.plot(x, pdf, 'C1', alpha=0.2)

u_mean = trace_h['μ'].mean()
k_mean = trace_h['κ'].mean()
dist = stats.beta(u_mean*k_mean, (1.0-u_mean)*k_mean)
pdf = dist.pdf(x)
mode = x[np.argmax(pdf)]
mean = dist.moment(1)
plt.plot(x, pdf, lw=3, label=f'mode = {mode:.2f}\nmean = {mean:.2f}')
plt.yticks([])

plt.legend()
plt.xlabel('$\theta_{prior}$')
plt.tight_layout()
```

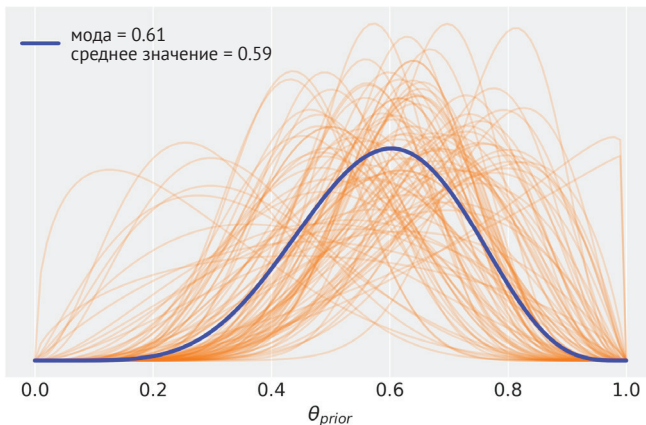


Рис. 2.21

Предполагалось, что этот пример будет последним в текущей главе, но по многочисленным просьбам я представляю еще одну иерархическую модель «на бис» – переходим к следующему разделу.

Еще один пример

Снова воспользуемся данными о химическом сдвиге. Эти данные получены из набора молекул протеинов, подготовленного лично мною. Точнее говоря, речь идет о химических сдвигах с участием ядер $^{13}\text{C}_\alpha$ (изотоп углерода) в протеинах, поскольку это наблюдаемый процесс. Мы будем измерять только определенные типы атомных ядер. Протеины состоят из цепочек размером в 20 элементов, известных как остатки или радикалы аминокислот. Каждая аминокислота может появляться в такой цепочке ноль или более раз, поэтому длина цепочек варьируется от нескольких до сотен и даже тысяч остатков аминокислот. Каждая аминокислота содержит один и только один изотоп углерода $^{13}\text{C}_\alpha$, поэтому можно с уверенностью связывать каждый химический сдвиг с конкретным остатком аминокислоты в протеине. Более того, каждая из 20 аминокислот имеет различные химические свойства, определяющие биологические характеристики протеина, некоторые могут обладать суммарными электрическими зарядами, другие абсолютно нейтральны. Некоторые аминокислоты окружены молекулами воды, другие же предпочитают компанию аминокислот того же или аналогичного типа и т. д. Здесь основной факт состоит в том, что аминокислоты похожи, но не одинаковы, следовательно, вполне разумным или даже естественным будет обоснование любого вывода, относящегося к химическому сдвигу, в терминах 20 групп в соответствии с определением типов аминокислот. Более подробно о протеинах можно узнать из превосходного видеоролика, размещенного на YouTube: <https://www.youtube.com/watch?v=wwTv8TqWC48>.

Для рассматриваемого здесь примера я ввел некоторые упрощения. Реальные эксперименты слишком сложны, и всегда существует вероятность того, что не удастся получить полную информацию о химическом сдвиге. Часто возникает проблема наложения сигналов, то есть в эксперименте не обеспечена степень разрешения, достаточная для различения двух и более похожих друг на друга сигналов. Для рассматриваемого примера такие сигналы были исключены, и можно считать исследуемый набор данных полноценным.

В приведенном ниже фрагменте кода данные загружаются в DataFrame, структуру которого следует объяснить предварительно. Структура содержит четыре столбца: в первом хранится идентификатор (ID) протеина (если вас интересует дополнительная информация об идентификаторах протеинов, то ее можно найти на сайте <https://www.rcsb.org/>). Во втором столбце – названия аминокислот в виде стандартного трехбуквенного кода, следующие столбцы содержат теоретически вычисленные значения химического сдвига (с использованием квантовохимических методов вычислений) и экспериментально измеренные химические сдвиги соответственно. Одна из целей этого примера – сравнение различий, чтобы узнать, насколько точно теоретические вычисления соответствуют экспериментальным измерениям. Поэтому вычисляются различия в последовательностях `diff` из библиотеки `pandas`:

```
cs_data = pd.read_csv('../data/chemical_shifts_theo_exp.csv')
diff = cs_data.theo.values - cs_data.exp.values
```

```
idx = pd.Categorical(cs_data['aa']).codes
groups = len(np.unique(idx))
```

Для наблюдения различий между иерархической и неиерархической моделями создадим две такие модели. Первая в основном совпадает с ранее используемой моделью `comparing_groups`:

```
with pm.Model() as cs_nh:
    μ = pm.Normal('μ', mu=0, sd=10, shape=groups)
    σ = pm.HalfNormal('σ', sd=10, shape=groups)

    y = pm.Normal('y', mu=μ[idx], sd=σ[idx], observed=diff)

    trace_cs_nh = pm.sample(1000)
```

Теперь создадим иерархическую версию модели. Для этого добавляются два априорных гиперраспределения: одно для среднего значения μ , второе для стандартного отклонения μ . Параметр σ остается без априорного гиперраспределения. Решение о выборе такой упрощенной модели принято исключительно в учебных целях. Могут возникать проблемы, из-за которых такая модель становится неприемлемой, но при необходимости можно рассмотреть вариант добавления априорного гиперраспределения для σ . Вы можете сделать это самостоятельно.

```
with pm.Model() as cs_h:
    # hyper_priors - априорные гиперраспределения
    μ_μ = pm.Normal('μ_μ', mu=0, sd=10)
    σ_μ = pm.HalfNormal('σ_μ', 10)

    # priors - априорные распределения
    μ = pm.Normal('μ', mu=μ_μ, sd=σ_μ, shape=groups)
    σ = pm.HalfNormal('σ', sd=10, shape=groups)

    y = pm.Normal('y', mu=μ[idx], sd=σ[idx], observed=diff)

    trace_cs_h = pm.sample(1000)
```

Мы будем сравнивать результаты с помощью функции `plot_forest` из библиотеки `ArviZ`. В эту функцию можно передавать более одной модели. Это удобно, если необходимо сравнивать значения параметров из различных моделей, как в нашем примере. Отметим, что в данном случае передается несколько аргументов в функцию `plot_forest` для получения требуемого графика, например `combined=True` для объединения результатов из всех цепочек. Выяснение смысла остальных аргументов предлагается читателю в качестве задания для самостоятельного выполнения:

```
_, axes = az.plot_forest([trace_cs_nh, trace_cs_h], model_names=['n_h', 'h'],
                        var_names='μ', combined=False, colors='cycle')
y_lims = axes[0].get_ylim()
axes[0].vlines(trace_cs_h['μ_μ'].mean(), *y_lims)
```

Что получается в итоге на рис. 2.22? Здесь мы видим график для 40 оцениваемых средних значений, по одному для каждой из 20 аминокислот с учетом

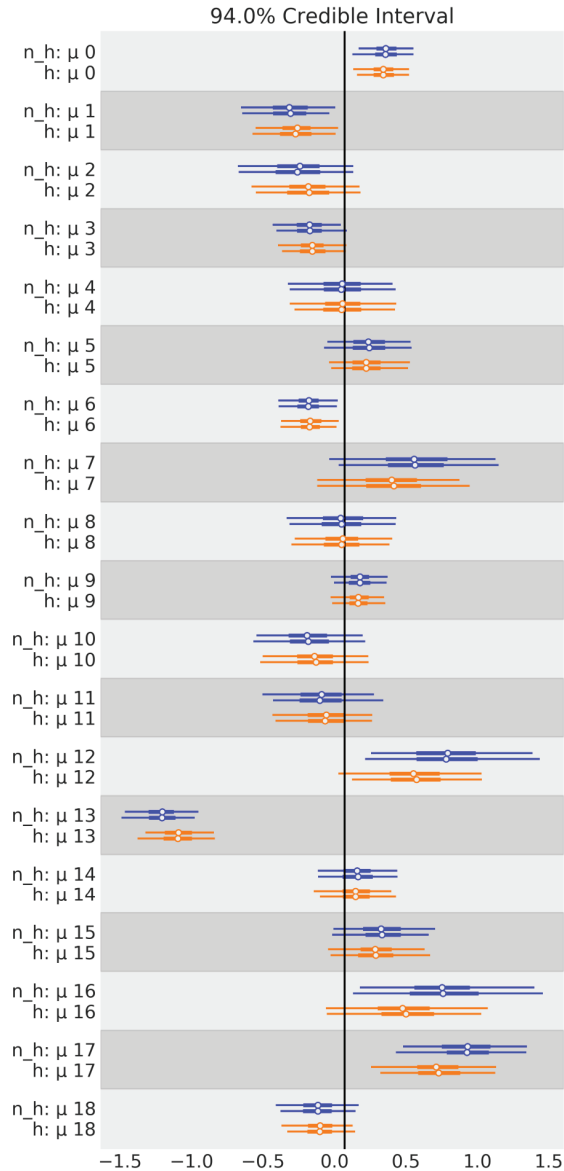


Рис. 2.22

использования двух моделей. Также имеются соответствующие 94%-ные доверительные интервалы и интерквартильный размах (центральная 50%-ная часть распределения). Вертикальная (черная) линия представляет глобальное среднее значение, соответствующее иерархической модели. Это значение,

близкое к нулю, как и прогнозировалось при вычислении теоретических значений, воспроизводящих результаты эксперимента.

Самым важным на этом графике является то, что оценки, полученные от иерархической модели, подтягиваются к частично объединенному среднему значению, или, что одно и то же, редуцируются с учетом необъединенных оценок. Также обратите внимание на то, что этот эффект более заметен для групп, расположенных дальше от среднего значения (например, 13), и что неопределенность равна или меньше, чем неопределенность в неиерархической модели. Оценки являются частично объединенными, потому что мы получили по одной оценке для каждой группы, но оценки для отдельных групп отделены друг от друга с помощью априорного гиперраспределения.

Таким образом, мы получаем промежуточное состояние между наличием одной группы, объединяющей все химические сдвиги, и наличием 20 отдельных групп, по одной для каждой аминокислоты. В этом и заключается главное достоинство иерархических моделей (рис. 2.22).

Перефразируя один из 19 основных принципов дзен Python¹, можно с уверенностью утверждать: «Иерархические модели – это воистину великая идея – так будем использовать их как можно больше». В следующих главах мы продолжим создавать иерархические модели и узнаем, как использовать их для получения еще более качественных моделей. Мы также рассмотрим, как иерархические модели связаны с вездесущей проблемой переподгонки/недоподгонки в статистике и машинном обучении, в главе 5 «Сравнение моделей». В главе 8 «Механизмы статистического вывода» будут рассматриваться некоторые технические проблемы, которые могут возникать при выборках из иерархических моделей, и методы выявления и устранения этих проблем.

РЕЗЮМЕ

Байесовская статистика с теоретической точки зрения проста, но полностью вероятностные модели часто приводят к формулированию аналитически необъяснимых выражений. В течение многих лет это представляло собой труднопреодолимое препятствие, мешающее широкому распространению байесовских методов. К счастью, помогли математика, статистика, физика и информационные технологии в форме численных методов, которые способны, по крайней мере теоретически, решить любую задачу статистического вывода. Возможность автоматизации процесса статистического вывода привела к разработке вероятностных языков программирования, позволяющих полностью разделить процедуры определения модели и процедуры статистического вывода.

PyMC3 – это библиотека языка Python для вероятностного программирования с чрезвычайно простым, понятным и легким для чтения синтаксисом,

¹ Zen of Python: «Namespaces are one honking great idea—let's do more of those!» – «Пространства имен – это воистину великая идея – так создадим их (как можно) больше». – *Прим. перев.*

весьма близким к синтаксису математической статистики, используемому для описания вероятностных моделей. Здесь библиотека PyMC3 была представлена при пересмотре модели подбрасывания монеты из главы 1, но уже без аналитического вывода апостериорного распределения. Модели PyMC3 определяются в управляемом контексте. Для добавления распределения вероятности в модель достаточно написать лишь одну строку кода. Распределения могут комбинироваться и использоваться как априорные распределения (ненаблюдаемые переменные) или правдоподобия (наблюдаемые переменные). Если данные передаются в распределение, то оно становится правдоподобием (likelihood). Кроме того, с помощью лишь одной строки кода можно создавать выборки. Библиотека PyMC3 позволяет получать выборки из апостериорного распределения. Если все сделано правильно, то такие выборки будут репрезентативными по отношению к корректному апостериорному распределению, следовательно, будут являться представлением логических последствий (выводов) из используемой модели и данных.

Апостериорное распределение, сгенерированное с помощью библиотеки PyMC3, можно исследовать с применением ArviZ, библиотеки языка Python, которая тесно связана с PyMC3 и может использоваться для решения множества разнообразных задач, в том числе для того, чтобы помочь в интерпретации и визуализации апостериорных распределений. Одним из способов применения апостериорного распределения как вспомогательного инструмента для принятия решений, управляемых статистическим выводом, является сравнение пространства практической равнозначности (ППР) с плотностью апостериорного распределения (ПАР). Также были кратко описаны функции потерь, представляющие формальный метод численной оценки компромиссов и издержек, связанных с принятием решений в условиях неопределенности. Узнали, что функции потерь и точечные оценки тесно связаны между собой.

До определенного момента мы ограничивались рассмотрением только простых моделей с одним параметром. Обобщение до произвольного количества параметров выполняется чрезвычайно просто с помощью библиотеки PyMC3: рассматривался пример использования нескольких параметров для гауссовой модели и *t*-модели Стьюдента. Гауссово распределение – это особый случай *t*-распределения Стьюдента, поэтому было показано, как использовать *t*-распределение для выполнения надежных статистических выводов при наличии промахов (выбросов). В следующей главе будет подробно рассматриваться применение этих моделей как частей моделей линейной регрессии.

Гауссова модель использовалась также для сравнения общей совокупности данных в задаче анализа по группам. Иногда такой подход применяется в контексте проверки предположений (гипотез), но мы выбрали другой путь и определили этот случай как задачу статистического вывода размера эффекта, то есть методику, которую в общем случае можно считать более продуктивной и эффективной. Также были продемонстрированы различные способы интерпретации и отчетов по результатам оценки размера эффекта.

В заключительной части главы рассматривалась одна из наиболее важных концепций, излагаемых в данной книге: иерархические модели. Мы можем создавать иерархические модели в каждом случае, когда возможно определение подгрупп исследуемых данных. В подобных случаях вместо интерпретации подгрупп как отдельных объектов или отказа от разделения на подгруппы и интерпретации их как единой группы можно создать модель, частично распределяющую информацию по группам. Главным эффектом такого частичного распределения является то, что оценки по каждой подгруппе будут подвержены влиянию оценок по остальным подгруппам. Такой эффект называют редуцированием (shrinkage), и в целом это весьма полезный прием, который помогает улучшить качество статистических выводов, делая их более консервативными (поскольку каждая подгруппа информирует другие подгруппы и подтягивает их оценки к своей) и более информативными. Мы получаем оценки на уровне подгрупп и на уровне общей группы. В следующих главах также будут рассматриваться другие примеры иерархических моделей. Каждый пример даст нам лучшее понимание этой концепции с несколько иной точки зрения.

УПРАЖНЕНИЯ

1. Используя библиотеку PyMC3, измените параметры априорного распределения в модели `our_first_model`, чтобы они соответствовали параметрам из примера в предыдущей главе. Сравните полученные результаты с результатами в предыдущей главе. Замените бета-распределение на равномерное распределение в интервале $[0, 1]$. Равнозначны ли полученные результаты результатам для $\text{Beta}(\alpha = 1, \beta = 1)$? Выполняется ли выборка быстрее, медленнее или за то же время? Как изменяется результат при использовании более широкого интервала, например $[-1, 2]$? Остается ли модель работоспособной? Какие сообщения об ошибках выводятся?
2. Внимательно изучите информацию о модели катастроф в угледобывающей промышленности, которая входит в состав документации библиотеки PyMC3: http://pymc-devs.github.io/pymc3/notebooks/getting_started.html#Case-study-2:-Coal-mining-disasters. Попробуйте самостоятельно реализовать и применить на практике эту модель.
3. Отредактируйте модель `model_g`, изменив априорное распределение для среднего значения гауссова распределения, центрированного в эмпирически определенной точке среднего значения, и поэкспериментируйте с несколькими разумными значениями для стандартного отклонения этого априорного распределения. Насколько надежными/осмысленными являются статистические выводы при этих изменениях? Что вы думаете об использовании гауссова распределения (которое является неограниченным распределением от $-\infty$ до $+\infty$) для моделирования ограниченных данных, как в рассматриваемом случае? Напомним, что ранее было сказано о невозможности получить значения меньше 0 и больше 100.

4. Используя данные из файла *chemical_shift.csv*, вычислите эмпирическое среднее значение и стандартное отклонение с учетом и без учета промахов. Сравните полученные результаты с байесовской оценкой, использующей гауссово распределение и t-распределение Стьюдента. Повторите вычисления с добавлением большего количества промахов.
5. Измените пример с оценкой получения чаевых, сделав его устойчивым к промахам. Попробуйте ввести один совместно используемый параметр ν для всех групп и по одному отдельному параметру ν для каждой группы. Выполните проверки прогнозируемого апостериорного распределения для оценки этих трех моделей.
6. Вычислите вероятность превосходства непосредственно из апостериорного распределения (без предварительного вычисления d-меры Коэна). Можно воспользоваться функцией `pm.sample_posterior_predictive()` для получения выборки из каждой группы. Действительно ли полученный результат отличается от результата вычислений с предположением о нормальности? Можете ли вы объяснить полученный результат?
7. Повторите упражнение, выполненное с моделью `model_h`. Но в этот раз без использования иерархической структуры, а с использованием упрощенного (плоского) априорного распределения, такого как $\text{Beta}(\alpha = 1, \beta = 1)$. Сравните результаты применения обеих моделей.
8. Создайте иерархическую версию примера оценки получения чаевых с частичным распределением по дням недели. Сравните полученные результаты с результатами, вычисленными без применения иерархической структуры.
9. Библиотека PyMC3 позволяет создавать из моделей направленные ациклические графы (directed acyclic graphs – DAG), очень похожие на диаграммы Крушке. Для получения таких графов можно воспользоваться функцией `pm.model_to_graphviz()`. Сгенерируйте направленные ациклические графы для каждой модели в этой главе.

Кроме примеров, приведенных в конце каждой главы, вы можете (и, вероятно, должны) обдумать подход к реальным задачам, в решении которых вы заинтересованы, и попытаться применить изученный материал при решении конкретной задачи. Возможно, потребуется определить задачу по-другому или возникнет необходимость в расширении или изменении изученных здесь моделей. Если вам покажется, что ваши реальные навыки и знания не достаточны для решения такой задачи, то отложите ее на время и вернитесь к ней после изучения очередной главы книги. И даже если после прочтения всей книги вы не нашли ответы на свои вопросы, то попробуйте подробно разобрать примеры применения библиотеки PyMC3 (<http://pymc-devs.github.io/pymc3/examples.html>) или задайте четко сформулированный вопрос на сайте <https://discourse.pymc.io/>.

Глава 3

.....

Моделирование с использованием линейной регрессии

«За три с лишним века науки изменилось все,
за исключением единственной вещи: стремле-
ния к простоте».

– Хорхе Вагенсберг Любински

Музыка – от классических произведений до песни «Sheena is a Punk Rocker» панк-группы Ramones, включая неизвестный хит любительской группы и либертанго Астора Пьяццолы, – создается из периодически повторяющихся звуковых шаблонов (фрагментов). Одни и те же гаммы, сочетания аккордов, риффы, мотивы и т. п. звучат снова и снова, создавая великолепный звуковой фон, способный вызывать широчайший спектр человеческих эмоций. Подобным образом мир статистики и машинного обучения формируется из многократно повторяющихся шаблонов (образцов), небольших элементов, появляющихся снова и снова. В этой главе будет рассматриваться один из самых распространенных и полезных шаблонов – линейная модель (или «лейтмотив», если угодно). Это чрезвычайно полезная модель сама по себе и как структурный компонент многих других моделей. Если вы изучали какой-либо курс по статистике (даже не обязательно по байесовской), то, возможно, уже слышали о простой и множественной линейной регрессии, логистической регрессии, дисперсионном анализе (ANOVA – ANalysis Of VAriance), ковариационном анализе (ANCOVA – ANalysis of COVAriance) и т. п. Все эти методы представляют собой вариации одного и того же основного «лейтмотива» – модели линейной регрессии. В этой главе рассматриваются следующие темы:

- простая (или парная) линейная регрессия;
- робастная линейная регрессия;
- иерархическая линейная регрессия;
- полиномиальная регрессия;

- множественная линейная регрессия;
- взаимодействия;
- переменная дисперсия.

ПРОСТАЯ ЛИНЕЙНАЯ РЕГРЕССИЯ

Многие задачи в науке, технике и бизнесе могут быть представлены в следующей форме. Имеется переменная x , необходимо смоделировать или спрогнозировать переменную y . Здесь важно, что эти переменные объединены в пары, то есть $\{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_n, y_n)\}$. В самом простом случае, известном под названием простой линейной регрессии, обе переменные x и y являются одномерными (линейными) непрерывными случайными переменными. Под непрерывностью подразумевается представление значений переменных с помощью действительных чисел (или чисел с плавающей точкой, если выражаться языком программистов). Поэтому при использовании библиотеки NumPy переменные x и y представляются в виде одномерных массивов. Поскольку это наиболее обобщенная модель, переменные обозначаются соответствующими именами. Переменную y называют зависимой, прогнозируемой или выходной переменной, а переменная x именуется независимой, прогнозирующей или входной переменной. Если X является матрицей (то есть имеется набор различных переменных), то такой вариант называют множественной линейной регрессией. В этой и следующей главах мы будем подробно изучать разнообразные модели линейной регрессии.

Ниже кратко описаны ситуации, наиболее подходящие для применения моделей линейной регрессии:

- модель взаимосвязи между такими факторами, как дождь, засоленность почвы и наличие/отсутствие удобрений для повышения урожайности сельскохозяйственных культур. Затем обсуждаются следующие вопросы: является ли эта взаимосвязь линейной? насколько сильна эта взаимосвязь? какие факторы оказывают самое мощное воздействие?
- поиск взаимосвязей между средним потреблением шоколада в стране и количеством нобелевских лауреатов в ней, затем выяснение, почему эта взаимосвязь может оказаться ложной;
- прогнозирование суммы счета за газоснабжение (для обогрева и приготовления пищи) вашего дома (квартиры) с использованием данных о величине солнечного излучения из отчетов о погоде, предоставляемых местным метеоцентром. Насколько точен такой прогноз?

Связь с машинным обучением

Перефразируя Кевина Мерфи (Kevin P. Murphy), машинное обучение – это всеобъемлющий термин для обозначения набора методов для автоматизированного изучения (выявления) типовых шаблонов в данных и последующего использования результатов такого обучения для прогнозирования будущих

данных или для принятия решений в условиях неопределенности. Машинное обучение и статистика – это действительно тесно связанные темы, их взаимосвязь становится очевидной, если взглянуть с вероятностной точки зрения, как это сделал Кевин Мерфи в своей великолепной книге «Machine learning: A probabilistic perspective». Эти области деятельности глубоко проникают друг в друга на концептуальном и математическом уровнях, но специфическая терминология может сделать эту взаимосвязь не столь явной. Поэтому в текущей главе вводятся термины из словаря машинного обучения при решении конкретных задач. Используя терминологию машинного обучения, мы говорим о задаче регрессии как о примере обучения с учителем (supervised learning). В рабочей среде машинного обучения задача регрессии возникает, когда необходимо обучение процедуре отображения X в Y , при этом Y является непрерывной переменной.

Специалист по машинному обучению обычно говорит о признаках (features) вместо переменных. Мы говорим, что процесс обучения управляется учителем, потому что знаем значения пар X - Y , то есть в определенном смысле нам известен правильный ответ. Все последующие вопросы относятся к тому, как обобщить эти наблюдения (или этот набор данных) для любого возможного будущего наблюдения, то есть для ситуации, когда известно значение X , но неизвестно значение Y .

Сущность моделей линейной регрессии

После обсуждения некоторых общетеоретических принципов линейной регрессии и установления терминологического соответствия между статистикой и машинным обучением можно приступить к изучению процесса создания линейных моделей.

Весьма полезным может оказаться тот факт, что вам уже известно следующее уравнение:

$$y_i = \alpha + x_i\beta. \quad (3.1)$$

Это уравнение сообщает о существовании линейной зависимости между переменной x и переменной y . Параметр β управляет углом наклона графика этой линейной зависимости, следовательно, интерпретируется как величина изменения переменной y по отношению к изменению величины переменной x . Другой параметр α известен под названием отсекаемого отрезка и сообщает значение y_i при $x_i = 0$. Графически отсекаемый отрезок представляет значение y_i в точке пересечения линейным графиком оси y .

Существует несколько методов подбора параметров для линейной модели. Одним из таких методов является метод наименьших квадратов, который возвращает значения α и β , дающие наименьшую среднюю квадратичную ошибку между наблюдаемым значением y и прогнозируемым значением y . Выраженная таким способом задача оценки α и β является задачей оптимизации, то есть задачей, в которой выполняются попытки найти минимум или максимум

некоторой функции. Альтернативой методу оптимизации является генерация полностью вероятностной модели. Вероятностный подход дает несколько преимуществ: можно получить наилучшие значения α и β (аналогичные полученным оптимизационными методами) вместе с оценкой неопределенности, присущей значениям этих параметров. Методы оптимизации требуют дополнительной работы по получению и подтверждению этой информации. Кроме того, вероятностный подход, особенно при использовании таких инструментальных средств, как библиотека PyMC3, предоставляет большую гибкость в адаптации моделей к решению конкретных задач, в чем мы сможем убедиться на протяжении этой главы.

При вероятностном подходе модель линейной регрессии можно выразить следующей формулой:

$$y \sim \mathcal{N}(\mu = \alpha + x\beta, \epsilon). \quad (3.2)$$

Таким образом, вектор данных y определяется гауссовым распределением со средним значением $\alpha + \beta x$ и стандартным отклонением ϵ .



Модель линейной регрессии – это расширение гауссовой модели, где среднее значение не оценивается напрямую, а вычисляется как линейная функция прогнозирующей переменной и некоторых дополнительных параметров.

Поскольку нам неизвестны значения α , β и ϵ , необходимо установить для них априорные распределения. В общем случае разумным выбором является следующий:

$$\begin{aligned} \alpha &\sim \mathcal{N}(\mu_\alpha, \sigma_\alpha); \\ \beta &\sim \mathcal{N}(\mu_\beta, \sigma_\beta); \\ \epsilon &\sim |\mathcal{N}(0, \sigma_\epsilon)|. \end{aligned} \quad (3.3)$$

Для априорного распределения параметра α можно использовать самое простое плоское гауссово распределение с установкой для σ_α относительно большого значения, чтобы масштабировать данные. В общем случае нам неизвестна возможная точка пересечения (отрезок отсечения), поэтому в разных задачах это значение может значительно изменяться в зависимости от конкретной области знаний. Для многих задач, над которыми мне приходилось работать, α обычно центрируется около нуля, при этом σ_α не больше 10, но это только мой личный опыт (не слишком убедительный) на основе небольшого подмножества задач. Такой опыт не всегда просто переносится на другие задачи. Что касается угла наклона, то, возможно, проще понять его основной смысл по сравнению с отсекаемым отрезком. Во многих задачах предварительно из-

вестен, по крайней мере, знак угла наклона, например можно ожидать, что переменная, характеризующая вес (массу), в среднем увеличивается вместе с ростом значения переменной, определяющей рост. Для параметра эpsilon (ϵ) можно определить значение σ_ϵ , большее по сравнению со значением переменной y , например в десять раз большее значение для стандартного отклонения. Эти весьма неопределенные предварительные условия обеспечивают чрезвычайно малое воздействие априорного распределения на апостериорное распределение, которое с легкостью компенсируется данными.



Точечная оценка, получаемая с использованием метода наименьших квадратов, согласовывается с оценкой апостериорного максимума (maximum a posteriori – MAP) (модой апостериорного распределения), полученной из байесовской простой линейной регрессии с плоскими априорными распределениями.

Альтернативами полугауссову распределению для ϵ является равномерное распределение и полураспределение Коши. В общем случае полураспределение Коши успешно работает как эффективное априорное распределение регуляризации (подробнее см. главу 6 «Сравнение моделей»), но равномерные распределения не всегда являются удачным выбором, за исключением тех случаев, когда известно, что параметр действительно строго ограничен. При необходимости применения весьма жестко ограниченных априорных распределений, сконцентрированных около некоторого конкретного значения для стандартного отклонения, можно воспользоваться гамма-распределением. Определенная по умолчанию параметризация гамма-распределения во многих программных пакетах может сначала показаться немного сложной, но, к счастью, библиотека PyMC3 позволяет определять ее параметры с использованием формы (shape) и нормы (rate) (вероятно, это наиболее обобщенный способ параметризации) или среднего значения и стандартного отклонения (вероятно, это более понятный способ параметризации, по крайней мере для начинающих).

Чтобы наглядно представить, как выглядит гамма-распределение и другие распределения, можно обратиться к соответствующему разделу документации по библиотеке PyMC3: <https://docs.pymc.io/api/distributions/continuous.html>.

Для наглядного представления и лучшего понимания модели линейной регрессии можно воспользоваться все той же диаграммой Крушке, которая показана на рис. 3.1. Следует напомнить, что в предыдущей главе на диаграммах Крушке символ $=$ (знак равенства) использовался для определения детерминированных переменных (таких как μ), а символ $\sim$ (тильда) – для определения случайных переменных, таких как α , β и ϵ .

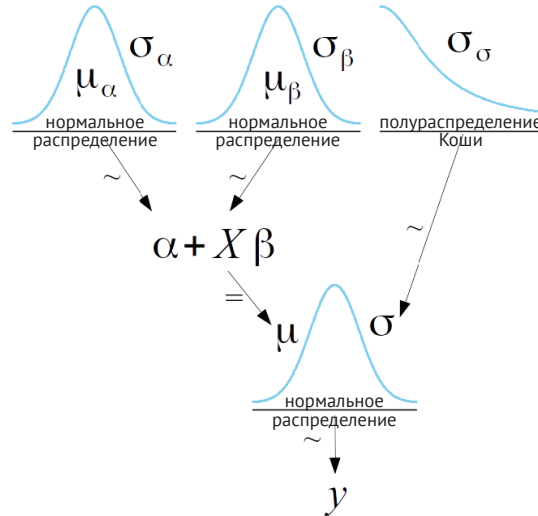


Рис. 3.1

После завершения определения модели необходимы данные для ее наполнения. И снова воспользуемся сгенерированным набором данных для лучшего понимания модели. Одним из преимуществ сгенерированного набора данных является то, что нам известны правильные значения параметров и доступна проверка возможности их восстановления с помощью выбранных моделей:

```
np.random.seed(1)
N = 100
alpha_real = 2.5
beta_real = 0.9
eps_real = np.random.normal(0, 0.5, size=N)

x = np.random.normal(10, 1, N)
y_real = alpha_real + beta_real * x
y = y_real + eps_real

_, ax = plt.subplots(1, 2, figsize=(8, 4))
ax[0].plot(x, y, 'C0.')
ax[0].set_xlabel('x')
ax[0].set_ylabel('y', rotation=0)
ax[0].plot(x, y_real, 'k')
ax.plot_kde(y, ax=ax[1])
ax[1].set_xlabel('y')
plt.tight_layout()
```

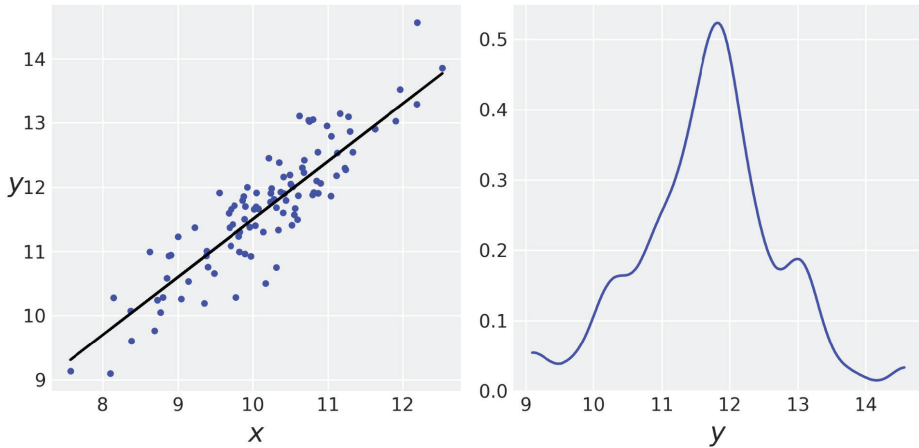


Рис. 3.2

Теперь воспользуемся библиотекой PyMC3 для создания модели и передачи в нее данных, но здесь мы видим кое-что новое. В используемой модели μ определена как детерминированная переменная, отображая то, что ранее было записано в математической нотации и показано на диаграмме Крушке. Если детерминированная переменная определяется по правилам библиотеки PyMC3, то мы сообщаем библиотеке о необходимости вычисления и сохранения этой переменной в процессе выполнения:

```
with pm.Model() as model_g:
     $\alpha$  = pm.Normal('a', mu=0, sd=10)
     $\beta$  = pm.Normal('b', mu=0, sd=1)
     $\epsilon$  = pm.HalfCauchy('e', 5)

     $\mu$  = pm.Deterministic('mu',  $\alpha + \beta * x$ )
    y_pred = pm.Normal('y_pred', mu= $\mu$ , sd= $\epsilon$ , observed=y)

    trace_g = pm.sample(2000, tune=1000)
```

В другом варианте – вместо включения детерминированной переменной в модель – можно отказаться от нее. В этом случае такая переменная вычисляется, но не сохраняется в процессе выполнения. Например, можно было бы написать следующую строку кода:

```
y_pred = pm.Normal('y_pred', mu=  $\alpha + \beta * x$ , sd= $\epsilon$ , observed=y)
```

Для исследования результатов нашего статистического вывода необходимо сгенерировать график трассировки, но без детерминированной переменной μ . Это можно сделать, передавая имена переменных, которые нужно включить в график, как список в аргументе `var_names`. Многие функции библиотеки ArviZ принимают аргумент `var_names`:

```
az.plot_trace(trace_g, var_names=['a', 'b', 'e'])
```

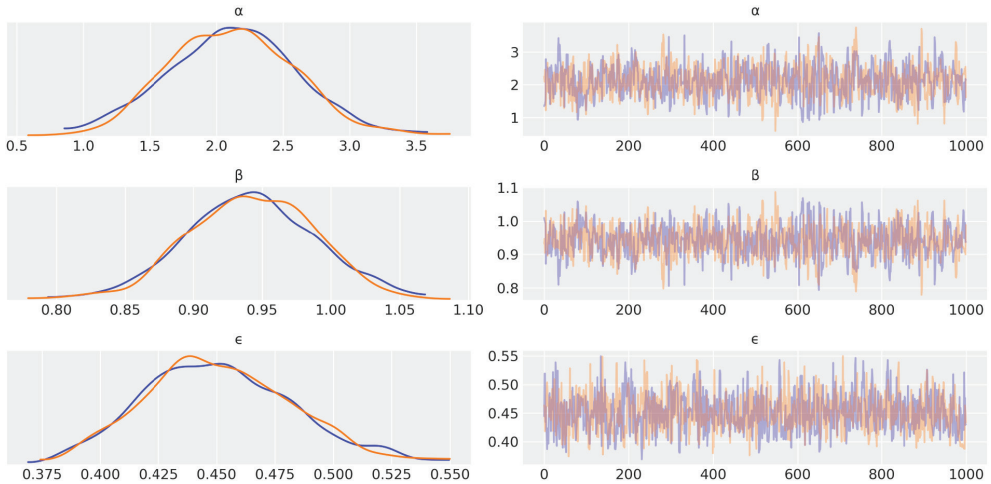


Рис. 3.3

Попробуйте поэкспериментировать с другими графиками ArviZ для более подробного исследования апостериорного распределения. В следующем разделе рассматривается одно из свойств линейной модели и его возможное воздействие на процесс выборки и на интерпретацию модели. Затем будут описаны способы интерпретации и наглядного представления апостериорного распределения.

Линейные модели и сильная автокорреляция

Использование линейных моделей приводит к апостериорному распределению, где α и β тесно взаимосвязаны, то есть между ними существует сильная корреляция. Рассмотрим следующий исходный код и рис. 3.4 для этого примера:

```
az.plot_pair(trace_g, var_names=[' $\alpha$ ', ' $\beta$ '], plot_kwargs={'alpha': 0.1})
```

Корреляция, наблюдаемая на рис. 3.4, является прямым следствием наших предположений. Не имеет значения, какая из линий полностью соответствует исследуемым данным, все они должны проходить через одну точку, то есть через среднее значение переменной x и среднее значение переменной y . Следовательно, процесс подгонки линии в некоторой степени равнозначен вращению прямой линии, закрепленной в центре данных, подобно колесу фортуны. Увеличение угла наклона означает уменьшение области пересечения, и наоборот. Оба параметра взаимосвязаны (коррелируют) по определению используемой модели. Таким образом, форма апостериорного распределения (исключая ϵ) представляет собой весьма диагональное пространство (very diagonal space). Это может стать проблемой для сэмплеров, таких как алгоритм Метрополиса–Гастингса, или в меньшей степени для NUTS (территориальных статисти-

ческих регионов). Более подробно это объясняется в главе 8 «Механизмы статистического вывода».

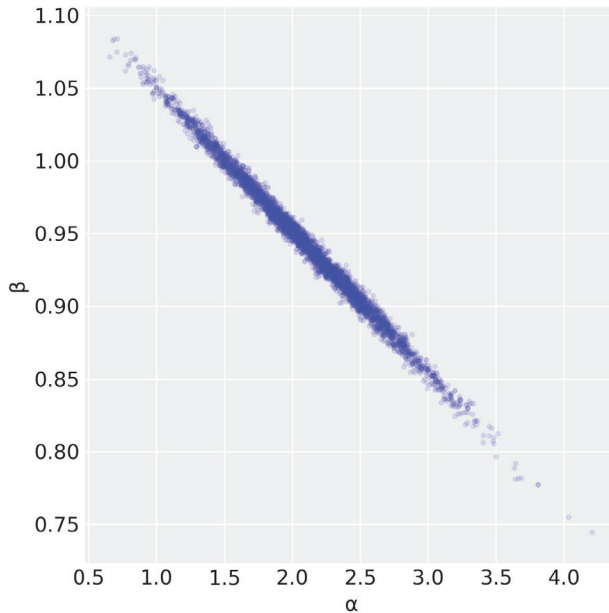


Рис. 3.4

Прежде чем продолжить, необходимо завершить выяснение одного обстоятельства. Тот факт, что для линии установлено ограничивающее условие прохождения через точку, соответствующую среднему значению данных, является истинным только для метода наименьших квадратов (и соответствующих предварительных предположений). При использовании байесовских методов это ограничение ослабляется. В следующих примерах мы увидим, что в общем случае линии располагаются в непосредственной близости от средних значений x и y , а не в точности проходят через точку, соответствующую средним значениям x и y . Тем не менее общее представление о том, что автокорреляция связана с линией, вращающейся относительно более или менее определенной точки, остается истинным, следовательно, необходимо лишь понять степень корреляции между параметрами α и β .

Изменение данных перед обработкой

Одним из простых способов устранения корреляции между α и β является центрирование переменных x . Из каждой точки данных x_i вычитается среднее значение переменной x ($\bar{x}$):

$$x' = x - \bar{x}. \quad (3.4)$$

В результате x' центрируется относительно 0, таким образом центр вращения (осевая точка) при изменении угла наклона неизменно является точкой пересечения, следовательно, вероятное пространство параметров теперь в большей степени приобретает форму круга и становится менее коррелированным. Рекомендуется выполнить упражнение 6 (см. раздел «Упражнения» к текущей главе), чтобы наблюдать различия между центрированными и нецентрированными данными.

Центрирование данных – это не только вычислительный прием, центрирование также может помочь при интерпретации результатов. Отрезок отсеечения – это значение y_i при $x_i = 0$. Для многих задач такая интерпретация не имеет реального смысла. Например, для числовых переменных, таких как рост и масса (тела), значение ноль бессмысленно, следовательно, отрезок отсеечения не соответствует значению, которое помогло бы осмыслению данных. Вместо этого при центрировании переменных отрезком отсеечения всегда является значение y_i при среднем значении x .

В некоторых задачах может оказаться полезной точная оценка отсекаемого отрезка, так как невозможно экспериментально измерить значение при $x_i = 0$, поэтому оцениваемый отсекаемый отрезок может предоставить ценную информацию. Но при этом надо действовать внимательно и осторожно, поскольку экстраполяции могут оказаться ошибочными.

Возможно, потребуется отчет по оцениваемым параметрам как с применением центрирования данных, так и без центрирования данных в зависимости от конкретной задачи и целевой аудитории. Если необходим отчет о параметрах в том виде, в каком они определены в исходном масштабе, то можно выполнить следующее преобразование для возврата к исходному масштабу:

$$\alpha = \alpha' - \beta' \bar{x}. \quad (3.5)$$

Это корректирующее преобразование является результатом следующих алгебраических обоснований:

$$\begin{aligned} y &= \alpha' + \beta' x' + \epsilon; \\ y &= \alpha' + \beta' (x - \bar{x}) + \epsilon; \\ y &= \alpha' - \beta' \bar{x} + \beta' x + \epsilon. \end{aligned} \quad (3.6)$$

Из этого следует, что выражение 3.5 истинно, а также равенство:

$$\beta = \beta'. \quad (3.7)$$

Можно даже не ограничиваться только центрированием x и преобразовать данные, нормализуя их перед использованием в модели. Нормализация является часто применяемым практическим методом для моделей линейной регрессии в статистике и машинном обучении, так как многие алгоритмы работают лучше, когда данные нормализованы. Такое преобразование выполняется с помощью центрирования данных и последующего деления полученного результата на стандартное отклонение. Математически это описывается следующими формулами:

$$\begin{aligned}x' &= \frac{x - \bar{x}}{x_{sd}}; \\ y' &= \frac{y - \bar{y}}{y_{sd}}.\end{aligned}\tag{3.8}$$

Одним из преимуществ нормализации данных является то, что всегда можно воспользоваться теми же малоинформативными априорными распределениями без необходимости подбора масштаба данных. Для нормализованных данных отсекаемый отрезок всегда будет близким к нулю, а угол (коэффициент) наклона будет ограничен интервалом $[-1, 1]$. Кроме того, нормализация данных позволяет мыслить в терминах Z-оценок (стандартизированных оценок), то есть в единицах измерения стандартного отклонения. Если утверждается, что значение параметра равно -1.3 в единицах измерения Z-оценки, то сразу понятно, что для рассматриваемого значения определен размер стандартного отклонения на 1.3 меньше среднего значения вне зависимости от действительного среднего значения или действительного значения стандартного отклонения исследуемых данных. Изменение на одну единицу измерения Z-оценки – это изменение на одну величину стандартного отклонения, каким бы ни был масштаб исходных данных. Это может оказаться весьма полезным при работе с несколькими переменными. Обработка всех переменных в едином масштабе может упростить интерпретацию данных.

Интерпретация и визуальное представление апостериорного распределения

Как мы уже видели ранее, существует возможность исследования апостериорного распределения с использованием функций библиотеки ArviZ, таких как `plot_trace` и `summary`, но также можно написать и собственные функции. Для линейной регрессии может оказаться полезным построение средней линии, соответствующей данным, вместе с усредненными средними значениями α и β . Для отображения неопределенности апостериорного распределения можно использовать полупрозрачные линии, отображающие выборки из апостериорного распределения:

```
plt.plot(x, y, 'C0.')

alpha_m = trace_g['a'].mean()
beta_m = trace_g['b'].mean()

draws = range(0, len(trace_g['a']), 10)
plt.plot(x, trace_g['a'][draws] + trace_g['b'][draws]
         * x[:, np.newaxis], c='gray', alpha=0.5)

plt.plot(x, alpha_m + beta_m * x, c='k',
         label=f'y = {alpha_m:.2f} + {beta_m:.2f} * x')

plt.xlabel('x')
plt.ylabel('y', rotation=0)
plt.legend()
```

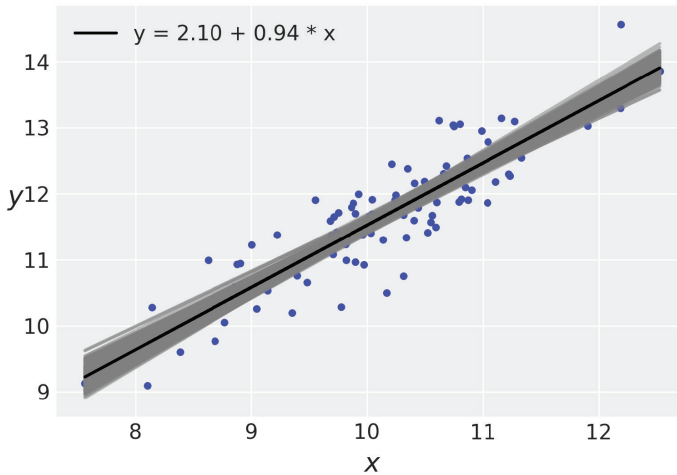


Рис. 3.5

Отметим, что неопределенность меньше в средней части графика, хотя и не сходится к одной точке, то есть апостериорное распределение совмещается с линиями, не проходящими в точности через среднее значение данных, как уже было сказано выше.

Использование полупрозрачных линий чрезвычайно удобно, но может потребоваться добавление более впечатляющих эффектов в график, например использование полупрозрачной полосы для отображения интервала плотности апостериорного распределения (ПАР) для μ . В действительности это было главной причиной определения детерминированной переменной μ в рассматриваемой модели. Наличие этой переменной упрощает следующий код:

```
plt.plot(x, alpha_m + beta_m * x, c='k',
         label=f'y = {alpha_m:.2f} + {beta_m:.2f} * x')
sig = az.plot_hpd(x, trace_g['μ'], credible_interval=0.98, color='k')
plt.xlabel('x')
plt.ylabel('y', rotation=0)
plt.legend()
```

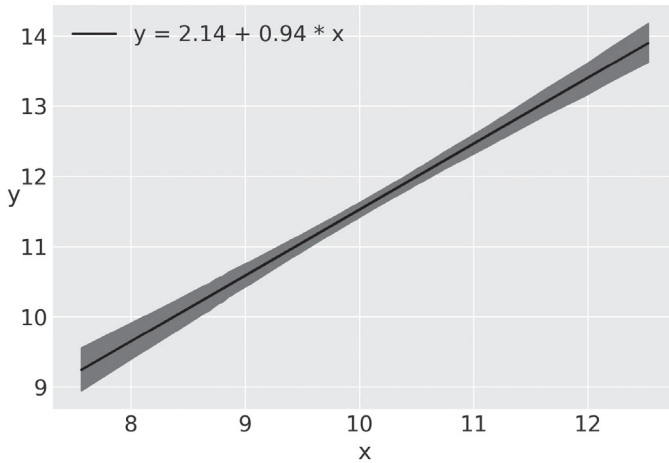


Рис. 3.6

Еще один вариант – построение графика плотности апостериорного распределения (ПАР) (например, 94 % и 50 %) прогнозируемых данных $\hat{y}$, то есть интервала, где ожидается наблюдение 94 % и 50 % будущих данных в соответствии с используемой моделью. На рис. 3.7 для отображения будет применяться более темный оттенок серого цвета для ПАР 50 % и более светлый оттенок серого цвета для ПАР 95 %.

Прогнозируемые выборки апостериорного распределения легко получить с помощью библиотеки PyMC3, если воспользоваться функцией `sample_posterior_predictive()`:

```
ppc = pm.sample_posterior_predictive(trace_g, samples=2000, model=model_g)
```

Теперь можно построить график по полученным результатам:

```
plt.plot(x, y, 'b.')
plt.plot(x, alpha_m + beta_m * x, c='k',
         label=f'y = {alpha_m:.2f} + {beta_m:.2f} * x')
az.plot_hpd(x, ppc['y_pred'], credible_interval=0.5, color='gray')
az.plot_hpd(x, ppc['y_pred'], color='gray')

plt.xlabel('x')
plt.ylabel('y', rotation=0)
```

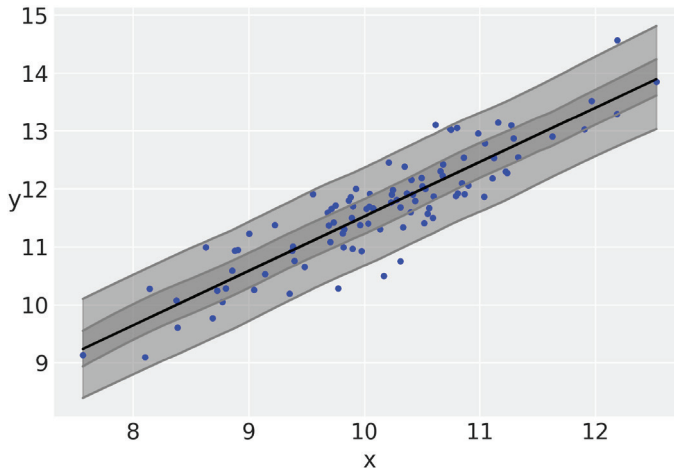



Рис. 3.7

Функция `az.plot_hpd` – это вспомогательное средство, которое можно применять для построения интервала ПАР для линейных регрессий. По умолчанию эта функция сглаживает интервал. Чтобы лучше понять смысл сглаживания, попробуйте передать в эту функцию аргумент `smooth=False`.

Коэффициент корреляции Пирсона

Иногда необходимо измерить степень (линейной) зависимости между двумя переменными. Наиболее часто используемой мерой линейной корреляции между двумя переменными является коэффициент корреляции Пирсона (или линейный коэффициент корреляции), часто обозначаемый строчной буквой r . При $r = +1$ мы имеем абсолютно положительную линейную корреляцию, то есть при увеличении значения одной переменной прогнозируется увеличение значения второй переменной. При $r = -1$ наблюдается абсолютно отрицательная линейная корреляция, когда при увеличении значения одной переменной прогнозируется уменьшение значения второй переменной. Если $r = 0$, то получаем нелинейную корреляцию. На практике мы будем использовать промежуточные значения. Важно всегда помнить о двух аспектах: коэффициент корреляции Пирсона ничего не сообщает о нелинейных корреляциях. Не следует путать r с углом наклона линии регрессии. Полезно всегда держать под рукой следующую иллюстрацию из Википедии: https://en.wikipedia.org/wiki/Correlation_and_dependence#/media/File:Correlation_examples2.svg.

Путаницу во взаимосвязи между r и углом наклона линии регрессии частично можно объяснить следующим отношением:

$$r = \beta \frac{\sigma_x}{\sigma_y}. \quad (3.9)$$

Из этой формулы следует, что угол наклона (β) и коэффициент корреляции Пирсона могут иметь одинаковые значения, но только в том случае, когда стандартные отклонения x и y равны. Отметим, что такое равенство возникает, например, после нормализации данных. Дополнительные уточнения:

- коэффициент корреляции Пирсона (r) – это мера степени корреляции (взаимосвязи) между двумя переменными, поэтому она всегда ограничена интервалом $[-1, 1]$. Этот интервал не зависит от масштаба данных;
- угол наклона линейной регрессии (β) показывает, на какую величину изменится y при изменении x на единицу измерения, и может принимать любое действительное значение.

Коэффициент корреляции Пирсона связан с количественной характеристикой, называемой коэффициентом детерминации, а в модели линейной регрессии это просто квадрат коэффициента корреляции Пирсона, то есть r^2 (иногда записывается как R^2) – произносится как «эр в квадрате» и может быть определен как дисперсия прогнозируемых значений, разделенная на дисперсию данных. Таким образом, эту характеристику можно интерпретировать как пропорциональное значение дисперсии в зависимой переменной, которая прогнозируется по независимой переменной. Для байесовской линейной регрессии дисперсия прогнозируемых значений может быть больше дисперсии данных. Поэтому R^2 может быть больше 1, и в качестве надежного решения по определению R^2 предлагается следующая формула:

$$R^2 = \frac{\mathbf{V}_{n=1}^N \mathbf{E}[\hat{y}^8]}{\mathbf{V}_{n=1}^N \mathbf{E}[\hat{y}^8] + \mathbf{V}_{n=1}^S (\hat{y}^8 - y)}. \quad (3.10)$$

В этом выражении $\mathbf{E}[\hat{y}^8]$ – это ожидаемое (или среднее) значение $\hat{y}$ по всем S выборкам апостериорного распределения. Это дисперсия прогнозируемых значений, разделенная на сумму дисперсий прогнозируемых значений и ошибок (или остатков, невязок). Преимущество такого определения состоит в том, что обеспечивается ограничение R^2 интервалом $[0, 1]$.

Проще всего вычислить R^2 с помощью функции `r2_score()` из библиотеки `ArviZ`. Для этого потребуются наблюдаемые значения y и предсказываемые значения $\hat{y}$. Как вы помните, значения $\hat{y}$ можно получить из функции `sample_posterior_predictive()`:

```
az.r2_score(y, ppc['y_pred'])
```

По умолчанию эта функция возвращает R^2 (в рассматриваемом примере 0.8) и стандартное отклонение (0.03).

Коэффициент корреляции Пирсона, вычисляемый по многомерному гауссову распределению

Другой способ вычисления коэффициента корреляции Пирсона – оценка ковариационной матрицы многомерного гауссова распределения. Многомерное гауссово распределение – это обобщение гауссова распределения для измере-

ния с размерностью, большей единицы. Сосредоточимся на варианте с двумя измерениями, потому что именно такой пример мы будем рассматривать прямо сейчас. Обобщение для измерений с более высокой размерностью становится не таким уж сложным после освоения двумерного варианта. Для полного описания двумерного гауссова распределения необходимы два средних значения (или вектор с двумя элементами), по одному для каждого предельного гауссова распределения. Кроме того, можно предположить, что потребуются два стандартных отклонения, но это не совсем так: нужна ковариационная матрица 2×2 , имеющая приблизительно такой вид:

$$\Sigma = \begin{bmatrix} \sigma_{x_1}^2 & \rho \sigma_{x_1} \sigma_{x_2} \\ \rho \sigma_{x_1} \sigma_{x_2} & \sigma_{x_2}^2 \end{bmatrix}. \quad (3.11)$$

Здесь Σ (прописная греческая сигма), обычно используемая для представления ковариационной матрицы. На главной диагонали расположены дисперсии каждой переменной, выраженные как квадраты соответствующих стандартных отклонений σ_{x_1} и σ_{x_2} . Остальными элементами матрицы являются ковариации (дисперсия между переменными), которые выражаются через соответствующие стандартные отклонения с коэффициентом корреляции Пирсона ρ (учитывающим корреляцию между переменными). Отметим, что здесь используется один коэффициент ρ , потому что рассматриваются только два измерения (то есть переменные). Для трех переменных пришлось бы использовать три коэффициента Пирсона.

В следующем коде генерируются контурные графики для двумерных гауссовых распределений с обоими фиксированными средними значениями в точке $(0, 0)$. Для одного из стандартных отклонений задано постоянное значение $\sigma_{x_1} = 1$, тогда как второе может принимать значения 1 или 2 при различных значениях коэффициента корреляции Пирсона:

```
sigma_x1 = 1
sigmas_x2 = [1, 2]
rhos = [-0.90, -0.5, 0, 0.5, 0.90]

k, l = np.mgrid[-5:5:.1, -5:5:.1]
pos = np.empty(k.shape + (2,))
pos[:, :, 0] = k
pos[:, :, 1] = l

f, ax = plt.subplots(len(sigmas_x2), len(rhos),
                    sharex=True, sharey=True, figsize=(12, 6),
                    constrained_layout=True)

for i in range(2):
    for j in range(5):
        sigma_x2 = sigmas_x2[i]
        rho = rhos[j]
        cov = [[sigma_x1**2, sigma_x1*sigma_x2*rho],
               [sigma_x1*sigma_x2*rho, sigma_x2**2]]
        rv = stats.multivariate_normal([0, 0], cov)
```

```

ax[i, j].contour(k, l, rv.pdf(pos))
ax[i, j].set_xlim(-8, 8)
ax[i, j].set_ylim(-8, 8)
ax[i, j].set_yticks([-5, 0, 5])
ax[i, j].plot(0, 0, label=f'$\\sigma_{{x2}}$ = {sigma_x2:3.2f}\\n$\\rho$ = {rho:3.2f}', alpha=0)
ax[i, j].legend()
f.text(0.5, -0.05, 'x_1', ha='center', fontsize=18)
f.text(-0.05, 0.5, 'x_2', va='center', fontsize=18, rotation=0)

```

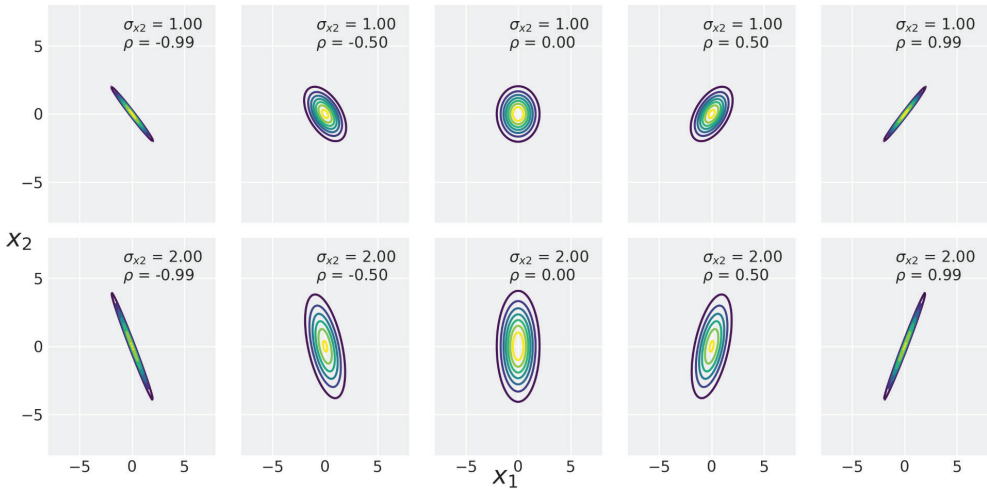


Рис. 3.8

Теперь нам известно многомерное гауссово распределение, поэтому можно оценить коэффициент корреляции Пирсона. Поскольку значения элементов ковариационной матрицы неизвестны, мы не устанавливаем для них априорные распределения. Один из вариантов решения – использование распределения Уишарта (Wishart distribution), которое является сопряженным априорным распределением инвертированной ковариационной матрицы по многомерной нормали. Распределение Уишарта можно считать обобщением для измерений более высоких порядков гамма-распределения, которые мы наблюдали ранее, или также обобщением распределения χ^2 . Второй вариант решения – использование априорного распределения LKJ (подробнее см. <https://docs.pymc.io/notebooks/LKJ.html>). Это априорное распределение для корреляционной матрицы (обратите внимание: не для ковариационной матрицы), которое может оказаться удобным с учетом того, что в общем случае более привычно рассуждать в терминах корреляции. Но мы подробно рассмотрим третий вариант решения: назначение априорных распределений непосредственно для σ_{x_1} , σ_{x_2} и ρ с последующим использованием этих значений для формирования ковариационной матрицы вручную:

```
data = np.stack((x, y)).T
with pm.Model() as pearson_model:

    μ = pm.Normal('μ', mu=data.mean(0), sd=10, shape=2)

    σ_1 = pm.HalfNormal('σ_1', 10)
    σ_2 = pm.HalfNormal('σ_2', 10)
    ρ = pm.Uniform('ρ', -1., 1.)
    r2 = pm.Deterministic('r2', ρ**2)

    cov = pm.math.stack([[σ_1**2, σ_1*σ_2*ρ],
                          [σ_1*σ_2*ρ, σ_2**2]])

    y_pred = pm.MvNormal('y_pred', mu=μ, cov=cov, observed=data)

    trace_p = pm.sample(1000)
```

Пока оставим без внимания все переменные, кроме $r2$:

```
az.plot_trace(trace_p, var_names=['r2'])
```

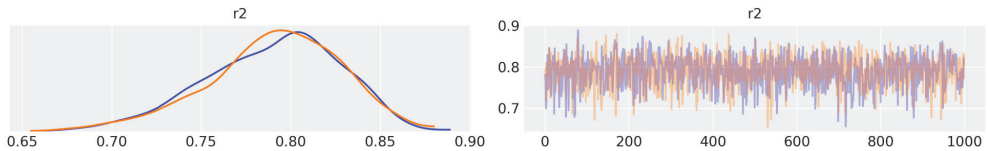


Рис. 3.9

Можно видеть, что распределение значений $r2$ концентрируется около значения, полученного в предыдущем примере с использованием функции `r2_score` из библиотеки `ArviZ`. Возможно, более простым способом сравнения значений, полученных из многомерного гауссова распределения с предыдущим результатом, является использование функции `summary`. Как видите, мы получили достаточно точное соответствие:

```
az.summary(trace_p, var_names=['r2'])
```

Таблица 3.1

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
r2	0.79	0.04	0.0	0.72	0.86	839.0	1.0

РОБАСТНАЯ ЛИНЕЙНАЯ РЕГРЕССИЯ

Предположение о том, что данные подчиняются гауссову распределению, абсолютно обосновано во многих ситуациях. Говоря о предполагаемой «гауссовости», мы не обязательно подразумеваем, что распределение данных действительно является гауссовым, а всего лишь допускаем, что это разумная

аппроксимация поставленной задачи. То же самое можно сказать и о других распределениях. Как мы видели в предыдущей главе, иногда предположение о гауссовом распределении является ошибочным, например при наличии промахов. Мы узнали, что применение t-распределения Стьюдента представляет собой эффективный способ обработки промахов и обеспечивает более надежный статистический вывод. Этот же принцип можно применять и для линейной регрессии.

Для подтверждения практическим примером надежности, которую привносит t-распределение Стьюдента в линейную регрессию, воспользуемся очень простым и удобным набором данных: третьей группой данных из квартета Энскомба (https://ru.wikipedia.org/wiki/Квартет_Энскомба). Эту группу можно сформировать с помощью библиотеки pandas. Данные центрируются, но только для того, чтобы упростить работу сэмплера, поскольку даже мощный механизм выборки, такой как NUTS, время от времени нуждается в небольшой помощи:

```
ans = pd.read_csv('../data/anscombe.csv')
x_3 = ans[ans.group == 'III']['x'].values
y_3 = ans[ans.group == 'III']['y'].values
x_3 = x_3 - x_3.mean()
```

Теперь посмотрим, как выглядит этот маленький набор данных:

```
_, ax = plt.subplots(1, 2, figsize=(10, 5))
beta_c, alpha_c = stats.linregress(x_3, y_3)[:2]
ax[0].plot(x_3, (alpha_c + beta_c * x_3), 'k',
            label=f'y = {alpha_c:.2f} + {beta_c:.2f} * x')
ax[0].plot(x_3, y_3, 'C0o')
ax[0].set_xlabel('x')
ax[0].set_ylabel('y', rotation=0)
ax[0].legend(loc=0)
az.plot_kde(y_3, ax=ax[1], rug=True)
ax[1].set_xlabel('y')
ax[1].set_yticks([])
plt.tight_layout()
```

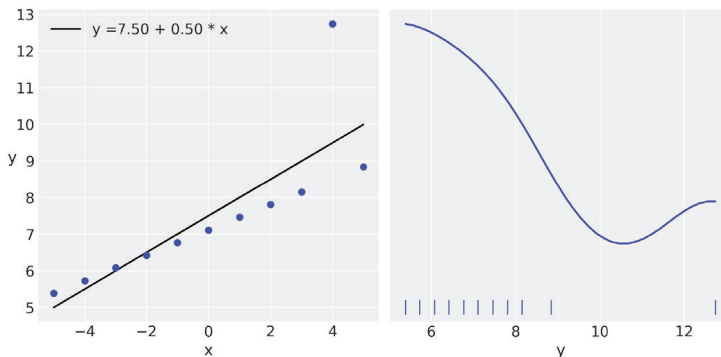


Рис. 3.10

Далее перепишем предыдущую модель (`model_g`), но на этот раз воспользуемся t -распределением Стьюдента вместо гауссова распределения. Это изменение также приводит к необходимости определения значения v , параметра нормальности. Чтобы вспомнить роль этого параметра, можно вернуться к содержанию главы 2.

В следующей модели используется экспоненциальный сдвиг, чтобы избежать значений v , близких к нулю. Без экспоненциального сдвига наблюдается слишком большой вес значений, близких к нулю. На основе собственного опыта могу отметить, что для данных без смягчения промахов близкие к нулю значения нормальны, но для данных с экстремальными промахами (или для данных с несколькими крупными массивами точек), такими как третий набор квартета Энскомба, лучше избегать слишком малых значений. Впрочем, эту рекомендацию, как и все прочие рекомендации по выбору априорных распределений, следует принимать с некоторой долей критики. Установки по умолчанию создают хорошие исходные условия, но нет необходимости всегда придерживаться значений по умолчанию. Другими часто применяемыми априорными распределениями для v являются `gamma(2, 0.1)` или `gamma(mu=20, sd=15)`:

```
with pm.Model() as model_t:
     $\alpha$  = pm.Normal('α', mu=y_3.mean(), sd=1)
     $\beta$  = pm.Normal('β', mu=0, sd=1)
     $\epsilon$  = pm.HalfNormal('ε', 5)
     $v_{-}$  = pm.Exponential('v_', 1/29)
     $v$  = pm.Deterministic('v',  $v_{-}$  + 1)

    y_pred = pm.StudentT('y_pred', mu= $\alpha$  +  $\beta$  * x_3,
                        sd= $\epsilon$ , nu=v, observed=y_3)

    trace_t = pm.sample(2000)
```

На рис. 3.11 можно видеть линию робастной подгонки `robust` и линию неробастной подгонки `non-robust` в соответствии с функцией `linregress` библиотеки SciPy (эта функция выполняет регрессию по методу наименьших квадратов). В качестве дополнительного упражнения можно попытаться добавить наилучшую линию, полученную с использованием модели `model_g`:

```
beta_c, alpha_c = stats.linregress(x_3, y_3)[:2]

plt.plot(x_3, (alpha_c + beta_c * x_3), 'k', label='non-robust', alpha=0.5)
plt.plot(x_3, y_3, 'C0o')
alpha_m = trace_t['α'].mean()
beta_m = trace_t['β'].mean()
plt.plot(x_3, alpha_m + beta_m * x_3, c='k', label='robust')

plt.xlabel('x')
plt.ylabel('y', rotation=0)
plt.legend(loc=2)
plt.tight_layout()
```

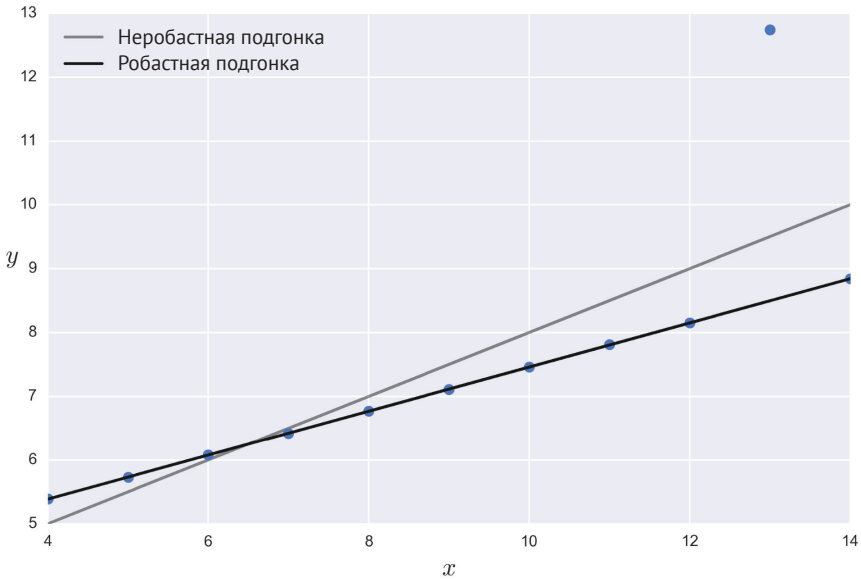


Рис. 3.11

Линия неробастной подгонки пытается найти компромисс для включения всех точек данных, в то время как робастная байесовская модель `model_t` автоматически отбрасывает одну точку и создает линию, которая точно проходит через все остальные точки. Очевидно, что исследуемый в примере набор данных весьма специфический, но полученный результат остается справедливым и для более сложных реальных наборов данных. Благодаря своему более медленно убывающему (более широкому) хвосту t -распределение Стьюдента способно снижать значимость точек, которые расположены слишком далеко от основного массива данных.

Прежде чем продолжить, задержимся ненадолго, чтобы осмыслить значения параметров (значения промежуточного параметра ν не принимаются во внимание, поскольку здесь они не представляют особого интереса):

```
az.summary(trace_t, var_names=varnames)
```

Таблица 3.2

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАП (hpd) 3 %	ПАП (hpd) 97 %	eff_n	r_hat
α	7.11	0.00	0.0	7.11	7.12	2216.0	1.0
β	0.35	0.00	0.0	0.34	0.35	2156.0	1.0
ε	0.00	0.00	0.0	0.00	0.01	1257.0	1.0
ν	1.21	0.21	0.0	1.00	1.58	3138.0	1.0

Можно видеть, что значения α , β и ϵ находятся в чрезвычайно узком интервале и в наибольшей степени это относится к параметру ϵ , который в большинстве случаев равен 0. Это абсолютно обосновано с учетом того, что выполняется подгонка линии к идеально выровненному набору точек данных (если исключить точку промаха).

Выполним проверку прогнозируемого апостериорного распределения, чтобы узнать, насколько эффективно наша модель захватывает данные. Можно передать библиотеке PyMC3 всю трудную вычислительную часть работы по созданию выборки из апостериорного распределения:

```
ppc = pm.sample_posterior_predictive(trace_t, samples=200, model=model_t, random_seed=2)
data_ppc = az.from_pymc3(trace=trace_t, posterior_predictive=ppc)
ax = az.plot_ppc(data_ppc, figsize=(12, 6), mean=True)
plt.xlim(0, 12)
```

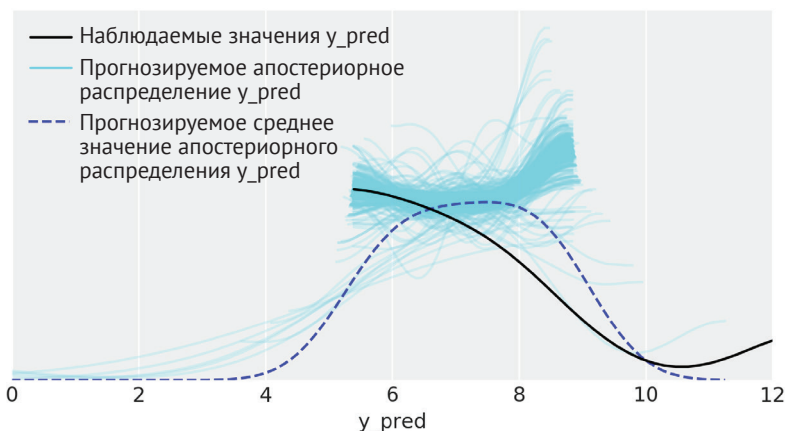


Рис. 3.12

Для основной части данных наблюдается очень хорошее соответствие. Также следует отметить, что используемая модель прогнозирует значения по обе стороны основной группы данных, а не только выше ее. Для наших текущих целей эта модель работает эффективно и не требует дальнейших изменений. Но отметим, что в некоторых задачах может возникнуть необходимость в отмене такого поведения. В этом случае, вероятнее всего, мы должны немного отступить и изменить модель так, чтобы ограничить возможные значения y_{pred} только положительными числами.

ИЕРАРХИЧЕСКАЯ ЛИНЕЙНАЯ РЕГРЕССИЯ

В предыдущей главе мы изучали основы иерархических моделей. Эту концепцию можно применить и к линейной регрессии. Такой подход позволяет моде-

лям выполнять статистические выводы на уровне групп, а оценки вычислять на более высоком уровне. Как уже было сказано ранее, это осуществляется с помощью введения априорных гиперраспределений.

Создадим восемь взаимосвязанных групп данных, включая одну группу с единственной точкой данных:

```
N = 20
M = 8
idx = np.repeat(range(M-1), N)
idx = np.append(idx, 7)
np.random.seed(314)

alpha_real = np.random.normal(2.5, 0.5, size=M)
beta_real = np.random.beta(6, 1, size=M)
eps_real = np.random.normal(0, 0.5, size=len(idx))

y_m = np.zeros(len(idx))
x_m = np.random.normal(10, 1, len(idx))
y_m = alpha_real[idx] + beta_real[idx] * x_m + eps_real

_, ax = plt.subplots(2, 4, figsize=(10, 5), sharex=True, sharey=True)
ax = np.ravel(ax)
j, k = 0, N
for i in range(M):
    ax[i].scatter(x_m[j:k], y_m[j:k])
    ax[i].set_xlabel('x_{i}')
    ax[i].set_ylabel('y_{i}', rotation=0, labelpad=15)
    ax[i].set_xlim(6, 15)
    ax[i].set_ylim(7, 17)
    j += N
    k += N
plt.tight_layout()
```

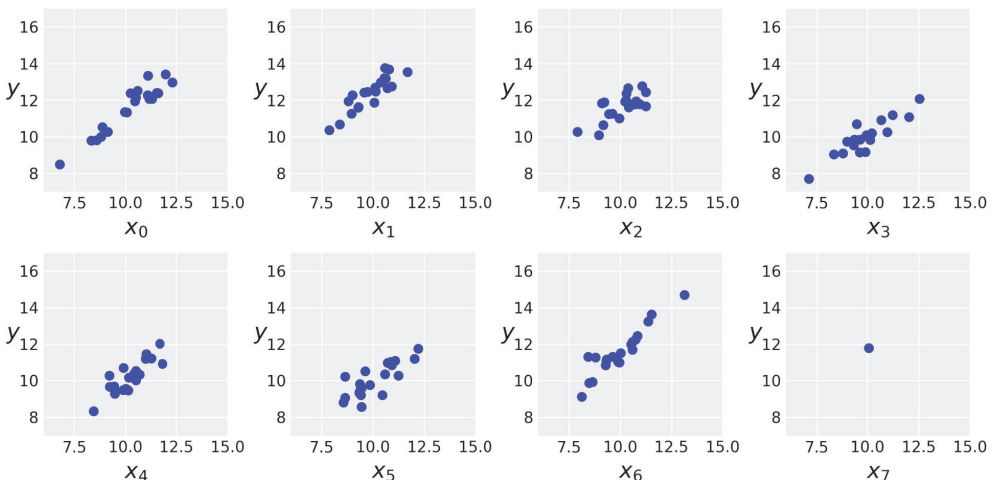


Рис. 3.13

Далее выполним центрирование данных до передачи их в модель:

```
x_centered = x_m - x_m.mean()
```

Сначала будет выполнена подгонка неиерархической модели точно так же, как мы уже наблюдали ранее. Единственное различие состоит в том, что сейчас добавляется код, возвращающий α к исходному масштабу:

```
with pm.Model() as unpooled_model:
    a_tmp = pm.Normal('a_tmp', mu=0, sd=10, shape=M)
    beta = pm.Normal('beta', mu=0, sd=10, shape=M)
    epsilon = pm.HalfCauchy('epsilon', 5)
    v = pm.Exponential('v', 1/30)
    y_pred = pm.StudentT('y_pred', mu=a_tmp[idx] + beta[idx] * x_centered,
                        sd=epsilon, nu=v, observed=y_m)
    alpha = pm.Deterministic('alpha', a_tmp - beta * x_m.mean())
    trace_up = pm.sample(2000)
```

На рис. 3.14 можно видеть, что оценки параметров α_7 и β_7 находятся в чрезвычайно широком интервале по сравнению с остальными параметрами $\alpha_{0:6}$ и $\beta_{0:6}$:

```
az.plot_forest(trace_up, var_names=['alpha', 'beta'], combined=True)
```



Рис. 3.14

Возможно, вы уже предсказали, что произойдет, – не имеет никакого смысла пытаться подогнать линию, проходящую через единственную точку. Необходимо, по меньшей мере, две точки, иначе параметры α и β становятся неограниченными. Это действительно так, если только не будет предоставлена некоторая дополнительная информация, например с использованием априорных распределений. Назначение строго определенного априорного распределения

для α может привести к генерации четко определенного набора линий даже для одной точки данных. Другой способ передачи информации – определение иерархических моделей, поскольку они позволяют совместно использовать информацию несколькими группам, редуцируя правдоподобные значения оцениваемых параметров. Это становится весьма полезным в тех случаях, когда имеются группы с разреженными (разбросанными) данными. В рассматриваемом примере разреженность данных была доведена до предела – одна из групп содержала единственную точку данных.

Теперь выполним реализацию иерархической модели, которая представляет собой ту же обычную модель линейной регрессии, но с априорными гиперраспределениями, которые можно наглядно изобразить на следующей диаграмме Крушке (см. рис. 3.15).

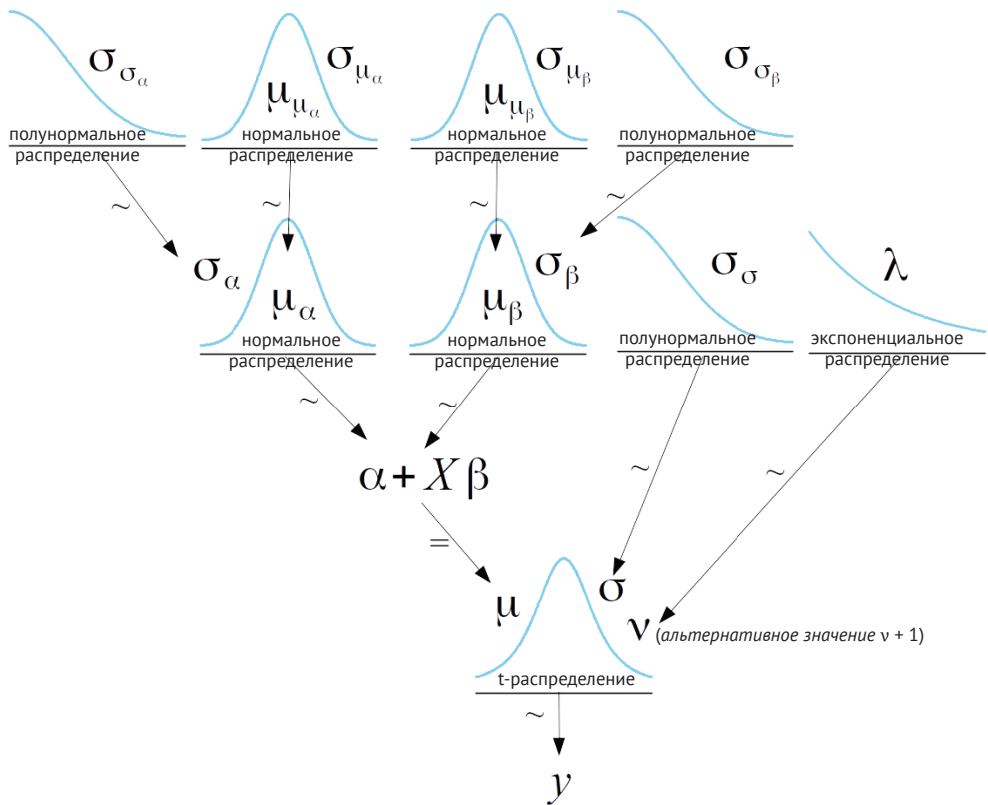


Рис. 3.15

Если сравнивать с ранее рассмотренными моделями, то в создаваемой для этого примера модели с использованием библиотеки РумСЗ можно выделить следующие основные отличия:

- добавляются априорные гиперраспределения;
- добавляется несколько строк кода для преобразования параметров обратно к исходному нецентрированному масштабу. Отметим, что это не обязательно – можно оставить параметры в измененном масштабе, но помнить об этом при интерпретации результатов:

```
with pm.Model() as hierarchical_model:
    # априорные гипер-распределения
     $\alpha_{\mu\_tmp}$  = pm.Normal('α_μ_tmp', mu=0, sd=10)
     $\alpha_{\sigma\_tmp}$  = pm.HalfNormal('α_σ_tmp', 10)
     $\beta_{\mu}$  = pm.Normal('β_μ', mu=0, sd=10)
     $\beta_{\sigma}$  = pm.HalfNormal('β_σ', sd=10)

    # априорные распределения
     $\alpha_{tmp}$  = pm.Normal('α_tmp', mu= $\alpha_{\mu\_tmp}$ , sd= $\alpha_{\sigma\_tmp}$ , shape=M)
     $\beta$  = pm.Normal('β', mu= $\beta_{\mu}$ , sd= $\beta_{\sigma}$ , shape=M)
     $\varepsilon$  = pm.HalfCauchy('ε', 5)
    v = pm.Exponential('v', 1/30)

    y_pred = pm.StudentT('y_pred',
                        mu= $\alpha_{tmp}[idx] + \beta[idx] * x\_centered$ ,
                        sd= $\varepsilon$ , nu=v, observed=y_m)

     $\alpha$  = pm.Deterministic('α',  $\alpha_{tmp} - \beta * x\_m.mean()$ )
     $\alpha_{\mu}$  = pm.Deterministic('α_μ',  $\alpha_{\mu\_tmp} - \beta_{\mu} * x\_m.mean()$ )
     $\alpha_{\sigma}$  = pm.Deterministic('α_σ',  $\alpha_{\sigma\_tmp} - \beta_{\sigma} * x\_m.mean()$ )

    trace_hm = pm.sample(1000)
```

Для сравнения результатов модели `unpooled_model` с результатами иерархической модели `hierarchical_model` необходимо сгенерировать еще один график в форме леса:

```
az.plot_forest(trace_hm, var_names=['α', 'β'], combined=True)
```

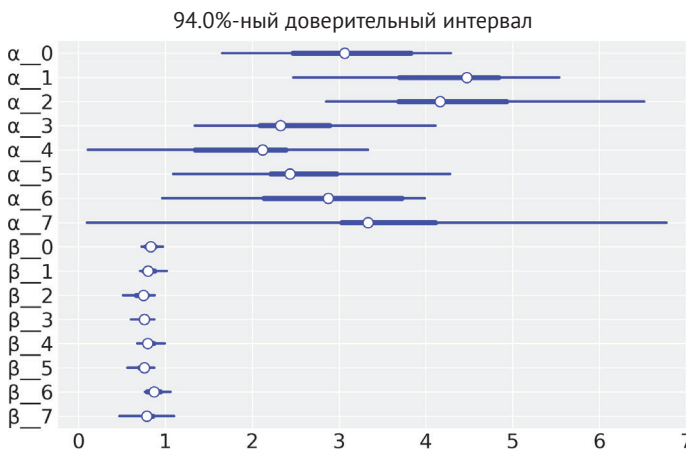


Рис. 3.16

Эффективный способ сравнения моделей с использованием метода `az.plot_forest()` – наглядное отображение параметров обеих моделей (`unpooled_model`, `hierarchical_model`) одновременно на одном и том же графике. Для этого необходимо только лишь передать список параметров трассировки.

Чтобы лучше понять, что именно модель узнает о данных, сгенерируем графики линий подгонки для каждой из восьми групп:

```
_, ax = plt.subplots(2, 4, figsize=(10, 5), sharex=True, sharey=True,
                    constrained_layout=True)
ax = np.ravel(ax)
j, k = 0, N
x_range = np.linspace(x_m.min(), x_m.max(), 10)
for i in range(M):
    ax[i].scatter(x_m[j:k], y_m[j:k])
    ax[i].set_xlabel(f'x_{i}')
    ax[i].set_ylabel(f'y_{i}', labelpad=17, rotation=0)
    alpha_m = trace_hm['a'][:, i].mean()
    beta_m = trace_hm['b'][:, i].mean()
    ax[i].plot(x_range, alpha_m + beta_m * x_range, c='k',
              label=f'y = {alpha_m:.2f} + {beta_m:.2f} * x')
    plt.xlim(x_m.min()-1, x_m.max()+1)
    plt.ylim(y_m.min()-1, y_m.max()+1)
    j += N
    k += N
```

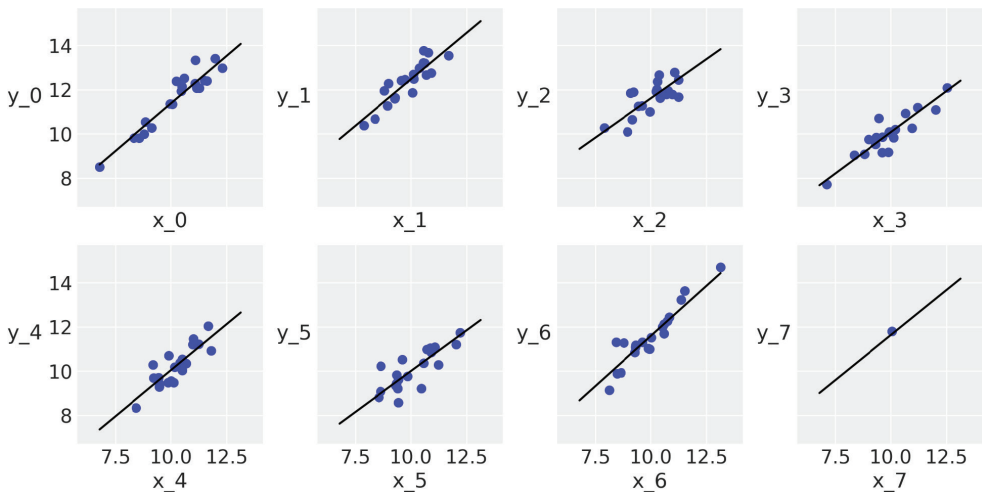


Рис. 3.17

Используя иерархическую модель, мы получаем возможность подгонки линии даже для одной точки данных, как можно видеть на рис. 3.17. На первый взгляд может показаться странным или даже неправдоподобным тот факт, что это является прямым следствием применения структуры иерархической мо-

дели. Каждая линия получает информацию от линий других групп, таким образом, в действительности подгонка линии выполняется не по единственной точке. Вместо этого регулирование положения линии происходит по одной точке, которая располагает информацией, полученной от точек в других группах.

Корреляция, причинно-следственная связь и беспорядочность жизни

Предположим, что необходимо спрогнозировать, сколько раз придется заплатить за газ для обогрева дома в течение зимы. Также предположим, что известен уровень солнечного излучения в той местности, где мы живем. В этом примере уровень солнечного излучения будет независимой переменной x , а сумма счета оплаты газа – зависимой переменной y . Весьма важно отметить, что нет никаких препятствий для изменения смысла вопроса на противоположный, то есть: каков уровень солнечного излучения с учетом суммы счета оплаты газа. Если установить линейную взаимозависимость (или любое другое отношение, соответствующее этому случаю), то можно по x определять y или наоборот. Переменную называют независимой, потому что ее значение невозможно предсказать с помощью модели. Независимая переменная является входными данными для модели, а зависимая переменная представляет выходные данные. Мы говорим, что значение одной переменной зависит от значения другой переменной, потому что создаем модель, определяющую такую зависимость. Мы не устанавливаем причинное (причинно-следственное) отношение между переменными, то есть мы не говорим, что x является причиной y . Всегда следует помнить следующую мантру: корреляция не означает наличия причинно-следственной связи. Попробуем объяснить этот принцип более подробно. Даже если имеется возможность прогнозирования суммы счета оплаты газа в зависимости от уровня солнечного излучения или уровня солнечного излучения по сумме счета оплаты газа, можно согласиться, что это никоим образом не подразумевает возможность управления мощностью солнечного излучения простым перемещением движка терморегулятора в нашем доме. Тем не менее истинным является тот факт, что более высокий уровень солнечного излучения может быть связан с уменьшением суммы в счете.

Таким образом, важно помнить, что создаваемая нами статистическая модель – это одно, а физический механизм, связывающий исследуемые переменные, – это совсем другое. Для утверждения того, что корреляция может интерпретироваться с точки зрения причинно-следственной связи, необходимо добавить достоверный физический механизм в описание задачи, поскольку одной лишь корреляции недостаточно. Существует весьма информативный веб-сайт с удачными примерами корреляции переменных без причинно-следственной взаимосвязи: <http://www.tylervigen.com/spurious-correlations>.

Бесполезна ли корреляция, когда ее пытаются применить для установления причинно-следственной связи? Разумеется, нет – в действительности корреляция может обеспечить причинно-следственную связь, если выполняется тщательно спланированный и подготовленный эксперимент. Например, известно,

что глобальное потепление тесно связано с повышением концентрации CO_2 в атмосфере. Из одного этого наблюдения невозможно сделать вывод о том, действительно ли более высокие температуры вызываются повышением концентрации CO_2 или более высокая концентрация углекислого газа вызвана повышением температуры.

Более того, возможно, что существует третья переменная, не принимаемая во внимание, которая является причиной как повышения температуры, так и увеличения концентрации CO_2 в атмосфере. Но обратите особое внимание на следующий факт: можно провести эксперимент, позволяющий получить более глубокое представление о внутренней сущности этой проблемы. Один из возможных вариантов эксперимента: создается группа стеклянных емкостей, заполненных различными количествами CO_2 . В одной из емкостей содержится обычный воздух (с концентрацией $\sim 0.04\%$ CO_2), в других емкостях содержание CO_2 постепенно увеличивается. Затем эти емкости нагреваются на солнце, например в течение трех часов. Проведя такой эксперимент, можно убедиться, что в емкостях с более высокой концентрацией CO_2 конечная температура выше. Следовательно, можно сделать вывод: CO_2 действительно создает парниковый эффект. В том же эксперименте можно также измерить концентрацию CO_2 в конце эксперимента, чтобы убедиться в том, что повышение температуры не вызывает увеличения содержания углекислого газа, по крайней мере в обычном воздухе. В этом случае экспериментальные исследования в совокупности со статистическими моделями предоставляют доказательство того, что выбросы CO_2 в атмосферу вносят свой вклад в глобальное потепление.

Другой важный факт, выводимый из рассмотренных примеров, состоит в том, что даже если уровень солнечного излучения и сумма счета оплаты газа взаимосвязаны и, вероятно, уровень солнечного излучения можно использовать для прогнозирования суммы счета, это отношение более сложное и требует привлечения дополнительных переменных. В действительности повышение температуры может в некоторой степени влиять на концентрацию CO_2 , так как океаны представляют собой огромное хранилище CO_2 , а углекислый газ хуже растворяется в воде при повышении температуры. Кроме того, более мощное солнечное излучение означает, что дом получает больше энергии. Часть этой энергии отражается, часть превращается в тепло, часть тепла поступает внутрь дома, другая часть рассеивается в окружающей среде. Количество потерь тепла зависит от нескольких факторов, например от температуры наружного воздуха и от скорости ветра. Кроме того, очевидно, что сумма счета оплаты газа также зависит от сторонних факторов, таких как международные цены на нефть и газ, цены и уровень прибыли конкретной компании (и степень ее алчности), а также степень регулирования правительством деятельности подобных компаний.

В общем, жизнь сложна и хаотична, многие проблемы трудны для понимания, поэтому всегда важен контекст. Статистические модели могут помочь в достижении более качественной интерпретации, они снижают риск появления бессмысленных утверждений и позволяют получить более точные прогнозы, но все это никогда не делается автоматически.

ПОЛИНОМИАЛЬНАЯ РЕГРЕССИЯ

Надеюсь, что практические навыки, освоенные в начальной части текущей главы, оказались полезными. Теперь мы рассмотрим, как выполнять подгонку кривых с использованием линейной регрессии. Одним из способов подгонки кривых с помощью модели линейной регрессии является формирование полиномиальных выражений, подобных следующему:

$$\mu = \beta_0 x^0 + \beta_1 x^1 + \dots + \beta_m x^m. \quad (3.12)$$

При внимательном рассмотрении можно заметить, что в этом полиномиальном выражении (многочлене) скрыта простая линейная модель. Чтобы понять это, достаточно лишь присвоить всем коэффициентам β_n с индексами больше единицы нулевое значение. Тогда получаем:

$$\mu = \beta_0 + \beta_1 x^1. \quad (3.13)$$

Полиномиальная регрессия остается линейной регрессией, поскольку линейность модели определяется тем, как вводятся в модель параметры, а не переменные. Попробуем создать полиномиальную регрессию со степенью 2:

$$\mu = \beta_0 + \beta_1 x^1 + \beta_2 x^2. \quad (3.14)$$

Третий член полинома управляет кривизной данного отношения.

В качестве исследуемого набора данных воспользуемся второй группой квартета Энскомба:

```
x_2 = ans[ans.group == 'II']['x'].values
y_2 = ans[ans.group == 'II']['y'].values
x_2 = x_2 - x_2.mean()

plt.scatter(x_2, y_2)
plt.xlabel('x')
plt.ylabel('y', rotation=0)
```

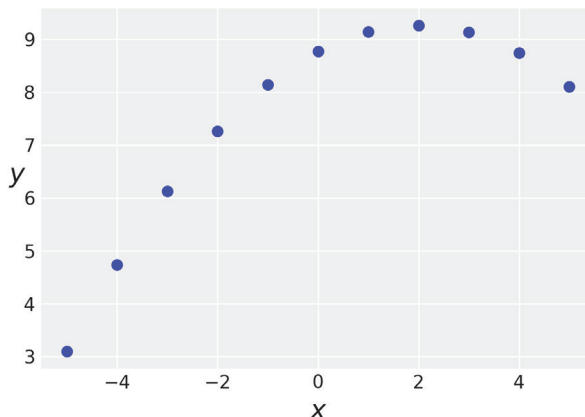


Рис. 3.18

Теперь создадим модель с использованием библиотеки PyMC3:

```
with pm.Model() as model_poly:
    α = pm.Normal('α', mu=y_2.mean(), sd=1)
    β1 = pm.Normal('β1', mu=0, sd=1)
    β2 = pm.Normal('β2', mu=0, sd=1)
    ε = pm.HalfCauchy('ε', 5)

    mu = α + β1 * x_2 + β2 * x_2**2

    y_pred = pm.Normal('y_pred', mu=mu, sd=ε, observed=y_2)

    trace_poly = pm.sample(2000)
```

И на этот раз пропустим некоторые проверки и обобщения, сразу перейдем к построению графика результатов, которые представляют собой плавную кривую, совпадающую с точками данных почти без ошибок. Но при этом надо учитывать минималистичность используемого набора данных.

```
x_p = np.linspace(-6, 6)
y_p = trace_poly['α'].mean() + trace_poly['β1'].mean() * \
    x_p + trace_poly['β2'].mean() * x_p**2
plt.scatter(x_2, y_2)
plt.xlabel('x')
plt.ylabel('y', rotation=0)
plt.plot(x_p, y_p, c='C1')
```

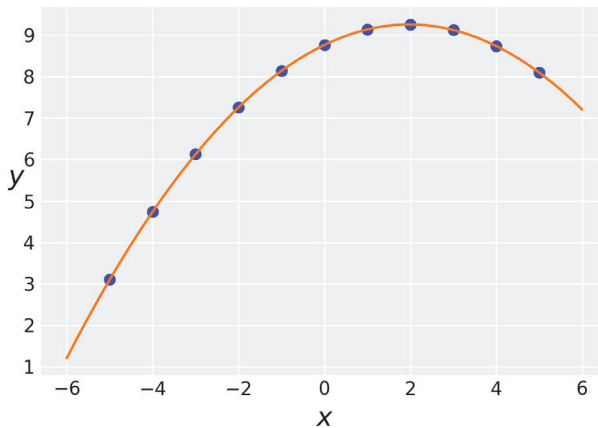


Рис. 3.19

Интерпретация параметров полиномиальной регрессии

Одной из проблем полиномиальной регрессии является интерпретация параметров. Если необходимо узнать, как изменяется y при изменении x на одну единицу измерения, то недостаточно просто проверить значение β_1 , потому что β_2 и следующие коэффициенты, если они присутствуют, оказывают воздействие на итоговое числовое значение. Таким образом, коэффициенты β уже

не являются углами наклона, они представляют собой нечто другое. В предыдущем примере коэффициент β_1 положителен, следовательно, начало кривой имеет положительный угол наклона, но коэффициент β_2 отрицательный, поэтому вскоре линия начинает изгибаться вниз. Похоже на то, что здесь действуют две силы – одна стремится изогнуть линию вверх, другая – вниз. Взаимодействие этих сил зависит от значения x . При $x \lesssim 11$ (в исходном масштабе или 2 в центрированном масштабе) преобладающее воздействие оказывает коэффициент β_1 , а при $x \gtrsim 11$ больше воздействует коэффициент β_2 .

Проблема интерпретации параметров – это не только математическая проблема. В подобной ситуации можно решить ее с помощью тщательного изучения и понимания конкретной модели. Во многих случаях проблема заключается в том, что параметры не преобразовываются в осмысленные числовые характеристики используемой области знаний. Мы не можем связать их с уровнем метаболизма клетки, или с энергией, излучаемой далекой галактикой, или с количеством спален в доме. Это всего лишь условные объекты, которые мы можем ввести для улучшения подгонки кривых, но они не имеют явного физического смысла. На практике большинство людей согласится с тем, что полиномы с порядком, большим, чем 2 или 3, в общем не являются сколько-нибудь полезными моделями, а более предпочтительны такие альтернативы, как гауссовы процессы, которые подробно рассматриваются в главе 7.

Является ли полиномиальная регрессия конечной моделью

Ранее уже отмечалось, что можно представить прямую линию как упрощенную (вырожденную) модель параболы с коэффициентом β_2 , равным нулю. Кроме того, прямая линия также является упрощенной моделью кубической модели, когда коэффициенты β_2 и β_3 равны нулю. Разумеется, парабола – это тоже упрощенная модель кубической кривой при β_3 , равном нулю. Здесь мы остановимся, но направление мысли уже понятно. Предполагается теоретически, что можно использовать полиномиальную регрессию для подгонки любой модели с произвольной сложностью. Нужно всего лишь сформировать полином правильного порядка. Это можно было бы сделать, постепенно увеличивая порядок многочлена на единицу, пока наблюдается улучшение качества подгонки, или взять полином бесконечного порядка и каким-то образом приравнять незначительные коэффициенты к нулю до тех пор, пока не получится идеально подогнанная к исследуемым данным кривая. Для проверки этой методики начнем с очень простого примера. Используем квадратичную модель для подгонки к данным третьей группы квартета Энскомба. Предлагается выполнить это задание самостоятельно.

Если вы действительно выполнили это задание, то на собственном опыте убедились в том, что вполне возможно использовать квадратичную модель для подгонки линии. Может даже показаться, что этот простой эксперимент убедительно подтверждает идею формирования полинома бесконечного порядка для подгонки к данным, но необходимо сдерживать энтузиазм и не под-

даваться слишком оптимистическому настроению. Вообще говоря, использование полиномов для подгонки к данным – не самая лучшая идея. Почему? Потому что не имеет значения, какими данными мы располагаем. Теоретически всегда возможно найти полином, идеально соответствующий исследуемым данным. На практике достаточно просто вычислить точный порядок, который должен иметь этот полином. Тогда почему процедура подгонки кривой к данным является проблематичной? Немного забегаая вперед, отметим, что это тема главы 6 «Сравнение моделей». Модель, которая точно соответствует текущим данным, в общем случае оказывает плохую услугу процессу подгонки/описания ранее ненаблюдаемых данных. Причина в том, что любой реальный набор данных содержит шумы (помехи) и иногда новые, заслуживающие внимания шаблоны (образцы). Произвольно выбранная сверхсложная модель выполнит подгонку с учетом шумов, что приведет к ухудшению прогнозов. Это явление называют переподгонкой (overfitting), оно весьма часто встречается в статистике и в машинном обучении. Полиномиальная регрессия является удобным оправданием при возникновении переподгонки, потому что эту проблему легко заметить. Это создает интуитивное представление о том, что мы можем выполнить преобразование в более сложные модели, а это приводит к тому, что мы лишаемся возможности обнаружить переподгонку. Один из этапов работы по анализу данных как раз и заключается в том, чтобы убедиться в отсутствии переподгонки в используемых моделях. Более подробно эта тема рассматривается в главе 6 «Сравнение моделей».

МНОЖЕСТВЕННАЯ ЛИНЕЙНАЯ РЕГРЕССИЯ

До настоящего момента мы работали с одной зависимой переменной и одной независимой переменной. Но нет ничего необычного в наличии нескольких независимых переменных, которые необходимо включить в применяемую модель. Приведем несколько примеров:

- различаемое на вкус качество вина (зависимая переменная) и кислотность, густота (концентрация), содержание алкоголя, остаточный сахар и содержание сульфатов (независимые переменные);
- средний балл (оценка) учащегося (зависимая переменная) и доход семьи, расстояние от дома до школы и уровень образования матери (категориальная переменная).

Модель простой линейной регрессии можно с легкостью расширить для работы с несколькими независимыми переменными. Такой вариант называют моделью множественной линейной регрессии, реже – линейной регрессией со многими переменными (не путать с многомерной линейной регрессией, при которой мы работаем с несколькими зависимыми переменными).

Применяя множественную линейную регрессию, мы моделируем среднее значение зависимой переменной следующим образом:

$$\mu = \alpha + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_m x_m. \quad (3.15)$$

Отметим, что данное выражение похоже на полиномиальную регрессию, но это совершенно другой метод. При множественной линейной регрессии используются различные переменные, а не постепенно возрастающие степени одной переменной. С точки зрения множественной линейной регрессии можно сказать, что полиномиальная регрессия похожа на множественную линейную регрессию, но использует переменные, возводимые в степень.

С помощью нотации линейной алгебры можно записать более компактный вариант:

$$\mu = \alpha + X\beta. \quad (3.16)$$

Здесь β – это вектор коэффициентов с длиной m , длина соответствует количеству независимых переменных. Переменная X – это матрица с размером $m \times n$, где n – число наблюдений, а m – количество независимых переменных. Если вы слегка подзабыли линейную алгебру, то можете обратиться к статьям Википедии, в которых описано скалярное произведение двух векторов и более обобщенная операция умножения матриц. В любом случае достаточно знать, что при этом используется более лаконичный и удобный способ записи модели:

$$X\beta = \sum_{i=1}^n \beta_i x_i = \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_m x_m. \quad (3.17)$$

Используя простую модель линейной регрессии, мы находим прямую линию, которая (как можно надеяться) описывает исследуемые данные. При помощи модели множественной линейной регрессии выполняется поиск гиперплоскости с размерностью m . Таким образом, сущность модели множественной линейной регрессии та же самая, что и сущность модели простой линейной регрессии, а единственным различием является то, что β становится вектором, а X – матрицей.

Определим исследуемые данные:

```
np.random.seed(314)
N = 100
alpha_real = 2.5
beta_real = [0.9, 1.5]
eps_real = np.random.normal(0, 0.5, size=N)

X = np.array([np.random.normal(i, j, N) for i, j in zip([10, 2], [1, 1.5])]).T
X_mean = X.mean(axis=0, keepdims=True)
X_centered = X - X_mean
y = alpha_real + np.dot(X, beta_real) + eps_real
```

Теперь переходим к определению подходящей функции для создания трех графиков разброса точек данных – два между каждой независимой переменной и зависимой переменной, а еще один – между обеими зависимыми переменными. В этом нет ничего необычного, просто функция, которой мы воспользуемся еще пару раз в оставшейся части главы:

```
def scatter_plot(x, y):
    plt.figure(figsize=(10, 10))
    for idx, x_i in enumerate(x.T):
        plt.subplot(2, 2, idx+1)
        plt.scatter(x_i, y)
        plt.xlabel(f'x_{idx+1}')
        plt.ylabel(f'y', rotation=0)

    plt.subplot(2, 2, idx+2)
    plt.scatter(x[:, 0], x[:, 1])
    plt.xlabel(f'x_{idx}')
    plt.ylabel(f'x_{idx+1}', rotation=0)
```

С помощью только что определенной функции `scatter_plot` можно сгенерировать визуальное представление данных, исследуемых в этом примере:

```
scatter_plot(X_centered, y)
```

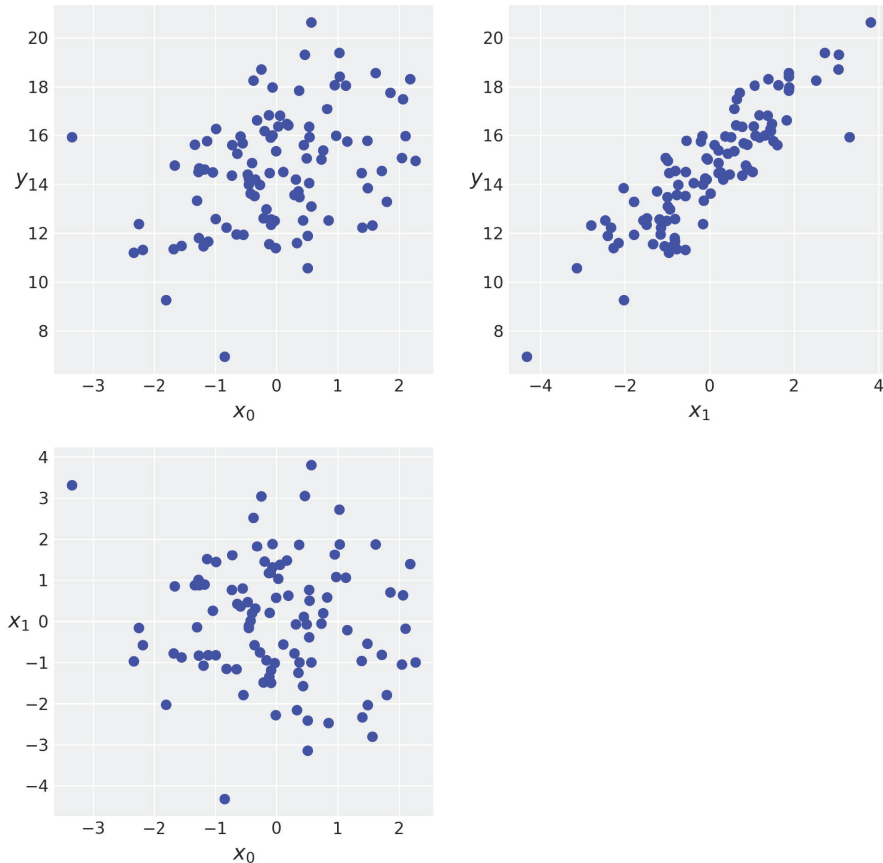


Рис. 3.20

Теперь используем библиотеку PyMC3 для определения модели, соответствующей множественной линейной регрессии. Как и предполагалось, исходный код выглядит почти так же, как при создании модели простой линейной регрессии. Тем не менее имеются существенные различия:

- переменная бета (β) представляет собой гауссово распределение с коэффициентом формы `shape=2`, то есть определяется по одному углу наклона для каждой независимой переменной;
- переменная μ определяется с применением функции скалярного произведения `pm.math.dot()`.

Если вы знакомы с библиотекой NumPy, то, вероятнее всего, знаете, что в нее включена функция скалярного произведения, а в версии Python 3.5 (и NumPy 1.10) и более поздних введен новый оператор для матриц `@`. Но здесь мы воспользуемся функцией скалярного произведения из библиотеки PyMC3, которая является всего лишь псевдонимом (алиасом) оператора умножения матриц из библиотеки Theano. Этот вариант выбран потому, что переменная β является тензором, определенным в библиотеке Theano, а не массивом NumPy:

```
with pm.Model() as model_mlr:
    a_tmp = pm.Normal('a_tmp', mu=0, sd=10)
    beta = pm.Normal('beta', mu=0, sd=1, shape=2)
    epsilon = pm.HalfCauchy('epsilon', 5)
    mu = a_tmp + pm.math.dot(X_centered, beta)
    alpha = pm.Deterministic('alpha', a_tmp - pm.math.dot(X_mean, beta))
    y_pred = pm.Normal('y_pred', mu=mu, sd=epsilon, observed=y)
    trace_mlr = pm.sample(2000)
```

Теперь объединим выведенные значения параметров для упрощения анализа результатов, чтобы узнать, насколько эффективно работает выбранная модель.

```
varnames = ['alpha', 'beta', 'epsilon']
az.summary(trace_mlr, var_names=varnames)
```

Таблица 3.3

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
$\alpha[0]$	1.86	0.46	0.0	0.95	2.69	5251.0	1.0
$\beta[0]$	0.97	0.04	0.0	0.89	1.05	5467.0	1.0
$\beta[1]$	1.47	0.03	0.0	1.40	1.53	5464.0	1.0
ϵ	0.47	0.03	0.0	0.41	0.54	4159.0	1.0

Здесь можно видеть, что используемая модель способна восстанавливать правильные значения (проверяем значения, которые были использованы для генерации синтезируемых данных).

В следующих разделах мы сосредоточимся на некоторых мерах предосторожности, которые следует предпринять при анализе результатов модели мно-

жественной линейной регрессии, особенно при интерпретации углов наклона. Особенно важно помнить о том, что при использовании множественной линейной регрессии каждый отдельный параметр имеет смысл только в контексте всех остальных параметров.

Спутывающие переменные и избыточные переменные

Представьте себе следующую ситуацию. Имеется переменная z , коррелирующая (связанная) с прогнозирующей переменной x и одновременно с прогнозируемой переменной y . Предположим, что эта переменная z является ответственной за причинно-следственные связи x и y . Например, z может быть промышленной революцией (это действительно очень сложная переменная), x – количеством пиратов, а y – концентрацией CO_2 в атмосфере. Этот пример должен быть очень хорошо знаком пастафарианам. Если исключить z из анализа, то можно получить превосходную линейную связь между x и y . Возможно, удастся даже прогнозировать y по x . Но если настоящий интерес вызывает понимание причин глобального потепления, то смысл того, что происходит в действительности, может быть полностью утерян при использовании применяемого в данном примере механизма, связывающего эти переменные.

Выше уже отмечалось, что корреляция не означает причинно-следственной связи. Одним из подтверждений этого факта является возможность исключения переменной z из анализа в вышеприведенном примере. В подобных случаях z называют спутывающей переменной или спутывающим фактором. Во многих реальных ситуациях z легко упустить. Возможно, эта переменная не измеряется или отсутствует в предоставленном нам наборе данных, или мы даже не задумывались над тем, что такая переменная может быть связана с решаемой задачей. Игнорирование спутывающих переменных при анализе может привести к установлению ложных корреляций. Это вечная проблема при попытках объяснения чего-либо, а кроме того, может становиться проблемой при попытках прогнозирования без уделения достаточного внимания пониманию внутреннего механизма явления. Понимание внутреннего механизма помогает перенести и применить полученные ранее знания в новые ситуации. Прогнозы, сделанные вслепую, не всегда обладают хорошей переносимостью. Например, количество спортивных тапочек, производимых в одной стране, можно использовать как легко измеряемый показатель мощности ее экономики, но, возможно, прогноз на основе этого показателя окажется совершенно неприемлемым для других стран с отличающейся структурой производства и культурной средой.

Воспользуемся искусственно сгенерированными данными для более глубокого понимания концепции спутывающей переменной. В приведенном ниже фрагменте кода имитируется спутывающая переменная x_1 . Отметим, что эта переменная оказывает воздействие на переменные x_2 и y .

```
np.random.seed(42)
N = 100
x_1 = np.random.normal(size=N)
```



```
x_2 = x_1 + np.random.normal(size=N, scale=1)
#x_2 = x_1 + np.random.normal(size=N, scale=0.01)
y = x_1 + np.random.normal(size=N)
X = np.vstack((x_1, x_2)).T
```

Обратите внимание на то, что благодаря способу создания этих переменных они уже являются центрированными, поэтому их можно легко проконтролировать с помощью удобной функции `scatter_plot`, которую мы написали ранее. Таким образом, не нужно выполнять центрирование данных для ускорения процесса статистического вывода:

```
scatter_plot(X, y)
```

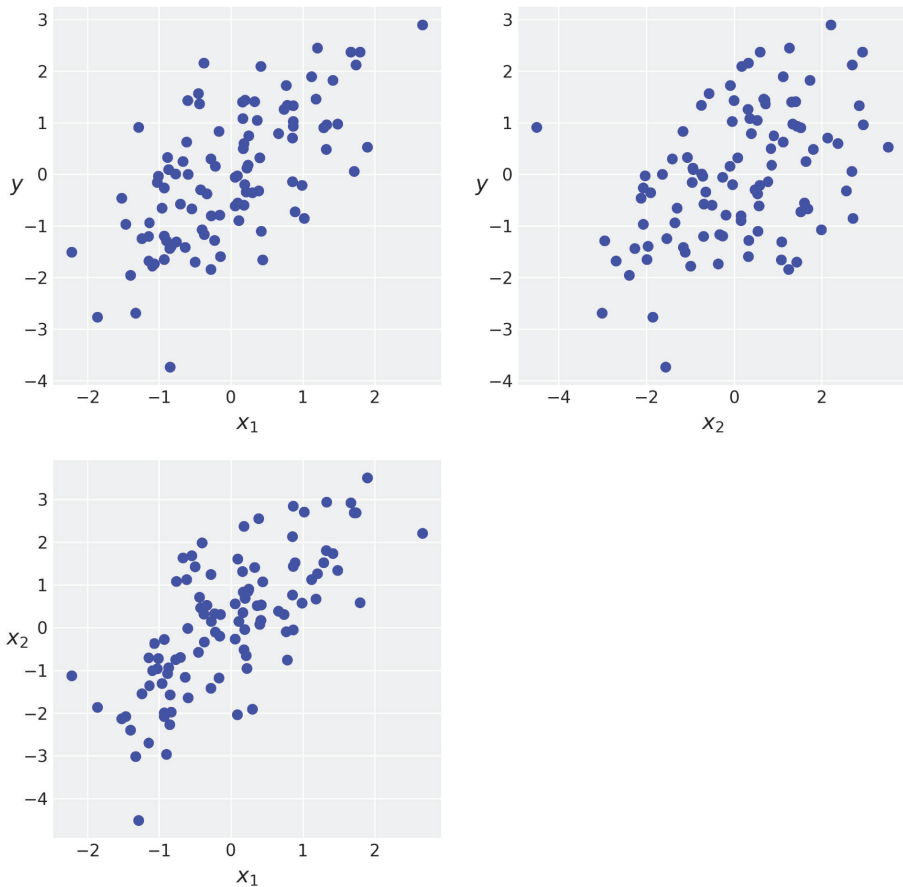


Рис. 3.21

Теперь создадим три связанные модели: первая `m_x1x2` – модель линейной регрессии с двумя независимыми переменными x_1 и x_2 (объединенными в пе-

ременной X). Вторая модель m_{x1} представляет собой простую линейную регрессию для переменной x_1 , а третья модель m_{x2} – простая линейная регрессия для x_2 .

```
with pm.Model() as m_x1x2:
     $\alpha$  = pm.Normal('α', mu=0, sd=10)
     $\beta_1$  = pm.Normal('β1', mu=0, sd=10)
     $\beta_2$  = pm.Normal('β2', mu=0, sd=10)
     $\varepsilon$  = pm.HalfCauchy('ε', 5)

     $\mu = \alpha + \beta_1 * X[:, 0] + \beta_2 * X[:, 1]$ 

    y_pred = pm.Normal('y_pred', mu= $\mu$ , sd= $\varepsilon$ , observed=y)

    trace_x1x2 = pm.sample(2000)

with pm.Model() as m_x1:
     $\alpha$  = pm.Normal('α', mu=0, sd=10)
     $\beta_1$  = pm.Normal('β1', mu=0, sd=10)
     $\varepsilon$  = pm.HalfCauchy('ε', 5)

     $\mu = \alpha + \beta_1 * X[:, 0]$ 

    y_pred = pm.Normal('y_pred', mu= $\mu$ , sd= $\varepsilon$ , observed=y)

    trace_x1 = pm.sample(2000)

with pm.Model() as m_x2:
     $\alpha$  = pm.Normal('α', mu=0, sd=10)
     $\beta_2$  = pm.Normal('β2', mu=0, sd=10)
     $\varepsilon$  = pm.HalfCauchy('ε', 5)

     $\mu = \alpha + \beta_2 * X[:, 1]$ 

    y_pred = pm.Normal('y_pred', mu= $\mu$ , sd= $\varepsilon$ , observed=y)

    trace_x2 = pm.sample(2000)
```

Рассмотрим подробнее параметры β в этих моделях. Используя функцию отображения леса, можно сравнить эти параметры на одном графике:

```
az.plot_forest([trace_x1x2, trace_x1, trace_x2],
               model_names=['m_x1x2', 'm_x1', 'm_x2'],
               var_names=['β1', 'β2'],
               combined=False, colors='cycle', figsize=(8, 3))
```

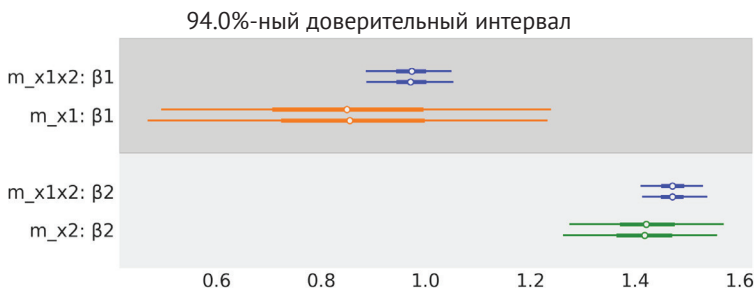


Рис. 3.22

Здесь можно видеть, что значение параметра β_2 в модели m_{x1x2} близко к нулю, что свидетельствует о почти нулевом вкладе переменной x_2 в объяснение y . Этот факт заслуживает внимания, поскольку уже известно (по результатам проверки искусственно сгенерированных данных), что действительно важной переменной является x_1 . Также отметим, что значение параметра β_2 в модели m_{x2} близко к 0.55 – и это очень важно. Это значение больше, чем для модели m_{x1x2} . Сила воздействия x_2 на прогнозирование y снижается, когда мы принимаем во внимание переменную x_1 , то есть информация в x_2 является избыточной при наличии x_1 .

Мультиколлинеарность или слишком сильная корреляция

В предыдущем примере рассматривалось, как модель множественной линейной регрессии реагирует на избыточные переменные, а также подчеркивалась важность принятия во внимание возможных спутывающих переменных. В этом разделе предыдущий пример будет доведен до предельного состояния, и мы увидим, что происходит, когда корреляция между двумя переменными слишком сильна. Чтобы исследовать эту проблему и ее последствия для статистического вывода, воспользуемся теми же искусственно сгенерированными данными и той же моделью, но увеличим степень корреляции между x_1 и x_2 , снизив уровень шума в гауссовом распределении, который был добавлен в x_1 для получения x_2 :

```
np.random.seed(42)
N = 100
x_1 = np.random.normal(size=N)
x_2 = x_1 + np.random.normal(size=N, scale=0.01)
y = x_1 + np.random.normal(size=N)
X = np.vstack((x_1, x_2)).T
```

Это изменение в коде генерации данных практически равнозначно добавлению нуля к x_1 , следовательно, обе переменные практически равны. В дальнейшем вы можете попробовать изменить значения масштабного коэффициента и использовать менее экстремальные значения, но нам необходима абсолютная ясность. После генерации новых данных посмотрим, как выглядят графики разброса точек данных:

```
scatter_plot(X, y)
```

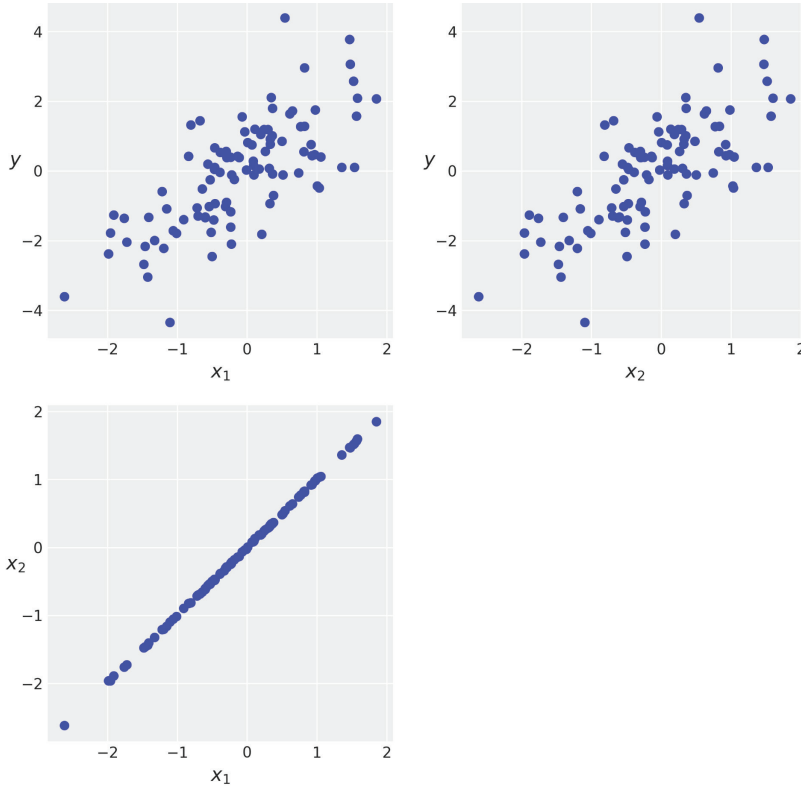


Рис. 3.23

Должна наблюдаться картина, похожая на рис. 3.23, где график для x_1 и x_2 представляет собой почти прямую линию с коэффициентом угла наклона, приблизительно равным 1.

Затем начинает работу модель множественной линейной регрессии:

```
with pm.Model() as model_red:
     $\alpha$  = pm.Normal('a', mu=0, sd=10)
     $\beta$  = pm.Normal('b', mu=0, sd=10, shape=2)
     $\epsilon$  = pm.HalfCauchy('e', 5)

     $\mu$  =  $\alpha$  + pm.math.dot(X,  $\beta$ )

    y_pred = pm.Normal('y_pred', mu= $\mu$ , sd= $\epsilon$ , observed=y)

    trace_red = pm.sample(2000)
```

После этого выполняется проверка результатов для параметров β с помощью графика леса:

```
az.plot_forest(trace_red, var_names=[' $\beta$ '], combined=True, figsize=(8, 2))
```

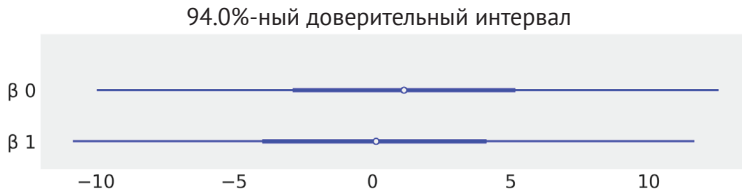


Рис. 3.24

Интервал ПАР для коэффициентов β подозрительно широк. Понять, что происходит, можно с помощью точечного графика для коэффициентов β :

```
az.plot_pair(trace_red, var_names=[' $\beta$ '])
```

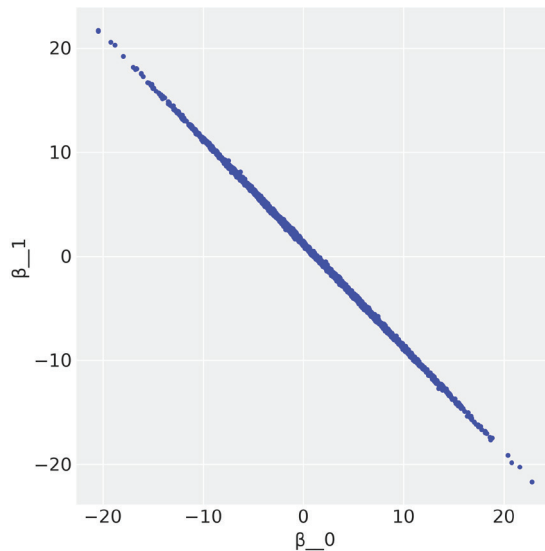


Рис. 3.25

Неожиданностью является то, что граничное апостериорное распределение для β представляет собой весьма тонкую диагональную линию. При возрастании значения одного коэффициента β другой должен убывать. Между этими двумя коэффициентами существует сильная корреляция. Это явное следствие применения конкретной выбранной модели и конкретных данных. В соответствии с примененной в рассматриваемом примере моделью среднее значение μ равно:

$$\mu = \alpha + \beta_1 x_1 + \beta_2 x_2. \quad (3.19)$$

Если предположить, что x_1 и x_2 не совсем равны с практической точки зрения, но математически идентичны, то можно переписать модель следующим образом:

$$\mu = \alpha + (\beta_1 + \beta_2)x. \quad (3.20)$$

При этом оказывается, что на среднее значение μ воздействует сумма коэффициентов β_1 и β_2 , а не их отдельные значения. Можно постепенно уменьшать значение β_1 , пока не будет получено значение β_2 . Таким образом, у нас практически нет двух переменных x , следовательно, практически нет двух параметров β . Такую модель можно назвать недетерминированной (или, что то же самое, данные невозможно ограничить параметрами в этой модели). В рассматриваемом примере существует две причины, по которым значение β не может свободно изменяться в интервале $[-\infty, +\infty]$. Во-первых, обе переменные почти одинаковы, но не равны в точности. Во-вторых, что наиболее важно, здесь применяется априорное распределение, ограничивающее возможные значения, которые может принимать β .

В рассмотренном выше примере заслуживает внимания еще несколько фактов. Прежде всего апостериорное распределение является не чем иным, как логическим следствием, выведенным из исследуемых данных и применяемой модели, поэтому нет ничего противоестественного в получении столь широкого размаха распределений для β , «такова жизнь». Во-вторых, на эту модель можно положиться при прогнозировании. Например, попробуйте выполнить проверки прогнозируемого апостериорного распределения – значения, прогнозируемые моделью, согласованы с имеющимися данными. Эта модель захватывает и обрабатывает данные весьма эффективно. В-третьих, возможно, это не самая лучшая модель для понимания и объяснения нашей задачи. Может быть, все стало бы более ясным, если удалить из модели одну из переменных. В конечном итоге получилась бы модель, которая прогнозирует данные не хуже, чем предыдущая, но с более простой (даже элементарной) интерпретацией.

В любом реальном наборе данных корреляции всегда будут существовать в той или иной степени. Насколько сильной должна быть корреляция между двумя или несколькими переменными, чтобы стать проблемой? Скажем, 0.9845. Но это всего лишь шутка. К сожалению, статистика – это раздел математики, в котором почти нет магических чисел, их всего лишь несколько. Всегда есть возможность сформировать корреляционную матрицу перед запуском любой байесовской модели и проверить переменные с сильной корреляцией, например большей 0.9 и т. д. Но в любом случае при таком подходе действительная проблема заключается не в корреляции, наблюдаемой между парами элементов корреляционной матрицы, а в корреляции переменных внутри модели. Как уже было отмечено выше, поведение переменных в изолированном контексте отличается от их поведения, когда они объединены в модели. Две

и более переменных могут увеличивать или уменьшать корреляцию между собой, когда они размещаются в контексте других переменных в модели множественной линейной регрессии. Как всегда, настоятельно рекомендуется внимательное исследование апостериорного распределения в совокупности с итеративным критическим подходом к созданию модели. Это может помочь выявить проблемы и лучше понять данные и модели.

В качестве краткого руководства ниже приведены рекомендации для тех случаев, когда обнаружены переменные с сильной корреляцией:

- если корреляция действительно сильная, то можно исключить одну переменную из анализа, особенно если допускается, что обе переменные содержат одинаковую или почти одинаковую информацию. В последнем случае не имеет значения, какая переменная исключается. Можно исключать переменные только из соображений удобства, например удалить переменную с наименее известным (значимым) смыслом в данной области знаний или переменную, которую труднее всего интерпретировать или измерять;
- можно создать новую переменную, усредняющую избыточные переменные. Более усовершенствованной версией этого подхода является использование алгоритма сокращения переменных, например метод главных компонентов (РСА). Проблема применения метода главных компонентов состоит в том, что получаемые в результате переменные являются линейными комбинациями исходных переменных, что в общем случае создает серьезные затруднения при интерпретации результатов;
- еще один вариант решения – применить более строгие априорные распределения для ограничения возможных значений, которые может принимать коэффициент. В главе 6 «Сравнение моделей» будут кратко рассмотрены некоторые варианты выбора таких априорных распределений, называемых регуляризирующими априорными распределениями.

Маскировочный эффект переменных

Хитроумным примером того, как переменные оказывают воздействие на выходные данные, является случай маскировочного эффекта переменных. Создадим небольшой искусственный набор данных для демонстрации этого явления. По существу, создаются две независимые переменные (x_1 и x_2). Между ними существует положительная корреляция, а кроме того, имеется корреляция с переменной y , но в противоположных направлениях: корреляция между x_1 и y положительная, а корреляция между x_2 и y отрицательная.

```
np.random.seed(42)
N = 126
r = 0.8
x_1 = np.random.normal(size=N)
x_2 = np.random.normal(x_1, scale=(1 - r ** 2) ** 0.5)
```

```

y = np.random.normal(x_1 - x_2)
X = np.vstack((x_1, x_2)).T
scatter_plot(X, y)

```

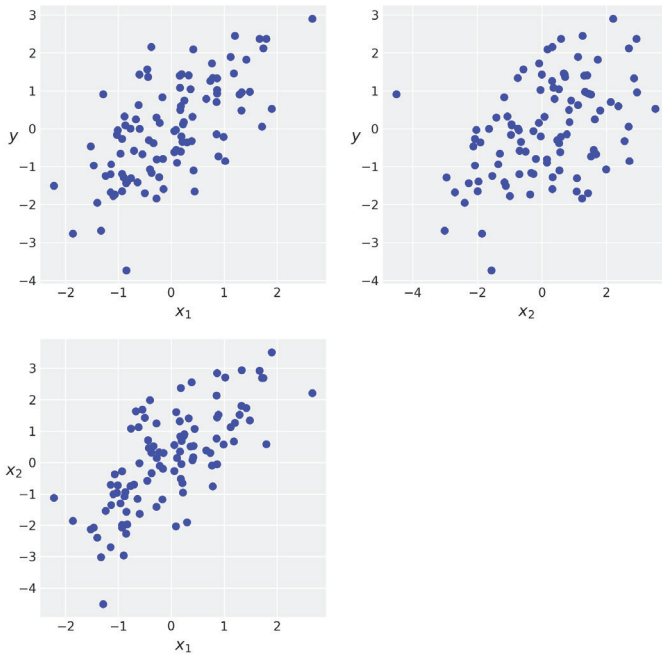


Рис. 3.26

Так же, как и в предыдущих примерах, создадим три взаимосвязанные модели. Первая модель m_{x1x2} – это модель линейной регрессии с двумя независимыми переменными x_1 и x_2 (объединенными в переменной X). Вторая модель m_{x1} – простая линейная регрессия для x_1 , третья – m_{x2} – простая линейная регрессия для x_2 . После выборки из этих моделей рассмотрим подробнее параметры β , используя отображение в форме леса для сравнения этих параметров на одном графике:

```

az.plot_forest([trace_x1x2, trace_x1, trace_x2],
               model_names=['m_x1x2', 'm_x1', 'm_x2'],
               var_names=['β1', 'β2'],
               combined=True, colors='cycle', figsize=(8, 3))

```

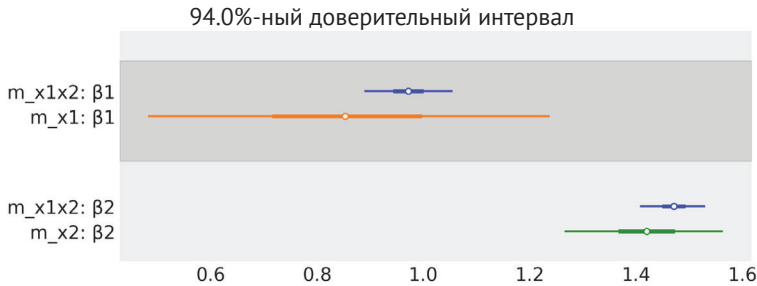



Рис. 3.27

В соответствии с апостериорным распределением значения β для модели m_{x1x2} близки к 1 и -1 (как и ожидалось в соответствии со способом генерации данных). Для модели простой линейной регрессии, то есть когда каждая переменная исследуется независимо от других, можно видеть, что значения β в этом случае ближе к нулю, свидетельствуя о более слабом эффекте.

Отметим, что x_1 коррелирует с x_2 . Фактически это означает, что при увеличении x_1 также увеличивается x_2 . Кроме того, отметим, что при увеличении y переменная x_1 тоже увеличивается, но переменная x_2 уменьшается. В результате применения этой конкретной схемы регулирования мы получаем частичную отмену эффектов, если только не включаем обе переменные в одну и ту же линейную регрессию. Модель линейной регрессии способна распространять эти эффекты, так как она обучается на каждой точке данных, то есть узнает, какой вклад вносит x_1 в значение y при определенном значении x_2 , а также величину вклада x_2 при определенном значении x_1 .

Добавление взаимодействий

До настоящего момента в определении модели линейной регрессии объявлялось (неявно), что любые изменения переменной x_i приводят к изменениям на постоянную величину значения переменной y при сохранении неизменными значений всех прочих прогнозирующих переменных. Но, разумеется, это положение не всегда истинно. Возможна ситуация, когда изменения переменной x_i воздействуют на y и при этом модулируются изменениями переменной x_j . Классическим примером такого поведения является совместное действие различных лекарственных препаратов. Например, увеличение дозы препарата А оказывает на пациента положительное воздействие. Это истинно при отсутствии препарата В (или при малой дозе препарата В), но эффект от применения А становится отрицательным (или даже может привести к смертельному исходу) при увеличении дозы В.

Во всех рассмотренных выше примерах зависимые переменные демонстрировали накопительный (суммируемый) вклад в значение прогнозируемой переменной. Мы просто добавляли переменные (с умножением на некоторый коэффициент). Если требуется сбор данных об эффектах, как в примере с ле-

карстенными препаратами, то необходимо включить в модель члены, которые не являются аддитивными. Часто применяется вариант умножения переменных, например:

$$\begin{aligned}\mu &= \alpha + \underbrace{(\beta_1 + \beta_3 x_2)}_{\text{коэффициент угла наклона } x_1} x_1 + \beta_2 x_2; \\ \mu &= \alpha + \beta_1 x_1 + \underbrace{(\beta_2 + \beta_3 x_1)}_{\text{коэффициент угла наклона } x_2} x_2.\end{aligned}\tag{3.22}$$

Эти формулы сообщают следующую информацию:

- член взаимодействия можно понимать как линейную модель. Таким образом, выражение для среднего значения μ представляет собой линейную модель, в состав которой включена еще одна линейная модель;
- взаимодействие симметрично – его можно интерпретировать как коэффициент угла наклона x_1 в форме функции от x_2 и в то же время как коэффициент угла наклона x_2 в форме функции от x_1 ;
- в модели множественной линейной регрессии без взаимодействий мы получаем гиперплоскость, то есть плоскую гиперповерхность. Член взаимодействия создает кривизну на этой гиперповерхности. Причина этого явления в том, что углы наклона перестают быть константами, а становятся функциями другой переменной;
- коэффициент β_1 описывает влияние прогнозирующей переменной x_1 только при $x_2 = 0$. Этот факт является истинным, потому что при этом $\beta_3 x_2 = 0$, следовательно, коэффициент угла наклона x_1 сокращается до $\beta_1 x_1$. По правилу симметричности взаимодействия то же самое обоснование можно применить и к коэффициенту β_2 .

ДИСПЕРСИЯ ПЕРЕМЕННОЙ

Мы воспользовались линейной последовательностью («линейным лейтмотивом») для моделирования среднего значения распределения, а в предыдущем разделе использовали ее для моделирования взаимодействий. Линейную последовательность можно также применять для моделирования дисперсии (или стандартного отклонения), когда предварительные предположения о постоянной дисперсии не имеют смысла. В этих случаях может потребоваться представление дисперсии в виде (линейной) функции от зависимой переменной.

Всемирная организация здравоохранения (ВОЗ) и другие организации, занимающиеся охраной здоровья, собирают данные по всему миру о новорожденных и о детях, начинающих ходить, и формируют графики стандартов роста и развития. Эти графики являются важнейшей частью рабочего комплекта каждого педиатра, а также средством измерения общего уровня здоровья населения при формировании стратегий развития здравоохранения, планирова-

нии лечебных и профилактических мер и для наблюдения за их эффективностью (<http://www.who.int/childgrowth/en/>).

Примером таких данных является рост (длина тела) новорожденных и начинающих ходить девочек как функция от возраста (в месяцах):

```
data = pd.read_csv('../data/babies.csv')
data.plot.scatter('Month', 'Lenght')
```

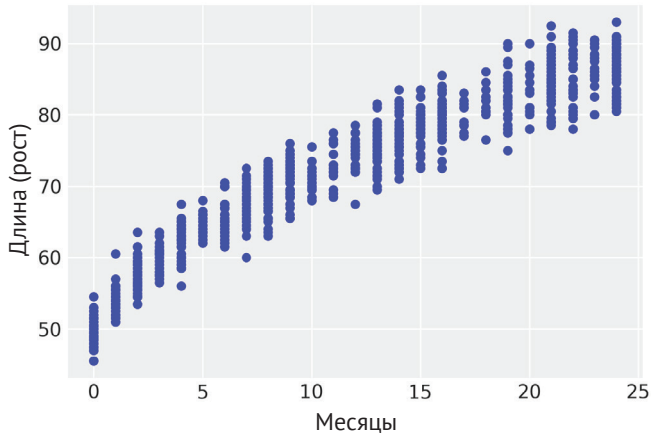


Рис. 3.28

Для моделирования этих данных введем три новых элемента, которых не было в ранее рассмотренных моделях:

- ε – теперь это линейная функция от x . Для этого добавляются два новых параметра γ и δ – прямые аналоги α и β ;
- линейная модель для среднего значения – это функция от $\sqrt{x}$. Это всего лишь обычный простой прием для подгонки линейной модели к некоторой кривой;
- определяется совместно используемая (разделяемая) переменная x_{shared} . Она будет использоваться для изменения значений переменной x (в рассматриваемом примере – Month) после подгонки модели без необходимости ее повторной подгонки. Скоро станет понятно, для чего это сделано:

```
with pm.Model() as model_vv:
    α = pm.Normal('α', sd=10)
    β = pm.Normal('β', sd=10)
    γ = pm.HalfNormal('γ', sd=10)
    δ = pm.HalfNormal('δ', sd=10)

    x_shared = shared(data.Month.values * 1.)

    μ = pm.Deterministic('μ', α + β * x_shared**0.5)
    ε = pm.Deterministic('ε', γ + δ * x_shared)
```

```
y_pred = pm.Normal('y_pred', mu=μ, sd=ε, observed=data.Lenght)
trace_vv = pm.sample(1000, tune=1000)
```

На рис. 3.29 показан результат работы этой модели. Среднее значение μ представлено кривой черного цвета, а две полупрозрачные оранжевые полосы отображают 1 и 2 стандартные отклонения соответственно:

```
plt.plot(data.Month, data.Lenght, 'C0.', alpha=0.1)
μ_m = trace_vv['μ'].mean(0)
ε_m = trace_vv['ε'].mean(0)
plt.plot(data.Month, μ_m, c='k')
plt.fill_between(data.Month, μ_m + 1 * ε_m, μ_m - 1 * ε_m, alpha=0.6, color='C1')
plt.fill_between(data.Month, μ_m + 2 * ε_m, μ_m - 2 * ε_m, alpha=0.4, color='C1')
plt.xlabel('x')
plt.ylabel('y', rotation=0)
```

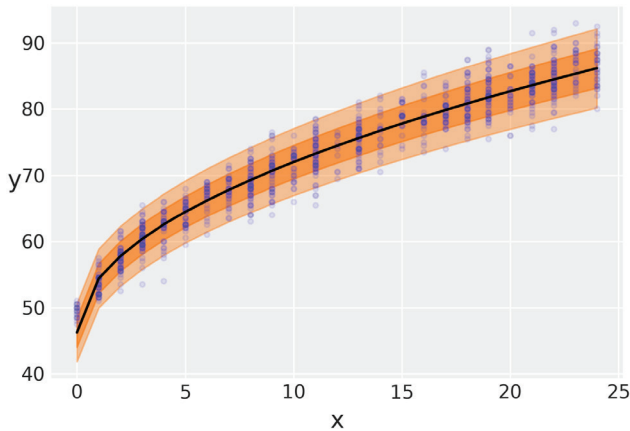


Рис. 3.29

Во время написания этого раздела книги моей дочери исполнилось две недели (приблизительно 0.5 месяца), и я заинтересовался тем, насколько ее рост (длина тела) окажется сравнимым с графиком роста, который мы только что создали. Одним из способов ответа на этот вопрос является запрос к модели о распределении переменной роста для детей в возрасте 0.5 месяца. Используя библиотеку PyMC3, можно получить ответ на такой запрос с помощью функции `sample_posterior_predictive`.

Эта функция выдает выборку значений y , обусловленную наблюдаемыми данными, и оцениваемое распределение параметров, которое включает неопределенности. Единственная проблема состоит в том, что по определению эта функция возвращает прогнозы для y по наблюдаемым значениям x , а 0.5 месяца (интересующее меня значение) не является наблюдаемым. Все указанные

измерения были выполнены для целочисленных значений месяцев. Простейший способ получения прогнозов для ненаблюдаемых значений x – определение совместно используемой (разделяемой) переменной (как части модели) и последующего обновления этой переменной непосредственно перед выполнением выборки из прогнозируемого апостериорного распределения:

```
x_shared.set_value([0.5])
ppc = pm.sample_posterior_predictive(trace_vv, 2000, model=model_vv)
y_ppc = ppc['y_pred'][:, 0]
```

Теперь можно построить график ожидаемого распределения роста (длины тела) для детей в возрасте двух недель и вычислить дополнительные числовые характеристики, например процентиль детей с такой длиной тела. Внимательно изучите фрагмент кода и рис. 3.30 для этого примера:

```
ref = 47.5
density, l, u = az._fast_kde(y_ppc)
x_ = np.linspace(l, u, 200)
plt.plot(x_, density)
percentile = int(sum(y_ppc <= ref) / len(y_ppc) * 100)
plt.fill_between(x_[x_ < ref], density[x_ < ref], label='percentile = {}'.format(percentile))
plt.xlabel('length')
plt.yticks([])
plt.legend()
```

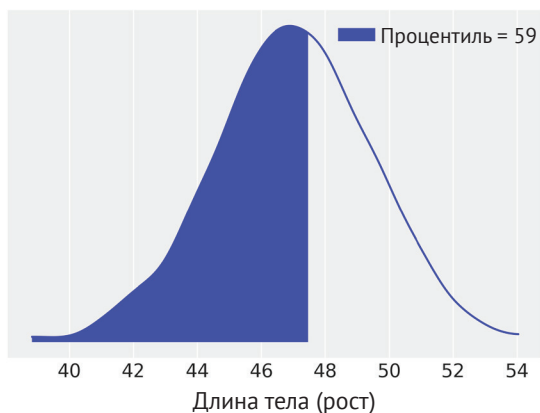


Рис. 3.30

РЕЗЮМЕ

Простая линейная регрессия – это модель, которую можно использовать для прогнозирования и/или объяснения значений одной переменной в зависимости от значений другой переменной. Говоря языком машинного обучения, это один из вариантов обучения с учителем. С вероятностной точки зрения модель

линейной регрессии – это расширение гауссовой модели, в которой среднее значение не оценивается напрямую, а вычисляется как линейная функция прогнозирующей переменной и некоторых дополнительных параметров. Гауссово распределение представляет собой наиболее часто применяемый вариант выбора для зависимой переменной, но мы свободно можем выбирать любые другие распределения. Одним из альтернативных вариантов, особенно удобным при обработке потенциально предполагаемых промахов, является t -распределение Стьюдента. В следующей главе будут рассматриваться и другие варианты.

В этой главе также обсуждался коэффициент корреляции Пирсона, наиболее часто применяемая мера линейной корреляции между двумя переменными. Мы узнали, как вычисляется байесовская версия этого коэффициента корреляции по данным и по выборкам прогнозируемого апостериорного распределения с использованием многомерного гауссова распределения. Полезным способом расширения модели линейной регрессии является ее иерархическая версия, предоставляющая преимущества редуцирования. Это очень просто реализуется с помощью библиотеки PyMC3. Кроме того, кратко обсуждалась важность строгого различения корреляции и причинно-следственной связи, по крайней мере при отсутствии механистической модели. Мы можем использовать линейные модели для подгонки к кривым, каким бы странным это не казалось. Такой подход был продемонстрирован на двух примерах: с использованием полиномиальной регрессии и с извлечением квадратного корня из независимой переменной. Еще одним расширением простой линейной регрессии является работа с несколькими независимыми переменными, то есть метод, который обычно называют множественной линейной регрессией. При этом необходимы некоторые меры предосторожности, чтобы избежать ошибок и проблем в процессе интерпретации моделей этого типа, что и было продемонстрировано на нескольких примерах. Кроме того, рассматривались некоторые способы использования линейного лейтмотива для моделирования взаимодействий, а также еще один метод применения непостоянной дисперсии для зависимой переменной.

УПРАЖНЕНИЯ

1. Проверьте следующее определение вероятностной модели. Определите правдоподобие, априорное распределение и апостериорное распределение:

$$y_i \sim \text{Normal}(\mu, \sigma);$$

$$\mu \sim \text{Normal}(0, 10);$$

$$\sigma \sim |\text{Normal}(0, 25)|$$
2. Сколько параметров модели из упражнения 1 имеют апостериорное распределение? Другими словами, какова размерность этой модели?
3. Запишите теорему Байеса для модели из упражнения 1.

4. Внимательно изучите приведенную ниже модель. Найдите определения линейной модели и правдоподобия. Сколько параметров имеют апостериорные распределения:

$$y \sim \text{Normal}(\mu, \epsilon);$$

$$\mu = \alpha + \beta x;$$

$$\alpha \sim \text{Normal}(0, 10);$$

$$\beta \sim \text{Normal}(0, 1);$$

$$\epsilon \sim \text{Normal}(0, 25)?$$

5. Для модели из упражнения 1 предположите, что имеется набор данных с 57 точками, полученными из гауссова распределения, со средним значением 4 и со стандартным отклонением 0.5. Используя библиотеку PyMC3, вычислите:

- апостериорное распределение;
- априорное распределение;
- прогнозируемое апостериорное распределение;
- прогнозируемое априорное распределение.

Совет: в библиотеке PyMC3, кроме функции `pm.sample()`, есть и другие функции формирования выборок.

6. Выполните модель `model_g` с использованием NUTS (сэмплер по умолчанию), затем с использованием сэмплера Metropolis. Сравните полученные результаты с помощью функций библиотеки ArviZ, таких как `plot_trace` и `plot_pairs`. Центрируйте переменную x и повторите оба вычисления. Какой вывод можно сделать по результатам сравнения всех вариантов вычислений?
7. Используйте набор данных `howell` (доступен на сайте <https://github.com/aloctavodia/BAP>) для создания линейной модели зависимости массы тела (веса) (x) от роста (y). Исключите лица моложе 18 лет. Объясните полученные результаты.
8. Для четырех лиц имеются данные о массе тела (весе) (45.73, 65.8, 54.2, 32.59), но нет данных о росте. Используя модель из предыдущего упражнения, выполните прогнозирование роста каждого лица вместе с интервалом ПАР 50 % и 94 %.
- Совет 1: внимательно изучите пример использования модели катастроф в угледобывающей промышленности из документации к библиотеке PyMC3.
- Совет 2: воспользуйтесь совместно используемыми (разделяемыми) переменными.
9. Повторите упражнение 7, но теперь включите в вычисления лица младше 18 лет. Объясните полученные результаты.
10. Известно, что для многих биологических видов масса тела (вес) не связана масштабным коэффициентом с ростом, но зависит от логарифма массы (веса). Используйте эту информацию для подгонки набора данных

howell (включая лица всех возрастов). Создайте еще одну модель, но на этот раз используйте не логарифм, а многочлен (полином) второго порядка. Сравните и объясните оба полученных результата.

11. Придумайте модель, которая способна подогнать первые три набора данных из квартета Энскомба. Также придумайте модель для подгонки четвертого набора данных.
12. Внимательно изучите код, соответствующий модели `model_t2` (и связанные с этой моделью данные). Поэкспериментируйте с априорными распределениями для v , например с бессдвиговым экспоненциальным и гамма априорными распределениями (в коде они закомментированы). Создайте график априорного распределения, чтобы убедиться в том, что вы понимаете смысл этих распределений. Проще всего это можно сделать, закомментировав строку определения правдоподобия в коде модели и активировав график трассировки. Более эффективный способ – использовать функцию `pm.sample_prior_predictive()` вместо функции `pm.sample()`.
13. Для модели `unpooled_model` измените значение sd априорного распределения β – попробуйте значения 1 и 100. Исследуйте, как изменяются оцениваемые коэффициенты угла наклона для каждой группы. Какая группа в наибольшей степени подвержена воздействию вносимых изменений?
14. Используя модель `hierarchical_model`, воспроизведите график с рис. 3.18 с восьмью группами и восьмью линиями, но теперь введите неопределенность для линейной подгонки.
15. Еще раз выполните пример с моделью `model_mlr`, но в этот раз без центрирования данных. Сравните неопределенность для параметра α в первом и во втором вариантах. Можете ли вы объяснить полученные результаты?
Совет: вспомните определение параметра α (также называемого отрезком отсечения).
16. Изучите и выполните следующий блокнот (notebook) из документации к библиотеке PyMC3: <https://pymc-devs.github.io/pymc3/notebooks/LKJ.html>.
17. Выберите набор данных, который вам действительно интересен, и используйте его в модели простой линейной регрессии. Внимательно исследуйте полученные результаты с помощью функций библиотеки ArviZ и вычислите коэффициент корреляции Пирсона. Если у вас нет собственного подходящего набора, то попробуйте поискать в интернете, например на сайте <http://data.worldbank.org/> или <http://www.stat.ufl.edu/~winner/datasets.html>.

Глава 4

.....

Обобщение линейных моделей

«Мы мыслим обобщениями, но живем в подробностях».

– Алфред Норт Уайтхед

В предыдущей главе использовалась линейная комбинация входных переменных для прогнозирования среднего значения выходной переменной. При этом предполагалось, что выходная переменная имеет гауссово распределение. Гауссово распределение применимо во многих случаях, но существуют многочисленные ситуации, когда более разумным может быть выбор другого распределения. Ранее уже приводился такой пример, когда гауссово распределение было заменено t -распределением Стьюдента. В этой главе будут рассматриваться иные примеры, демонстрирующие разумность применения распределений, отличающихся от гауссова. Вы сами убедитесь в том, что существует общий лейтмотив или шаблон, который можно использовать для обобщения линейной модели при решении многих задач.

В этой главе рассматриваются следующие темы:

- обобщенные линейные модели;
- логистическая регрессия и функции обратной связи;
- простая логистическая регрессия;
- множественная логистическая регрессия;
- функция softmax и мультиномиальная логистическая регрессия;
- регрессия Пуассона;
- регрессия Пуассона с дополнением нулевыми значениями.

ОБОБЩЕННЫЕ ЛИНЕЙНЫЕ МОДЕЛИ

Одна из основных тем, обсуждаемых в этой главе, достаточно проста: для прогнозирования среднего значения выходной переменной можно применить произвольно выбранную функцию к линейной комбинации входной переменной:

$$\mu = f(\alpha + X\beta). \quad (4.1)$$

Здесь f – функция, которую мы будем называть функцией обратной связи (inverse link function). Существует множество функций обратной связи, которые можно выбрать. Вероятно, самой простой является функция тождественного отображения, то есть функция, возвращающая значение, которое было передано в нее как аргумент. Все модели в главе 3 использовали тождественное отображение, но для упрощения она просто не упоминалась. Само по себе тождественное упрощение не очень полезно, но оно позволяет интерпретировать различные модели более обобщенным способом.



Почему f называется функцией обратной связи, а не просто функцией связи? Потому что обычно функции применяются в другой части формулы 4.1 и, к сожалению для нас, термин «функция связи» уже занят. Поэтому, чтобы избежать путаницы, здесь будет использоваться термин «функция обратной связи».

Одним из случаев, когда требуется использование функции обратной связи, является работа с категориальными переменными, такими как обозначение цвета, пол, биологические виды или политические партии/принадлежность к ним. Ни одну из таких переменных невозможно правильно смоделировать с помощью гауссова распределения. Теоретически гауссово распределение эффективно работает для непрерывной переменной, принимающей любое значение на оси действительных чисел, но вышеупомянутые переменные дискретны и принимают только несколько конкретных значений (например, красный, зеленый, синий – для цвета). При изменении распределения, используемого для моделирования данных, необходимо также изменить способ моделирования вероятных значений для средней величины этих распределений. Например, если используется биномиальное распределение, как в главах 1 и 2, то потребуются линейная модель, которая возвращает среднее значение в интервале $[0, 1]$. Одним из способов достижения этой цели является сохранение линейной модели с использованием функции обратной связи для ограничения выходного значения требуемым интервалом. Этот прием может применяться не только к дискретным переменным – может возникнуть необходимость моделирования данных, принимающих только положительные значения, следовательно, линейная модель должна быть ограничена, чтобы возвращать только положительные значения средней величины распределения, как, например, гамма-распределение или экспоненциальное распределение.

Прежде чем продолжить, отметим, что некоторые переменные могут кодироваться как числовые или качественные, а вариант кодирования выбирается на основе контекста конкретной решаемой задачи. Например, можно говорить о категориальных переменных «красный» и «зеленый», имея в виду обозначение цвета, или определить непрерывные переменные «650 нм» и «510 нм» при работе с соответствующими длинами волн.

ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ

Задачи регрессии относятся к прогнозированию непрерывного значения для выходной переменной на основе значений одной или нескольких входных переменных. В противоположность этому классификация занимается присваиванием дискретного значения (представляющего дискретный класс) выходной переменной на основе значений некоторых входных переменных. В обоих случаях задача состоит в том, чтобы получить модель, которая правильно формирует отображение соответствий между входными и выходными переменными. Чтобы добиться этого, необходимо иметь в своем распоряжении выборку с корректно определенными парами входных-выходных переменных. С точки зрения машинного обучения регрессии и классификации являются экземплярами алгоритмов обучения с учителем.

Моя мама готовит очень вкусное блюдо *sopa seca*, которое представляет собой особый способ приготовления спагетти и дословно означает «сухой суп». Возможно, такое название кажется неправильным или даже выглядит как оксюморон, но его можно понять в полной мере, если узнать, как готовится это блюдо. Нечто подобное происходит и с логистической регрессией, когда эта модель, несмотря на ее название, обобщенно позиционируется как метод решения задач классификации.

Модель логистической регрессии – это обобщение модели линейной регрессии, описанной в главе 3, отсюда такое название. Такое обобщение производится посредством замены функции f в формуле 4.1 на логистическую функцию как функцию обратной связи:

$$\text{logistic}(z) = \frac{1}{1 + e^{-z}}. \quad (4.2)$$

Для достижения нашей цели главным свойством этой логистической функции является то, что она фактически независима от значений ее аргумента z , поэтому результатом всегда будет число в интервале $[0,1]$. Таким образом, эту функцию можно рассматривать как удобный способ сжатия или упаковки значений, вычисляемых с помощью линейной модели, в диапазон значений, которые можно передать в распределение Бернулли. Эта логистическая функция также известна под названием сигмоидальная функция или просто сигма-функция, поскольку обладает характерной S-образной формой, которую можно наблюдать, если выполнить следующий фрагмент кода:

```
z = np.linspace(-8, 8)
plt.plot(z, 1 / (1 + np.exp(-z)))
plt.xlabel('z')
plt.ylabel('logistic(z)')
```

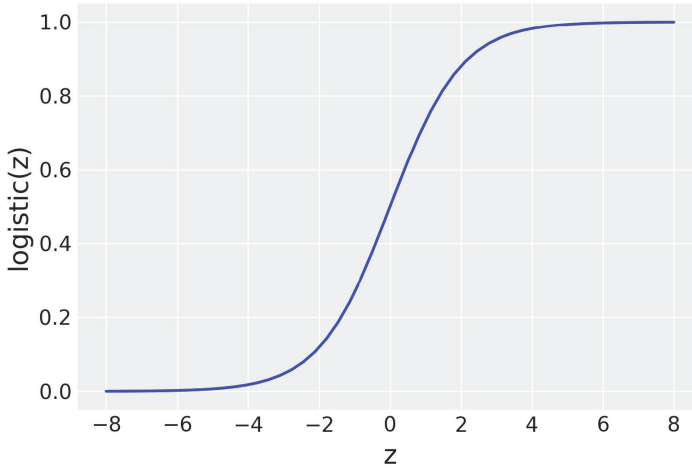


Рис. 4.1

Логистическая модель

Теперь у нас есть почти все необходимые элементы для превращения простой линейной регрессии в простую логистическую регрессию. Начнем с варианта, в котором существуют только два класса (два типа экземпляров), например не-спам/спам, безопасный/небезопасный, облачно/солнечно, здоровый/больной или хотдог/не-хотдог. Во-первых, необходимо закодировать эти классы, определив, скажем, что прогнозируемая переменная y может принимать только два значения 0 и 1, то есть $y \in \{0, 1\}$. После принятия этого предварительного условия задача становится похожей на пример с подбрасыванием монеты из глав 2 и 3.

Здесь можно вспомнить, что мы использовали распределение Бернулли как функцию правдоподобия. Отличие от задачи с подбрасыванием монеты состоит в том, что теперь θ не генерируется из бета-распределения, а определяется линейной моделью с логистической функцией, используемой как функция обратной связи. В итоге получаем (без известных априорных распределений):

$$\begin{aligned}\theta &= \text{logistic}(\alpha + x\beta); \\ y &= \text{Bern}(\theta).\end{aligned}\tag{4.3}$$



Еще раз отметим, что главное отличие от модели простой линейной регрессии из главы 3 состоит в использовании распределения Бернулли вместо гауссова распределения и в применении логистической функции вместо тождественного отображения.

Набор данных iris

Применим логистическую регрессию к набору данных *iris*. Это классический набор данных, содержащий информацию о цветах, принадлежащих к весьма близким биологическим видам: *setosa* (ирис щетинистый), *virginica* (верони-

ка виргинская) и versicolor (ирис разноцветный). Эти виды будут зависимыми переменными, то есть прогнозируемыми классами. Имеется 50 отдельных экземпляров каждого вида, а для каждого экземпляра предоставлен набор данных из четырех переменных, которые мы будем использовать как независимые переменные (или признаки): длина лепестка, ширина лепестка, длина чашелистика, ширина чашелистика. Если вам неизвестны ботанические термины, поясню: чашелистики – это видоизмененные листья, функцией которых является защита лепестков в бутонах (почках). Загрузить набор данных iris в фрейм (dataframe) можно с помощью следующих команд:

```
iris = pd.read_csv('../data/iris.csv')
iris.head()
```

Таблица 4.1

	Длина чашелистика (sepal_length)	Ширина чашелистика (sepal_width)	Длина лепестка (petal_length)	Ширина лепестка (petal_width)	Вид цветка (specie)
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Теперь построим график длин чашелистиков sepal_length для всех трех видов с помощью функции stripplot из библиотеки seaborn:

```
sns.stripplot(x="species", y="sepal_length", data=iris, jitter=True)
```

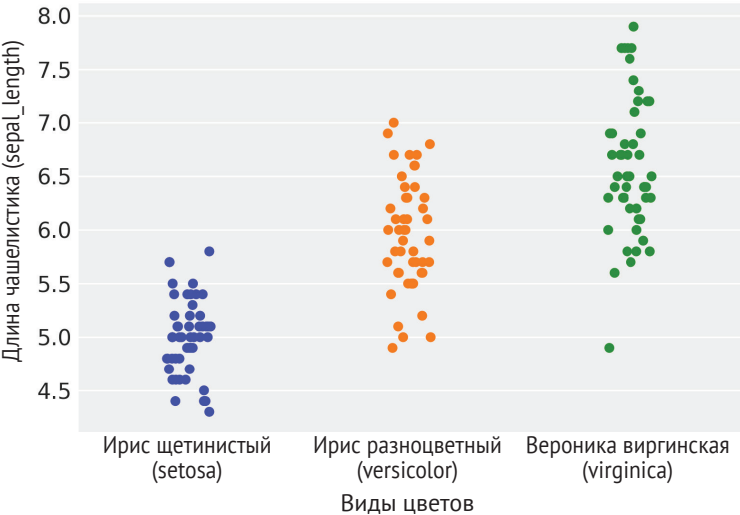


Рис. 4.2

На рис. 4.2 по оси y размещены непрерывные значения, тогда как ось x является категориальной. Дисперсия или разброс (джиттер) точек вдоль оси x не имеет никакого значения, это просто прием, добавляемый с помощью аргумента `jitter`, чтобы избежать слияния всех точек в одну непрерывную линию. Чтобы понять смысл этого приема, попробуйте установить для аргумента `jitter` значение `False`. При исследовании по оси x значение имеет только принадлежность точек к классам *setosa*, *versicolor* или *virginica*. Можно попробовать построить другие типы графиков для этих данных, например *violin*, которые также доступны как однострочный вызов функции из библиотеки *seaborn*.

Другой способ исследования данных – формирование матричной диаграммы разброса с помощью функции `pairplot`. Для нашего примера формируются диаграммы разброса на сетке 4×4, так как мы работаем с четырьмя признаками в наборе данных *iris*. Сетка симметричная с верхним и нижним треугольниками, показывающими одинаковую информацию. Диаграмма разброса по главной диагонали должна соответствовать самой исследуемой переменной. Приняв во внимание, что такая диаграмма абсолютно неинформативна, стандартные диаграммы рассеяния мы заменили на ядерную оценку плотности (ЯОП) для каждого признака. В каждой поддиаграмме отображено три вида (или класса) теми же цветами, что и на рис. 4.2.

```
sns.pairplot(iris, hue='species', diag_kind='kde')
```

Прежде чем продолжить чтение, рекомендуется уделить немного времени внимательному изучению рис. 4.3, попытаться поближе познакомиться с набором данных *iris* и выяснить, каким образом взаимосвязаны отображенные на диаграммах признаки и классы.

Логистическая модель, применяемая к набору данных *iris*

Начнем с наипростейшей задачи классификации: два класса *setosa* и *versicolor* и только одна независимая переменная или признак *sepal_length*. В соответствии с обычным образом действий закодируем категориальные переменные *setosa* и *versicolor* числами 0 и 1 соответственно. Используя библиотеку *pandas*, можно выполнить следующие операции:

```
df = iris.query("species == ('setosa', 'versicolor')")
y_0 = pd.Categorical(df['species']).codes
x_n = 'sepal_length'
x_0 = df[x_n].values
x_c = x_0 - x_0.mean()
```

Как и при работе с другими линейными моделями, центрирование данных может помочь при формировании выборок. Сейчас исследуемые данные представлены в корректном формате, поэтому можно сразу перейти к заключительному этапу создания модели с использованием библиотеки *PyMC3*.

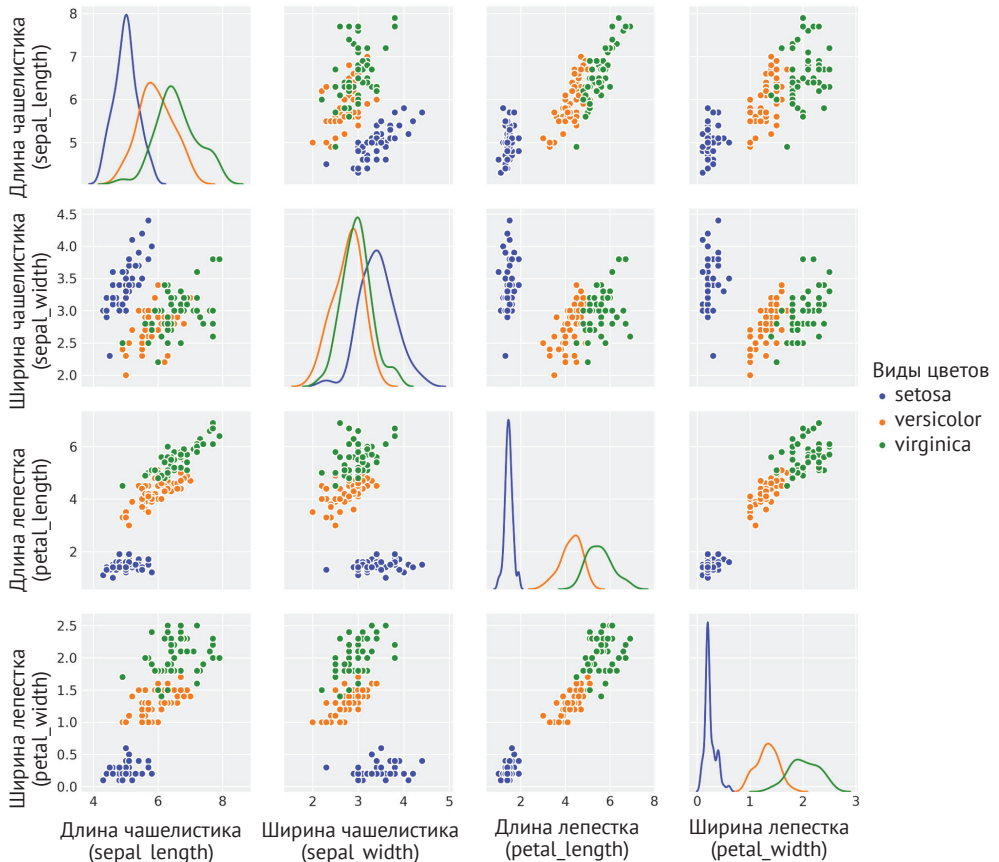


Рис. 4.3

Отметим, что первая часть модели `model_0` напоминает модель линейной регрессии. Также обратите внимание на две детерминированные переменные θ и bd . Переменная θ – это выходные данные логистической функции, примененной к переменной μ , а bd – граница решения, то есть значение, используемое для разделения классов. Более подробно граница решения будет рассмотрена немного позже. Кроме того, заслуживает внимания тот факт, что вместо явной записи логистической функции мы воспользовались функцией `pm.math.sigmoid` (это всего лишь псевдоним (алиас) для функции из библиотеки Theano с тем же именем):

```
with pm.Model() as model_0:
     $\alpha$  = pm.Normal('a', mu=0, sd=10)
     $\beta$  = pm.Normal('b', mu=0, sd=10)
     $\mu$  =  $\alpha$  + pm.math.dot(x_c,  $\beta$ )
```

```

theta = pm.Deterministic('theta', pm.math.sigmoid(mu))
bd = pm.Deterministic('bd', -alpha/beta)
yl = pm.Bernoulli('yl', p=theta, observed=y_0)

trace_theta = pm.sample(1000)
    
```

Чтобы сэкономить место и не заставлять вас скучать, снова и снова изучая один и тот же тип графика, мы не будем формировать график трассировки и прочие аналогичные сводные отчеты, тем не менее я предлагаю читателям самостоятельно создавать графики и сводные отчеты для всех последующих примеров в книге. А сейчас мы перейдем сразу к генерации рис. 4.4, то есть диаграммы данных вместе с подогаданной кривой сигмоидой и границей решения:

```

theta = trace_theta['theta'].mean(axis=0)
idx = np.argsort(x_c)
plt.plot(x_c[idx], theta[idx], color='C2', lw=3)
plt.vlines(trace_theta['bd'].mean(), 0, 1, color='k')
bd_hpd = az.hpd(trace_theta['bd'])
plt.fill_betweenx([0, 1], bd_hpd[0], bd_hpd[1], color='k', alpha=0.5)

plt.scatter(x_c, np.random.normal(y_0, 0.02), marker='.', color=[f'C{x}' for x in y_0])
az.plot_hpd(x_c, trace_theta['theta'], color='C2')

plt.xlabel(x_n)
plt.ylabel('theta', rotation=0)
# используется первоначальный масштаб для xticks
locs, _ = plt.xticks()
plt.xticks(locs, np.round(locs + x_0.mean(), 1))
    
```

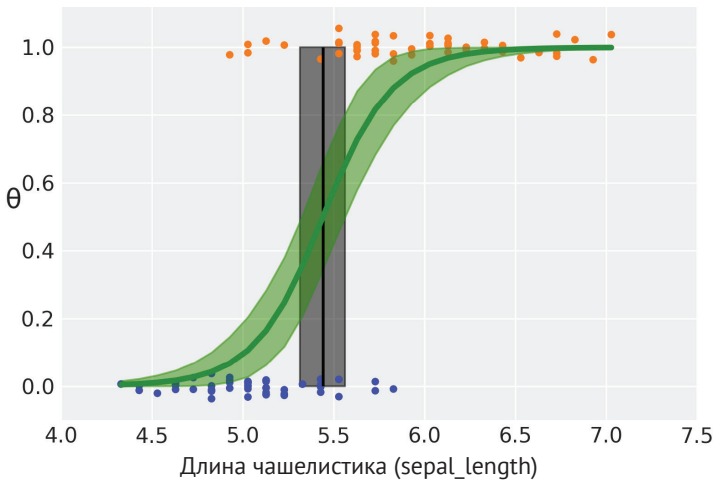


Рис. 4.4

На рис. 4.4 показаны данные о длине чашелистика `sepal_length` по двум видам ирисов (`setosa` = 0, `versicolor` = 1). Чтобы избежать чрезмерной плотности

и перегруженности диаграммы, бинарные переменные результатов слегка смещены (разбросаны). Линия S-образной формы (зеленого цвета) – это среднее значение θ . Эту линию можно интерпретировать как вероятность того, что цветок принадлежит к виду ирис разноцветный, полагая, что нам известно значение длины чашелистиков. Полупрозрачная S-образная полоса (зеленого цвета) – это интервал ПАР 94 %. Граница решения представлена в виде (черной) вертикальной прямой линии с полупрозрачной полосой для соответствующего интервала ПАР 94 %. Значения x_i (в данном примере длины чашелистиков), расположенные слева от границы решения, относятся к классу 0 (setosa), а значения справа от границы решения – к классу 1 (versicolor).

Граница решения определяется как значение x_i , для которого $y = 0.5$. Выясняется, что это отношение $-\alpha/\beta$, которое можно вывести следующим образом:

- из определения модели очевидно следующее равенство:

$$\theta = \text{logistic}(\alpha + x\beta); \quad (4.4)$$

- из определения логистической функции получаем $\theta = 0.5$, когда аргумент логистической регрессии равен 0:

$$0.5 = \text{logistic}(\alpha + x_i\beta) \Leftrightarrow 0 = \alpha + x_i\beta; \quad (4.5)$$

- после преобразования равенства 4.5 получаем значение x_i , для которого $\theta = 0.5$, то есть следующее выражение:

$$x_i = -\frac{\alpha}{\beta}. \quad (4.6)$$

Необходимо особо отметить следующие важные факты:

- вообще говоря, значение θ определяется как $p(y = 1|x)$. В этом смысле логистическая регрессия является истинной регрессией. Главный момент состоит в том, что мы регрессируем вероятность того, что некоторая точка данных принадлежит классу 1 с учетом линейной комбинации признаков;
- здесь моделируется среднее значение дихотомической переменной (переменной, которая может принимать только два значения), то есть целое число в интервале $[0, 1]$. Затем вводится правило преобразования этой вероятности в назначение принадлежности одному из двух классов. В рассматриваемом примере если $p(y = 1) \geq 0.5$, то определяется принадлежность классу 1, в противном случае – принадлежность классу 0;
- значение 0.5 не является каким-то особенным, кроме того что это число, находящееся в середине интервала между 0 и 1. Можно возразить, что эта граница обоснована только в том случае, если ошибки равновероятны в обоих направлениях относительно середины интервала, другими словами, если нет никакой разницы между неправильной классификацией ириса щетинистого как ириса разноцветного и ириса разноцветного как ириса щетинистого. Но это не всегда так, и издержки, связанные с неправильной классификацией, не обязательно являются симметрич-

ными – возможно, вы помните ситуацию в главе 2, когда рассматривались функции потерь.

МНОЖЕСТВЕННАЯ ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ

По аналогии с множественной линейной регрессией множественная логистическая регрессия предполагает использование более одной независимой переменной. Попробуем объединить данные о длине и ширине чашелистиков. Напомним, что требуется небольшая предварительная обработка данных:

```
df = iris.query("species == ('setosa', 'versicolor')")
y_1 = pd.Categorical(df['species']).codes
x_n = ['sepal_length', 'sepal_width']
x_1 = df[x_n].values
```

Граница решения

Этот раздел можно пропустить и сразу перейти к реализации модели (следующий раздел), если вас не слишком интересует процесс формирования границы решения.

По определению модели верно следующее равенство:

$$\theta = \text{logistic}(\alpha + \beta_1 x_1 + \beta_2 x_2). \quad (4.7)$$

А из определения логистической функции известно, что $\theta = 0.5$, когда аргумент логистической регрессии равен нулю:

$$0.5 = \text{logistic}(\alpha + \beta_1 x_1 + \beta_2 x_2) \Leftrightarrow 0 = \alpha + \beta_1 x_1 + \beta_2 x_2. \quad (4.8)$$

После преобразования равенства 4.8 находим значение x_2 , для которого $\theta = 0.5$, то есть следующее выражение:

$$x_2 = -\frac{\alpha}{\beta_2} + \left(-\frac{\beta_1}{\beta_2} x_1\right). \quad (4.9)$$

Это выражение для границы решения имеет математическую форму, равнозначную уравнению прямой линии, где первый член (свободный член) представляет отрезок отсечения, а второй член – коэффициент угла наклона. Скобки используются для полной ясности, поэтому их можно опустить. Граница, представленная прямой линией, кажется вполне обоснованным вариантом. Если исследуется один признак, то данные одномерны, и их можно разделить на две группы с помощью точки. Если рассматриваются два признака, то формируется двумерное пространство данных, которое можно разделить линией. Для трех измерений границей будет плоскость, для измерений более высоких порядков мы рассматриваем гиперплоскости. В действительности гиперплоскость – это обобщенная концепция, приближенно определяемая как подпространство размерности $n - 1$ в n -мерном пространстве, поэтому можно сказать, что мы всегда имеем в виду гиперплоскость.

Реализация модели

Для записи модели множественной логистической регрессии с использованием библиотеки РумСЗ воспользуемся преимуществами, предоставляемыми возможностями векторизации и позволяющими внести только минимальные изменения в предыдущую простую логистическую модель `model_0`:

```
with pm.Model() as model_1:
    α = pm.Normal('α', mu=0, sd=10)
    β = pm.Normal('β', mu=0, sd=2, shape=len(x_n))
    μ = α + pm.math.dot(x_1, β)
    θ = pm.Deterministic('θ', 1 / (1 + pm.math.exp(-μ)))
    bd = pm.Deterministic('bd', -α/β[1] - β[0]/β[1] * x_1[:,0])
    y_l = pm.Bernoulli('y_l', p=θ, observed=y_1)

    trace_1 = pm.sample(2000)
```

По аналогии с ранее используемой одной прогнозирующей переменной сформируем диаграмму данных и график границы решения:

```
idx = np.argsort(x_1[:,0])
bd = trace_1['bd'].mean(0)[idx]
plt.scatter(x_1[:,0], x_1[:,1], c=[f'C{x}' for x in y_0])
plt.plot(x_1[:,0][idx], bd, color='k');

az.plot_hpd(x_1[:,0], trace_1['bd'], color='k')

plt.xlabel(x_n[0])
plt.ylabel(x_n[1])
```

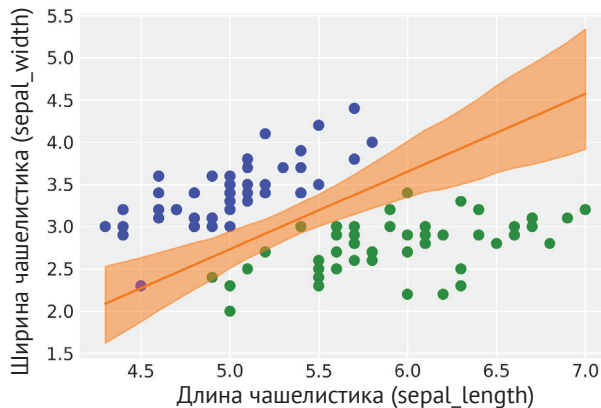


Рис. 4.5

Граница решения представлена прямой линией, как уже наблюдалось в предыдущем примере. Пусть вас не смущают криволинейные границы полосы интервала ПАР 94 %. Явная кривизна является результатом наличия нескольких линий, вращающихся относительно центральной области (приблизительно: относительно среднего значения x и среднего значения y).

Интерпретация коэффициентов логистической регрессии

При интерпретации коэффициентов β логистической регрессии необходимо быть внимательным и осторожным. В этом случае интерпретация не так проста и очевидна, как для линейных моделей, которые рассматривались в главе 3. Использование логистической функции обратной связи приводит к нелинейности, которую необходимо учитывать. Если коэффициент β положительный, то при увеличении x увеличивается вероятность $p(y = 1)$ на некоторую величину, но эта величина не является линейной функцией от x , а зависит нелинейно от значения x . Этот факт можно отчетливо наблюдать на рис. 4.4 – вместо линии с постоянным углом наклона здесь изображена S-образная линия, угол наклона которой изменяется как функция от x . Несколько алгебраических выкладок помогут лучше понять, как изменяется вероятность $p(y = 1)$ при изменении β .

Формула основной логистической модели:

$$\theta = \text{logistic}(\alpha + X\beta). \quad (4.11)$$

Обратной к логистической функции является логит-функция (logit):

$$\text{logit}(z) = \log\left(\frac{z}{1-z}\right). \quad (4.12)$$

Таким образом, если взять первую формулу в этом разделе и применить логит-функцию к обоим ее членам, то получится следующая формула:

$$\text{logit}(\theta) = \alpha + X\beta. \quad (4.13)$$

Или равнозначная ей формула:

$$\log\left(\frac{\theta}{1-\theta}\right) = \alpha + X\beta. \quad (4.14)$$

Вспомним, что θ в нашей модели равна $p(y = 1)$, тогда:

$$\log\left(\frac{p(y = 1)}{1 - p(y = 1)}\right) = \alpha + X\beta. \quad (4.15)$$

Значение под логарифмом $(p(y = 1))/(1 - p(y = 1))$ известно как шансы (odds).

Шансы на успех определяются как отношение вероятности успешного результата к вероятности неудачного результата. Вероятность выпадения 2 при броске симметричной игральной кости равна $1/6$, таким образом, шансы наступления этого события определяются как $(1/6)/(5/6) \approx 0.2$ или как одно благоприятное событие к пяти неблагоприятным событиям. Понятие шансов часто используют участники азартных игр, в основном потому, что шансы представляют собой более интуитивно понятный инструмент, чем чистые вероятности, когда речь идет о правильной методике ставок.



В логистической регрессии коэффициент β соответствует (кодирует) увеличению в единицах логарифма шансов по отношению к единице увеличения переменной x .

Преобразование вероятности в шансы является монотонным, то есть шансы возрастают при увеличении вероятности и уменьшаются, если вероятность снижается. Вероятность ограничена интервалом $[0, 1]$, в то время как шансы существуют в интервале $[0, \infty)$. Логарифмирование – это тоже монотонное преобразование, поэтому значения логарифмов шансов располагаются в интервале $(-\infty, +\infty)$. На рис. 4.6 показана взаимосвязь вероятностей с шансами и логарифмами шансов.

```
probability = np.linspace(0.01, 1, 100)
odds = probability / (1 - probability)

_, ax1 = plt.subplots()
ax2 = ax1.twinx()
ax1.plot(probability, odds, 'C0')
ax2.plot(probability, np.log(odds), 'C1')

ax1.set_xlabel('probability')
ax1.set_ylabel('odds', color='C0')
ax2.set_ylabel('log-odds', color='C1')
ax1.grid(False)
ax2.grid(False)
```

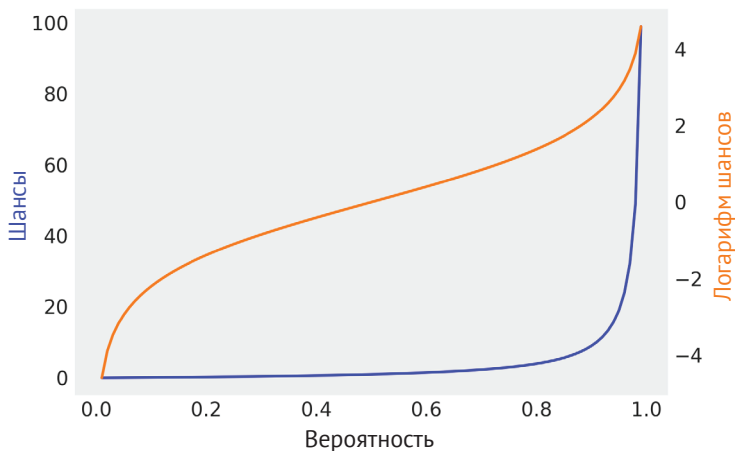


Рис. 4.6

Таким образом, значения коэффициентов, получаемых с помощью функции `summary`, указываются в масштабе логарифмов шансов.

```
df = az.summary(trace_1, var_names=varnames)
df
```

Таблица 4.2

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
α	-9.12	4.61	0.15	-17.55	-0.42	1353.0	1.0
$\beta[0]$	4.65	0.87	0.03	2.96	6.15	1268.0	1.0
$\beta[1]$	-5.16	0.95	0.01	-7.05	-3.46	1606.0	1.0

Весьма полезным практическим способом глубокого понимания моделей является изменение параметров и наблюдение за происходящим. В приведенном ниже фрагменте кода вычисляются логарифмы шансов для ириса разноцветного (versicolor) как $\log_odds_versicolor_i = \alpha + \beta_1 x_1 + \beta_2 x_2$, затем вероятность принадлежности к классу versicolor с помощью логистической функции. После этого вычисления повторяются при том же значении x_2 , но с увеличением значения x_1 на 1:

```
x_1 = 4.5 # sepal_length
x_2 = 3   # sepal_width

log_odds_versicolor_i = (df['mean'] * [1, x_1, x_2]).sum()
probability_versicolor_i = logistic(log_odds_versicolor_i)

log_odds_versicolor_f = (df['mean'] * [1, x_1 + 1, x_2]).sum()
probability_versicolor_f = logistic(log_odds_versicolor_f)

log_odds_versicolor_f - log_odds_versicolor_i, probability_versicolor_f - probability_versicolor_i
```

Если выполнить этот код, то можно видеть, что увеличение логарифма шансов ≈ 4.66 , то есть практически равно значению β_0 (проверьте по сводному отчету trace_1). Этот результат совпадает с предыдущим наблюдением и выводом о том, что коэффициент β кодирует увеличение в единицах логарифма шансов по отношению к единице увеличения переменной x . Увеличение вероятности ≈ 0.70 .

Обработка коррелирующих переменных

Из главы 3 известно, что при работе с переменными с (сильной) корреляцией могут возникать неожиданные ситуации. Коррелирующие переменные превращаются в широкое разнообразие сочетаний коэффициентов, предназначенных для объяснения данных. С несколько другой точки зрения коррелирующие данные обладают меньшим воздействием на ограничения модели. Аналогичная проблема возникает, когда классы становятся совершенно неразделяемыми, то есть когда не существует пересечений между классами при линейной комбинации переменных в выбранной модели.

Используя набор данных iris, можно попробовать выполнить модель model_1, но на этот раз с переменными petal_length и petal_width. После выполнения выяснится, что диапазон значений коэффициентов β расширился по сравнению

с предыдущим примером, а кроме того, черная полоса ПАР 94 % на рис. 4.5 также стала значительно шире:

```
corr = iris[iris['species'] != 'virginica'].corr()
mask = np.tri(*corr.shape).T
sns.heatmap(corr.abs(), mask=mask, annot=True, cmap='viridis')
```

На рис. 4.7 представлена тепловая карта, показывающая, что для переменных `sepal_length` и `sepal_width`, использованных в первом примере, корреляция не столь высока, как корреляция между переменными `petal_length` и `petal_width` во втором примере.

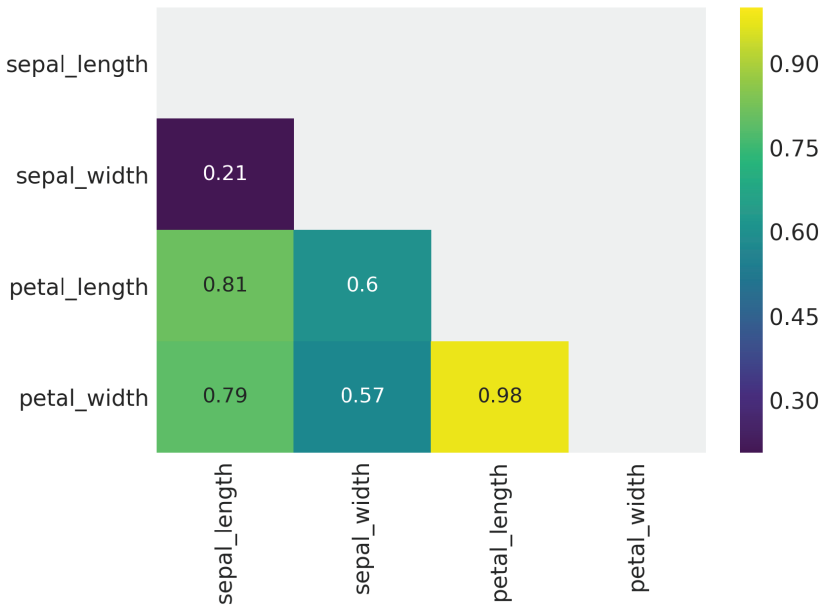


Рис. 4.7

При формировании диаграммы на рис. 4.7 использовалась маска для удаления верхнего треугольника и диагональных элементов тепловой карты, поскольку они не предоставляют полезной информации, в отличие от нижнего треугольника. Также отметим, что на диаграмме отображено абсолютное значение корреляции, поскольку в настоящий момент нас не интересует знак корреляции между переменными, а только ее величина.

Одним из вариантов решения при обработке переменных с (сильной) корреляцией является простое удаление одной коррелирующей переменной (или нескольких коррелирующих переменных). Другой вариант – передача большего объема информации в априорное распределение. Этого можно достичь, используя информативные априорные распределения, если мы располагаем

полезной информацией. Для слабоинформативных априорных распределений Эндрю Гелман (Andrew Gelman) и Стэн Тим (Stan Team) рекомендуют (<https://github.com/stan-dev/stan/wiki/Prior-Choice-Recommendations>) масштабирование всех небинарных переменных для получения среднего значения 0 и последующего использования t-распределения Стьюдента:

$$\beta \sim \text{StudentT}(0, v, \text{sd}). \quad (4.10)$$

Здесь стандартное отклонение sd должно выбираться так, чтобы предоставлять минимальную (слабую) информацию об ожидаемых значениях при установленном масштабе. Для параметра нормальности v предполагается значение в интервале 3–7. Такое априорное распределение в общем означает, что ожидаются малые значения коэффициентов, но при этом используется распределение с широкими хвостами, потому что могут встретиться и некоторые большие значения коэффициентов. Как мы уже видели в главе 3, применение t-распределения Стьюдента позволяет создать более надежную модель по сравнению с использованием гауссова распределения.

Работа с несбалансированными классами

Набор данных `iris` полностью сбалансирован в том смысле, что в каждой категории имеется абсолютно одинаковое число наблюдений: 50 ирисов щетиных, 50 ирисов разноцветных, 50 вероник виргинских. За это следует поблагодарить Роналда Фишера (Ronald Fisher), но только за это, а не за его деятельность по популяризации использования p-значений.

С другой стороны, многие наборы состоят из несбалансированных данных, то есть в них содержится намного больше точек данных одного класса по сравнению с другими классами. В такой ситуации логистическая регрессия может «впасть в панику», потому что границу невозможно определить с такой же точностью, как при исследовании более сбалансированного набора данных.

Для наблюдения этого поведения снова воспользуемся набором данных `iris`, но теперь удалим несколько случайно выбранных точек данных из класса `setosa`:

```
df = iris.query("species == ('setosa', 'versicolor')")
df = df[45:]
y_3 = pd.Categorical(df['species']).codes
x_n = ['sepal_length', 'sepal_width']
x_3 = df[x_n].values
```

Затем выполним модель множественной логистической регрессии так же, как в предыдущих примерах:

```
with pm.Model() as model_3:
    alpha = pm.Normal('alpha', mu=0, sd=10)
    beta = pm.Normal('beta', mu=0, sd=2, shape=len(x_n))
    mu = alpha + pm.math.dot(x_3, beta)
    theta = 1 / (1 + pm.math.exp(-mu))
```



```

bd = pm.Deterministic('bd', -a/β[1] - β[0]/β[1] * x_3[:,0])
yl = pm.Bernoulli('yl', p=θ, observed=y_3)

trace_3 = pm.sample(1000)

```

На рис. 4.8 можно видеть, что граница решения теперь сдвинута в направлении класса с меньшим количеством точек данных, а неопределенность выше, чем раньше. Это обычное поведение логистической модели при несбалансированных данных. Но не следует спешить с выводами. Вы можете возразить, что я здесь схитрил, поскольку более широкая зона неопределенности могла стать результатом уменьшения общего количества точек данных, а не только уменьшением числа членов класса *setosa* относительно класса *versicolor*. Возможно, это вполне допустимая точка зрения, можно попытаться проверить ее, выполнив упражнение 6 к этой главе. Проверим также, что объясняет следующая диаграмма для несбалансированных данных:

```

idx = np.argsort(x_3[:,0])
bd = trace_3['bd'].mean(0)[idx]
plt.scatter(x_3[:,0], x_3[:,1], c=[f'C{x}' for x in y_3])
plt.plot(x_3[:,0][idx], bd, color='k')

az.plot_hpd(x_3[:,0], trace_3['bd'], color='k')

plt.xlabel(x_n[0])
plt.ylabel(x_n[1])

```

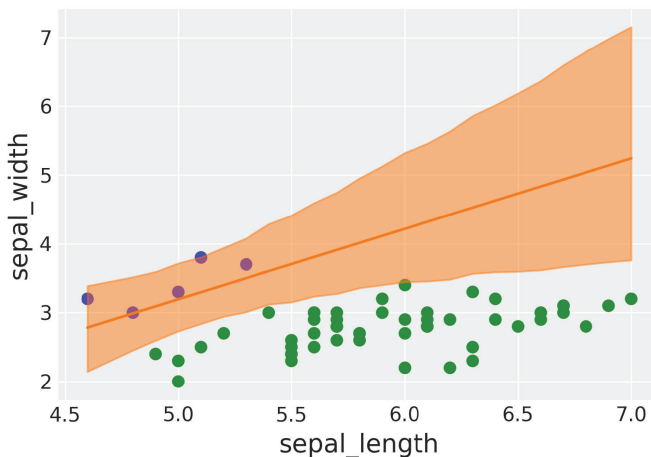


Рис. 4.8

Что делать при обнаружении несбалансированных данных? Самым очевидным выглядит получение набора данных с приблизительно одинаковым числом точек данных в каждом классе. Об этом важно помнить при сборе или генерации данных. Но если нет возможности управлять формированием набора данных, то потребуется особая внимательность при интерпретации несба-

лансированных данных. Выполните проверку неопределенности выбранной модели, а также несколько проверок прогнозируемого апостериорного распределения, чтобы увидеть, будут ли действительно полезными результаты. Другой вариант решения – ввод большего объема информации в априорное распределение, если это возможно, и/или использование альтернативной модели, о чем речь пойдет немного позже в этой же главе.

Регрессия с использованием функции softmax

Одним из способов обобщения логистической регрессии для более чем двух классов является регрессия с применением функции softmax. Необходимо внести два изменения по сравнению с логистической регрессией: во-первых, заменить логистическую функцию на функцию softmax:

$$\text{softmax}_i(\mu) = \frac{\exp(\mu_i)}{\sum \exp(\mu_k)}. \quad (4.16)$$

Другими словами, для получения выходных данных функции softmax для i -го элемента вектора μ берется экспонента от значения μ_i и делится на сумму экспонент всех значений, содержащихся в векторе μ .

Функция softmax гарантирует получение положительных значений, которые в сумме дают 1. Эта функция представляет собой сокращенную версию логистической функции при $k = 2$. Дополнительное замечание: функция softmax имеет ту же форму, что и распределение Больцмана, используемое в статистической механике – весьма значимом разделе физики, занимающимся вероятностным описанием атомных и молекулярных систем. Распределение Больцмана¹ (и softmax в некоторых областях) имеет параметр, называемый температурой T , на который делится μ . Когда $T \rightarrow \infty$, распределение вероятности становится плоским (равномерным) и все состояния одинаково вероятны, а когда $T \rightarrow 0$, заполняется только наиболее вероятное состояние, следовательно, softmax начинает вести себя как функция max .

Второе изменение заключается в замене распределения Бернулли на категориальное распределение. Категориальное распределение – это обобщение распределения Бернулли для получения более двух выходных результатов. Кроме того, так же как распределение Бернулли (подбрасывание одной монеты) представляет собой особый случай биномиального распределения (подбрасывание n монет), так и категориальное распределение (один бросок игральной кости) является особым случаем мультиномиального распределения (n бросков игральной кости). Попробуйте предложить эту головоломку своим родственникам.

Чтобы привести пример практического применения регрессии с использованием функции softmax, продолжим работу с набором данных *iris*, но в этот раз воспользуемся тремя классами (*setosa*, *versicolor*, *virginica*) и четырьмя

¹ Чаше его называют распределением Максвелла–Больцмана. – *Прим. перев.*

признаками (sepal length, sepal width, petal length и petal width). Также будет выполнена стандартизация данных, поскольку это поможет сэмплеру работать более эффективно (это еще можно сделать с помощью простого центрирования данных):

```
iris = sns.load_dataset('iris')
y_s = pd.Categorical(iris['species']).codes
x_n = iris.columns[:-1]
x_s = iris[x_n].values
x_s = (x_s - x_s.mean(axis=0)) / x_s.std(axis=0)
```

В этом коде, написанном с использованием библиотеки PyMC3, отображены различия между логистической и softmax моделями. Обратите внимание на формы коэффициентов α и β . Мы используем функцию softmax из библиотеки Theano, то есть `import theano.tensor` как идиому `tt`, в соответствии с соглашением, принятым разработчиками библиотеки PyMC3.

```
with pm.Model() as model_s:
     $\alpha$  = pm.Normal('a', mu=0, sd=5, shape=3)
     $\beta$  = pm.Normal('b', mu=0, sd=5, shape=(4,3))
     $\mu$  = pm.Deterministic('mu',  $\alpha$  + pm.math.dot(x_s,  $\beta$ ))
     $\theta$  = tt.nnet.softmax( $\mu$ )
    y_l = pm.Categorical('yl', p= $\theta$ , observed=y_s)
    trace_s = pm.sample(2000)
```

Проверим, насколько эффективна созданная модель. Для этого определим количество правильных прогнозов. В следующем фрагменте кода используются только средние значения параметров для вычисления вероятности каждой точки данных, принадлежащей одному из трех классов, затем присваивается класс с помощью функции `argmax`. Результаты сравниваются с наблюдаемыми значениями.

```
data_pred = trace_s['mu'].mean(0)
y_pred = [np.exp(point)/np.sum(np.exp(point), axis=0)
for point in data_pred]
f'{np.sum(y_s == np.argmax(y_pred, axis=1)) / len(y_s):.2f}'
```

В итоге приблизительно 98 % точек данных классифицированы правильно, то есть промахи произошли только в трех случаях. Это очень хороший результат. Но настоящая проверка эффективности этой модели будет выполнена на данных, которые ей пока еще неизвестны. В противном случае может иметь место переоценка возможностей модели с точки зрения обобщения для других данных. Эта тема более подробно будет рассматриваться в главе 5. Но сейчас мы просто выполним тест на автоматическую целостность, который подтвердит эффективность модели.

Возможно, вы заметили, что апостериорные распределения, или – более точно – граничные распределения, каждого параметра чрезвычайно широки. В действительности их ширина определяется априорными распределениями. Даже при возможности выполнять правильные прогнозы это выглядит не

слишком хорошо. Возникла та же проблема неидентифицируемости, с которой мы уже сталкивались при обработке коррелирующих данных в других моделях регрессии или при работе с абсолютно разделенными (непересекающимися) классами. В этом случае слишком широкое апостериорное распределение является следствием условия, по которому все вероятности должны быть просуммированы, чтобы получить одно значение вероятности. Принимая это условие, мы используем больше параметров, чем необходимо для полного определения модели. Проще говоря, если имеется десять чисел, которые суммируются для получения одного результата, то мне необходимо передать только девять из них, десятое я могу вычислить. Вариантом решения этой проблемы является присваивание лишним параметрам некоторого фиксированного значения, например нуля. В приведенном ниже фрагменте кода демонстрируется этот способ с использованием библиотеки PyMC3:

```
with pm.Model() as model_sf:
    α = pm.Normal('α', mu=0, sd=2, shape=2)
    β = pm.Normal('β', mu=0, sd=2, shape=(4,2))
    α_f = tt.concatenate([[0], α])
    β_f = tt.concatenate([np.zeros((4,1)) , β], axis=1)
    μ = α_f + pm.math.dot(x_s, β_f)
    θ = tt.nnet.softmax(μ)
    y_l = pm.Categorical('y_l', p=θ, observed=y_s)
    trace_sf = pm.sample(1000)
```

Дискриминативные и порождающие модели

До настоящего момента мы рассматривали логистическую регрессию и некоторые ее расширения. Во всех случаях мы пытались напрямую вычислить вероятность $p(y|x)$, то есть вероятность выбора определенного класса при известном x , где x – признак (или несколько признаков), который измеряется (оценивается) для членов этих классов. Другими словами, выполнялась попытка прямого моделирования отображения независимых переменных в зависимые переменные с последующим использованием порогового значения для преобразования непрерывно вычисляемой вероятности в дискретную границу, которая позволяет распределять данные по классам.

Такая методика не оригинальна. Другой вариант – сначала моделируется $p(x|y)$, то есть распределение x для каждого класса, затем данные распределяются по классам. Этот тип модели называется порождающим классификатором (generative classifier), так как модель создается на основе возможности генерировать (порождать) выборки данных из каждого класса. С другой стороны, логистическая регрессия представляет собой тип дискриминативного классификатора (discriminative classifier), поскольку такая модель пытается различить (discriminate) классы, но при этом сохраняется возможность генерации выборок данных из каждого класса модели. Сейчас мы не будем подробно изучать возможности применения порождающих моделей как классификаторов, но рассмотрим один пример, демонстрирующий основной принцип

использования этого типа модели для классификации. В пример включены два класса и только один признак, как и в самой первой модели, созданной в текущей главе (`model_0`), кроме того, и данные взяты те же самые.

Ниже показана реализация порождающего классификатора с использованием библиотеки PyMC3. В этом коде можно видеть, что теперь граница решения определена как среднее арифметическое по оцениваемым гауссовым средним значениям. Это правильная граница решения, когда распределения нормальные и их стандартные отклонения равны. Такие предварительные предположения соответствуют модели, называемой линейным дискриминантным анализом (ЛДА). Несмотря на характерное название, модель ЛДА является порождающей.

```
with pm.Model() as lda:
    μ = pm.Normal('μ', mu=0, sd=10, shape=2)
    σ = pm.HalfNormal('σ', 10)
    setosa = pm.Normal('setosa', mu=μ[0], sd=σ, observed=x_0[:50])
    versicolor = pm.Normal('versicolor', mu=μ[1], sd=σ, observed=x_0[50:])
    bd = pm.Deterministic('bd', (μ[0] + μ[1]) / 2)
    trace_lda = pm.sample(1000)
```

Теперь необходимо сформировать диаграмму, отображающую два класса (`setosa = 0` и `versicolor = 1`) вместе со значениями длины чашелистиков (`sepal_length`), а также границу решения в виде красной линии и соответствующий интервал плотности апостериорного распределения (ПАР) 94 % в виде полупрозрачной красной полосы.

```
plt.axvline(trace_lda['bd'].mean(), ymax=1, color='C1')
bd_hpd = az.hpd(trace_lda['bd'])
plt.fill_betweenx([0, 1], bd_hpd[0], bd_hpd[1], color='C1', alpha=0.5)

plt.plot(x_0, np.random.normal(y_0, 0.02), '.', color='k')
plt.ylabel('θ', rotation=0)
plt.xlabel('sepal_length')
```

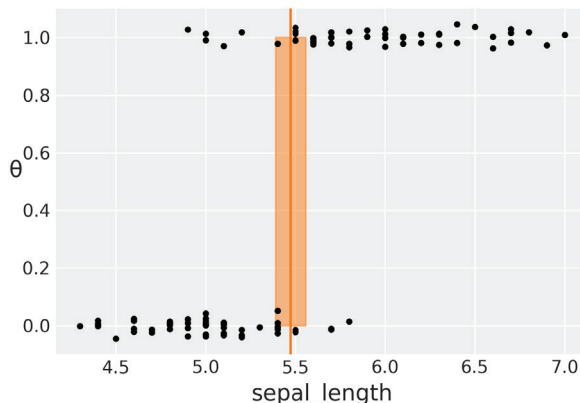


Рис. 4.9

Если сравнить рис. 4.9 с рис. 4.4, то они покажутся почти одинаковыми. Но проверим также значения границы решения в следующем итоговом отчете:

```
az.summary(trace_lda)
```

Таблица 4.3

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
μ[0]	5.01	0.06	0.0	4.89	5.13	2664.0	1.0
μ[1]	5.94	0.06	0.0	5.82	6.06	2851.0	1.0
σ	0.45	0.03	0.0	0.39	0.51	2702.0	1.0
bd	5.47	0.05	0.0	5.39	5.55	2677.0	1.0

Модель ЛДА и логистическая регрессия дают практически одинаковые результаты. Линейную дискриминантную модель можно расширить для использования более одного признака, если моделировать классы как многомерные гауссовы распределения. Кроме того, можно сделать менее строгим предварительное предположение о том, что все классы совместно используют общую дисперсию (или ковариацию). В результате мы приходим к модели квадратичного линейного дискриминантного анализа (КДА), поскольку теперь граница решения не линейная, а квадратичная (парабола или параболическая поверхность).

В общем модели ЛДА и КДА работают лучше, чем логистическая регрессия, когда используемые признаки являются более или менее распределенными по Гауссу, в противном случае более эффективна логистическая регрессия. Одним из преимуществ порождающей модели для классификации является то, что в нее проще и естественнее вводится априорная информация. Например, в нашем распоряжении может находиться информация о среднем значении и дисперсии данных, передаваемых в модель.

Важно отметить, что границы решений ЛДА и КДА имеют замкнутую форму, следовательно, обычно вычисляются соответствующим способом. При использовании ЛДА для двух классов и одного признака необходимо вычислить только среднее значение каждого распределения и найти среднее арифметическое этих двух значений – это и будет граница решения. В предыдущей рассматриваемой модели мы так и поступили, но с поворотом в сторону байесовского анализа. Параметры оценивались по двум гауссовым распределениям, затем две полученные оценки были переданы в предварительно определенную формулу. Откуда взялась эта формула? Если оставить в стороне слишком мелкие подробности, то для получения этой формулы необходимо допустить, что данные являются распределенными по Гауссу, следовательно, эта формула будет работать, только если данные не существенно отклоняются от нормальности. Разумеется, может возникнуть проблема, если необходимо смягчить допущение о нормальности, как, например, с помощью использования

t-распределения Стюдента (или многомерного t-распределения Стюдента, или какого-либо другого распределения). В этом случае мы уже не можем воспользоваться замкнутой формой для модели ЛДА (или КДА), тем не менее мы можем продолжать вычисление границы решения в числовом виде с применением библиотеки Руним3.

РЕГРЕССИЯ ПУАССОНА

Еще одной весьма распространенной обобщенной линейной моделью является регрессия Пуассона. Эта модель предполагает, что данные распределены, как можно легко догадаться, в соответствии с распределением Пуассона.

Один из вариантов полезного применения распределения Пуассона – подсчет объектов, например: количество частиц при распаде радиоактивных ядер, количество детей у супружеской пары, количество подписчиков в «Твиттере». Все эти (и многие другие примеры подсчета) объединяет то, что они обычно моделируются с использованием дискретных неотрицательных чисел: {0, 1, 2, 3, ...}. Этот тип переменной получил название дискретные, или счетные, данные (count data).

Распределение Пуассона

Предположим, что мы подсчитываем количество красных машин, проезжающих по некоторой улице за один час. Для описания этих данных можно было бы воспользоваться распределением Пуассона. Вообще, распределение Пуассона применяется для описания вероятности определенного числа событий, возникающих в фиксированном временном или пространственном интервале. Таким образом, распределение Пуассона предполагает, что события происходят независимо друг от друга и возникают в строго определенном интервале времени и/или пространства. Это дискретное распределение параметризуется с использованием только одного значения μ (средняя частота возникновения событий, которую часто обозначают греческой буквой λ). Параметр μ соответствует среднему значению, а также дисперсии этого распределения. Функция вероятности (probability mass function – PMF) распределения Пуассона определяется следующей формулой:

$$f(x|\mu) = \frac{e^{-\mu} \mu^x}{x!}. \quad (4.17)$$

Здесь:

- μ – среднее арифметическое количества событий в единицу времени/пространства;
- x – положительное целое значение {0, 1, 2, ...};
- $x!$ – факториал x , то есть $k! = k \times (k - 1) \times (k - 2) \times \dots \times 2 \times 1$.

На рис. 4.10 можно наблюдать несколько примеров из семейства распределения Пуассона для различных значений μ :

```

mu_params = [0.5, 1.5, 3, 8]
x = np.arange(0, max(mu_params) * 3)
for mu in mu_params:
    y = stats.poisson(mu).pmf(x)
    plt.plot(x, y, 'o-', label=f'μ = {mu:3.1f}')
plt.legend()
plt.xlabel('x')
plt.ylabel('f(x)')

```

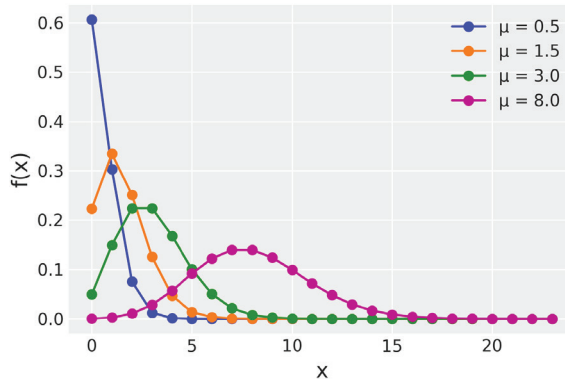


Рис. 4.10

Отметим, что μ может быть числом с плавающей точкой, но выходные данные распределения Пуассона всегда являются целыми числами. На рис. 4.10 точки представляют значения этого распределения, в то время как непрерывные линии выполняют вспомогательную роль, помогая без затруднений узнать форму конкретного распределения. Следует всегда помнить о том, что распределение Пуассона – дискретное распределение.

Распределение Пуассона может рассматриваться как особый случай биномиального распределения, когда число испытаний n весьма велико, но вероятность успешного исхода p чрезвычайно мала. Попробуем объяснить это утверждение без углубления в математические подробности. Возвращаясь к примеру с подсчетом машин, мы можем утверждать, что видим красную машину или не видим ее, следовательно, здесь можно использовать биномиальное распределение. Тогда получаем следующую формулу:

$$x \sim \text{Bin}(n, p). \quad (4.18)$$

Далее среднее значение биномиального распределения вычисляется следующим образом:

$$E[x] = np. \quad (4.19)$$

Дисперсия:

$$V[x] = np(1 - p). \quad (4.20)$$

Но следует отметить, что даже если вы находитесь на улице с очень интенсивным движением, шанс увидеть красную машину по сравнению с общим количеством машин в городе весьма мал, следовательно, получаем:

$$n \gg p \Rightarrow np \approx np(1 - p). \quad (4.21)$$

Поэтому можно применить следующую приближенную формулу:

$$V[x] = np. \quad (4.22)$$

Теперь среднее значение и дисперсия представлены одним и тем же числом, поэтому можно с уверенностью утверждать, что исследуемая переменная определяется распределением Пуассона:

$$x \sim \text{Poisson}(\mu = np). \quad (4.23)$$

Модель Пуассона с дополнением нулевыми значениями

При подсчете объектов одним из возможных вариантов является отсутствие объекта счета, то есть получение нулевого значения. Ноль может встречаться по нескольким причинам: красные машины не проезжали по наблюдаемой улице или мы не заметили их (возможно, красная машина быстро проехала за большим грузовиком-трейлером). Поэтому при использовании распределения Пуассона вы заметите, например при выполнении проверки прогнозируемого апостериорного распределения, что модель генерирует меньше нулей по сравнению со значимыми данными. Как исправить эту ситуацию? Можно попытаться найти точную причину, по которой модель прогнозирует меньше нулей, чем наблюдается в действительности, и включить этот фактор в модель. Но чаще применяется вариант, при котором для наших целей достаточно просто предположить, что наблюдается смешанное выполнение двух процессов:

- процесс, моделируемый распределением Пуассона с вероятностью ψ ;
- процесс, генерирующий дополнительные нули с вероятностью $1 - \psi$.

Это модель Пуассона с дополнением нулевыми значениями (zero-inflated Poisson – ZIP). В некоторых публикациях определяется, что ψ представляет дополнительные нули, а $1 - \psi$ – вероятность распределения Пуассона. Это не существенно, главное – помнить о том, что есть что, при решении конкретной задачи.

В общем виде модель Пуассона с дополнением нулевыми значениями выглядит так:

$$p(u_i = 0) = 1 - \psi + (\psi)e^{-\mu_i}; \quad (4.24)$$

$$p(y_j = k_i) = \psi \frac{\mu_i^{k_i} e^{-\mu_i}}{k_i!}. \quad (4.25)$$

Здесь $1 - \psi$ – вероятность дополнительных нулевых значений.

Чтобы рассмотреть пример применения распределения Пуассона с дополнением нулевыми значениями, создадим несколько искусственно сгенерированных точек данных:

```

n = 100
θ_real = 2.5
ψ = 0.1
# Имитация некоторых случайных данных
counts = np.array([(np.random.random() > (1-ψ)) * np.random.poisson(θ_real) for i in
range(n)])

```

Можно было бы с легкостью реализовать формулы 4.24 и 4.25 в модели с применением методов библиотеки РумСЗ. Но это можно сделать даже проще: воспользуемся реализацией распределения Пуассона с дополнением нулевыми значениями из той же библиотеки РумСЗ:

```

with pm.Model() as ZIP:
    ψ = pm.Beta('ψ', 1, 1)
    θ = pm.Gamma('θ', 2, 0.1)
    y = pm.ZeroInflatedPoisson('y', ψ, θ, observed=counts)
    trace = pm.sample(1000)

```

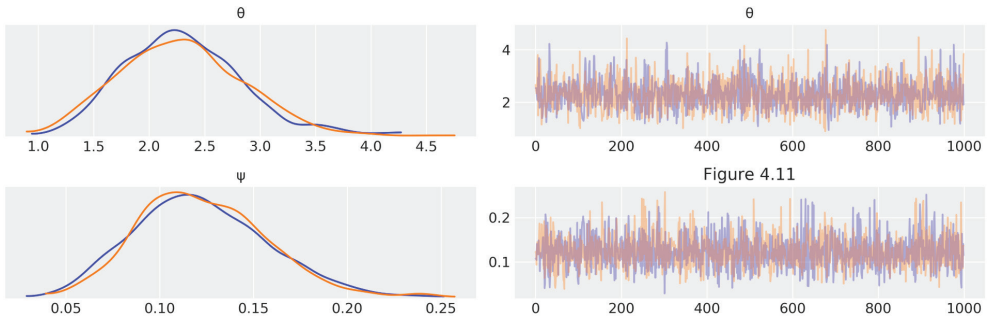


Рис. 4.11

Регрессия Пуассона и модель Пуассона с дополнением нулевыми значениями

Модель Пуассона с дополнением нулевыми значениями может показаться немного непонятной, но иногда необходимо давать оценку простым распределениям, таким как в приведенном выше примере, или таким как распределения Пуассона или Гаусса. В любом случае, распределение Пуассона или модель Пуассона с дополнением нулевыми значениями можно использовать как часть линейной модели. Как уже наблюдалось при изучении логистической регрессии и softmax, можно воспользоваться функцией обратной связи для преобразования результата выполнения линейной модели в переменную с диапазоном значений, более подходящим для использования в других распределениях, отличающихся от нормального. Следуя этому принципу, теперь можно выполнять регрессионный анализ, когда выходная переменная представляет собой счетную (дискретную) переменную, и использовать для этого распределение Пуассона или модель Пуассона с дополнением нулевыми значениями. В этот

раз в качестве функции обратной связи можно взять экспоненциальную функцию e^x . Такой выбор обеспечивает возврат из линейной модели только положительных значений:

$$\theta = e^{(\alpha + X\beta)}. \quad (4.26)$$

Для примера реализации модели Пуассона с дополнением нулевыми значениями будем работать с набором данных, предоставляемым отделением калифорнийского университета в Лос-Анджелесе Institute for Digital Research and Education (<http://www.ats.ucla.edu/stat/data>). Имеется 250 групп посетителей парка. Для каждой группы собраны некоторые фрагменты данных:

- количество пойманных рыб (count);
- количество детей в группе (child);
- факт приезда в парк на туристическом автофургоне (camper).

Используя эти данные, создадим модель, прогнозирующую количество пойманных рыб как функцию от переменных, определяющих количество детей и факт приезда в парк на туристическом автофургоне. Для загрузки данных можно воспользоваться библиотекой `pandas`:

```
fish_data = pd.read_csv('../data/fish.csv')
```

В качестве самостоятельного упражнения предлагается выполнить исследование этого набора данных с помощью диаграмм и/или функций библиотеки `pandas`, например `describe`. Но сейчас продолжим реализацию модели ZIP_reg с использованием библиотеки `PyMC3`:

```
with pm.Model() as ZIP_reg:
    ψ = pm.Beta('ψ', 1, 1)
    α = pm.Normal('α', 0, 10)
    β = pm.Normal('β', 0, 10, shape=2)
    θ = pm.math.exp(α + β[0] * fish_data['child'] + β[1] * fish_data['camper'])
    y_l = pm.ZeroInflatedPoisson('y_l', ψ, θ, observed=fish_data['count'])
    trace_ZIP_reg = pm.sample(1000)
```

Переменная `camper` является бинарной, то есть принимающей значение 0, если посетители прибыли без туристического автофургона, и 1, если посетители приехали на туристическом автофургоне. Переменная, обозначающая наличие/отсутствие какого-либо атрибута, обычно называется фиктивной переменной (dummy variable) или характеристической переменной (indicator variable). Отметим, что когда переменная `camper` принимает значение 0, член, содержащий коэффициент β_1 , также будет равен 0, и модель сводится к регрессии с одной независимой переменной.

Чтобы лучше понять результаты этого статистического вывода, построим диаграмму:

```
children = [0, 1, 2, 3, 4]
fish_count_pred_0 = []
fish_count_pred_1 = []
for n in children:
    without_camper = trace_ZIP_reg['α'] + trace_ZIP_reg['β'][:,0] * n
```

```

with_camper = without_camper + trace_ZIP_reg['β'][:,1]
fish_count_pred_0.append(np.exp(without_camper))
fish_count_pred_1.append(np.exp(with_camper))
plt.plot(children, fish_count_pred_0, 'C0.', alpha=0.01)
plt.plot(children, fish_count_pred_1, 'C1.', alpha=0.01)

plt.xticks(children);
plt.xlabel('Number of children')
plt.ylabel('Fish caught')
plt.plot([], 'C0o', label='without camper')
plt.plot([], 'C1o', label='with camper')
plt.legend()

```

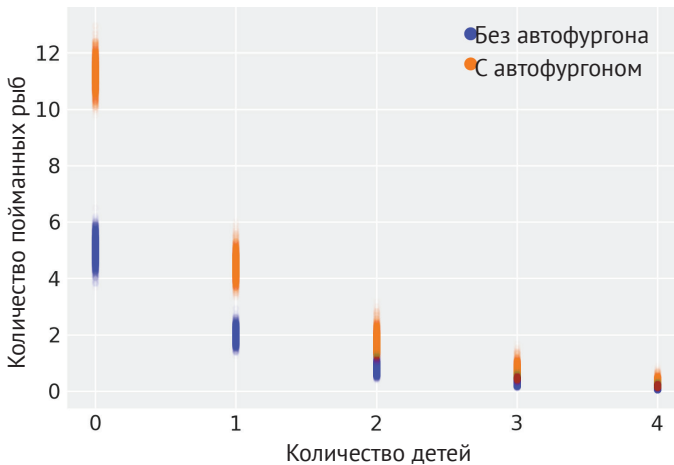


Рис. 4.12

На рис. 4.12 можно видеть, что чем больше количество детей, тем меньше поймано рыбы. Кроме того, прибывшие на собственном туристическом автофургоне в целом вылавливают больше рыбы. Если проверить коэффициенты β для количества детей и наличия автофургона, то можно утверждать следующее:

- для каждого дополнительного ребенка ожидаемое количество пойманных рыб уменьшается на ≈ 0.4 ;
- наличие туристического автофургона увеличивает ожидаемое количество пойманных рыб на ≈ 2 .

Эти значения получены при взятии экспоненты от коэффициентов β_1 и β_2 соответственно.

РОБАСТНАЯ ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ

В предыдущем разделе рассматривалась методика корректировки избыточных нулевых значений без прямого моделирования фактора, порождающего эту проблему. Похожая методика, предложенная Крушке (Kruschke), может

применяться для выполнения более надежной (робастной) версии логистической регрессии. Напомним, что при использовании логистической регрессии мы моделируем данные как биномиальные, то есть нули и единицы. Поэтому может возникнуть ситуация, когда в наборе данных обнаружатся ненужные нули и/или единицы. Для примера воспользуемся уже знакомым набором данных `iris`, но добавим в него несколько чужеродных экземпляров:

```
iris = sns.load_dataset("iris")
df = iris.query("species == ('setosa', 'versicolor')")
y_0 = pd.Categorical(df['species']).codes
x_n = 'sepal_length'
x_0 = df[x_n].values
y_0 = np.concatenate((y_0, np.ones(6, dtype=int)))
x_0 = np.concatenate((x_0, [4.2, 4.5, 4.0, 4.3, 4.2, 4.4]))
x_c = x_0 - x_0.mean()
plt.plot(x_c, y_0, 'o', color='k');
```

Здесь мы получили несколько ирисов многоцветных (значения 1) с преднамеренно укороченной (то есть неправильной) длиной чашелистика. Эту ситуацию можно исправить с помощью смешанной модели. Можно сказать, что значение выходной переменной получается с вероятностью π при случайном угадывании (предсказании) или с вероятностью $1 - \pi$ из модели логистической регрессии. Это утверждение записывается математически следующим образом:

$$p = \pi 0.5 + (1 - \pi) \text{logistic}(\alpha + X\beta). \quad (4.26)$$

Отметим, что при $\pi = 1$ получаем $p = 0.5$, а при $\pi = 0$ возвращаемся к формуле логистической регрессии. Реализация этой модели представляет собой очевидную модификацию самой первой модели, рассматриваемой в текущей главе:

```
with pm.Model() as model_rlg:
    alpha = pm.Normal('alpha', mu=0, sd=10)
    beta = pm.Normal('beta', mu=0, sd=10)
    mu = alpha + x_c * beta
    theta = pm.Deterministic('theta', pm.math.sigmoid(mu))
    bd = pm.Deterministic('bd', -alpha/beta)
    pi = pm.Beta('pi', 1., 1.)
    p = pi * 0.5 + (1 - pi) * theta
    y_l = pm.Bernoulli('y_l', p=p, observed=y_0)

    trace_rlg = pm.sample(1000)
```

Если сравнить полученные здесь результаты с результатами модели `model_0` (первой модели в начале главы), то можно видеть, что получена почти та же самая граница решения. Чтобы убедиться в этом, сравните рис. 4.13 с рис. 4.4.

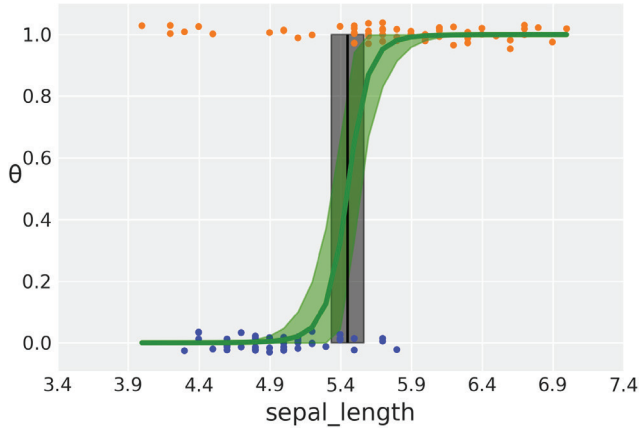


Рис. 4.13

Возможно, вы захотите самостоятельно сформировать итоговый отчет по моделям `model_0` и `model_glg` для сравнения значений границ в каждой модели.

Модуль GLM

В начале главы отмечалось, что линейные модели являются чрезвычайно полезными статистическими инструментами. Расширения, рассмотренные в текущей главе, делают эти инструменты еще более обобщенными, то есть широко применимыми. Именно поэтому в библиотеку PyMC3 включен специальный модуль для упрощения создания линейных моделей – модуль обобщенной линейной модели GLM (Generalized Linear Model). Например, простая линейная регрессия создается так:

```
with pm.Model() as model:
    glm.glm('y ~ x', data)
    trace = sample(2000)
```

В приведенном выше фрагменте кода вторая строка отвечает за добавление априорных распределений для свободного члена и коэффициента угла наклона. По умолчанию для свободного члена назначается плоское (линейное) априорное распределение, а для коэффициента угла наклона априорное распределение определяется как $N(0, 1 \times 10^6)$. Отметим, что оценка апостериорного максимума (maximum a posteriori – MAP) в модели по умолчанию в точности равна оценке, полученной с использованием обыкновенного метода наименьших квадратов. Это вполне соответствует используемой по умолчанию линейной регрессии, но принимаемые по умолчанию параметры можно изменить с помощью аргумента `priors`. Кроме того, модуль GLM по умолчанию добавляет гауссово правдоподобие. Его можно изменить, воспользовавшись аргументом

family и выбрав один из следующих возможных вариантов: Normal (по умолчанию), StudentT, Binomial, Poisson или NegativeBinomial.

Для описания статистической модели модуль GLM использует Patsy (<https://patsy.readthedocs.io/en/latest/index.html>) – библиотеку Python, которая предоставляет синтаксис мини-языка формул, похожий на язык, используемый в программных средах R и S. В приведенном выше фрагменте кода выражение $y \sim x$ определяет, что выходную переменную y необходимо оценивать как линейную функцию от x .

РЕЗЮМЕ

Основная идея, обсуждаемая в этой главе, достаточно проста: для прогнозирования среднего значения выходной переменной можно применять произвольную функцию для линейной комбинации входных переменных. Об этом уже говорилось в начале главы, но это настолько важно, что заслуживает повторения. Такая произвольная функция называется функцией обратной связи. Единственное ограничение при выборе этой функции – результат должен быть подходящим для использования в качестве параметра распределения выборки (в общем случае – среднего значения). Функция обратной связи может потребоваться при работе с категориальными переменными, или когда данные могут принимать только положительные значения, или если необходима переменная, значения которой ограничены интервалом $[0, 1]$. Все перечисленные случаи становятся различными моделями, многие из них постоянно используются как статистические инструменты, а их практические приложения и статистические свойства изучаются.

Модель логистической регрессии – это обобщение модели линейной регрессии из предыдущей главы. Логистическая регрессия может использоваться для классификации или в более общих целях для прогнозирования биномиальных данных. Отличительный признак логистической регрессии – использование логистической функции как функции обратной связи и распределения Бернулли как правдоподобия. Использование функции обратной связи вводит нелинейность, которую необходимо учитывать при интерпретации коэффициентов логистической регрессии. Коэффициент обозначает увеличение в логарифмических единицах отношения шансов на единицу увеличения значения переменной x .

Одной из методик обобщения логистической регрессии для обработки более двух классов является регрессия с использованием функции softmax. В этом случае функцией обратной связи становится функция softmax – обобщение логистической функции для многомерного случая (более двух значений), а в качестве правдоподобия используется категориальное распределение.

Логистическая регрессия и softmax – это примеры дискриминативных моделей, когда выполняется попытка моделирования $p(y|x)$ без явного моделирования $p(x)$. Порождающая модель для классификации сначала моделирует $p(x|y)$,

то есть распределение x для каждого класса y , затем выполняет присваивание классов. Этот тип модели называют порождающим классификатором, потому что создается модель, которая может генерировать выборки из каждого класса. Мы рассмотрели один пример порождающего классификатора с использованием гауссовых распределений.

Набор данных `iris` использовался для демонстрации практического применения всех вышеперечисленных моделей, а также при кратком рассмотрении коррелирующих переменных, абсолютно разделяемых и несбалансированных классов.

Еще одной широко используемой обобщенной линейной моделью является регрессия Пуассона. Эта модель предполагает, что данные характеризуются распределением Пуассона, а функция обратной связи представлена экспоненциальной функцией. Распределение и регрессия Пуассона удобны для моделирования счетных (дискретных) данных, то есть данные принимают только неотрицательные целые значения, получаемые по итогам подсчета, а не ранжирования. Большинство распределений связано друг с другом. Один из примеров – гауссово распределение и t -распределение Стьюдента. Еще одним примером является распределение Пуассона, которое можно рассматривать как особый случай биномиального распределения, когда число испытаний очень велико, а вероятность успешного исхода чрезвычайно мала.

Полезным способом расширения распределения Пуассона является модель Пуассона с дополнительными нулевыми значениями (ZIP), которую можно интерпретировать как сочетание двух других распределений: распределения Пуассона и бинарное распределение, генерирующее дополнительные нули. Еще одно полезное расширение, которое можно интерпретировать как комбинированное, – отрицательное биномиальное распределение – в этом случае объединяются распределения Пуассона, где норма или отношение (μ) является случайной переменной с гамма-распределением. Отрицательное биномиальное распределение – полезная альтернатива распределению Пуассона, когда исследуются данные с очень большим разбросом (дисперсией), то есть для таких данных дисперсия больше их среднего значения.

УПРАЖНЕНИЯ

1. Повторно выполните вычисления по самой первой модели с использованием переменных длины лепестка (`petal_length`) и ширины лепестка (`petal_width`). Какие основные различия наблюдаются в результатах? Насколько широк или узок интервал ПАР 94 % в каждом случае?
2. Повторите выполнение упражнения 1, но теперь используйте t -распределение Стьюдента как мало (слабо) информативное априорное распределение. Поэкспериментируйте с разными значениями v .
3. Вернитесь к первому примеру использования логистической регрессии для классификации ириса щетилистного и ириса многоцветного по дли-

не чашелистика. Попробуйте решить ту же задачу с использованием модели простой линейной регрессии, которая рассматривалась в главе 3. Насколько хорошо подходит здесь линейная регрессия по сравнению с логистической регрессией? Можно ли полученный результат интерпретировать как вероятность? Совет-подсказка: проверьте, не ограничены ли значения у интервалом $[0, 1]$.

4. В примере из раздела «Интерпретация коэффициентов логистической регрессии» мы изменили значение переменной `sepal_length` на единицу. Используя рис. 4.6, подтвердите, что значение `log_odds_versicolor_i` соответствует значению `probability_versicolor_i`. Если принять во внимание только тот факт, что значение `log_odds_versicolor_i` отрицательно, то что можно сказать о вероятности? Рисунок 4.6 поможет вам ответить на этот вопрос. Понятен ли вам этот результат с учетом определения логарифма отношения шансов?
5. Используйте пример из предыдущего упражнения. Для модели `model_1` проверьте, на какую величину изменяется логарифм отношения шансов при увеличении значения `sepal_length` с 5.5 до 6.5 (подсказка: должно получиться значение 4.66). На какую величину изменяется вероятность? Сравните это увеличение с вариантом, при котором `sepal_length` увеличивается с 4.5 до 5.5.
6. В примере с обработкой несбалансированных данных измените `df = df[45:]` на `df = df[22:78]`. При этом сохранится приблизительно то же количество точек данных, но классы станут сбалансированными. Сравните новый полученный результат с полученными ранее. Какой из этих результатов больше всего похож на пример с использованием полного набора данных?
7. Предположим, что вместо регрессии с использованием функции `softmax` мы применяем простую линейную модель и обозначаем классы как `setosa = 0`, `versicolor = 1` и `virginica = 2`. Что происходит в модели простой линейной регрессии после изменения кодирования классов? Получим ли мы те же самые или другие результаты?
8. Сравните правдоподобие логистической модели с правдоподобием модели ЛДА. Используйте функцию `sample_posterior_predictive` для генерации прогнозируемых данных и сравните типы данных, полученных в обоих случаях. Убедитесь в том, что вы действительно понимаете различия между типами данных, прогнозируемых моделями.
9. Используя набор данных `fish`, выполните расширение модели `ZIP_reg`, включив в нее переменную `persons` как часть линейной модели. Введите эту переменную для моделирования количества дополнительных нулей. Должна получиться модель, включающая две линейные модели: одна объединяет количество детей и наличие/отсутствие туристического автофургона с коэффициентом Пуассона (как в рассмотренном примере),

- другая устанавливает связь (общего) количества людей с переменной ψ . Для второго случая потребуется логистическая функция обратной связи.
10. Используйте данные из примера применения робастной логистической регрессии для передачи в модель неробастной логистической регрессии и для проверки действительности воздействия промахов на результаты. Может потребоваться добавление или удаление промахов для лучшего понимания эффекта оценки по логистической регрессии и робастности модели, представленной в этой главе.
 11. Изучите и выполните следующие блокноты (notebooks) из документации по библиотеке PyMC3:
 - <https://pymc-devs.github.io/pymc3/notebooks/GLM-linear.html>;
 - <https://pymc-devs.github.io/pymc3/notebooks/GLM-robust.html>;
 - <https://pymc-devs.github.io/pymc3/notebooks/GLM-hierarchical.html>.

Глава 5

.....

Сравнение моделей

«Карта не есть территория, которую она представляет, но если карта правильная, то она обладает структурой, похожей на эту территорию».

– Альфред Коржибски (*Alfred Korzybski*)

Модели должны проектироваться как приближения (аппроксимации), чтобы помочь понять конкретную задачу или класс взаимосвязанных задач. Модели не проектируются как точные копии объектов и явлений реального мира. Следовательно, все модели неверны в том же смысле, что и «карта не есть территория». Даже если предварительно мы считаем каждую модель неверной, не все модели одинаково неверны – некоторые модели лучше других описывают поставленную задачу. Во всех последующих главах мы сосредоточим внимание на задаче статистического вывода, то есть на том, как узнать значения параметров по исходным данным. В этой главе главной темой является решение дополнительной задачи: как сравнить две и более модели, используемые для описания одних и тех же данных. Вы сами убедитесь в том, что это совсем не простая задача, которая в то же время является одной из главных задач в анализе данных.

В этой главе рассматриваются следующие темы:

- проверки прогнозируемого апостериорного распределения;
- лезвие Оккама – простота и точность;
- переподгонка и недоподгонка;
- информационные критерии;
- коэффициенты Байеса;
- регуляризация априорных распределений.

ПРОВЕРКИ ПРОГНОЗИРУЕМОГО АПОСТЕРИОРНОГО РАСПРЕДЕЛЕНИЯ

В главе 1 была представлена концепция проверок прогнозируемого апостериорного распределения, а в последующих главах эта методика использовалась

для того, чтобы оценить, насколько хорошо модели объясняют данные, передаваемые как входные в модель. Цель проверок прогнозируемого апостериорного распределения заключается не в том, чтобы объявить модель неправильной – это уже известно заранее. Выполняя проверки прогнозируемого апостериорного распределения, мы надеемся лучше узнать и понять ограничения, присущие модели, чтобы правильно их учитывать, или попытаться улучшить качество модели. В предыдущем предложении был неявно выражен тот факт, что модели в общем случае не с одинаковой точностью воспроизводят все аспекты решаемой задачи. Вообще говоря, это не та проблема, для решения которой создаются конкретные модели. Проверка прогнозируемого апостериорного распределения – это способ оценки модели в контексте достижения конкретной цели, следовательно, если у нас имеется более одной модели, то можно воспользоваться проверками прогнозируемого апостериорного распределения для их сравнения.

Возьмем очень простой набор данных и сформируем его диаграмму:

```
dummy_data = np.loadtxt('../data/dummy.csv')
x_1 = dummy_data[:, 0]
y_1 = dummy_data[:, 1]

order = 2
x_1p = np.vstack([x_1**i for i in range(1, order+1)])
x_1s = (x_1p - x_1p.mean(axis=1, keepdims=True)) / x_1p.std(axis=1, keepdims=True)
y_1s = (y_1 - y_1.mean()) / y_1.std()
plt.scatter(x_1s[0], y_1s)
plt.xlabel('x')
plt.ylabel('y')
```

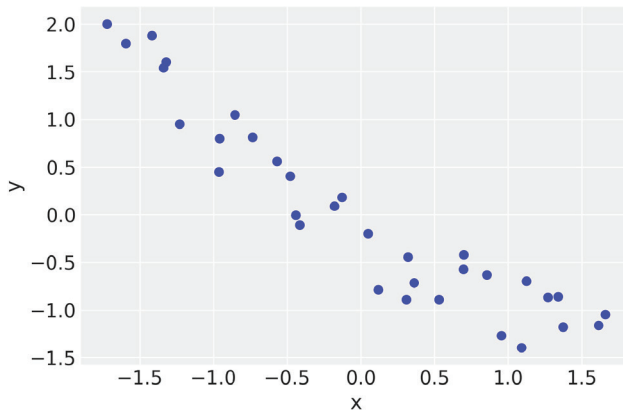


Рис. 5.1

Теперь передадим эти данные в две слегка различные модели – линейную и полиномиальную второго порядка, также известную как параболическая, или квадратическая, модель:

```

with pm.Model() as model_l:
    α = pm.Normal('α', mu=0, sd=1)
    β = pm.Normal('β', mu=0, sd=10)
    ε = pm.HalfNormal('ε', 5)
    μ = α + β * x_1s[0]
    y_pred = pm.Normal('y_pred', mu=μ, sd=ε, observed=y_1s)
    trace_l = pm.sample(2000)

with pm.Model() as model_p:
    α = pm.Normal('α', mu=0, sd=1)
    β = pm.Normal('β', mu=0, sd=10, shape=order)
    ε = pm.HalfNormal('ε', 5)
    μ = α + pm.math.dot(β, x_1s)
    y_pred = pm.Normal('y_pred', mu=μ, sd=ε, observed=y_1s)
    trace_p = pm.sample(2000)

```

Сформируем диаграмму среднего значения для обеих моделей:

```

x_new = np.linspace(x_1s[0].min(), x_1s[0].max(), 100)

α_l_post = trace_l['α'].mean()
β_l_post = trace_l['β'].mean(axis=0)
y_l_post = α_l_post + β_l_post * x_new

plt.plot(x_new, y_l_post, 'C1', label='linear model')

α_p_post = trace_p['α'].mean()
β_p_post = trace_p['β'].mean(axis=0)
idx = np.argsort(x_1s[0])
y_p_post = α_p_post + np.dot(β_p_post, x_1s)

plt.plot(x_1s[0][idx], y_p_post[idx], 'C2', label=f'model order {order}')

α_p_post = trace_p['α'].mean()
β_p_post = trace_p['β'].mean(axis=0)
x_new_p = np.vstack([x_new**i for i in range(1, order+1)])
y_p_post = α_p_post + np.dot(β_p_post, x_new_p)

plt.scatter(x_1s[0], y_1s, c='C0', marker='.')
plt.legend()

```

По рис. 5.2 можно видеть, что модель порядка 2 работает лучше, но и линейная модель не так уж плоха. Воспользуемся библиотекой PyMC3 для получения выборок прогнозируемого апостериорного распределения для обеих моделей:

```

y_l = pm.sample_posterior_predictive(trace_l, 2000, model=model_l)['y_pred']
y_p = pm.sample_posterior_predictive(trace_p, 2000, model=model_p)['y_pred']

```

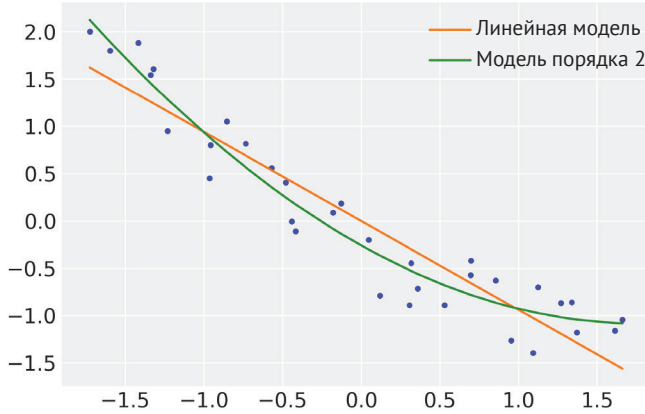


Рис. 5.2

Как уже было отмечено выше, проверки прогнозируемого апостериорного распределения часто выполняются с использованием их визуальных представлений, как в следующем примере:

```
plt.figure(figsize=(8, 3))
data = [y_1s, y_l, y_p]
labels = ['data', 'linear model', 'order 2']
for i, d in enumerate(data):
    mean = d.mean()
    err = np.percentile(d, [25, 75])
    plt.errorbar(mean, -i, xerr=[[-err[0]], [err[1]]], fmt='o')
    plt.text(mean, -i+0.2, labels[i], ha='center', fontsize=14)
plt.ylim([-i-0.5, 0.5])
plt.yticks([])
```

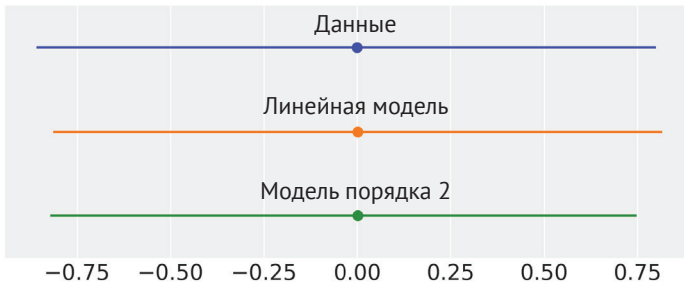


Рис. 5.3

Диаграмма на рис. 5.3 отображает среднее значение и интерквартильный размах (interquartile range – IQR) для данных, исследуемых с помощью линейной и квадратической модели. По этой диаграмме вычисляется среднее арифметическое выборки прогнозируемого апостериорного распределения для

каждой модели. Здесь можно видеть, что среднее значение (как среднее арифметическое) успешно воспроизведено обеими моделями, а интерквартильный размах не очень широк. Тем не менее наблюдаются некоторые небольшие различия, которые в реальной задаче, вероятнее всего, потребуют более пристального внимания. Можно было бы сформировать несколько различных диаграмм, чтобы более подробно исследовать прогнозируемое апостериорное распределение. Например, можно построить диаграмму вариации (дисперсии) обоих средних значений и интерквартильного размаха как противоположность средним значениям. Следующий пример демонстрирует создание такой диаграммы:

```
fig, ax = plt.subplots(1, 2, figsize=(10, 3), constrained_layout=True)

def iqr(x, a=0):
    return np.subtract(*np.percentile(x, [75, 25], axis=a))

for idx, func in enumerate([np.mean, iqr]):
    T_obs = func(y_1s)
    ax[idx].axvline(T_obs, 0, 1, color='k', ls='--')
    for d_sim, c in zip([y_l, y_p], ['C1', 'C2']):
        T_sim = func(d_sim, 1)
        p_value = np.mean(T_sim >= T_obs)
        az.plot_kde(T_sim, plot_kwargs={'color': c}, label=f'p-value {p_value:.2f}',
ax=ax[idx])
    ax[idx].set_title(func.__name__)
    ax[idx].set_yticks([])
    ax[idx].legend()
```

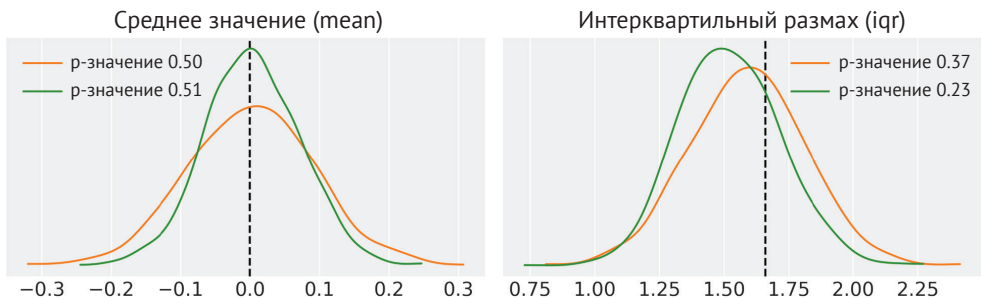


Рис. 5.4

На рис. 5.4 черная штриховая линия представляет статистическую характеристику, вычисленную по исследуемым данным (либо среднее значение, либо интерквартильный размах). Поскольку исследуемый набор данных один и тот же, мы получаем одно значение по результатам статистических вычислений (но не одно распределение). Кривые (цветовые обозначения те же, что и на рис. 5.3) представляют распределение среднего значения (левая диаграмма) и интерквартильного размаха (правая диаграмма), вычисленное по выборкам прогнозируемого апостериорного распределения. Возможно, вы заметили,

что на рис. 5.4 также указаны р-значения. Эти р-значения вычисляются при сравнении имитационных данных с реальными данными. Для обоих наборов данных вычислены обобщающие статистические характеристики (среднее значение и интерквартильный размах в рассматриваемом примере), затем было определено пропорциональное отношение количества случаев, когда обобщенные статистические характеристики больше или равны статистическим характеристикам, вычисленным по реальным данным. Если реальные и имитационные данные согласованы, то мы должны ожидать р-значение, близкое к 0.5, в противном случае наблюдается отклонение прогнозируемого апостериорного распределения.



Байесовские р-значения – это всего лишь способ получения числовой меры соответствия проверкам прогнозируемого апостериорного распределения.

Если вам знакомы методы частотной вероятности и вы читаете эту книгу, потому что слышали, что «настоящие крутые статистики» уже не используют р-значения, то, вероятно, чрезвычайно удивлены и даже разочарованы тем, что сказал автор, до сих пор вызывавший доверие. Но сохраняйте спокойствие и продолжайте чтение. Эти байесовские р-значения действительно являются р-значениями (критериями значимости), потому что они изначально определены тем же способом, что и их частотно-вероятностные собратья:

$$\text{Bayesian p-value} \triangleq p((T_{\text{sim}} \geq T_{\text{obs}})|y). \quad (5.1)$$

То есть мы получаем вероятность получения имитационной статистической характеристики T_{sim} большей или равной, чем статистическая характеристика реальных данных T_{obs} . Здесь T может быть практически любой характеристикой, которая предоставляет возможность получения обобщенного результата по исследуемым данным. На рис. 5.4 это среднее значение на левой диаграмме и стандартное отклонение на правой диаграмме. Характеристика T должна выбираться с учетом ответа на вопрос, который стал главной побудительной причиной выполнения статистического вывода.

Рассматриваемые здесь р-значения являются байесовскими, потому что для распределения сэмплирования (выборки) мы используем прогнозируемое апостериорное распределение. Также отметим, что здесь мы не принимаем во внимание какие-либо нулевые гипотезы (предположения), фактически мы имеем полное апостериорное распределение для θ и рассматриваем только условия, связанные с наблюдаемыми реальными данными. Еще одно отличие состоит в том, что мы не используем какое-либо предварительно определенное пороговое значение для объявления статистической значимости, а кроме того, не проводим проверки гипотез – мы лишь пытаемся найти числовое значение для оценки соответствия прогнозируемого апостериорного распределения наблюдаемому исходному набору данных.

Проверки прогнозируемого апостериорного распределения – с использованием диаграмм, или числовых итоговых отчетов, таких как байесовские р-значения, или даже совокупность этих двух способов – чрезвычайно гибкая

и универсальная методика. Эта концепция является достаточно обобщенной для того, чтобы позволить аналитику воспользоваться собственным воображением и прийти тем или иным путем к тщательному исследованию прогнозируемого апостериорного распределения и применить наиболее подходящий метод, позволяющий полностью объяснить управляемую данными ситуацию, в том числе и сравнение моделей, но не ограничиваясь только этим. В следующих разделах мы будем подробно рассматривать другие методы сравнения моделей.

ЛЕЗВИЕ ОККАМА – ПРОСТОТА И ТОЧНОСТЬ

При выборе из нескольких вариантов рекомендуется использовать правило, известное как лезвие Оккама (Occam), которое в свободном и кратком изложении выглядит так:



При наличии двух и более равнозначных объяснений одного явления следует выбрать самое простое.

Существует множество подтверждений этого эвристического правила. Одно из них связано с критерием фальсифицируемости (опровергаемости), введенным Карлом Поппером (Karl Popper). Другое выражает практическую точку зрения и утверждает, что «использование простых моделей легче понять, чем более сложные модели, поэтому следует выбирать наиболее простые модели». Еще одно подтверждение основано на байесовской статистике, и мы познакомимся с ним при обсуждении коэффициентов Байеса. Без подробного изучения подтверждений и обоснований правила лезвия Оккама мы принимаем этот критерий как полезное практическое правило, которое для наших целей выглядит весьма разумно.

При сравнении моделей следует всегда принимать во внимание и другой фактор – точность, то есть насколько точно модели соответствуют данным. Ранее мы уже встречались с некоторыми мерами точности, такими как коэффициент детерминации R^2 , который можно интерпретировать как пропорцию объясняемой дисперсии в линейной регрессии. Кроме того, проверки прогнозируемого апостериорного распределения основаны на понятии точности данных. Если имеются две модели и одна из них лучше объясняет данные, чем другая, то мы должны выбрать первую модель, то есть нам нужна модель с более высокой точностью.

При сравнении моделей мы интуитивно стремимся к выбору моделей с высокой точностью, которые являются более простыми. Пока все идет хорошо, но что делать, если самая простая модель обладает наихудшей точностью (или более простая модель дает более низкую точность)? В общем, как сохранить баланс между этими двумя критериями?

На протяжении оставшейся части главы мы будем обсуждать принцип сохранения баланса между точностью и простотой, поэтому здесь больше теории, чем в предыдущих главах (даже несмотря на то, что мы лишь поверхностно рассматриваем эту тему). Тем не менее в этой главе будет использоваться программ-

ный код, диаграммы и примеры, которые помогут перейти от (правильного) интуитивного соблюдения баланса между точностью и сложностью к более теоретически (или, по крайней мере, эмпирически) обоснованному подходу.

Начнем с подгонки полиномов с увеличивающейся сложностью к очень простому набору данных. Вместо использования байесовского механизма применим аппроксимацию методом наименьших квадратов для подгонки линейных моделей. Напомним, что линейную модель с байесовской точки зрения можно интерпретировать как модель с равномерными априорными распределениями. В некотором смысле мы остаемся на позиции байесовской статистики, но делаем небольшое упрощение:

```
x = np.array([4., 5., 6., 9., 12, 14.])
y = np.array([4.2, 6., 6., 9., 10, 10.])

plt.figure(figsize=(10, 5))
order = [0, 1, 2, 5]
plt.plot(x, y, 'o')
for i in order:
    x_n = np.linspace(x.min(), x.max(), 100)
    coeffs = np.polyfit(x, y, deg=i)
    ffit = np.polyval(coeffs, x_n)
    p = np.poly1d(coeffs)
    yhat = p(x)
    ybar = np.mean(y)
    ssreg = np.sum((yhat-ybar)**2)
    sstot = np.sum((y - ybar)**2)
    r2 = ssreg / sstot

    plt.plot(x_n, ffit, label=f'order {i}, $R^2$= {r2:.2f}')

plt.legend(loc=2)
plt.xlabel('x')
plt.ylabel('y', rotation=0)
```

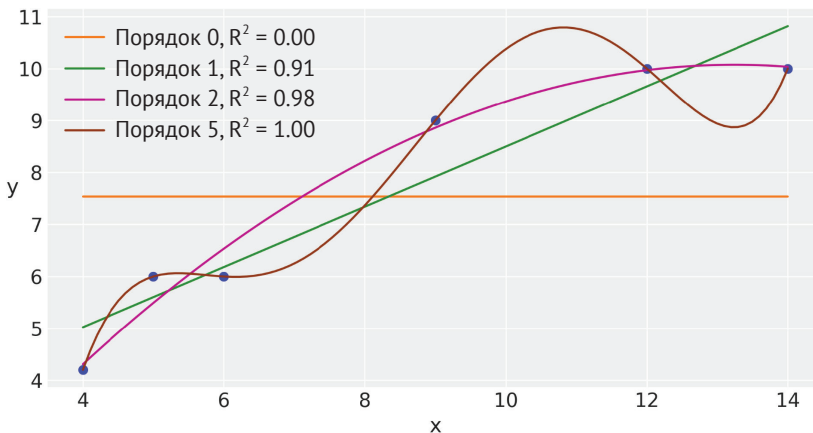


Рис. 5.5

Лишние параметры приводят к переподгонке

На рис. 5.5 можно видеть, что увеличение сложности модели сопровождается повышением точности, которое отображается коэффициентом детерминации R^2 . Очевидно, что полином порядка 5 точно соответствует данным. Вероятно, вы помните, что мы кратко рассматривали такое поведение полиномов в главе 3 и пришли к выводу, что в общем случае использование полиномов для решения реальных задач является не самой лучшей идеей.

Почему полином пятого порядка способен захватывать данные без единого пропуска какой-либо точки? Причина в том, что число параметров 6 равно количеству точек данных (тоже 6), следовательно, модель просто кодирует данные другим способом. Такая модель не обучается чему-либо на этих данных, а просто запоминает все, что ей предлагают. На этом примере можно видеть, что модель с более высокой точностью – это не всегда то, что нам действительно нужно.

Предположим, что у нас появились дополнительные ресурсы и время, поэтому мы собрали больше точек данных для включения в рассматриваемый набор данных. Например, получены точки [(10,9), (7,7)] (см. рис. 5.6). Насколько точно теперь модель полинома порядка 5 объясняет эти точки данных по сравнению с моделями 1-го и 2-го порядка? Очевидно, что не очень точно. Модель порядка 5 не обучается каким-либо заслуживающим внимания шаблонам в исследуемых данных, вместо этого она просто запоминает все, что ей предлагают (прошу извинить за настойчивое повторение этой мысли). Таким образом, модель полинома порядка 5 очень плохо подходит для будущего обобщения ранее не встречавшихся, но потенциально наблюдаемых данных.

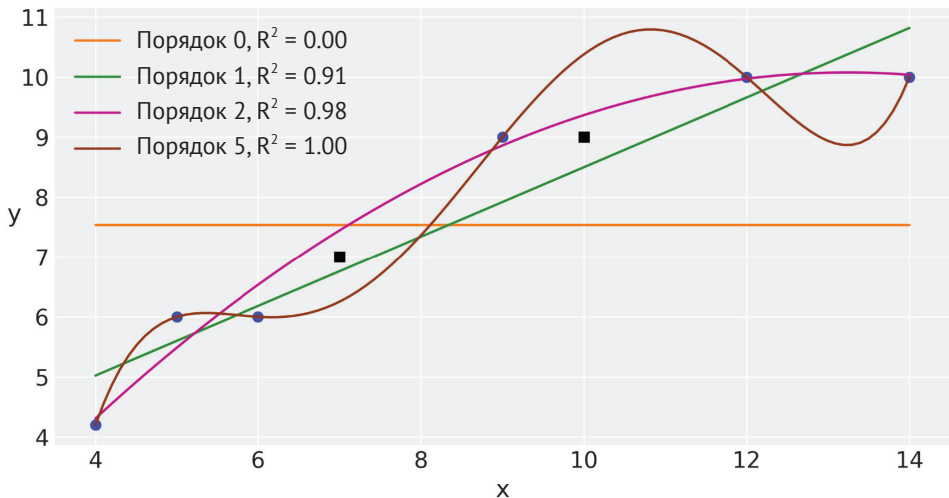


Рис. 5.6

Когда модель очень хорошо подогнана к набору данных, который использовался для обучения его параметрам на первом этапе, но очень плохо соответствует другим наборам данных, говорят, что произошла переподгонка (*overfitting*). Это чрезвычайно часто возникающая проблема в статистике и машинном обучении. Весьма полезным способом наглядного представления проблемы переподгонки является интерпретация наборов данных как состоящих из двух компонентов: сигнал и шум. Сигнал – это то, что необходимо для обучения на входных данных. Если мы используем некоторый набор данных, то считаем, что в нем содержится сигнал, в противном случае это бесполезная трата времени. С другой стороны, шум не является полезным компонентом, это результат ошибок измерений, следствие ограничений методов генерации или сбора данных, повреждений данных и т. д. Переподгонка модели происходит, если модель настолько гибка, что обучается шуму, надежно скрывающему сигнал. Это практическое подтверждение правила лезвия Оккама. Теоретически всегда можно создать настолько сложную модель, что она будет объяснять буквально все в подробностях, как в империи, описанной Борхесом (Jorge Luis Borges), где картографы достигли такого уровня мастерства, что создали карту размером с саму империю, и эта карта абсолютно точно соответствовала территории империи.

Недостаточное количество параметров приводит к недоподгонке

Продолжим рассмотрение примера из предыдущего раздела, но теперь в противоположном крайнем случае – минимальной сложности соответствует модель порядка 0. Такая модель в действительности является гауссовой моделью переменной y , выдающей себя за линейную модель. На рис. 5.5 можно заметить, что эта модель захватывает только среднее арифметическое зависимой переменной y и полностью независима от значений переменной x . Поведение такой модели называют недоподгонкой (*underfitting*; также часто используется термин «недообучение»).

Баланс между простотой и точностью

«Все должно быть изложено так просто, как только возможно, но не проще» – эту цитату часто приписывают Эйнштейну, она похожа на лезвие Оккама, но имеет обратный смысл. В идеальном случае нам хотелось бы получить модель без переподгонки и недоподгонки. В реальности приходится идти на компромисс и каким-либо образом оптимизировать и настраивать используемые модели.

Компромисс обычно выражается в терминах, обозначающих понятия дисперсии (*variance*) и смещения (*bias*):

- большое смещение является результатом работы модели со слабой возможностью согласования данных. Большое смещение может стать при-

чиной того, что модель пропускает значимые шаблоны, что приводит к недоподгонке (недообучению);

- высокий уровень дисперсии – результат работы модели с высокой чувствительностью к подробностям в данных. Высокий уровень дисперсии может стать причиной того, что модель захватывает шум в данных, что приводит к перепогонке.

На рис. 5.5 модель порядка 0 – это модель с большим смещением (и низким уровнем дисперсии), потому что она смещена по направлению к прямой линии, соответствующей среднему значению переменной y независимо от значения переменной x . Полином порядка 5 – модель с высоким уровнем дисперсии (и малым смещением). Простейший способ наблюдать поведение этой модели – наглядное представление различных наборов из шести точек данных. Эти точки можно размещать любыми разнообразными способами, а модель будет адаптироваться к каждому варианту размещения. Она будет точно подгоняться к большинству вариантов (за исключением некоторых, например окружностей). Более подробно об этом см. упражнение 6.

Модель с большим смещением является моделью с преувеличенной предвзятостью (прошу извинить за присваивание модели человеческих качеств) или с большей инерцией мышления (еще раз прошу извинить за небольшой уклон в область психологии), тогда как модель с высоким уровнем дисперсии – это более свободно мыслящая модель. Проблема слишком большого смещения (предвзятости) состоит в том, что вы не подготовлены к восприятию новых доказательств. Проблема слишком свободного («широкого») мышления заключается в том, что в конце концов вы начинаете верить в абсурдные вещи, такие как гравилеты или тайный союз антивакцинистов (anti-Vaxxers)¹. В общем случае при увеличении уровня одной из этих характеристик уровень другой снижается, то есть необходим компромисс между смещением и дисперсией. Следует еще раз подчеркнуть, что в итоге нам необходима сбалансированная модель.

Измерения прогнозируемой точности

В предыдущем примере достаточно легко заметить, что модель порядка 0 очень проста, а модель порядка 5 чрезмерно сложна для исследуемых данных, но что можно сказать о двух других моделях? Для ответа на этот вопрос необходима более строгая методика оценки точности, с одной стороны, и простоты – с другой. Чтобы определить такую методику, необходимо ввести несколько новых концепций. Две первые концепции:

- точность в пределах выборки – точность, измеряемая для данных, используемых для подгонки (обучения) модели;

¹ Vaxxed – псевдонаучный фильм (2016 г.) о предполагаемой связи MMR-вакцины и аутизма. – *Прим. перев.*

- точность вне пределов выборки – точность модели, измеряемая для данных, которые не использовались при подгонке (обучении) модели, – ее также называют прогнозируемой точностью.

Для любого сочетания данных и моделей точность в пределах выборки в среднем будет меньше, чем точность вне пределов выборки. Следовательно, точность в пределах выборки может создать ложное представление о том, что используемая модель лучше, чем она есть на самом деле. Поэтому измерения вне пределов выборки более предпочтительны, чем измерения в пределах выборки. Но при этом возникает глобальная проблема. Необходима возможность отделения части данных, предназначенных не для подгонки модели, а для ее проверки (тестирования). Для большинства аналитиков чаще всего это непозволительная роскошь. Для устранения этой проблемы люди затрачивают немалые усилия, разрабатывая методы оценки прогнозируемой точности вне пределов выборки при использовании только данных из выборки. Два таких метода кратко описаны ниже:

- перекрестная проверка (кросс-валидация) – эмпирический метод, основанный на разделении доступных данных на подмножества, которые используются для подгонки (обучения) и вычисления оценки, при этом подмножества данных все время меняются ролями;
- информационные критерии – это гипероним (обобщающее, более широкое понятие) для нескольких относительно простых выражений, которые могут рассматриваться как методы аппроксимации результатов, которые мы могли бы получить при выполнении перекрестной проверки.

Перекрестная проверка

Перекрестная проверка – это простое и в большинстве случаев эффективное решение для вычисления оценки модели без выхода за пределы имеющихся данных. В обобщенном виде этот процесс показан на рис. 5.7. Данные разделяются на K частей. Эти части мы пытаемся сделать более или менее равными (по размеру и иногда по другим признакам, например части должны содержать равное количество классов). Затем $K - 1$ частей используются для тренировки (предварительного обучения) модели, а оставшаяся часть – для тестирования. Далее эта процедура повторяется с использованием другой комбинации $K - 1$ частей для тренировки и другой оставшейся части для тестирования (то есть часть для тестирования постоянно меняется). Процесс состоит из K циклов. По результатам $A_1, A_2, A_3, \dots, A_K$ вычисляется среднее арифметическое по K циклам выполнения. Этот метод называют k -кратной перекрестной проверкой. Если K равно числу точек данных, то метод называется LOOCV (leave-one-out cross validation – перекрестная проверка с выделением одной точки). Иногда при использовании метода LOOCV количество циклов может быть меньше общего числа точек данных, если точек чрезвычайно много.

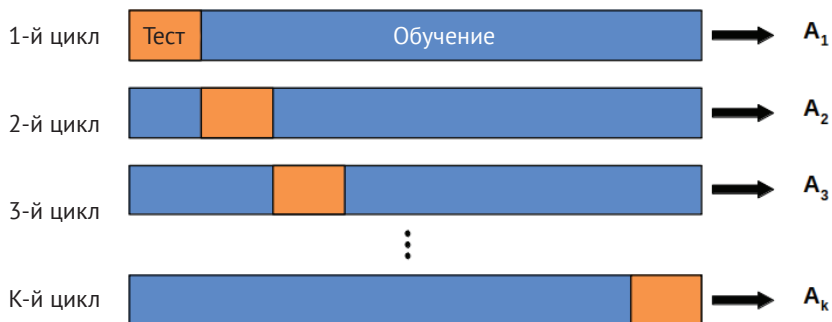


Рис. 5.7

Перекрестная проверка – это «хлеб с маслом» специалистов-практиков по машинному обучению. Здесь мы оставили без внимания некоторые подробности, но сейчас этой информации вполне достаточно. Более подробно о перекрестной проверке можно прочесть в книге «Python Machine Learning» Себастьяна Рашки (Sebastian Raschka) или в книге «Python Data Science Handbook» Джейка Вандерплаца (Jake Vanderplas).

Перекрестная проверка представляет собой весьма простую и полезную методику, но для некоторых моделей или для слишком больших объемов данных издержки на вычисления при перекрестной проверке могут превзойти наши возможности. Было совершено много попыток найти более простые для вычислений числовые характеристики, которые аппроксимируют результаты перекрестной проверки, и этот подход действует в ситуациях, когда перекрестную проверку не так-то просто выполнить. Но это уже тема следующего раздела.

ИНФОРМАЦИОННЫЕ КРИТЕРИИ

Информационные критерии – это набор различных, иногда взаимосвязанных инструментальных средств, которые используются для сравнения моделей в отношении того, насколько точно они соответствуют конкретным данным, при этом сложность моделей учитывается через штрафные факторы. Другими словами, информационные критерии формализуют интуитивное сравнение, которое рассматривалось в начале текущей главы. Нам необходим надежный способ достижения баланса между степенью точности объяснения данных моделью, с одной стороны, и степенью сложности модели – с другой.

Эти числовые характеристики должны быть выведены с использованием теории информации, но эта научная дисциплина выходит за рамки тематики данной книги, поэтому мы ограничимся ее рассмотрением только с практической точки зрения.

Логарифмическая функция правдоподобия и отклонение

Интуитивным способом измерения степени соответствия модели исходным данным является вычисление среднеквадратической ошибки между данными и прогнозами, выполненными этой моделью:

$$\frac{1}{n} \sum_{i=1}^n (y_i - E(y_i|\theta))^2. \quad (5.2)$$

Здесь $E(y_i|\theta)$ – прогнозируемое значение с учетом оцениваемых параметров.

Следует отметить, что это значение по существу является средним арифметическим разностей между наблюдаемыми и прогнозируемыми данными. Возведение в квадрат значений ошибок гарантирует, что эти разности не будут сведены на нет, а также придает особое значение большим значениям ошибок по сравнению с другими способами вычисления похожих числовых характеристик, например использование абсолютного значения. Более обобщенной мерой является вычисление логарифмической функции правдоподобия:

$$\sum_{i=1}^n \log p(y_i|\theta). \quad (5.3)$$

При нормальном значении правдоподобия оно становится пропорциональным среднеквадратической ошибке.

На практике и по историческим причинам обычно напрямую не используют логарифмическую функцию правдоподобия. Вместо нее применяется числовая характеристика, называемая отклонением (deviance):

$$-2 \sum_{i=1}^n \log p(y_i|\theta). \quad (5.4)$$

Отклонение используется как в байесовской, так и в небайесовской статистике. Различие состоит в том, что в байесовском анализе θ оценивается по апостериорному распределению, поэтому точно так же, как любая числовая характеристика, выводимая из апостериорного распределения, θ обладает собственным распределением. С другой стороны, в небайесовском анализе θ – это точечная оценка. Чтобы научиться использовать отклонение, необходимо знать две главные особенности этой числовой характеристики:

- чем меньше отклонение, тем больше значение логарифмической функции правдоподобия и тем выше степень согласования прогнозов модели с данными. Следовательно, необходимо стремиться к малым значениям отклонения;
- отклонение измеряет точность в пределах выборки модели, следовательно, сложные модели в общем случае будут давать меньшие значения отклонения, чем более простые модели. Таким образом, в сложные модели необходимо включать некоторый штрафной коэффициент или член.

В следующих разделах подробно рассматриваются различные информационные критерии. Их объединяет использование отклонения и штрафного коэффициента (члена). Но они отличаются способами вычисления отклонения и штрафного коэффициента (члена).

Информационный критерий Акаике

Это весьма широко известный и часто применяемый информационный критерий, особенно в небайесовских методиках. Информационный критерий Акаике (AIC) определяется следующей формулой:

$$\text{AIC} = -2 \sum_{i=1}^n \log p(y_i | \hat{\theta}_{\text{mle}}) + 2p_{\text{AIC}}. \quad (5.5)$$

Здесь p_{AIC} – количество параметров, а $\hat{\theta}_{\text{mle}}$ – максимальная оценка правдоподобия значения θ . Максимальная оценка правдоподобия является общепринятым практическим методом для небайесовской рабочей среды и в общем случае равнозначна байесовской оценке апостериорного максимума (maximum a posteriori – MAP) при использовании равномерных априорных распределений. Отметим, что $\hat{\theta}_{\text{mle}}$ – это точечная оценка, а не распределение.

Еще раз подчеркнем, что множитель -2 введен в формулу по историческим причинам. Важное замечание с практической точки зрения: первый член формулы учитывает степень соответствия модели данным, а второй член штрафует сложные модели. Следовательно, если две модели объясняют данные одинаково хорошо, но в одной модели параметров больше, чем в другой, то AIC сообщает нам, что следует выбрать модель с меньшим количеством параметров.

AIC хорошо работает для небайесовских методик, но в байесовском анализе его применение проблематично. Одна из причин – AIC не использует апостериорное распределение, следовательно, отказывается от информации о неопределенности в оценке, кроме того, AIC предполагает, что априорные распределения равномерны, поэтому такой метод измерения несовместим с информативными и малоинформативными априорными распределениями, которые рассматриваются в данной книге.

Часто применяемый информационный критерий

Это полноценная байесовская версия критерия AIC. Как и AIC, часто применяемый информационный критерий (widely applicable information criterion – WAIC)¹ содержит два элемента: первый измеряет степень соответствия данных и модели, второй – штрафует сложные модели:

$$\text{WAIC} = -2\text{lppd} + 2p_{\text{WAIC}}. \quad (5.6)$$

¹ Аббревиатура WAIC также означает Watanabe-Aikake information criterion, то есть информационный критерий Ватанабе–Аикаке. – *Прим. перев.*

Если необходимо лучше понять, что означают эти слагаемые, то рекомендуется внимательно изучить раздел «Информационный критерий WAIC в подробностях» (далее в этой главе). С практической точки зрения достаточно знать, что предпочтительны наименьшие значения WAIC.

Парето-сглаженная выборка по значимости для перекрестной проверки LOOCV

Парето-сглаженная выборка по значимости для перекрестной проверки LOOCV – это метод, используемый для аппроксимации результатов перекрестной проверки LOOCV, но без действительного выполнения K итераций. Это не информационный критерий, но на практике дает результаты, очень похожие на результаты WAIC, и при некоторых определенных обобщенных условиях методы WAIC и LOOCV сходятся асимптотически. Не углубляясь в подробности, отметим, что основная идея заключается в возможности аппроксимации LOOCV при помощи соответствующего перерасчета весовых коэффициентов правдоподобий. Это можно сделать, используя весьма широко распространенную и полезную статистическую методику, известную как выборка по значимости. Проблема заключается в том, что результаты нестабильны. Для устранения нестабильности был введен новый метод с применением подхода, названного парето-сглаженной выборкой по значимости для перекрестной проверки LOOCV (Pareto-smoothed importance sampling – PSIS), который может применяться для вычисления более надежных оценок LOOCV. Интерпретация этого метода похожа на интерпретацию критериев AIC и WAIC: чем меньше значение, тем выше оцениваемая прогнозируемая точность рассматриваемой модели. Таким образом, наиболее предпочтительны модели с более низкими значениями.

Другие информационные критерии

Распространенным информационным критерием является информационный критерий отклонения (Deviance Information Criterion – DIC). Если используется *bayes-o-meter*[™], то следует знать, что этот критерий представляет собой нечто среднее между AIC и WAIC. Оставаясь широко распространенным, критерий WAIC доказал, что теоретически и эмпирически более полезен, чем DIC, поэтому рекомендуется применять WAIC, а не DIC.

Еще один представитель этой группы – байесовский информационный критерий (Bayesian Information Criterion – BIC), который похож на логистическую регрессию и на «сухой суп» моей матери. Название может ввести в заблуждение. Байесовский информационный критерий был предложен как способ устранения некоторых проблем с информационным критерием Аикаке, и для этого автор воспользовался байесовским обоснованием. Но BIC в действительности не является байесовским методом, хотя фактически он похож на AIC. Байесовский информационный критерий также использует равномерные априорные распределения и оценку максимального правдоподобия. Но более важно то,

что BIC отличается от AIC и WAIC и в большей степени связан с концепцией байесовских коэффициентов, которые будут рассматриваться несколько позже в этой главе.

Сравнение моделей с помощью библиотеки PyMC3

Сравнение моделей с помощью библиотеки ArviZ проще выполнить, чем описать:

```
waic_l = az.waic(trace_l)
waic_l
```

Таблица 5.1

	waic	waic_se	p_waic	warning
0	28.750381	5.303983	2.443984	0

При необходимости вычислять LOO вместо WAIC следует использовать метод `az.loo`. Отчет о сравнении информационных критериев WAIC и LOO, созданный с помощью библиотеки PyMC3, содержит четыре значения:

- точечная оценка;
- стандартная ошибка точечной оценки (вычисляется, исходя из предположения о нормальности, следовательно, может быть не очень надежной, когда размер выборки мал);
- действительное количество параметров;
- предупреждение `warning` (подробности см. ниже в подразделе «Замечание о надежности вычислений WAIC и LOO»).

Поскольку значения WAIC/LOO всегда интерпретируются как относительные, то есть эти значения сравниваются для нескольких различных моделей, библиотека ArviZ предоставляет две вспомогательные функции для удобства сравнения. Первая функция `az.compare`:

```
cmp_df = az.compare({'model_l':trace_l, 'model_p':trace_p}, method='BB-pseudo-BMA')
cmp_df
```

Таблица 5.2

	waic	pwaic	dwaic	weight	se	dse	warning
1	9.07	2.59	0	1	5.11	0	0
2	28.75	2.44	19.68	0	4.51	5.32	0

Ниже приведено последовательное описание всех столбцов табл. 5.2 (нулевой столбец использовался для нумерации).

1. Первый столбец содержит значения информационного критерия WAIC. Объект `DataFrame` всегда сортирует эти значения по возрастанию. Числа в нулевом столбце соответствуют порядку передачи моделей в функцию сравнения.

2. Во втором столбце указано оцениваемое действительное количество параметров. В общем случае модели с большим количеством параметров более пригодны для подгонки к данным, но в то же время это может привести к перепогонке. Таким образом, значение $pWAIC$ можно интерпретировать как штрафующий элемент. Кроме того, можно также интерпретировать это значение как меру приспособленности каждой модели к подгонке к данным.
3. Третий столбец – относительная разность между значением $WAIC$ для модели с наивысшей оценкой и значениями $WAIC$ для каждой последующей модели. Поэтому для первой модели в этом столбце всегда находится значение 0.
4. Иногда при сравнении моделей не требуется выбирать самую лучшую модель. Вместо этого необходимы прогнозы по усредненным характеристикам всех моделей (или нескольких моделей). В идеальном варианте желательно получить взвешенное среднее арифметическое с приданием большего веса той модели, которая, как предполагается, лучше объясняет/прогнозирует данные. Для выполнения этой задачи применяются различные методы. Один из них – весовые коэффициенты Акаике на основе значений $WAIC$ для каждой модели. Эти весовые коэффициенты можно приблизительно интерпретировать как вероятность выбора каждой модели (из сравниваемых моделей) с учетом наблюдаемых данных. Одним из недостатков этого метода является то, что весовые коэффициенты вычисляются на основе точечных оценок $WAIC$, то есть не учитывается неопределенность.
5. В пятом столбце содержатся значения стандартной ошибки при вычислениях $WAIC$. Стандартная ошибка может оказаться полезной для установления неопределенности оценок $WAIC$.
6. Точно так же, как стандартную ошибку для каждого значения $WAIC$, можно вычислить стандартную ошибку разностей между двумя значениями $WAIC$. Отметим, что обе эти числовые характеристики не являются обязательными, потому что неопределенность $WAIC$ коррелирует между моделями. Для модели с наивысшей оценкой это значение всегда равно 0.
7. Последний столбец называется `warning`. Значение 1 сообщает, что вычисление $WAIC$, возможно, является ненадежным. Более подробно об этом можно прочесть ниже в подразделе «Замечание о надежности вычислений $WAIC$ и LOO ».

Практически ту же самую информацию можно получить в наглядном графическом виде с помощью функции `az.plot_compare`. Эта вторая полезная вспомогательная функция принимает вывод функции `az.compare` и формирует сводную диаграмму в стиле, используемом в книге Ричарда МакЭлрайта (Richard McElreath) *Statistical Rethinking*:

```
az.plot_compare(cmp_df)
```

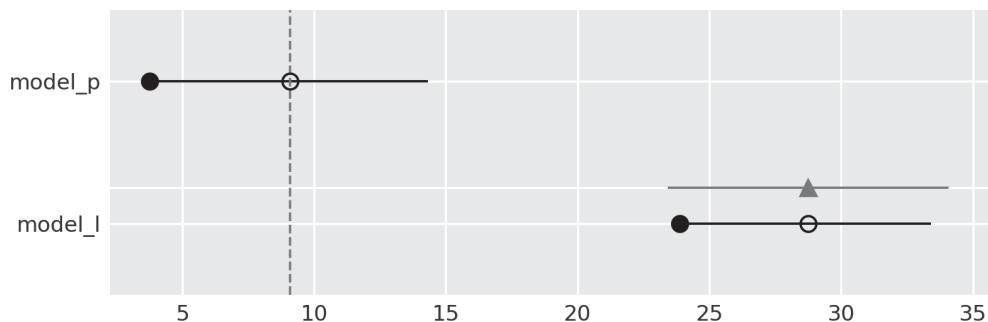


Рис. 5.8

Диаграмма на рис. 5.8 требует подробного описания:

- пустой (незакрашенный) кружок представляет значения WAIC, а черные полоски ошибок, связанные с ним, – это значения стандартного отклонения WAIC;
- наименьшее значение WAIC изображено вертикальной штриховой серой линией для упрощения сравнения с другими значениями WAIC;
- черные (закрашенные) кружки – отклонение в пределах выборки для каждой модели: для WAIC – значение $2p\text{WAIC}$ соответствующего значения WAIC;
- для всех моделей, за исключением модели с наивысшей оценкой, также показан значок треугольника, отображающий значение разности WAIC между рассматриваемой моделью и наилучшей моделью, а серая полоска ошибок, связанная с этим значком, – это стандартная ошибка разностей между WAIC с наивысшей оценкой и значением WAIC для каждой модели.

Простейший способ использования информационных критериев – выполнение процедуры выбора модели. Нужно просто выбрать модель с наименьшим значением информационного критерия и забыть обо всех прочих моделях. В такой интерпретации для рассматриваемого выше примера это очень простой выбор – квадратичная модель самая лучшая. Отметим, что стандартные ошибки не перекрываются, что прибавляет уверенности в этом варианте выбора. Если бы стандартные ошибки перекрывались, то мы получили бы менее определенный ответ.

Замечание о надежности вычислений WAIC и LOO

При вычислении WAIC и LOO иногда выводится предупреждающее сообщение о том, что результат одного из вычислений может оказаться ненадежным. Причиной этого предупреждения является пренебрегаемая малозначимая величина, которая была определена эмпирически (ссылку см. в разделе о дальнейшем чтении). Это не всегда критично, тем не менее может свидетельствовать о проблеме с вычислением указанных характеристик. WAIC и LOO – относительно но-

вые критерии, поэтому пока еще остается необходимость в разработке более эффективных методов их вычисления, позволяющих повысить надежность. В любом случае, если вы получили такое предупреждение, в первую очередь убедитесь в том, что у вас достаточное количество выборок и вы используете правильно скомбинированную (уравновешенную) выборку (см. главу 8 «Механизмы статистического вывода»). Если продолжается вывод предупреждающих сообщений, то авторы метода LOO рекомендуют использовать более робастную модель, например применяющую t -распределение Стьюдента вместо гауссова распределения. Если ни одна из перечисленных выше рекомендаций не подействовала, то, возможно, необходимо подумать об использовании другого метода, например о прямом применении k -кратной перекрестной проверки.

В более обобщенном смысле информационные критерии WAIC и LOO могут помочь только в том случае, когда выбор происходит из заданного набора сравниваемых моделей, но вряд ли эти критерии помогут определить, что какая-либо модель действительно является правильным решением конкретной задачи. Поэтому критерии WAIC и LOO должны быть дополнены проверками прогнозируемого апостериорного распределения, а также другой информацией и тестами, помогающими увидеть модели и данные с точки зрения области знаний, связанной с задачей, которую вы пытаетесь решить.

Усреднение моделей

Процедура выбора модели привлекает своей простотой, но мы пренебрегаем информацией о неопределенности моделей. Это в некоторой степени похоже на вычисление полного апостериорного распределения, после чего сохраняется только его среднее значение, то есть возможно, что мы становимся чрезмерно уверенными в том, что нам действительно известно. При выборе модели гораздо лучше формировать отчеты и обсуждать различные модели с учетом всех вычисленных значений информационных критериев, стандартных ошибок и даже с учетом результатов проверок прогнозируемых апостериорных распределений. Важно рассматривать все эти числовые характеристики и результаты тестов в контексте конкретной решаемой задачи, чтобы мы и наши оппоненты смогли лучше понять возможные ограничения и недостатки сравниваемых моделей. Если вы работаете в академической среде, то можете использовать этот подход для добавления элементов в дискуссионный раздел публикаций, презентаций, тезисов докладов и т. п.

Еще один подход – полный охват неопределенности при сравнении моделей и выполнение усреднения моделей. Основная идея такой методики заключается в генерации метамодели (и метапрогнозов) с использованием взвешенных усредненных характеристик каждой модели. В качестве одного из способов вычисления весовых коэффициентов предлагается следующая формула:

$$w_i = \frac{e^{1/2(dE_i)}}{\sum_j^M e^{-1/2(dE_j)}}. \quad (5.7)$$

Здесь dE_i – разность между значением WAIC i -й модели и модели с наименьшим значением WAIC. Вместо WAIC можно использовать любой другой информационный критерий, например AIC, или другие числовые характеристики, например LOO. Эта формула представляет собой эвристический метод вычисления относительной вероятности каждой модели (в строго определенном наборе моделей) по значениям критерия WAIC (или любым другим аналогичным числовым характеристикам). Знаменатель – это нормализационный элемент, гарантирующий, что все весовые коэффициенты в сумме равны единице. Можно вспомнить похожее выражение из главы 4, поскольку это та же функция softmax. Применение весовых коэффициентов, вычисляемых по формуле 5.7, для усреднения моделей известно как псевдоусреднение байесовского моделирования. Истинное усреднение байесовского моделирования выполняется с использованием предельных правдоподобий вместо критериев WAIC или LOO. Но даже если использование предельных правдоподобий выглядит теоретически более привлекательным, существуют теоретические и эмпирические обоснования предпочтения критерия WAIC или LOO и для сравнения, и для усреднения моделей. Более подробно об этом вы узнаете в разделе «Коэффициенты Байеса».

Используя библиотеку PyMC3, можно вычислить весовые коэффициенты по формуле 5.7, передавая аргумент `method='pseudo-BMA'` (псевдоусреднение байесовского моделирования) в функцию `az.compare`. Одним из недостатков формулы 5.7 является то, что она не учитывает неопределенность при вычислении значений E_i . Это ошибки, возвращаемые функциями `az.waic`, `az.loo`, а также функцией `az.compare` при передаче ей аргумента `method='pseudo-BMA'`. Оценку неопределенности также можно вычислить, используя байесовский бутстрэппинг. Это более надежный метод, чем предположение о нормальности. Библиотека PyMC3 позволяет вычислять оценку неопределенности с использованием байесовского бутстрэппинга, если передать аргумент `method='BB-pseudo-BMA'` в функцию `az.compare`.

Еще один способ вычисления весовых коэффициентов для усреднения моделей известен как стогование прогнозируемых распределений, или просто стогование (stacking). В библиотеке PyMC3 этот метод применяется при передаче аргумента `method='stacking'` в функцию `az.compare`. Основная идея состоит в объединении нескольких моделей в метамодель посредством минимизации отклонения между метамоделью и реальной генерируемой моделью. При использовании правила логарифмической оценки (функции) это равнозначно следующей формуле:

$$\max(1/n) \sum_{i=1}^n \log \sum_{k=1}^K w_k p(y_i | y_{-i}, M_k). \quad (5.8)$$

Здесь n – количество точек данных, а K – количество моделей. Для принятия решения вводятся ограничения для w : $w_k \geq 0$ и $\sum w_k = 1$. Числовое значение $p(y_i | y_{-i}, M_k)$ – это LOO (leave-one-out) прогнозируемое распределение для мо-

дели M_k . Как уже отмечалось ранее, вычисление этого значения требует подгонки каждой модели n раз, с отбрасыванием одной точки данных при каждой итерации. К счастью, можно аппроксимировать точное LOO прогнозируемое распределение с помощью критериев WAIC или LOO – такой способ также поддерживается библиотекой PyMC3.

Существуют и другие способы усреднения моделей, например явное создание метамодели, включающей все рассматриваемые при сравнении модели как субмодели. Такую модель можно создать тем же способом, которым выполняется вывод параметра каждой субмодели. Одновременно вычисляется относительная вероятность каждой модели (пример такого вычисления приведен в следующем разделе «Коэффициенты Байеса»).

Помимо усреднения дискретных моделей, иногда можно мысленно представить их непрерывные версии. Простейшим примером такого подхода является мысленное представление решения задачи о подбрасывании монеты при наличии двух различных моделей: одна с априорным отклонением в сторону орлов, другая с априорным отклонением в сторону решек. В этом случае непрерывной версией будет иерархическая модель, в которой априорное распределение оценивается непосредственно по данным. Такая иерархическая модель включает дискретные модели как особые случаи.

Какая методика лучше? Ответ зависит от конкретной задачи. Есть ли у нас действительно важная причина задуматься о дискретных моделях, или наша задача будет лучше представлена в виде непрерывной модели? Важно ли для нашей задачи выбрать одну из моделей, поскольку мы рассуждаем в категориях состязательных обоснований и объяснений, или усреднение станет более плодотворной идеей, так как мы более заинтересованы в прогнозах, или же мы можем всерьез задуматься о процессе, генерирующем процесс как усреднение нескольких подпроцессов? На все эти вопросы статистика не отвечает. Мы лишь получаем информацию от статистических методов в контексте некоторой области знаний.

Ниже приведен очень простой пример получения взвешенного прогнозируемого апостериорного распределения выборки, взятый из библиотеки PyMC3. Здесь используется функция `pm.sample_posterior_predictive_w` (обратите внимание на букву *w* в конце имени функции). Различие между функциями `pm.sample_posterior_predictive` и `pm.sample_posterior_predictive_w` заключается в том, что вторая принимает более одной трассировки и модели, а также список весовых коэффициентов (по умолчанию список весовых коэффициентов одинаков для всех моделей). Эти весовые коэффициенты можно получить из функции `az.compare` или из любого выбранного вами источника.

```
w = 0.5
y_lp = pm.sample_posterior_predictive_w([trace_l, trace_p],
                                       samples=1000,
                                       models=[model_l, model_p],
                                       weights=[w, 1-w])
_, ax = plt.subplots(figsize=(10, 6))
```



```

az.plot_kde(y_l, plot_kwargs={'color': 'C1'}, label='linear model', ax=ax)
az.plot_kde(y_p, plot_kwargs={'color': 'C2'}, label='order 2 model', ax=ax)
az.plot_kde(y_lp['y_pred'], plot_kwargs={'color': 'C3'}, label='weighted model', ax=ax)

plt.plot(y_1s, np.zeros_like(y_1s), '|', label='observed data')
plt.yticks([])
plt.legend()

```

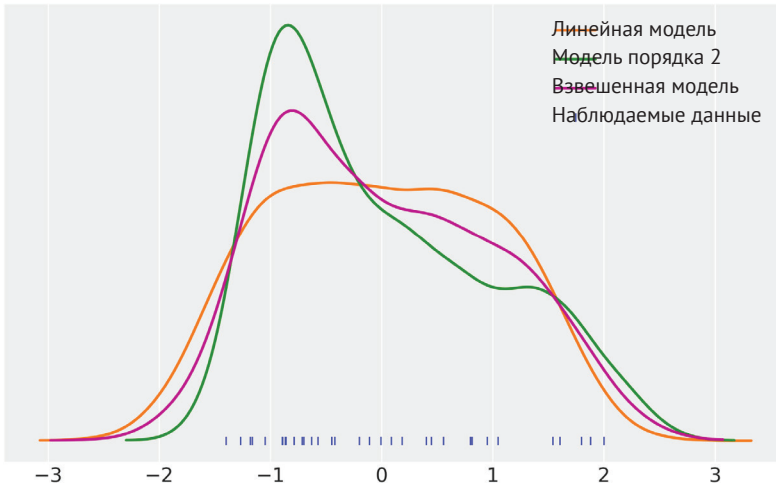


Рис. 5.9

Выше было отмечено, что это чрезвычайно упрощенный пример, потому что квадратичная модель имеет настолько низкое значение критерия WAIC по сравнению с линейной моделью, что весовой коэффициент изначально приравнивается к 1 для первой модели и 0 для второй модели, а для генерации диаграммы на рис. 5.9 я предположил, что обе модели имеют одинаковые весовые коэффициенты.

КОЭФФИЦИЕНТЫ БАЙЕСА

Другим общеизвестным решением задачи оценки и сравнения моделей в среде байесовского анализа (во всяком случае, в некоторых его областях) являются коэффициенты Байеса. Чтобы понять, что такое коэффициенты Байеса, еще раз запишем формулу теоремы Байеса (мы уже давно этого не делали):

$$p(\theta|y) = \frac{p(y|\theta)p(\theta)}{p(y)}. \quad (5.9)$$

Здесь y представляет данные. Можно сформулировать зависимость статистического вывода от конкретной модели M в явной форме и записать ее следующим образом:

$$p(\theta|y, M_k) = \frac{p(y|\theta, M_k)p(\theta, M_k)}{p(y|M_k)}. \quad (5.10)$$

Выражение в знаменателе известно как предельное правдоподобие (или доказательство), как вы, вероятно, помните из главы 1. При выполнении статистического вывода нет необходимости вычислять эту константу нормализации, поэтому на практике часто вычисляют апостериорное распределение в соответствии с постоянным коэффициентом. Но для сравнения и усреднения моделей предельное правдоподобие является важной числовой характеристикой. Если главная цель – выбор только одной наилучшей модели из набора k моделей, то можно выбрать ту модель, для которой получено максимальное значение $p(y|M_k)$. В качестве обобщенного критерия $p(y|M_k)$ являются весьма малыми числами и сами по себе дают не очень много информации, как и при использовании информационных критериев, здесь важны их относительные значения. Поэтому на практике часто вычисляют отношение двух предельных правдоподобий, которое называют коэффициентом Байеса:

$$BF = \frac{p(y|M_0)}{p(y|M_1)}. \quad (5.11)$$

Если $BF > 1$, то модель M_0 объясняет данные лучше, чем модель M_1 .

Некоторые авторы предложили таблицы с диапазонами значений для дискретизации и упрощения интерпретации коэффициентов Байеса. Например, в следующем списке показаны степени силы доказательства (подтверждения) того, что модель 0 лучше, чем модель 1:

- 1–3: неправдоподобно;
- 3–10: умеренно;
- 10–30: убедительно;
- 30–100: весьма убедительно;
- > 100: чрезвычайно (экстремально) убедительно.

Но следует помнить, что такие правила являются всего лишь соглашениями, в лучшем случае – ориентировочными рекомендациями. Тем не менее и эти результаты всегда должны вводиться в контекст задачи и сопровождаться достаточно подробной информацией, которую могли бы проверить другие исследователи, если они согласны с нашими выводами. Доказательство, необходимое для публичного объявления результатов, не может быть одним и тем же в физике элементарных частиц, в суде или в плане эвакуации города для предотвращения сотен жертв.

Использование $p(y|M_k)$ для сравнения моделей является удачным решением, если предполагается, что все модели имеют одинаковую априорную вероятность. В противном случае необходимо вычислять апостериорные шансы:

$$\underbrace{\frac{p(M_0|y)}{p(M_1|y)}}_{\text{апостериорные шансы}} = \underbrace{\frac{p(y|M_0)}{p(y|M_1)}}_{\text{коэффициенты Байеса}} \underbrace{\frac{p(M_0)}{p(M_1)}}_{\text{априорные шансы}}. \quad (5.12)$$

Некоторые дополнительные замечания

Итак, мы кратко рассмотрели некоторые наиболее важные аспекты предельного правдоподобия. При внимательном изучении определения предельного правдоподобия можно лучше понять его свойства и последствия практического использования:

$$p(y|M_k) = \int_{\theta_k} p(y|\theta_k, M_k) p(\theta_k, M_k) d\theta_k. \quad (5.13)$$

- Положительный факт: модели с большим количеством параметров получают больший штраф, чем модели с меньшим количеством параметров. В коэффициент Байеса встроено лезвие Оккама. Очевидная причина такого поведения состоит в том, что чем больше количество параметров, тем больше разброс априорного распределения с учетом правдоподобия. Таким образом, при вычислении интеграла по формуле 5.13 вы получаете меньшее значение при более концентрированном априорном распределении.
- Отрицательный факт: в общем случае вычисление предельного правдоподобия представляет собой трудную задачу, поскольку формула 5.13 содержит интеграл от быстро изменяющейся функции, определенной на многомерном пространстве параметров. В общем случае этот интеграл требует решения численными методами, которые могут быть весьма не-тривиальными.
- Чрезвычайно плохой факт: предельное правдоподобие очень сильно зависит от значений априорных распределений.

Использование предельного правдоподобия для сравнения моделей – удачное решение, так как штрафование сложных моделей уже включено в этот метод (следовательно, предотвращает переподргонку модели). В то же время любое изменение в априорном распределении воздействует на вычисление предельного правдоподобия. На первый взгляд это утверждение кажется немного наивным – ведь мы уже знаем, что априорные распределения влияют на вычисления (иначе мы могли бы просто оставить их без внимания), но в данном случае самое важное словосочетание – «очень сильно зависит». Здесь имеется в виду, что при изменениях в априорном распределении статистический вывод θ будет оставаться более или менее одинаковым, но те же изменения могут оказывать значительное воздействие на значение предельного правдоподобия. Возможно, в рассмотренных выше примерах вы заметили, что в общем случае наличие нормального априорного распределения со стандартным отклонением 100 практически то же самое, что наличие нормального априорного распределения со стандартным отклонением 1000. Но этот тип изменений будет весьма существенно воздействовать на коэффициенты Байеса.

Другим уязвимым для критики аспектом коэффициентов Байеса является то, что они могут использоваться как байесовский способ выполнения проверки гипотез. В этом нет ничего неправильного по существу, но многие авторы особо отметили, что методика статистического вывода, подобная используемой в дан-

ной и некоторых других книгах, например Statistical Rethinking Ричарда МакЭлрета (Richard McElreath), лучше подходит для решения большинства задач, чем методика проверки гипотез (вне зависимости, байесовская или нет).

Закончив с теоретическими замечаниями, рассмотрим, как можно вычислить коэффициенты Байеса.

Вычисление коэффициентов Байеса

Вычисление коэффициентов Байеса можно организовать в форме иерархической модели, где параметр верхнего уровня – это индекс, присваиваемый каждой модели и выбираемый из категориального распределения. Другими словами, статистический вывод одновременно выполняется из двух (или нескольких) конкурирующих моделей, а дискретная переменная используется для перемещения между моделями. Время, затраченное на выборку в каждой модели, пропорционально $p(M_k|y)$. Затем применяется формула 5.10 для вычисления коэффициентов Байеса.

В качестве примера вычисления коэффициентов Байеса еще раз возьмем задачу о подбрасывании монеты (рис. 5.10).

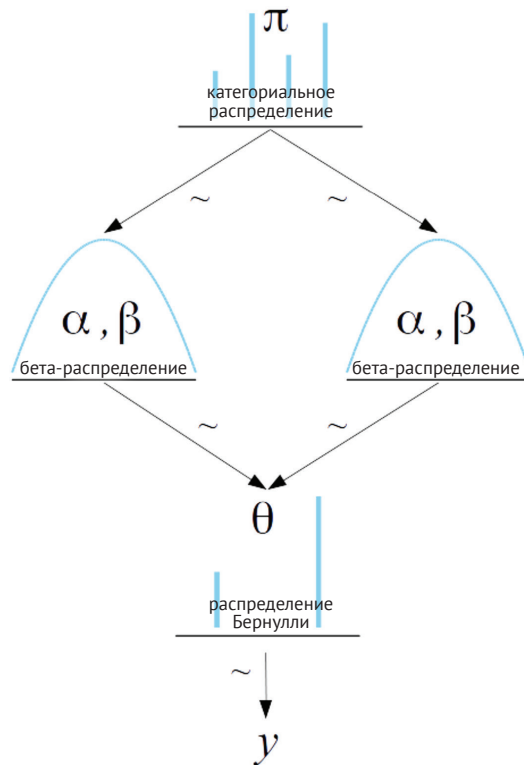


Рис. 5.10

Отметим, что хотя мы вычисляем в этом примере коэффициенты Байеса между моделями, отличающимися друг от друга только по априорному распределению, модели могли бы различаться по правдоподобию или даже по обеим этим характеристикам. Общий принцип вычисления остается тем же самым.

Создадим некоторые данные, которые можно использовать в любом примере:

```
coins = 30
heads = 9
y_d = np.repeat([0, 1], [coins-heads, heads])
```

Теперь подробнее рассмотрим модель с использованием библиотеки PyMC3. Для переключения между распределениями (моделей) применяется функция `pm.math.switch`. Если первый аргумент этой функции вычисляется как истинное значение (`true`), то возвращается второй аргумент, в противном случае возвращается третий аргумент. Отметим также использование функции `pm.math.eq` для проверки равенства 0 переменной `model_index`.

```
with pm.Model() as model_BF:
    p = np.array([0.5, 0.5])
    model_index = pm.Categorical('model_index', p=p)

    m_0 = (4, 8)
    m_1 = (8, 4)
    m = pm.math.switch(pm.math.eq(model_index, 0), m_0, m_1)

    # априорное распределение
    theta = pm.Beta('theta', m[0], m[1])
    # правдоподобие
    y = pm.Bernoulli('y', theta, observed=y_d)

    trace_BF = pm.sample(5000)
    az.plot_trace(trace_BF)
```

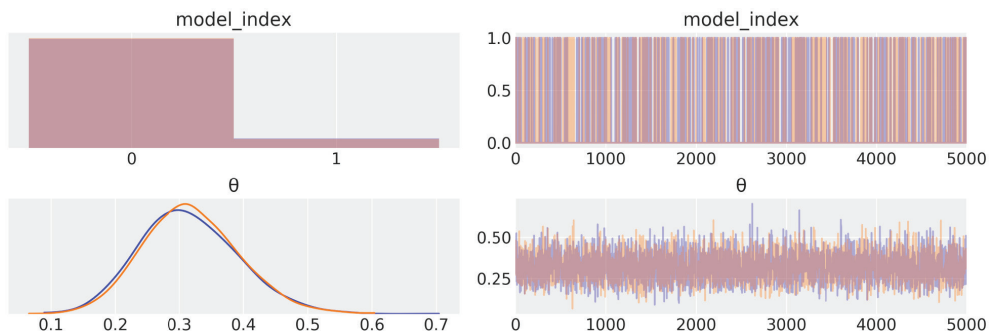


Рис. 5.11

Далее необходимо вычислить коэффициент Байеса с учетом значения переменной `model_index`. Отметим, что в эти вычисления мы включили значения априорных распределений для каждой модели:

```
pM1 = trace_BF['model_index'].mean()
pM0 = 1 - pM1
BF = (pM0 / pM1) * (p[1] / p[0])
```

В результате получаем значение ≈ 11 , означающее, что модель 0 предпочтительнее модели 1 на целый порядок. Этот результат имеет обобщенный смысл, поскольку данные содержат меньше значений орлов (лицевых сторон монеты), чем ожидалось при $\theta = 5$, а единственное различие между этими двумя моделями заключается в том, что априорное распределение модели 0 более совместимо с $\theta < 0.5$ (больше решек, чем орлов), а модель 1 более совместима с $\theta > 0.5$ (больше орлов, чем решек).

Общие проблемы при вычислении коэффициентов Байеса

Некоторые общие проблемы при вычислении коэффициентов Байеса рассматриваемым и применяемым здесь способом заключаются в том, что если одна модель лучше другой по определению, то мы потратим больше времени на выборку из лучшей модели, чем из другой. Это может стать проблематичным, так как возможна субдискретизация (undersampling) одной из моделей. Другая проблема – значения параметров обновляются, даже если эти параметры не используются для подгонки модели. То есть если выбрана модель 0, то обновляются параметры в модели 1, но поскольку эти параметры не используются для объяснения данных, они ограничены только априорным распределением. Если априорное распределение слишком неопределенное, то возможно, что при выборе модели 1 значения рассматриваемых параметров окажутся весьма далекими от предварительно принятых значений, следовательно, этот этап отвергается. Таким образом, мы заканчиваем процесс вычислений с проблемой выборки.

В том случае, когда возникают обе вышеописанные проблемы, можно внести в исследуемую модель два изменения, чтобы улучшить качество выборки:

- в идеальном варианте можно получить более качественную выборку из обеих моделей при одинаковом обращении к ним, поэтому мы можем отрегулировать априорное распределение для каждой модели (переменная p в предыдущей модели) в пользу менее предпочтительной модели за счет более предпочтительной модели. Это не повлияет на вычисление коэффициента Байеса, потому что априорные распределения включены в это вычисление;
- использовать псевдоаприорные распределения, как предложено Крушке (Kruschke) и другими специалистами. Идея проста: если проблема в том, что смещение параметров не ограничено, когда модель, которой они принадлежат, не выбрана, то решением является попытка ограничить их искусственно, но только в том случае, когда они не используются. Пример практического применения псевдоаприорных распределений можно найти в модели, используемой Крушкой в его книге и перенесенной мною в среду Python/PyMC3 (см. https://github.com/alocstavodia/Doing_bayesian_data_analysis).

Использование последовательного метода Монте-Карло для вычисления коэффициентов Байеса

Другой способ вычисления коэффициентов Байеса заключается в использовании метода выборки, известного как последовательный метод Монте-Карло (Sequential Monte Carlo – SMC). Более подробно этот метод будет рассматриваться в главе 8 «Механизмы статистического вывода». Сейчас достаточно знать, что этот механизм выборки вычисляет оценку предельного правдоподобия как побочный (промежуточный) результат, который можно напрямую использовать для вычисления коэффициентов Байеса. Чтобы воспользоваться реализацией последовательного метода Монте-Карло из библиотеки PyMC3, нужно просто передать метод `pm.SMC()` в аргументе `step` в функцию `sample`:

```
with pm.Model() as model_BF_0:
    θ = pm.Beta('θ', 4, 8)
    y = pm.Bernoulli('y', θ, observed=y_d)
    trace_BF_0 = pm.sample(2500, step=pm.SMC())

with pm.Model() as model_BF_1:
    θ = pm.Beta('θ', 8, 4)
    y = pm.Bernoulli('y', θ, observed=y_d)
    trace_BF_1 = pm.sample(2500, step=pm.SMC())

model_BF_0.marginal_likelihood / model_BF_1.marginal_likelihood
```

При вычислении последовательным методом Монте-Карло коэффициент Байеса также приблизительно равен 11. При необходимости вычисления коэффициентов Байеса с использованием библиотеки PyMC3 настоятельно рекомендуется применять последовательный метод Монте-Карло. Другой метод, представленный в данной книге, требует более громоздких вычислений и большого объема ручной работы, главным образом потому, что для организации перехода между моделями пользователь должен выполнять точную настройку методом проб и ошибок. А последовательный метод Монте-Карло является более автоматизированным методом.

Коэффициенты Байеса и информационные критерии

Отметим, что если мы берем логарифм от коэффициентов Байеса, то можем превратить отношение предельных правдоподобий в разность. Сравнение разностей предельных правдоподобий похоже на сравнение разностей информационных критериев. Более того, можно интерпретировать коэффициенты Байеса, точнее – предельные правдоподобия, как имеющийся в наличии член подгонки и штрафующий член. Член, определяющий, насколько хорошо модель соответствует данным, относится к части правдоподобия, а штрафующий фактор определяется усреднением априорного распределения. Чем боль-

ше количество параметров, тем больше объем априорного распределения по сравнению с объемом правдоподобия, следовательно, в конечном итоге мы получим средние арифметические характеристики из областей, в которых правдоподобие имеет весьма малые значения. Чем больше параметров, тем более разреженным или рассеянным становится априорное распределение, следовательно, тем больше штраф при вычислении доказательства (обоснования). Это как раз та причина, по которой говорят, что теорема Байеса ведет к естественному штрафованию сложных моделей, то есть в теорему Байеса изначально встроено лезвие Оккама.

Ранее уже было отмечено, что коэффициенты Байеса более чувствительны к априорным распределениям, чем предполагают (или даже уверенно полагают) многие люди. Словно бы имеющиеся различия практически не имеют значения при выполнении статистического вывода, но вдруг становятся важными при вычислении коэффициентов Байеса. Если предположить существование бесконечной мультивселенной, то я почти уверен, что в каждой из входящих в нее вселенных существовало бы и ток-шоу Херальдо¹ с антибайесовской критикой и проклятиями, в особенности по адресу коэффициентов Байеса. В такой вселенной (отбросим иллюзии – и в нашей единственной вселенной) я бы выражал одобрение и поддержку на антибайесовской стороне. Тем не менее сейчас мы рассмотрим пример, который поможет разобраться, что именно делают коэффициенты Байеса, какие информационные критерии работают и каким образом, и хотя они схожи, особое внимание будет уделено двум различным аспектам. Вернемся к определению данных для примера с подбрасыванием монеты и на этот раз зададим 300 монет и 90 орлов. Это та же пропорция, что в исходном примере, но теперь данных в 10 раз больше. Затем выполним каждую модель отдельно:

```
traces = []
waics = []
for coins, heads in [(30, 9), (300, 90)]:
    y_d = np.repeat([0, 1], [coins-heads, heads])
    for priors in [(4, 8), (8, 4)]:
        with pm.Model() as model:
            θ = pm.Beta('θ', *priors)
            y = pm.Bernoulli('y', θ, observed=y_d)
            trace = pm.sample(2000)
            traces.append(trace)
            waics.append(az.waic(trace))
```

¹ В последнее время – шоу Херальдо Риверы (The Geraldo Rivera Show) – дискуссионное телевизионное шоу журналиста, писателя и адвоката Херальдо Риверы (США). – Прим. перев.



Рис. 5.12

Увеличивая объем данных, мы получаем почти полностью охватывающее априорное распределение, и теперь обе модели выдают похожие прогнозы. При использовании 30 монет и 9 орлов наблюдался коэффициент Байеса 11. Если повторить вычисления (сделайте это самостоятельно) с данными, содержащими 300 монет и 90 орлов, то получим коэффициент Байеса ≈ 25 . Этот коэффициент Байеса сообщает, что модель 0 предпочтительнее модели 1 даже в большей степени, чем в исходном примере. При увеличении объема данных решение о выборе модели становится более очевидным. Этот вывод обретает общий смысл, поскольку теперь мы более уверены в том, что модель 1 имеет распределение, не соответствующее данным.

Также отметим, что при увеличении объема данных в обеих моделях будет проявляться тенденция к сближению значений θ , по факту получим $\theta \approx 0.3$ для обеих моделей. Таким образом, если мы решили воспользоваться переменной θ для прогнозирования новых выходных данных, то при этом непременно воз-

никнут какие-либо отличия от той модели, по которой мы вычисляем распределение θ .

Теперь сравним с результатом, показанным информационным критерием WAIC (см. рис. 5.13). WAIC ≈ 368.4 для модели 0 и ≈ 368.6 для модели 1. Это кажется весьма небольшим различием. Здесь более важно не это конкретное различие, а тот факт, что если вычислить информационные критерии для 30 монет и 9 орлов, то получим значения WAIC ≈ 38.1 для модели 0 и ≈ 39.4 для модели 1. То есть относительная разность при увеличении объема данных уменьшается – чем более схожи оценки θ , тем более близки значения прогнозируемой точности, оцениваемой по информационным критериям. Та же картина наблюдается еще более наглядно, если вместо WAIC использовать LOO:

```
fig, ax = plt.subplots(1, 2, sharey=True)
labels = model_names
indices = [0, 0, 1, 1]
for i, (ind, d) in enumerate(zip(indices, waics)):
    mean = d.waic
    ax[ind].errorbar(mean, -i, xerr=d.waic_se, fmt='o')
    ax[ind].text(mean, -i+0.2, labels[i], ha='center')
ax[0].set_xlim(30, 50)
ax[1].set_xlim(330, 400)
plt.ylim([-i-0.5, 0.5])
plt.yticks([])
plt.subplots_adjust(wspace=0.05)
fig.text(0.5, 0, 'Deviance', ha='center', fontsize=14)
```

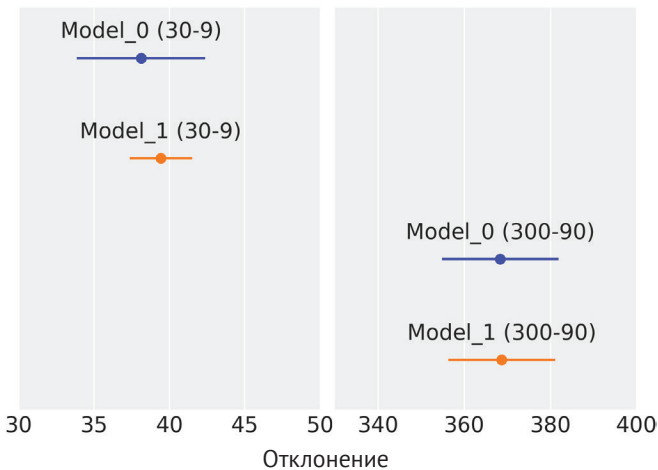


Рис. 5.13

Коэффициенты Байеса ориентированы на определение лучшей модели, в то время как информационные критерии WAIC и LOO направлены на то, чтобы

узнать, какая модель дает более точные прогнозы. Вы можете сами увидеть эти различия, если вернетесь к формулам 5.6 и 5.11 и тщательно их проанализируете. WAIC, как и любой другой информационный критерий, так или иначе использует логарифмическую функцию правдоподобия, а априорные распределения не участвуют непосредственно в вычислениях. Только косвенное участие априорных распределений помогает оценить значение θ . Коэффициенты Байеса напрямую используют априорные распределения, поскольку необходимо получить среднее арифметическое правдоподобие по всему интервалу априорных значений.

РЕГУЛЯРИЗАЦИЯ АПРИОРНЫХ РАСПРЕДЕЛЕНИЙ

Использование информативных и малоинформативных априорных распределений – это способ ввода смещения в модель, и если это сделано корректно, то может стать действительно положительным фактором, поскольку смещение предотвращает перепогонку, следовательно, вносит вклад в способность модели генерировать хорошо обобщаемые прогнозы. Добавление смещения для сокращения числа ошибок обобщения без воздействия на способность модели правильно моделировать данные, используемые для подгонки, называют регуляризацией. Регуляризация часто принимает форму штрафования слишком больших значений параметров модели. Это способ сокращения объема информации, которую модель способна представить, следовательно, снижается и вероятность того, что модель захватит шум вместо сигнала.

Идея регуляризации настолько мощна и полезна, что возникала и реализовывалась неоднократно, в том числе и за пределами байесовского анализа. В некоторых областях деятельности эта идея известна как регуляризация Тихонова. В небайесовской статистике идея регуляризации принимает форму двух модификаций метода наименьших квадратов: гребневая регрессия и Lasso-регрессия¹. С байесовской точки зрения гребневую регрессию можно интерпретировать как использование нормальных распределений для бета-коэффициентов (линейной модели) с малым стандартным отклонением, которое принудительно смещает эти коэффициенты к нулю. В этом смысле мы уже проделывали нечто, подобное гребневой регрессии для каждой отдельной линейной модели, рассматриваемой в данной книге (за исключением примеров в текущей главе, в которых использовалась библиотека SciPy). С другой стороны, Lasso-регрессию с байесовской точки зрения можно интерпретировать как оценку апостериорного максимума, вычисленную из модели с априорными распределениями Лапласа для бета-коэффициентов. Распределение Лапласа выглядит похожим на гауссово распределение, но его первая производная не определена в нулевой точке, поскольку в этой точке имеется чрезвычайно

¹ Lasso – least absolute shrinkage and selection operator. – *Прим. перев.*

острый пик (см. рис. 5.14). В распределении Лапласа основная масса вероятностей концентрируется намного ближе к нулю по сравнению с нормальным распределением. Идея использования такого распределения возникла потому, что оно обеспечивает и регуляризацию, и отбор переменных (признаков) (variable selection). Поскольку при нуле имеется острый пик кривой, можно ожидать разреженности априорного распределения, то есть мы создаем модель с большим количеством параметров, но такое априорное распределение автоматически сделает большинство этих параметров нулевыми, сохраняя только важные переменные, действительно вносящие вклад в выходные данные модели. К сожалению, байесовская Lasso-регрессия в действительности не работает так, как описано здесь, главным образом потому, что при наличии множества параметров априорное распределение Лапласа принудительно устанавливает для ненулевых параметров малые значения. К счастью, не все так плохо – существуют байесовские модели, которые могут использоваться и для создания разреженности, и для отбора переменных (признаков), например модель подковы (horseshoe) и финской подковы (Finnish horseshoe).

Важно отметить, что классические версии гребневой регрессии и Lasso-регрессии соответствуют одноточечным оценкам, тогда как байесовские версии дают в результате полное апостериорное распределение:

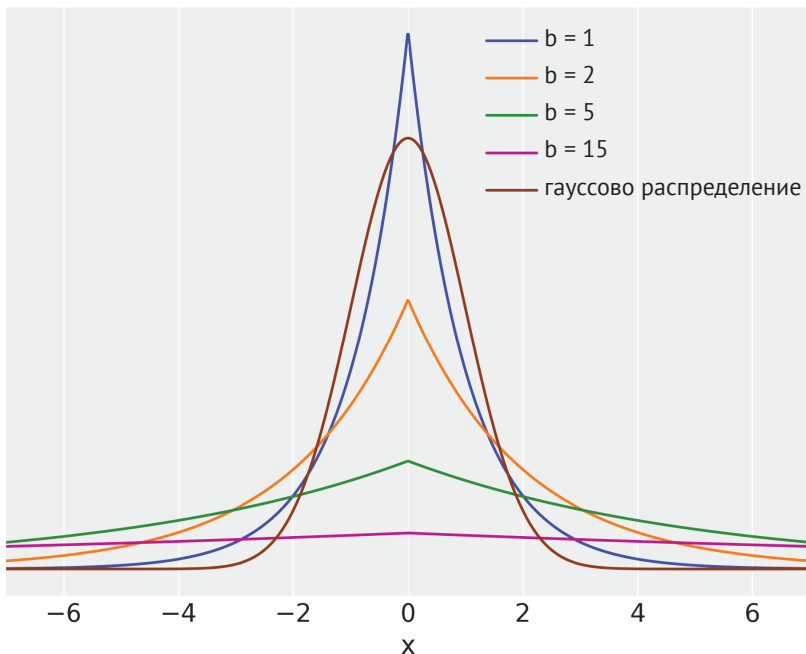


Рис. 5.14

БОЛЕЕ ПОДРОБНО ОБ ИНФОРМАЦИОННОМ КРИТЕРИИ WAIC

Если раскрыть все выражения в формуле 5.6, то получим следующую формулу:

$$\text{WAIC} = -2 \sum_i^n \log \left(\frac{1}{S} \sum_{s=1}^S p(y_i | \theta^s) \right) + 2 \sum_i^n \left(\sum_{s=1}^S (\log p(y_i | \theta^s)) \right). \quad (5.14)$$

Оба члена этой формулы очень похожи друг на друга. Первый член – логарифм поточечно определяемой прогнозируемой плотности ($\log$ pointwise predictive density – lppd) – вычисляет среднее значение правдоподобия по S выборкам апостериорного распределения. Это делается для каждой точки данных, затем берется логарифм, и полученные результаты суммируются по всем точкам данных. Сравните этот член с формулами 5.3 и 5.4. Это то, что называют отклонением (deviance), но вычисляемым с учетом апостериорного распределения. Таким образом, если мы принимаем тот факт, что вычисление логарифмической функции правдоподобия является корректным способом измерения степени пригодности подгонки модели, то ее вычисление по апостериорному распределению представляет собой вполне логичный метод при байесовском подходе. Как уже было отмечено выше, логарифм поточечно определяемой прогнозируемой плотности (lppd) наблюдаемых данных y – это преувеличенная оценка lppd для будущих данных. Из-за этого вводится второй член для корректировки этой преувеличенной оценки. Во втором члене вычисляется дисперсия логарифмической функции правдоподобия по S выборкам апостериорного распределения. Сначала вычисления выполняются для каждой точки данных, затем все результаты суммируются по всем точкам данных. Почему дисперсия становится штрафующим элементом? Интуиция подсказывает, что такой подход похож на встроенное в коэффициенты Байеса лезвие Оккама. Чем больше количество действительных (активных) параметров, тем более широким будет размах апостериорного распределения. При добавлении в модель структуры, например, с использованием информативных/регуляризирующих априорных распределений или иерархических зависимостей мы ограничиваем апостериорное распределение, следовательно, уменьшаем количество действующих (активных) параметров по сравнению с аналогичной нерегуляризованной или менее структурированной моделью.

Энтропия

Теперь хотелось бы сказать несколько слов о концепции энтропии (entropy). Математически можно определить энтропию следующей формулой:

$$H(p) = - \sum_i p_i \log(p_i). \quad (5.15)$$

Из этой формулы можно понять, что чем больше размах распределения, тем больше его энтропия. Это можно наблюдать, выполняя следующий фрагмент кода и внимательно изучая рис. 5.15.

```

np.random.seed(912)
x = range(0, 10)
q = stats.binom(10, 0.75)
r = stats.randint(0, 10)

true_distribution = [list(q.rvs(200)).count(i) / 200 for i in x]

q_pmf = q.pmf(x)
r_pmf = r.pmf(x)

_, ax = plt.subplots(1, 3, figsize=(12, 4), sharey=True, constrained_layout=True)
for idx, (dist, label) in enumerate(zip([true_distribution, q_pmf, r_pmf],
                                       ['true_distribution', 'q', 'r'])):
    ax[idx].vlines(x, 0, dist, label=f'entropy = {stats.entropy(dist):.2f}')
    ax[idx].set_title(label)
    ax[idx].set_xticks(x)
    ax[idx].legend(loc=2, handlelength=0)
    
```

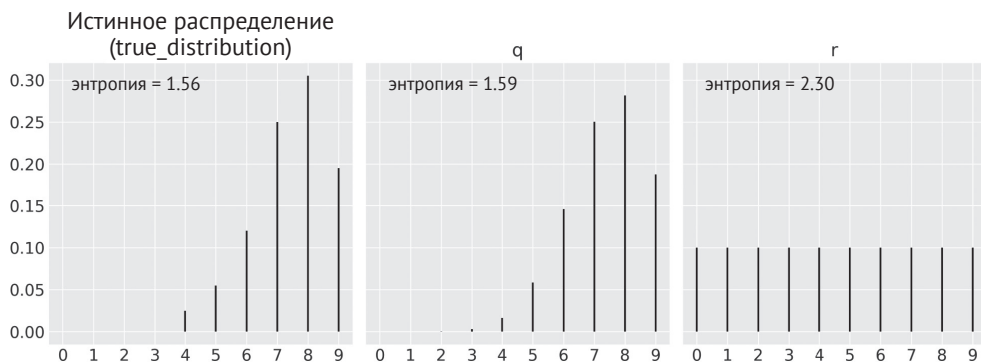


Рис. 5.15

На рис. 5.15 можно видеть, что распределение r на соответствующей диаграмме (справа) имеет наиболее широкий размах из трех распределений, а также наибольшую энтропию. Читателю предлагается поэкспериментировать с исходным кодом и понаблюдать, как изменяется энтропия (более подробно см. упражнение 10).

После рассмотрения приведенного выше примера, вероятно, появляется соблазн объявить энтропию некоторой причудливой формой измерения дисперсии распределения. Эти концепции связаны друг с другом, но все же они разные. При некоторых условиях увеличение энтропии означает увеличение дисперсии. Так будет в случае гауссова распределения. Тем не менее можно привести примеры, когда дисперсия увеличивается, а энтропия нет. Понять причину этого явления можно без особых затруднений. Предположим, что мы исследуем распределение, представляющее собой объединение двух гауссовых распределений (более подробно объединение распределений будет рассматриваться в главе 6 «Смешанные модели»). При увеличении расстояния

между модами увеличивается расстояние основного массива точек данных от среднего значения, а дисперсия в точности равна среднему арифметическому расстояний всех точек данных до среднего значения. Поэтому если расстояние продолжает увеличиваться, то дисперсия тоже будет возрастать без ограничения. На энтропию будет оказано меньшее воздействие, поскольку при увеличении расстояния между модами для точек между модами вероятность становится все меньше и меньше, следовательно, их вклад в общую энтропию будет ничтожным. С точки зрения энтропии, если мы начинаем наблюдение с двух перекрывающихся гауссовых распределений, а затем перемещаем одно из них относительно другого, то в любой произвольной точке мы получим два отдельных гауссовых распределения.

Энтропия также связана с концепцией информации или с противоположной концепцией неопределенности. В самом деле, на протяжении всей этой книги неоднократно отмечается, что чем больше размах и уплощенность априорного распределения, тем оно менее информативно. Это верно не только с интуитивной точки зрения, но также имеет теоретическую поддержку со стороны концепции энтропии. В действительности среди байесовских аналитиков существует группа, которая использует энтропию для обоснования малоинформативных или регуляризируемых априорных распределений. Такой подход известен под названием принципа максимальной энтропии. Необходимо найти распределение с наибольшей возможной энтропией (то есть наименее информативное), но при этом также необходимо учитывать ограничения, предварительно определенные в условии решаемой задачи. Это задача оптимизации, которую можно решить математически, но здесь мы не будем подробно рассматривать весь ход вычислений, а вместо этого приведем несколько примеров распределений с наибольшей энтропией при указанных конкретных ограничениях:

- без ограничений – равномерное распределение (непрерывное или дискретное, в соответствии с типом переменной);
- положительное среднее значение – экспоненциальное распределение;
- с заданной дисперсией – нормальное распределение;
- только две неупорядоченные выходные переменные и постоянное среднее значение – биномиальное распределение или распределение Пуассона, если исследуются редкие события (напомним, что распределение Пуассона – это особый случай биномиального распределения).

Заслуживает внимания замечание о том, что многие обобщенные линейные модели, подобные рассмотренным в главе 4, обычно определяются с использованием распределений с максимальной энтропией и с учетом ограничений конкретных моделей.

Расхождение Кульбака–Лейблера

Необходимо также кратко рассмотреть расхождение Кульбака–Лейблера (РКЛ) (Kullback–Leibler (KL) divergence). Эта концепция часто встречается при чтении публикаций по статистике, машинному обучению, теории информации или

статистической механике. Можно утверждать, что обоснование обращения к теме РКЛ, как и к другим концепциям, таким как энтропия или предельное правдоподобие, слишком упрощено, тем не менее все перечисленные дисциплины в той или иной мере рассматривают одни и те же группы задач, хотя и с несколько различных точек зрения.

Полезность расхождения Кульбака–Лейблера заключается в том, что это способ измерения степени близости двух распределений, определяемый следующей формулой:

$$D_{\text{KL}}(p||q) = \sum_i p_i \log \frac{p_i}{q_i}. \quad (5.16)$$

Это означает расхождение Кульбака–Лейблера от q до p (необходимо читать в обратном порядке), где p и q – два распределения вероятностей. Для непрерывных переменных вместо суммы необходимо вычислить интеграл, но основной принцип остается неизменным.

Расхождение $D_{\text{KL}}(p||q)$ можно интерпретировать как дополнительную энтропию или неопределенность, которая вводится применением распределения вероятностей q для аппроксимации распределения p . В действительности РКЛ представляет собой разность между двумя энтропиями:

$$D_{\text{KL}}(p||q) = \underbrace{\sum_i p_i \log p_i}_{\text{энтропия } p} - \underbrace{\sum_i p_i \log q_i}_{\text{перекрестная энтропия } p, q} = \sum_i p_i (\log p_i - \log q_i). \quad (5.17)$$

Используя свойства логарифмов, можно преобразовать формулу 5.17 в формулу 5.16. Поэтому можно также считать расхождение $D_{\text{KL}}(p||q)$ относительной энтропией распределения p по отношению к распределению q (в этом случае выражение читается в прямом порядке).

В качестве простого примера можно использовать расхождение Кульбака–Лейблера, для того чтобы определить, какое распределение, q или r , является более точной аппроксимацией истинного распределения `true_distribution`. Воспользовавшись библиотекой SciPy, можно вычислить расхождение $D_{\text{KL}}(\text{true_distribution}||q)$ и $D_{\text{KL}}(\text{true_distribution}||r)$:

```
stats.entropy(true_distribution, q_pmf), stats.entropy(true_distribution, r_pmf)
```

Если выполнить этот фрагмент кода, то получим значения ≈ 0.0096 , ≈ 0.7394 . Можно сделать вывод: q является более точной аппроксимацией истинного распределения `true_distribution`, чем r , потому что вводит меньше дополнительной неопределенности. Надеюсь, что читатели согласятся со мной с учетом того, что полученные числовые результаты соответствуют результатам наблюдения диаграмм на рис. 5.15.

Возможно, возникает соблазн описать РКЛ как расстояние, но это расхождение не симметрично, следовательно, не является реальным расстоянием. При выполнении приведенного ниже фрагмента кода будут получены результаты

≈ 2.7 , ≈ 0.7 . Очевидно, что эти числа отличаются от результатов предыдущего вычисления. В этом примере можно видеть, что g лучше аппроксимирует q – это другой способ оценки:

```
stats.entropy(r_pmf, q_pmf), stats.entropy(q_pmf, r_pmf)
```

Расхождение $D_{KL}(p||q)$ показывает, насколько точно распределение q аппроксимирует распределение p , и мы можем представить это с точки зрения неожиданности, то есть насколько неожиданным будет появление q , если ожидается p . Степень неожиданности возникновения некоторого события зависит от информации, которая используется для оценки этого события. Я вырос в городе, расположенном в чрезвычайно засушливой климатической зоне, где настоящие штормовые ливни случаются один-два раза в год. Затем я переехал в другую провинцию, чтобы поступить в колледж, и был в буквальном смысле шокирован тем, что штормовые ливни в среднем шли каждую неделю, по крайней мере в сезон дождей. Некоторые из моих одноклассников приехали из Буэнос-Айреса, одной из провинций Аргентины с самым влажным и дождливым климатом. Для них частые дожди были более или менее ожидаемым явлением. Более того, они считали, что дождь мог бы идти и немного чаще, потому что воздух не был настолько влажным.

Можно было бы воспользоваться расхождением Кульбака–Лейблера для сравнения моделей, так как в результате этого вычисления мы получим меру близости к истинному распределению апостериорных распределений из каждой модели. Проблема заключается в том, что нам неизвестно истинное распределение. Следовательно, расхождение Кульбака–Лейблера невозможно применить напрямую. Но можно воспользоваться РКЛ как аргументом для регулирования использования отклонения (формула 5.3). Если предположить, что истинное распределение существует, как показано в приведенной ниже формуле 5.18, то это истинное распределение не зависит от каких-либо моделей и констант, следовательно, оно будет воздействовать на расхождение Кульбака–Лейблера также, независимо от (апостериорного) распределения, используемого для аппроксимации истинного распределения. Таким образом, можно использовать отклонение, то есть часть, которая зависит от каждой модели, для оценки степени близости к истинному распределению, даже если оно неизвестно. Из формулы 5.17 с помощью несложных алгебраических преобразований можно вывести следующую формулу:

$$\begin{aligned} D_{KL}(p||q) - D_{KL}(p||r) &= \left(\sum_i p_i \log p_i - \sum_i p_i \log q_i \right) - \\ &\quad - \left(\sum_i p_i \log p_i - \sum_i p_i \log r_i \right) = \\ &= \sum_i p_i \log q_i - \sum_i p_i \log r_i. \end{aligned} \quad (5.18)$$

Даже если нам неизвестно p , можно сделать вывод о том, что распределение с большим значением $\log(\cdot)$, или $\log$ -правдоподобия или отклонения (по

вашему выбору), ближе по расхождению Кульбака–Лейблера к истинному распределению. На практике логарифмическая функция правдоподобия/отклонения получается из модели, которая была подогнана к конечной выборке. Таким образом, необходимо также добавить штрафной член для корректировки преувеличенной оценки отклонения, а это приводит нас к WAIC и другим информационным критериям.

РЕЗЮМЕ

Проверки прогнозируемого апостериорного распределения – обобщенная концепция и практика, которая помогает понять, насколько точно модели соответствуют данным и насколько правильно модели охватывают аспекты решаемой задачи. Проверки прогнозируемого апостериорного распределения можно выполнять как для одной модели, так и для нескольких моделей, то есть можно пользоваться этими проверками как методом сравнения моделей. Проверки прогнозируемого апостериорного распределения в общем случае выполняются в форме визуальных представлений, но полезными также могут оказаться и обобщающие отчеты по числовым характеристикам, таким как байесовские p -значения.

Эффективные модели соблюдают правильный баланс между сложностью и точностью прогнозов. Мы рассмотрели соотношение этих характеристик на примере с использованием классической задачи полиномиальной регрессии. Обсуждались два метода оценки точности вне пределов выборки без исключения данных: перекрестная проверка и информационные критерии. Основное внимание было уделено второму методу. С практической точки зрения информационные критерии – это семейство методов, обеспечивающих соблюдение баланса двух компонентов: один измеряет качество подгонки модели к данным, второй штрафует сложные модели. Из множества доступных информационных критериев для байесовских моделей наиболее полезен WAIC. Еще один полезный критерий измерения, который на практике показывает весьма похожие на WAIC результаты, – PSIS-LOO-CV (или просто LOO). Метод LOO используется для аппроксимации перекрестной проверки LOO без больших вычислительных издержек на реальную повторную подгонку модели, выполняемую несколько раз. WAIC и LOO можно использовать для выбора модели, а кроме того, полезна для усреднения моделей. Вместо выбора одной наилучшей модели усреднение моделей позволяет объединить все доступные модели, приняв их взвешенные средние арифметические характеристики.

Другим подходом к выбору, сравнению и усреднению моделей являются коэффициенты Байеса, представляющие собой отношение предельного правдоподобия двух моделей. Вычисление коэффициентов Байеса может оказаться достаточно сложной процедурой. В этой главе были продемонстрированы два способа вычислений коэффициентов Байеса с помощью библиотеки PyMC3: иерархическая модель, в которой мы напрямую попытались оценить относи-

тельную вероятность каждой модели с использованием дискретного индекса, и метод выборки, известный как последовательный метод Монте-Карло, который рекомендуется для практического применения.

Кроме сложности вычисления коэффициентов Байеса, для них характерны проблемы применения, связанные с высокой чувствительностью к определению априорного распределения. Мы также сравнивали коэффициенты Байеса и информационные критерии и подробно разобрали пример, в котором решались два различных, но все же взаимосвязанных вопроса: первый относится к идентификации правильной модели, второй касается наилучших прогнозов или наименьших потерь при обобщении. В обоих методах существуют свои проблемы, но на практике гораздо более устойчивы и надежны информационные критерии WAIC и LOO.

Мы кратко рассмотрели связь априорных распределений с проблемами перепогонки, отклонения и с задачей регуляризации с учетом важной темы создания моделей с качественными обобщающими свойствами.

Завершило главу более подробное обсуждение информационного критерия WAIC с рассмотрением связанных с ним концепций энтропии, принципа максимальной энтропии и расхождения Кульбака–Лейблера.

УПРАЖНЕНИЯ

1. В этом упражнении используются априорные распределения регуляризации. В коде, генерирующем данные, измените значение `order=2` на другое значение, например `order=5`. Затем выполните подгонку модели `model_p` и постройте итоговую кривую. Повторите упражнение с использованием априорного распределения для бета-распределения с `sd=100` вместо `sd=1` и постройте итоговую кривую. Насколько различны полученные кривые? Попробуйте еще раз выполнить упражнение с `sd=np.array([10, 0.1, 0.1, 0.1, 0.1])`.
2. Повторите выполнение предыдущего упражнения с увеличением объема данных до 500 точек данных.
3. Выполните подгонку кубической модели (порядок 3), вычислите WAIC и LOO, постройте диаграмму результатов и сравните их с результатами линейной и квадратической моделей.
4. Используйте `pm.sample_posterior_predictive()` для повторного выполнения примера проверок прогнозируемого апостериорного распределения, но на этот раз сформируйте диаграмму значений `y` вместо средних значений.
5. Изучите и выполните пример проверок прогнозируемого апостериорного распределения из документации к библиотеке PyMC3 (https://pymc-devs.github.io/pymc3/notebooks/posterior_predictive.html). Особое внимание обратите на применение совместно используемых переменных из библиотеки Theano.

6. Воспользуйтесь исходным кодом, который генерирует диаграммы на рис. 5.5 и 5.6, и внесите изменения, чтобы получить новые наборы из шести точек данных. Внимательно исследуйте по сгенерированным диаграммам, насколько точно различные полиномиальные распределения соответствуют этим новым наборам данных. Попробуйте связать полученные результаты с материалом, изложенным в книге.
7. Изучите и выполните пример усреднения моделей из документации к библиотеке PyMC3 (https://docs.pymc.io/notebooks/model_averaging.html).
8. Вычислите коэффициент Байеса для задачи о подбрасывании монеты с использованием равномерного априорного бета-распределения (1, 1) и априорных распределений, например бета-распределения (0.5, 0.5). В качестве условия определите 30 монет и 15 орлов. Сравните полученный результат со статистическим выводом, который был сделан в первой главе книги.
9. Повторите выполнение примера, в котором сравнивались коэффициенты Байеса и информационные критерии, но с уменьшением размера выборки.
10. В примере вычисления энтропии измените распределение q . Попробуйте повторить вычисления с такими распределениями, как `stats.binom(10, 0.5)` и `stats.binom(10, 0.25)`.

Глава 6

.....

Смешанные модели

«...Отец его лев, а мать – муравей. Его отец питается мясом, но мать ест растительную пищу. А потомство их – муравьиный лев...»

– *Хорхе Луис Борхес, Маргарита Герреро,
«Книга вымышленных существ»*

Ривер-Плэйт (также известная под названиями Ла Плата Ривер или Рио де ла Плата) – самая широкая река на Земле и естественная природная граница между Аргентиной и Уругваем. На протяжении второй половины XIX века в прибрежной области вдоль всей реки происходило слияние местного населения с африканцами (в большинстве своем – рабами) и иммигрантами из Европы. Одним из последствий этого объединения стало смешение европейской музыки, такой как вальс и мазурка, с африканской музыкой кандомбе и аргентинским жанром милонга (который, в свою очередь, представляет собой смесь афроамериканских ритмов). В результате этого смешения стилей появилась музыка и танец, который мы называем танго.

Объединение ранее существующих элементов – это превосходный способ создания новых вещей и объектов, не только музыки. В статистике смешанные модели являются широко распространенной методикой создания моделей. Такие модели создаются посредством объединения (смешивания) более простых распределений для получения более сложных моделей. Например, можно объединить два гауссовых распределения для описания бимодального распределения или несколько гауссовых распределений для описания произвольных случайных распределений. Гауссовы распределения используются чаще всего, но теоретически можно смешивать любые семейства распределений по нашему выбору. Смешанные модели используются для разнообразных целей, например для прямого моделирования отдельных групп населения или в качестве полезного приема для обработки сложносоставных распределений, которые невозможно описать с помощью более простых распределений.

В этой главе рассматриваются следующие темы:

- конечные (или конечномерные) смешанные модели;
- бесконечные смешанные модели (смешанные модели с бесконечной размерностью);
- непрерывные смешанные модели.

СМЕШАННЫЕ МОДЕЛИ

Смешанные модели появились естественным образом, когда всеобщее население стало объединением отдельных групп и подгрупп населения. Самый известный пример – распределение роста в выбранной группе взрослых людей, которое можно описать как смесь (объединение) подгрупп женщин и мужчин. Другим классическим примером является кластеризация рукописных цифр. В этом случае вполне обоснованно ожидается 10 подгрупп, по крайней мере в десятичной системе счисления. Если известно, к какой подгруппе принадлежит каждый наблюдаемый объект, то обобщенной эффективной методикой является использование этой информации для моделирования каждой подгруппы как отдельной группы. Но если прямого доступа к такой информации нет, то на помощь приходят смешанные модели.



Многие наборы данных невозможно правильно описать с использованием единственного распределения вероятностей, но есть возможность описать их как объединение (смесь) таких распределений. Модели, которые принимают на обработку данные, происходящие из смеси распределений, называют смешанными моделями.

При создании смешанных моделей не обязательна полная уверенность в том, что формируются абсолютно истинные подгруппы в исследуемых данных. Кроме того, смешанные модели можно использовать как статистический прием для добавления гибкости набору рабочих инструментов. Например, возьмем гауссово распределение. Его можно использовать как разумную аппроксимацию для многих унимодальных (однородных) и более или менее симметричных распределений. Но что делать с мультимодальными или асимметричными распределениями? Можно ли воспользоваться гауссовыми распределениями для их моделирования? Да, можно, если использовать объединение гауссовых распределений. В гауссовой смешанной модели каждый компонент будет гауссовым распределением с различным средним значением и в общем случае с различным стандартным отклонением (но это не обязательно). Объединяя гауссовы распределения, мы можем добавить в создаваемую модель гибкость, позволяющую правильно подогнать ее к сложным распределениям данных. В действительности можно аппроксимировать любое распределение, если правильно подобрать сочетание гауссовых распределений. Точное количество распределений зависит от требуемой точности аппроксимации и от подробной информации о данных. По факту мы уже применяли подход с объединением гауссовых распределений при построении многих диаграмм в этой книге. Метод ядерной оценки плотности (ЯОП) представляет собой небайесовскую (и непараметрическую) реализацию этой идеи. В принципе, при вызове `az.plot_kde` эта функция размещает гауссово распределение (с фиксированной дисперсией) поверх каждой точки данных, затем суммирует все отдельные гауссовы распределения для аппроксимации эмпирического распределения данных. На рис. 6.1 показан реальный пример того, как можно объединить восемь гауссовых распределений для представления распределения сложной формы, на-

поминающей удава боа-констриктора, переваривающего проглоченного слона (если это сравнение непонятно, то настоятельно рекомендую прочесть книгу «Маленький принц»). На рис. 6.1 все гауссовы распределения имеют одинаковую дисперсию и центрированы по оранжевым точкам, которые представляют точки выборки из возможно неизвестной совокупности. Если внимательно рассмотреть рис. 6.1, то можно заметить, что два гауссовых распределения размещены одно над другим весьма близко.

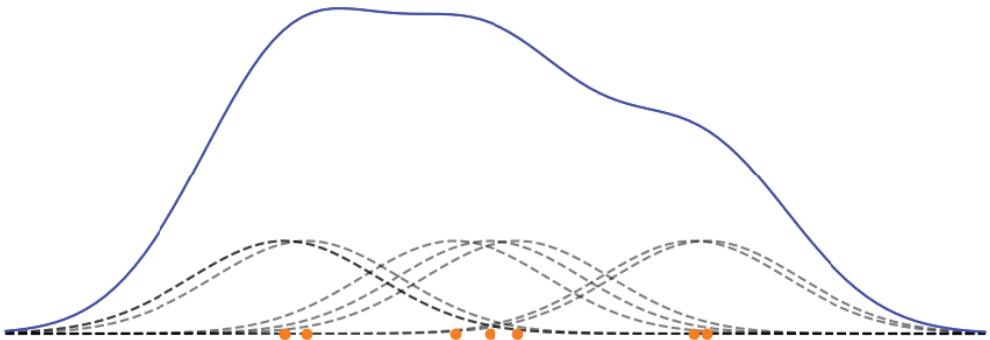


Рис. 6.1

Не важно, действительно ли мы уверены в существовании подгрупп из совокупности или используем их для удобства с математической точки зрения (или даже при некотором усредненном подходе), в любом случае, смешанные модели являются полезным способом добавления гибкости в применяемые модели с использованием объединения распределений для описания данных.

Конечные смешанные модели

Один из методов создания смешанных моделей – рассмотрение конечного взвешенного объединения двух и более распределений. Это конечная (или конечномерная) смешанная модель. В этом случае плотность вероятности наблюдаемых данных представляет собой взвешенную сумму плотностей вероятностей для K подгрупп данных:

$$p(y|\theta) = \sum_{i=1}^K w_i p_i(y|\theta_i). \quad (6.1)$$

Здесь w_i – весовой коэффициент каждого компонента (или класса). Весовой коэффициент w_i можно интерпретировать как вероятность компонента i , следовательно, его значения ограничены интервалом $[0,1]$, а сумма $\sum_i w_i = 1$. Компоненты $p_i(y|\theta)$ могут быть практически любыми элементами, признаваемыми полезными и перенесенными из простых распределений, таких как гауссово

распределение или распределение Пуассона, в более сложные объекты, например иерархические модели или нейронные сети. Для конечной смешанной модели K – конечное число (обычно, но не обязательно, это небольшое число $K \lesssim 20$). Для подгонки конечной смешанной модели необходимо определить значение K , потому что нам действительно заранее известно правильное значение или потому что имеется некоторое обоснованное предположение.

В принципе, для решения смешанной модели нужно только правильно установить соответствие между каждой точкой данных и одним из компонентов. В вероятностной модели это можно сделать с помощью ввода случайной переменной z , задача которой заключается в определении: какой компонент конкретного наблюдения назначается (присваивается). Такую переменную обычно называют скрытой (latent) переменной, потому что она не наблюдается напрямую.

Начнем с создания смешанных моделей с использованием данных о химическом сдвиге, которые уже рассматривались ранее в главе 2.

```
cs = pd.read_csv('../data/chemical_shifts_theo_exp.csv')
cs_exp = cs['exp']
az.plot_kde(cs_exp)
plt.hist(cs_exp, density=True, bins=30, alpha=0.3)
plt.yticks([])
```

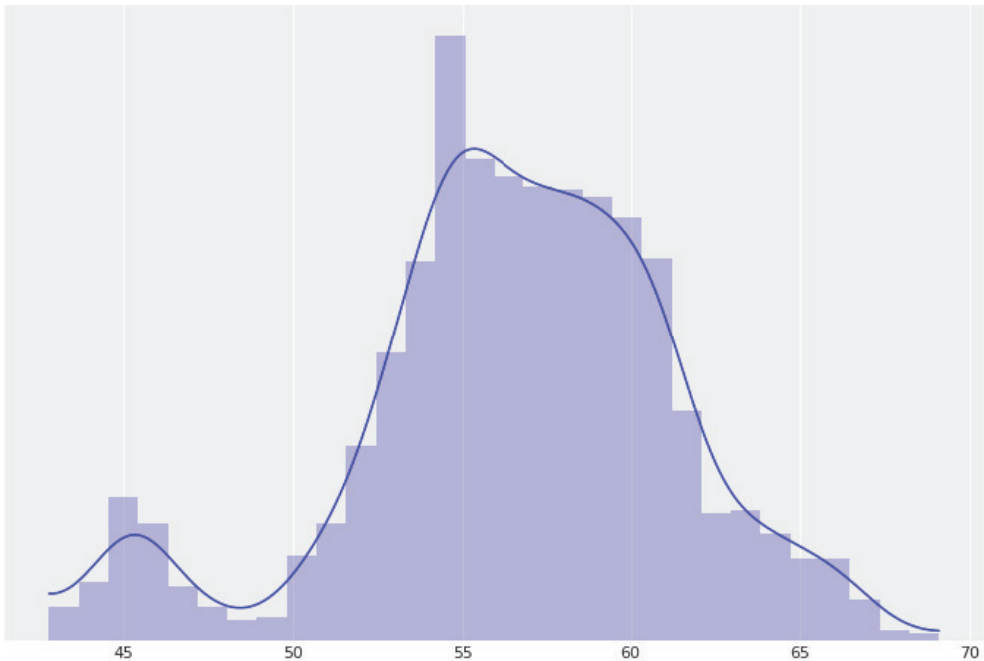


Рис. 6.2

На рис. 6.2 можно видеть, что эти данные невозможно правильно описать с помощью единственного гауссова распределения, но, возможно, три-четыре таких распределения помогут нам. В действительности существуют веские теоретические обоснования (которые здесь мы не будем рассматривать) происхождения этих данных – это около 40 подмножеств общей совокупности со значительными пересечениями между ними.

Для расширения интуитивного понимания смешанных моделей можно воспользоваться основными принципами из задачи о подбрасывании монеты. Для этой модели имеем два возможных итоговых результата, а для их описания применяется распределение Бернулли. Поскольку нам неизвестна вероятность выпадения орлов и решек, в качестве априорного распределения используется бета-распределение. Смешанная модель аналогична, за исключением того, что вместо двух итоговых результатов, то есть орел или решка, теперь мы получаем K итоговых результатов (или K -компонентов). Обобщением распределения Бернулли для K итоговых результатов является категориальное распределение, а обобщением бета-распределения становится распределение Дирихле. Теперь необходимо ближе познакомиться с этими двумя новыми распределениями.

Категориальное распределение

Категориальное распределение – это наиболее обобщенное дискретное распределение, которое использует параметр, определяющий вероятности каждого возможного итогового результата. На рис. 6.3 представлены два возможных экземпляра категориального распределения. Точки представляют значения категориального распределения, а непрерывные линии – это вспомогательные визуальные элементы, помогающие сразу понять форму распределения.

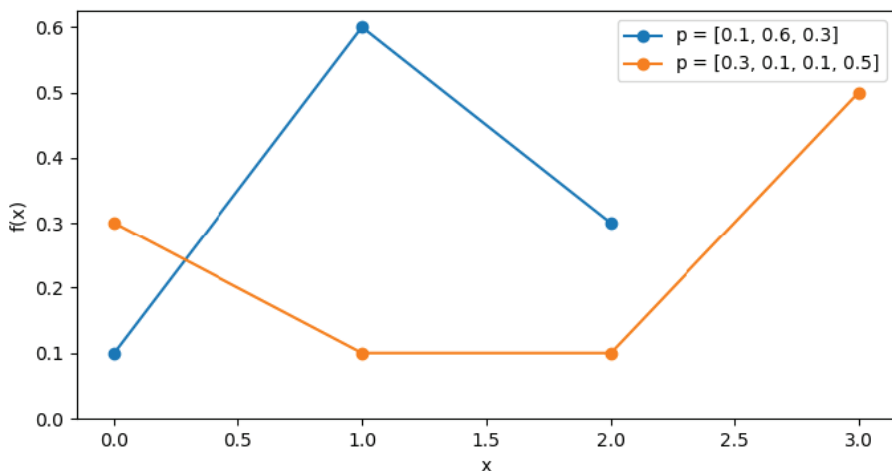


Рис. 6.3

Распределение Дирихле

Распределение Дирихле существует в симплексе как n -мерном обобщении треугольника: 1-симплекс – это прямая линия, 2-симплекс – треугольник, 3-симплекс – тетраэдр и т. д. Почему именно симплекс? Потому что выходные данные этого распределения представлены в виде вектора длиной K , значения элементов которого ограничены диапазоном чисел, больших или равных нулю, но в сумме должны давать 1. Как уже было отмечено выше, распределение Дирихле является обобщением бета-распределения. Таким образом, чтобы лучше понять распределение Дирихле, необходимо сравнить его с бета-распределением. Бета-распределение применяется для решения задач с двумя возможными итоговыми результатами: один с вероятностью p , другой с вероятностью $1 - p$. Вполне очевидно, что $p + 1 - p = 1$. Бета-распределение возвращает вектор с двумя элементами $(p, 1 - p)$, но на практике элемент $1 - p$ обычно опускают как абсолютно определенный результат, если известен элемент p . При необходимости расширения бета-распределения до трех итоговых результатов потребуется вектор из трех элементов (p, q, r) , где каждый элемент больше нуля и $p + q + r = 1$, следовательно, $r = 1 - (p + q)$. Можно было бы воспользоваться тремя скалярными величинами для параметризации такого распределения, скажем, α , β и γ , но набор из 24 греческих букв достаточно быстро будет исчерпан. Вместо этого разумнее использовать вектор с именем α и длиной K , где K – количество возможных итоговых результатов. Следует отметить, что можно воспринимать оба рассматриваемых здесь распределения как распределения по пропорциям. Чтобы глубже понять основной принцип такого распределения, внимательно изучите рис. 6.4 и попытайтесь установить связь каждой треугольной диаграммы с бета-распределением с соответствующими параметрами.

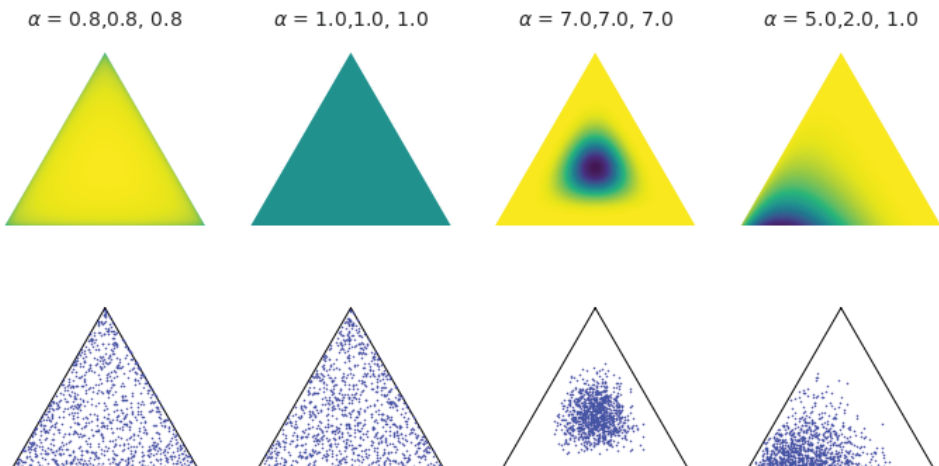


Рис. 6.4

Теперь, когда сущность распределения Дирихле более понятна, мы располагаем всеми элементами для создания смешанных моделей. Одним из способов наглядного представления смешанных моделей является модель подбрасывания монеты с K сторонами, выстроенная поверх модели гауссовой оценки. Снова воспользуемся диаграммой Крушке.

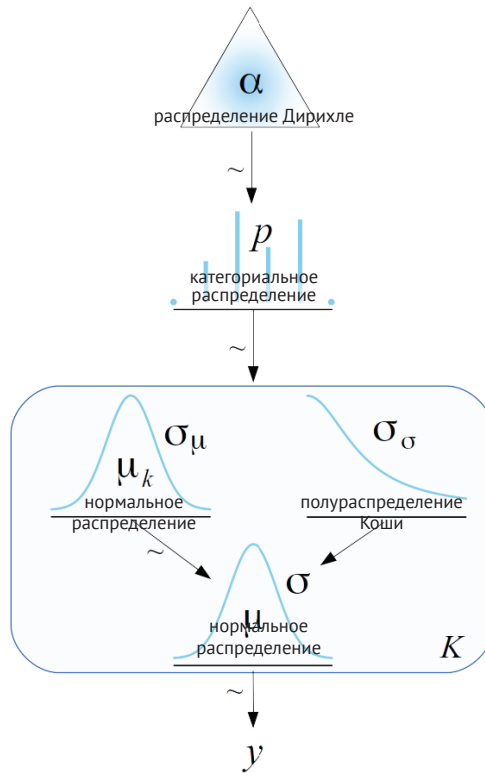


Рис. 6.5

Прямоугольник с закругленными углами обозначает, что имеется K компонентов и категориальные переменные, которые выборочно (по нашему решению) используются для описания заданной точки данных.



Следует отметить, что на рис. 6.5 только μ_k зависит от различных компонентов, в то время как σ_μ и σ_σ совместно используются всеми этими компонентами. При необходимости это состояние можно изменить и позволить принимать во внимание другие параметры каждого компонента.

Эту модель (предварительно предполагая, что `clusters = 2`) можно реализовать с использованием библиотеки РумСЗ, как показано ниже:

```
with pm.Model() as model_kg:
    p = pm.Dirichlet('p', a=np.ones(clusters))
    z = pm.Categorical('z', p=p, shape=len(cs_exp))
    means = pm.Normal('means', mu=cs_exp.mean(), sd=10, shape=clusters)
    sd = pm.HalfNormal('sd', sd=10)

    y = pm.Normal('y', mu=means[z], sd=sd, observed=cs_exp)
    trace_kg = pm.sample()
```

Если выполнить этот фрагмент кода, то обнаружится, что он работает очень медленно, а его трассировка выглядит весьма плохо (более подробно о диагностике см. главу 8 «Механизмы статистического вывода»). Причина таких затруднений заключается в том, что в модель `model_kg` явно включена скрытая переменная `z`. Одной из проблем этого подхода с явным включением является то, что выборка дискретной переменной `z` обычно приводит к медленному смешиванию (объединению) и неэффективному использованию хвостов применяемого распределения. Один из способов устранения подобных проблем с выборкой – репараметризация модели.

Отметим, что в смешанной модели наблюдаемая переменная `y` моделируется в соответствии с условиями, определяемыми скрытой переменной `z`. То есть $p(y|z, \theta)$. Можно мысленно интерпретировать скрытую переменную `z` как «неудобную» переменную (помеху), которую можно «убрать со сцены» и получить $p(y|\theta)$. К счастью, в библиотеку РумСЗ включена реализация распределения `NormalMixture`, которой можно воспользоваться для записи гауссовой смешанной модели в следующем виде:

```
clusters = 2
with pm.Model() as model_mg:
    p = pm.Dirichlet('p', a=np.ones(clusters))
    means = pm.Normal('means', mu=cs_exp.mean(), sd=10, shape=clusters)
    sd = pm.HalfNormal('sd', sd=10)
    y = pm.NormalMixture('y', w=p, mu=means, sd=sd, observed=cs_exp)
    trace_mg = pm.sample(random_seed=123)
```

Воспользуемся также библиотекой `ArviZ`, чтобы наблюдать, как выглядит трассировка, и сравним полученную трассировку с трассировкой для модели `model_mgr` из следующего раздела:

```
varnames = ['means', 'p']
az.plot_trace(trace_mg, varnames)
```

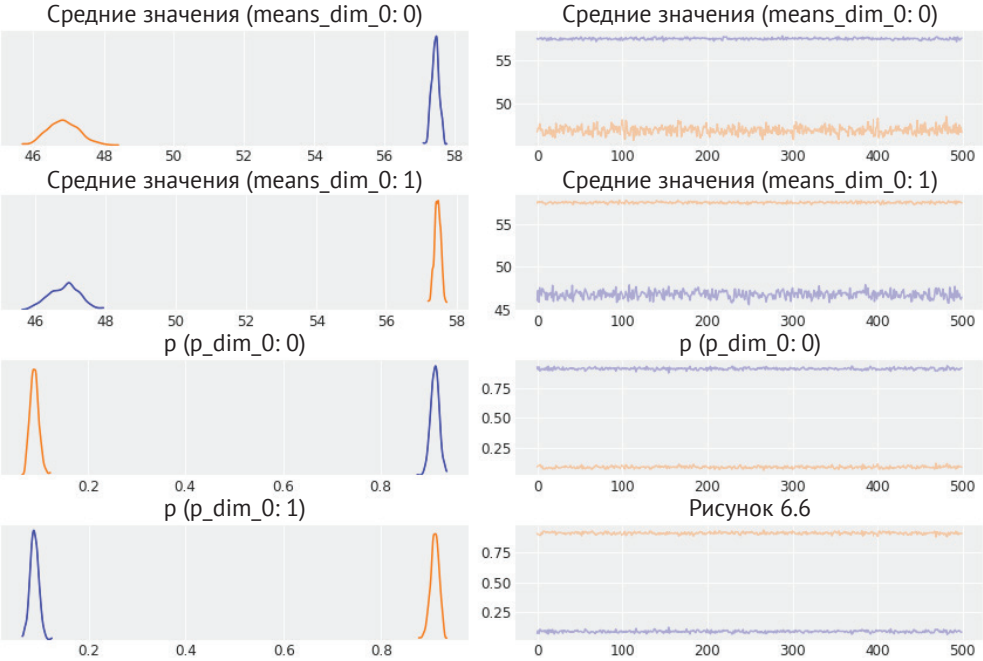


Рис. 6.6

Также вычислим обобщающие итоговые характеристики для рассматриваемой модели и сравним их с аналогичными обобщающими характеристиками для модели `model_mgp` из следующего раздела:

```
az.summary(trace_mgp, varnames)
```

Таблица 6.1

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
<code>means[0]</code>	52.12	5.35	2.14	46.24	57.68	1.0	25.19
<code>means[1]</code>	52.14	5.33	2.13	46.23	57.65	1.0	24.52
<code>p[0]</code>	0.50	0.41	0.16	0.08	0.92	1.0	68.91
<code>p[1]</code>	0.50	0.41	0.16	0.08	0.92	1.0	68.91

Неидентифицируемость смешанных моделей

Если внимательно рассмотреть рис. 6.6, то можно заметить некоторое интересное явление. Оба средних значения оцениваются как бимодальные распределения со значениями, приблизительно равными (47, 57.5), а если проверить итоговый обобщающий отчет, полученный с помощью `az.summary`, то мы видим,

что средние арифметические значения обоих средних значений почти одинаковы и приблизительно равны 52. Похожее явление наблюдается и для значений p . Это пример явления, которое в статистике называется неидентифицируемостью параметров (parameter non-identifiability). Это происходит потому, что модель остается той же самой, если компонент 1 имеет среднее значение 47, а компонент 2 имеет среднее значение 57.5, и наоборот. Обе ситуации абсолютно равнозначны. В контексте смешанных моделей это явление также известно под названием проблемы (самопроизвольного) перемещения меток (label-switching problem). Мы уже наблюдали пример неидентифицируемости параметров в главе 3, когда рассматривали линейные модели и переменные с высокой степенью корреляции.

По возможности модель должна определяться так, чтобы исключить неидентифицируемость. Для смешанных моделей существуют два способа сделать это:

- принудительное упорядочение компонентов. Например, размещение средних значений всех компонентов в строго возрастающем порядке;
- использование информативных априорных распределений.



Параметры в модели являются неидентифицируемыми, если одна и та же функция правдоподобия получена для более одного варианта выбора параметров модели.

При использовании библиотеки PyMC3 одним из простых способов принудительного упорядочения компонентов является применение функции `pm.potential()`. Потенциал – это произвольный коэффициент (фактор), который добавляется в функцию правдоподобия без добавления переменной в модель. Основное различие между правдоподобием и потенциалом заключается в том, что потенциал не обязательно зависит от данных, тогда как правдоподобие зависит всегда. Потенциал можно использовать для усиления ограничений. Например, можно определить потенциал таким способом: если ограничение не нарушено, то в функцию правдоподобия добавляется нулевой коэффициент (фактор), иначе добавляется фактор $-\infty$. В результате модель определяет параметр (или сочетание нескольких параметров), нарушающий заданные ограничения, как невозможный для использования, в то же время модель остается неизменной по отношению к остальным значениям:

```
clusters = 2
with pm.Model() as model_mgp:
    p = pm.Dirichlet('p', a=np.ones(clusters))
    means = pm.Normal('means', mu=np.array([.9, 1]) * cs_exp.mean(), sd=10, shape=clusters)
    sd = pm.HalfNormal('sd', sd=10)
    order_means = pm.Potential('order_means',
                               tt.switch(means[1]-means[0] < 0, -np.inf, 0))
    y = pm.NormalMixture('y', w=p, mu=means, sd=sd, observed=cs_exp)
    trace_mgp = pm.sample(1000, random_seed=123)

varnames = ['means', 'p']
az.plot_trace(trace_mgp, varnames)
```

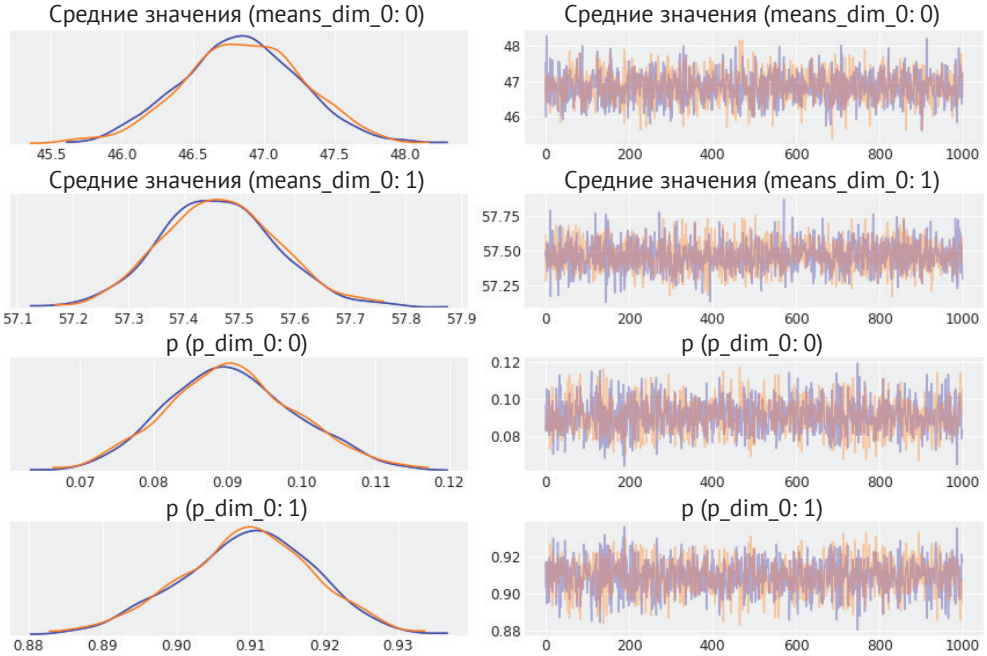


Рис. 6.7

Вычислим итоговые обобщающие характеристики для этой модели:

```
az.summary(trace_mgp)
```

Таблица 6.2

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
means[0]	46.84	0.42	0.01	46.04	47.61	1328.0	1.0
means[1]	57.46	0.10	0.00	57.26	57.65	2162.0	1.0
p[0]	0.09	0.01	0.00	0.07	0.11	1365.0	1.0
p[1]	0.91	0.01	0.00	0.89	0.93	1365.0	1.0
sd	3.65	0.07	0.00	3.51	3.78	1959.0	1.0

Другое ограничение, которое может оказаться полезным, – обеспечение ненулевой вероятности всех компонентов, другими словами, каждый компонент смешанной модели должен иметь, как минимум, одно наблюдение, связанное с ним. Это можно сделать, воспользовавшись следующим выражением:

```
p_min = pm.Potential('p_min', tt.switch(tt.min(p) < min_p, -np.inf, 0))
```

Здесь можно установить для `min_p` некоторое произвольно выбранное, но разумное значение, например 0.1 или 0.01.

На рис. 6.4 можно видеть, что значение α управляет концентрацией распределения Дирихле. Плоское априорное распределение в симплексе получается при $\alpha = 1$, как при использовании в модели `model_mgp`. Большие значения α означают более информативные априорные распределения. Эмпирическое доказательство подтверждает, что значения $\alpha \approx 4$ или 10 в общем случае являются вполне подходящим выбором по умолчанию, поскольку при таких значениях обычно получаются апостериорные распределения, когда каждый компонент имеет, как минимум, одну точку данных, связанную с ним, с одновременным снижением вероятности завышенной оценки числа компонентов.

Как правильно выбрать число K

Один из главных вопросов при использовании конечных смешанных моделей – как правильно определить число компонентов. Предлагается практическое правило: начать с относительно небольшого количества компонентов, затем увеличивать его для улучшения оценки качества подгонки модели. Как обычно качество подгонки модели оценивается с использованием проверок прогнозируемого апостериорного распределения, таких критериев измерения, как WAIC или LOO, а также на основе практического опыта эксперта (группы экспертов), выполняющего моделирование.

Сравним версии модели для $K = \{3, 4, 5, 6\}$. Для этого выполним подгонку модели четыре раза, сохраняя объекты `trace` и `model` для дальнейшего использования:

```
clusters = [3, 4, 5, 6]
models = []
traces = []
for cluster in clusters:
    with pm.Model() as model:
        p = pm.Dirichlet('p', a=np.ones(cluster))
        means = pm.Normal('means', mu=np.linspace(cs_exp.min(), cs_exp.max(), cluster),
                           sd=10, shape=cluster, transform=pm.distributions.transforms.
ordered)
        sd = pm.HalfNormal('sd', sd=10)
        y = pm.NormalMixture('y', w=p, mu=means, sd=sd, observed=cs_exp)
        trace = pm.sample(1000, tune=2000, random_seed=123)
        traces.append(trace)
        models.append(model)
```

Чтобы лучше отобразить воздействие значения K на результат статистического вывода, сравним оценку качества подгонки с аналогичной оценкой, полученной с помощью метода `az.plot_kde`. Также построим диаграмму гауссовых компонентов рассматриваемой смешанной модели:


```

_, ax = plt.subplots(2, 2, figsize=(11, 8), constrained_layout=True)
ax = np.ravel(ax)
x = np.linspace(cs_exp.min(), cs_exp.max(), 200)
for idx, trace_x in enumerate(traces):
    x_ = np.array([x] * clusters[idx]).T

    for i in range(50):
        i_ = np.random.randint(0, len(trace_x))
        means_y = trace_x['means'][i_]
        p_y = trace_x['p'][i_]
        sd = trace_x['sd'][i_]
        dist = stats.norm(means_y, sd)
        ax[idx].plot(x, np.sum(dist.pdf(x_) * p_y, 1), 'C0', alpha=0.1)

    means_y = trace_x['means'].mean(0)
    p_y = trace_x['p'].mean(0)
    sd = trace_x['sd'].mean(0)
    dist = stats.norm(means_y, sd)
    ax[idx].plot(x, np.sum(dist.pdf(x_) * p_y, 1), 'C0', lw=2)
    ax[idx].plot(x, dist.pdf(x_) * p_y, 'k--', alpha=0.7)
    az.plot_kde(cs_exp, plot_kwargs={'linewidth':2, 'color':'k'})

ax=ax[idx])
ax[idx].set_title('K = {}'.format(clusters[idx]))
ax[idx].set_yticks([])
ax[idx].set_xlabel('x')

```

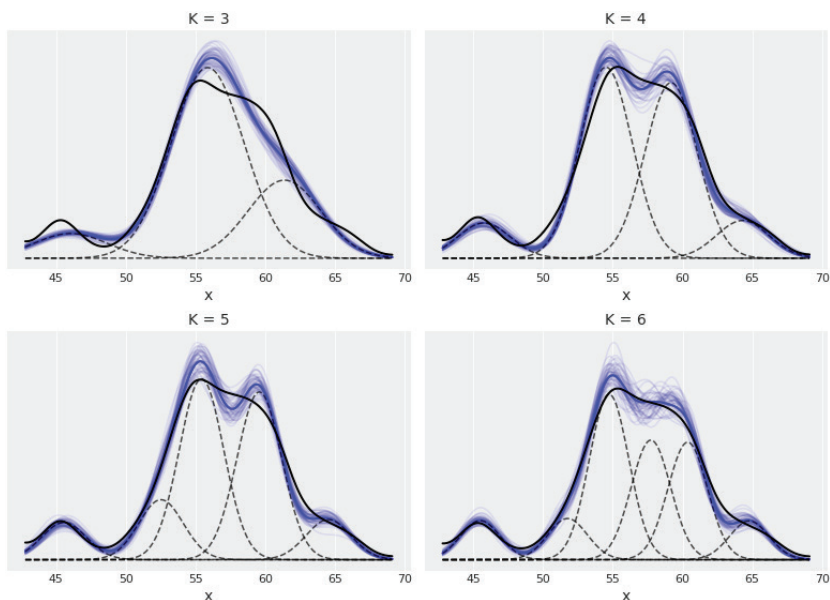


Рис. 6.8

На рис. 6.8 показана диаграмма ядерной оценки плотности (ЯОП) для наблюдаемых данных в виде сплошной черной линии вместе со средним значением подгонки, показанным более широкой (синей) линией, а также выборки из апостериорного распределения, изображаемые полупрозрачными (синими) линиями. Средние значения гауссовых компонентов представлены черными штриховыми линиями. По рис. 6.8 видно, что при $K = 3$ слишком велико несоответствие, а значения $K = 4, 5, 6$ представляют собой более подходящие варианты выбора.

Отметим, что гауссова смешанная модель на диаграмме показывает два центральных пика/выпуклости (с приблизительной абсциссой 55–60), в то время как ЯОП прогнозирует менее значительные (более плоские) пики. Следует подчеркнуть, что это не обязательно является следствием плохой подгонки гауссовой смешанной модели, поскольку ядерные оценки плотности в общем случае настроены на представление более сглаженных плотностей. Как уже отмечалось в главе 5, можно попытаться вычислить и построить диаграммы прогнозируемых апостериорных распределений интересующих нас тестовых количественных характеристик и вычислить байесовские p -значения. На рис. 6.9 показан пример таких вычислений и визуального представления результатов.

```
ppc_mm = [pm.sample_posterior_predictive(traces[i], 1000, models[i]) for i in range(4)]

fig, ax = plt.subplots(2, 2, figsize=(10, 6), sharex=True,
                      constrained_layout=True)
ax = np.ravel(ax)
def iqr(x, a=0):
    return np.subtract(*np.percentile(x, [75, 25], axis=a))

T_obs = iqr(cs_exp)
for idx, d_sim in enumerate(ppc_mm):
    T_sim = iqr(d_sim['y'][:100].T, 1)
    p_value = np.mean(T_sim >= T_obs)
    az.plot_kde(T_sim, ax=ax[idx])
    ax[idx].axvline(T_obs, 0, 1, color='k', ls='--')
    ax[idx].set_title(f'K = {clusters[idx]} \n p-value {p_value:.2f}')
    ax[idx].set_yticks([])
```

На рис. 6.9 можно видеть, что значение $K = 6$ является неплохим вариантом выбора с байесовским p -значением, очень близким к 0.5. В приведенном ниже представлении содержимого объекта DataFrame и на рис. 6.10 также можно видеть, что информационный критерий WAIC указывает на вариант $K = 6$ как на наилучшую версию модели (среди моделей, оцениваемых в этом примере):

```
comp = az.compare(dict(zip(clusters, traces)), method='BB-pseudo-BMA')
comp
```

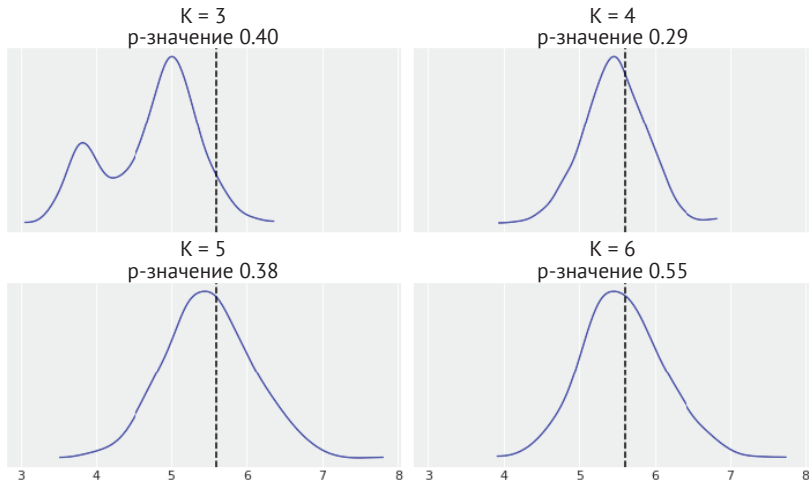


Рис. 6.9

Таблица 6.3

	waic	pwaic	dwaic	weight	se	dse	warning
6	1250	12.368	0	0.948361	62.7354	0	0
5	10259.7	10.3531	9.69981	0.0472388	61.3804	4.6348	0
4	10278.9	7.45718	28.938	0.00440011	60.7985	9.82746	0
3	10356.9	5.90559	106.926	3.19235e – 13	60.9242	18.5501	0

Гораздо чаще понять диаграмму проще, чем таблицу с числами, поэтому построим диаграмму степени различий для моделей в соответствии с информационными критериями WAIC. На рис. 6.10 видно, что модель с шестью компонентами имеет более низкий критерий WAIC по сравнению с остальными моделями, но при этом их графики значительно перекрываются, если рассматривать оцениваемую стандартную ошибку (se). В наибольшей степени это относится к модели с пятью компонентами:

```
az.plot_compare(comp)
```

Отклонение

Рис. 6.10

Смешанные модели и кластеризация

Кластеризация – это часть семейства методов обучения без учителя в задачах статистики и машинного обучения. Кластеризация похожа на классификацию, но все же немного отличается от нее тем, что нам неизвестны правильные метки.

Кластеризация, или кластерный анализ, – это задача анализа данных, заключающаяся в группировке объектов таким способом, чтобы объекты в одной группе были ближе друг к другу, чем объекты из других групп. Такие группы называются кластерами, а степень (мера) близости может вычисляться многими различными способами. Например, можно использовать такую метрику, как евклидово расстояние. Если вместо подобной метрики берется вероятностная характеристика, возникает смешанная модель, являющаяся естественно сформированным кандидатом для решения задач кластеризации.

Выполнение процесса кластеризации с использованием вероятностных моделей называют кластеризацией на основе моделей. Применение вероятностной модели позволяет вычислить вероятность каждой точки данных, принадлежащей какому-либо кластеру. Эта процедура известна под названием нечеткая кластеризация (soft clustering, fuzzy clustering), в отличие от строгой кластеризации (hard clustering), когда каждая точка данных принадлежит какому-либо кластеру с вероятностью 0 или 1. Нечеткую кластеризацию можно превратить в строгую, установив некоторое правило или границу. Вероятно, вы помните, что именно так мы поступили, превратив логистическую регрес-

сию в метод классификации, когда воспользовались в качестве граничного значения по умолчанию числом 0.5. Для кластеризации разумным выбором является распределение точки данных в кластер с наивысшей вероятностью.

В общем, когда говорят о кластеризации, то имеют в виду группировку объектов, а при обсуждении смешанных моделей подразумевается объединение простых распределений в модель с более сложным (составным) распределением, либо для идентификации подгрупп, либо просто для получения более гибкой модели для описания исследуемых данных.

СМЕШАННЫЕ МОДЕЛИ С БЕСКОНЕЧНОЙ РАЗМЕРНОСТЬЮ

Для некоторых задач, например при попытке кластеризации рукописных цифр, достаточно легко точно определить количество групп, ожидаемых при исследовании массива данных. В других задачах есть возможность сделать удачное предположение, например можно узнать, что выборка данных по цветам рода ирисов была выполнена по региону, где растут только три вида ирисов, следовательно, использование трех компонентов является разумным отправным пунктом. Когда нет уверенности в количестве компонентов, можно использовать методы выбора модели, которые помогут определить число групп. Но существуют и такие задачи, в которых предварительный выбор количества групп может оказаться нецелесообразным и вместо этого необходимо получить оценку этого количества по имеющимся данным. Байесовское решение задач такого типа связано с процессом Дирихле.

Процесс Дирихле

Все рассматриваемые до настоящего момента модели являлись параметрическими моделями. Эти модели имеют точно определенное число параметров, для которых необходимо вычислить оценку, например точное количество кластеров. Но модели могут быть и непараметрическими, возможно, более подходящим названием для таких моделей было бы «модели с произвольным числом параметров», но термин «непараметрические» появился раньше и уже стал общепринятым. Непараметрические модели – это модели с теоретически бесконечным количеством параметров. На практике с помощью самих исследуемых данных каким-либо образом сокращают теоретически бесконечное число параметров до некоторого конечного числа. Другими словами, сами данные определяют решение по реальному количеству параметров, поэтому непараметрические модели чрезвычайно гибки. В этой книге мы рассмотрим два примера непараметрических моделей: процесс Гаусса (это тема следующей главы) и процесс Дирихле, который мы начнем изучать прямо сейчас.

Распределение Дирихле представляет собой n -мерное обобщение бета-распределения, а процесс Дирихле – это бесконечномерное обобщение распределения Дирихле. Распределение Дирихле – это распределение вероятностей в пространстве вероятностей, в то время как процесс Дирихле – это распреде-

ление вероятностей в пространстве распределений. Это означает, что единственным выводом, извлекаемым из процесса Дирихле, в действительности является распределение. Для конечных смешанных моделей мы использовали распределение Дирихле, чтобы назначить априорное распределение для точно определенного количества кластеров или групп. Процесс Дирихле является способом присваивания априорного распределения для неопределенного (нефиксированного) числа кластеров, более того, мы можем мысленно интерпретировать процесс Дирихле как способ выборки из априорного распределения или из нескольких априорных распределений.

Прежде чем перейти к работе с реальной непараметрической смешанной моделью, необходимо рассмотреть некоторые подробности процесса Дирихле. Формальное определение процесса Дирихле может показаться несколько непонятным, если вы не очень хорошо знакомы с теорией вероятностей, поэтому вместо строгого определения позволю себе описать несколько свойств процесса Дирихле, наиболее важных для понимания его роли в создании смешанных моделей:

- процесс Дирихле – это распределение, реализацией которого являются распределения вероятностей в отличие, например, от гауссова распределения, результатом которого является действительное число;
- процесс Дирихле определяется базовым распределением $\mathcal{H}$ и положительным действительным числом α , которое называется параметром концентрации (это аналог параметра концентрации в распределении Дирихле);
- $\mathcal{H}$ – это ожидаемое значение процесса Дирихле, то есть процесс Дирихле генерирует распределения на основе базового распределения, являющегося в некоторой степени аналогом среднего значения гауссового распределения;
- параметр α регулирует степень концентрации – при его увеличении реализации распределений становятся менее концентрированными;
- на практике процесс Дирихле всегда генерирует дискретные распределения;
- при предельном значении $\alpha \rightarrow \infty$ реализации, генерируемые процессом Дирихле, будут равны базовому распределению, следовательно, если базовое распределение непрерывно, то процесс Дирихле сгенерирует непрерывное распределение. Поэтому профессиональные математики говорят, что распределения, генерируемые процессом Дирихле, *почти наверное* являются дискретными. На практике, поскольку α является конечным числом, мы всегда будем работать с дискретными распределениями.

Чтобы придать этим свойствам большую конкретность, еще раз рассмотрим диаграмму категориального распределения на рис. 6.3. Такое распределение можно полностью определить, указывая позицию по оси x и соответствующую ей высоту по оси y . Для категориального распределения позиции по оси x огра-

ничены целыми числами, а сумма всех соответствующих высот должна быть равна 1. Сохраним последнее ограничение, но немного смягчим первое. Для генерации позиций по оси x сформируем выборку из базового распределения $\mathcal{H}$. Теоретически $\mathcal{H}$ может быть любым распределением по нашему выбору, следовательно, если выбрано гауссово распределение, то позиции могут соответствовать любым значениям числовой оси (действительным числам). Если же выбрано бета-распределение, то позиции ограничены интервалом $[0, 1]$, а приняв в качестве базового распределение Пуассона, мы ограничим позиции неотрицательными целыми числами $\{0, 1, 2, \dots\}$.

До сих пор никаких затруднений не встречалось, но как выбрать значения по оси y ? Выполним мысленный эксперимент, известный под названием процесс постепенного разделения стержня (stick-breaking process). Предположим, что у нас есть стержень длиной 1 и мы разделяем (ломаем) его на две части (не обязательно равные). Одну часть отбросим, а другую еще раз разделим (разломим) на две, потом еще раз и т. д. На практике мы не можем продолжать этот процесс до бесконечности – стержень ломается только до некоторого предварительно определенного значения длины K , но общая идея понятна. Для управления процессом постепенного разделения стержня воспользуемся параметром α . При увеличении значения α стержень разделяется (ломается) на все более мелкие части. Таким образом, при $\lim_{\alpha \rightarrow 0}$ стержень не ломается совсем, а при $\lim_{\alpha \rightarrow \infty}$ стержень ломается на бесконечное количество частей. На рис. 6.11 показаны четыре диаграммы, представляющие процесс Дирихле, для четырех различных значений α . Код, генерирующий эти диаграммы, будет объяснен несколько позже, но сначала сосредоточимся на понимании того, что сообщают нам эти выборки о процессе Дирихле:

```
def stick_breaking_truncated( $\alpha$ , H, K):
```

```
    """
    Сокращенное представление процесса stick-breaking для визуализации процесса Дирихле
    Параметры
    -----
     $\alpha$  : float
        параметр концентрации
    H : scipy distribution
        Базовое распределение
    K : int
        количество компонентов
    Возвращаемые значения
    -----
    locs : array
        позиции
    w : array
        вероятности
    """
     $\beta$ s = stats.beta.rvs(1,  $\alpha$ , size=K)
    w = np.empty(K)
    w =  $\beta$ s * np.concatenate([1., np.cumprod(1 -  $\beta$ s[:-1])])
    locs = H.rvs(size=K)
    return locs, w
```

```
# Параметры процесса Дирихле
K = 500
H = stats.norm
alphas = [1, 10, 100, 1000]

_, ax = plt.subplots(2, 2, sharex=True, figsize=(10, 5))
ax = np.ravel(ax)
for idx, a in enumerate(alphas):
    locs, w = stick_breaking_truncated(a, H, K)
    ax[idx].vlines(locs, 0, w, color='C0')
    ax[idx].set_title('a = {}'.format(a))

plt.tight_layout()
```

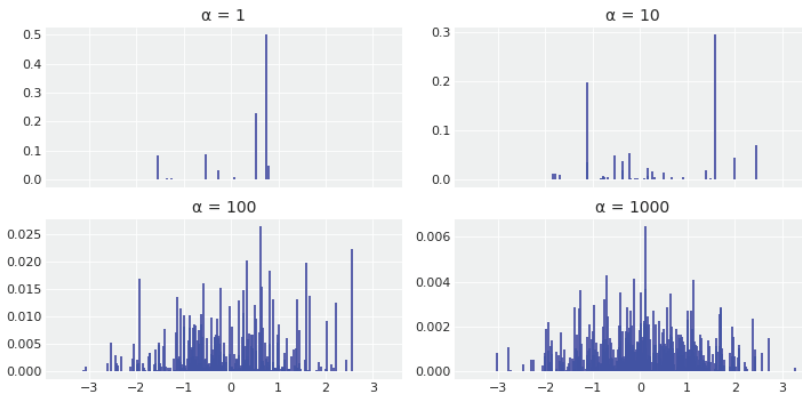


Рис. 6.11

На рис. 6.11 можно видеть, что процесс Дирихле является дискретным распределением. При увеличении α получаем распределение с большим размахом и меньшими частями стержня. Обратите внимание на изменение масштаба по оси y . Также напомним, что общая длина стержня строго определена, как равная 1. Базовое распределение управляет позициями по оси x , так как эти позиции выбираются из базового распределения. На рис. 6.11 можно видеть, что параметр α увеличивает сходство формы распределений процесса Дирихле с базовым распределением $\mathcal{H}$ все больше и больше. Можно надеяться, что при $\lim_{\alpha \rightarrow \infty}$ мы будем наблюдать точную форму базового распределения.



Можно мысленно интерпретировать процесс Дирихле как априорное распределение случайного распределения f , где базовое распределение $\mathcal{H}$ – это ожидаемая форма f , а параметр концентрации α определяет степень нашей уверенности в правильности выбора априорного распределения.

На рис. 6.1 показано, что если поместить гауссово распределение поверх каждой точки данных, а затем просуммировать все размещенные гауссовы распределения, то можно получить аппроксимацию распределения данных. Можно воспользоваться процессом Дирихле, чтобы проделать нечто подоб-

ное, но вместо размещения гауссова распределения поверх каждой точки данных можно поместить гауссово распределение в месте расположения каждого «фрагмента стержня» (substick) из реализации процесса Дирихле и промасштабировать или взвесить (отрегулировать с помощью весовых коэффициентов) это гауссово распределение по длине соответствующего фрагмента стержня. Описанная процедура дает обобщенный рецепт создания бесконечной гауссовой смешанной модели. В другом варианте можно заменить гауссово распределение на любое другое распределение, и получить обобщенный способ создания бесконечной смешанной модели. На рис. 6.12 показан пример такой модели, в которой использована комбинация из распределений Лапласа. Распределение Лапласа выбрано произвольно только для того, чтобы закрепить излагаемый здесь принцип и чтобы вы не подумали, что описанный подход ограничен только гауссовыми смешанными моделями.

```

a = 10
H = stats.norm
K = 5

x = np.linspace(-4, 4, 250)
x_ = np.array([x] * K).T
locs, w = stick_breaking_truncated(a, H, K)

dist = stats.laplace(locs, 0.5)
plt.plot(x, np.sum(dist.pdf(x_) * w, 1), 'C0', lw=2)
plt.plot(x, dist.pdf(x_) * w, 'k--', alpha=0.7)
plt.yticks([])

```

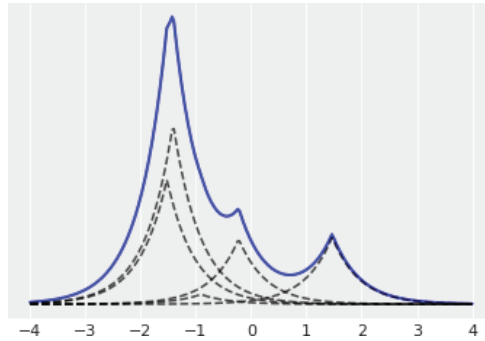


Рис. 6.12

Надеюсь, что читатели хорошо поняли сущность процесса Дирихле и остались неразобранными только мелкие подробности для полного понимания работы функции `stick_break_truncated`. Математически процесс постепенного разделения стержня как наглядная реализация процесса Дирихле может быть представлен в виде следующей формулы:

$$\sum_{k=1}^{\infty} w_k \cdot \delta_{\theta_k}(\theta) = f(\theta) \sim \text{DP}(\alpha, H). \quad (6.2)$$

Здесь:

- δ_{θ_k} – индикаторная (характеристическая) функция, дающая нулевой результат везде, кроме $\delta_{\theta_k}(\theta_k) = 1$, представляющего позиции выборки из базового распределения $\mathcal{H}$;
- вероятности w_k вычисляются по следующей формуле:

$$w_k = \beta'_k \cdot \prod_{i=1}^{k-1} (1 - \beta'_i), \quad (6.3)$$

где:

- w_k – длина фрагмента (части) стержня;
- $\prod_{i=1}^{k-1} (1 - \beta'_i)$ – длина оставшейся части стержня, необходимой для продолжения процесса его разделения (отламывания очередной части);
- β'_k определяет, как разделяется оставшаяся часть стержня;
- $\beta'_k \sim \text{Beta}(1, \alpha)$ – из этого выражения можно видеть, что при увеличении α значение β'_k будет в среднем уменьшаться.

Теперь мы в большей степени готовы к попытке реализации процесса Дирихле средствами библиотеки PyMC3. Сначала определим функцию `stick_breaking`, которая работает на основе PyMC3:

```
N = cs_exp.shape[0]
K = 20

def stick_breaking(a):
    beta = pm.Beta('beta', 1., a, shape=K)
    w = beta * pm.math.concatenate([[1.], tt.extra_ops.cumprod(1. - beta)[: -1]])
    return w
```

Необходимо также определить априорное распределение для параметра концентрации α . Общепринятым выбором для этого параметра является гамма-распределение:

```
with pm.Model() as model:
    alpha = pm.Gamma('alpha', 1., 1.)
    w = pm.Deterministic('w', stick_breaking(alpha))
    means = pm.Normal('means', mu=cs_exp.mean(), sd=10, shape=K)
    sd = pm.HalfNormal('sd', sd=10, shape=K)
    obs = pm.NormalMixture('obs', w, means, sd=sd, observed=cs_exp.values)
    trace = pm.sample(1000, tune=2000, nuts_kwargs={'target_accept': 0.9})
```

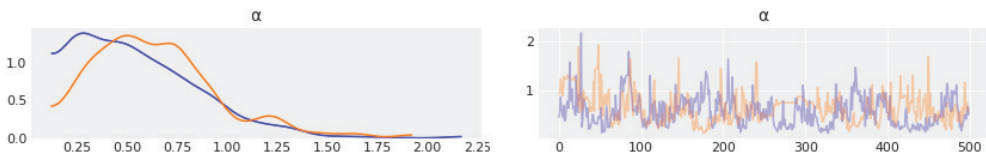


Рис. 6.13

На рис. 6.13 можно видеть, что при относительно малом значении α достаточно лишь нескольких компонентов для описания данных.

Поскольку мы аппроксимируем бесконечный процесс Дирихле с помощью процедуры постепенного разделения стержня, важно проверить тот факт, что значение количества разделяемых частей ($K = 20$ в нашем примере) не вводит какого-либо смещения. Простым способом выполнения такой проверки является построение диаграммы средних арифметических весовых коэффициентов для каждого компонента – для сохранения корректности необходимо иметь несколько компонентов с незначительными весовыми коэффициентами, иначе непременно придется увеличивать количество разделяемых частей. Пример диаграммы такого типа приведен на рис. 6.14. Здесь можно видеть, что только несколько первых компонентов важны, следовательно, можно быть уверенными в том, что выбранное значение верхней границы $K = 20$ достаточно велико для используемой модели и исследуемых данных:

```
plt.figure(figsize=(8, 6))
plot_w = np.arange(K)
plt.plot(plot_w, trace['w'].mean(0), 'o-')
plt.xticks(plot_w, plot_w+1)
plt.xlabel('Component')
plt.ylabel('Average weight')
```

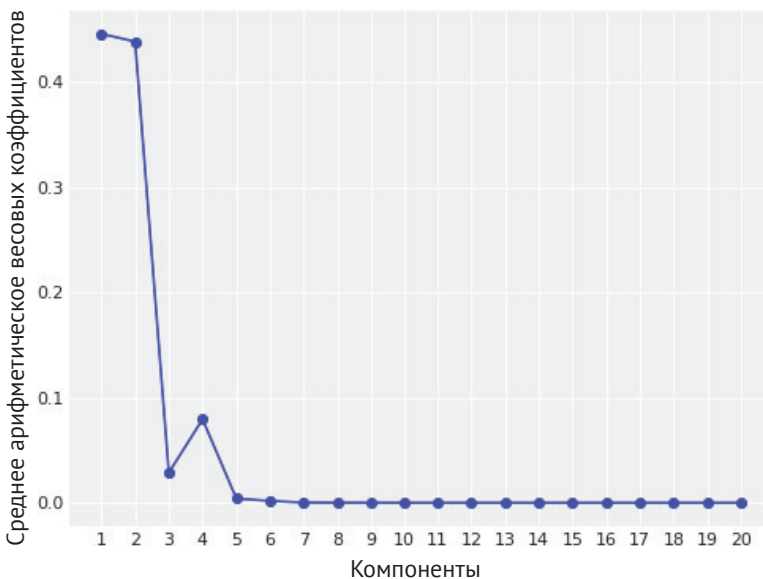


Рис. 6.14

На рис. 6.15 показана оцениваемая плотность среднего значения с использованием модели процесса Дирихле, изображаемая черной линией, вместе с выборками из апостериорного распределения (серые линии) для отобра-

жения неопределенности этой оценки. Эта модель также показывает менее сглаженную кривую плотности по сравнению с ядерной оценкой плотности на рис. 6.2 и рис. 6.8.

```
x_plot = np.linspace(cs.exp.min()-1, cs.exp.max()+1, 200)
post_pdf_contribs = stats.norm.pdf(np.atleast_3d(x_plot),
                                   trace['means'][:, np.newaxis, :],
                                   trace['sd'][:, np.newaxis, :])
post_pdfs = (trace['w'][:, np.newaxis, :] * post_pdf_contribs).sum(axis=-1)
plt.figure(figsize=(8, 6))

plt.hist(cs_exp.values, bins=25, density=True, alpha=0.5)
plt.plot(x_plot, post_pdfs[:,100].T, c='0.5')
plt.plot(x_plot, post_pdfs.mean(axis=0), c='k')

plt.xlabel('x')
plt.yticks([])
```

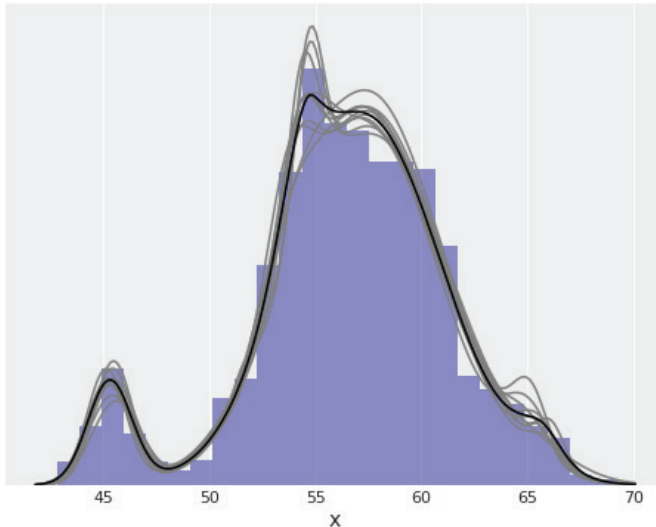


Рис. 6.15

НЕПРЕРЫВНЫЕ СМЕШАННЫЕ МОДЕЛИ

В этой главе основное внимание было уделено дискретным смешанным моделям, но можно использовать и непрерывные смешанные модели. Мы уже знакомы с некоторыми из них, например с распределением с дополнением нулевыми значениями из главы 4, где применялось сочетание распределения Пуассона и процесса генерации нулевых значений. Другой пример – модель робастной логистической регрессии из той же главы, представляющая собой комбинацию двух компонентов: логистической регрессии, с одной стороны,

и случайного прогнозирования – с другой. Отметим, что параметр π не является бинарным переключателем типа вкл/выкл, а в большей степени похож на ручку регулировки микширования, управляющую соотношением долей случайного прогнозирования и логистической регрессии в используемой комбинации. Только при граничных значениях π мы получаем чистое случайное прогнозирование или чистую логистическую регрессию.

Иерархические модели также можно интерпретировать как непрерывные смешанные модели, где параметры в каждой группе извлекаются из непрерывных распределений более высокого уровня. Более конкретный пример – выполнение линейной регрессии для нескольких групп. Можно предположить, что каждая группа обладает собственным коэффициентом угла наклона или что все группы совместно используют одинаковый угол наклона. В альтернативном варианте вместо ограничения задачи рамками двух экстремальных дискретных опций иерархическая модель позволяет эффективно моделировать непрерывное объединение этих экстремальных опций, таким образом, эти экстремальные опции становятся лишь конкретными частными случаями более обобщенной иерархической модели.

Биномиальное бета-распределение и отрицательное биномиальное распределение

Биномиальное бета-распределение – это дискретное распределение, в общем случае используемое для описания количества успешных (положительных) результатов у n испытаний Бернулли, когда вероятность успешного результата p в каждом испытании неизвестна и предполагается соответствующей бета-распределению с параметрами α и β :

$$\text{BetaBinomial}(y|n, \alpha, \beta) = \int_0^1 \text{Bin}(y|p, n) \text{Beta}(p|\alpha, \beta) dp. \quad (6.4)$$

То есть для определения вероятности получения наблюдаемого исходного результата y вычисляется среднее арифметическое по всем возможным (и непрерывным) значениям p . Таким образом, биномиальное бета-распределение может рассматриваться как непрерывная смешанная модель. Если название биномиальной бета-модели кажется вам знакомым, то причина этого заключается в том, что мы уже встречались с ней в первых двух главах книги. Это модель, которая использовалась в задаче о подбрасывании монеты, хотя мы явно применяли бета-распределение и биномиальное распределение вместо предварительно объединенного биномиального бета-распределения.

Подобным образом можно получить и отрицательное биномиальное распределение, определяя его как объединение гамма-распределения и распределения Пуассона. В этой модели формируется комбинация распределений Пуассона, а параметр средней интенсивности событий определяется гамма-распределением. Это распределение часто используется для устранения общеизвестной проблемы, возникающей при обработке счетных (дискретных) данных. Эта проблема известна под названием чрезмерная вариативность

(overdispersion). Предположим, что вы используете распределение Пуассона для моделирования дискретных данных и через некоторое время обнаруживаете, что уровень дисперсии в исследуемых данных превышает допустимый в модели уровень. Проблема применения распределения Пуассона в такой ситуации состоит в том, что среднее значение и дисперсия взаимосвязаны (фактически они описываются одним и тем же параметром). Поэтому одним из способов устранения такой проблемы является моделирование данных как (непрерывного) объединения распределений Пуассона с параметрами средней интенсивности событий, получаемыми из гамма-распределения. Это дает нам обоснование применения отрицательного биномиального распределения. Поскольку теперь мы рассматриваем объединение распределений, наша модель обладает большей гибкостью и способна лучше адаптироваться к реально наблюдаемому среднему значению и дисперсии данных. Биномиальное бета-распределение и отрицательное биномиальное распределение можно использовать как часть линейных моделей, а кроме того, для обоих этих распределений существуют версии с дополнением нулевыми значениями. Не менее важен тот факт, что оба распределения реализованы в библиотеке РумСЗ как полностью готовые к практическому применению распределения.

t-распределение Стьюдента

Ранее t-распределение Стьюдента мы представили как надежную альтернативу гауссову распределению. Но оказывается, что t-распределение Стьюдента также можно считать непрерывной смешанной моделью. В этом случае имеем следующую формулу:

$$t_v(y|\mu, \sigma) = \int_0^\infty N(y|\mu, \sigma) \text{Inv}\chi^2(\sigma|v) dv. \quad (6.5)$$

Отметим, что эта формула похожа на выражение для отрицательного биномиального распределения из предыдущего раздела, за исключением того, что здесь используется нормальное распределение с параметрами μ и σ и распределение $\text{Inv}\chi^2$ с параметром v . Из распределения $\text{Inv}\chi^2$ выполняется выборка значений σ – параметра, известного как степень свободы или, как мы предпочитаем называть его, параметр нормальности. Параметр v , как и параметр p для биномиального бета-распределения, равнозначен скрытой переменной z в конечных смешанных моделях. Для некоторых конечных смешанных моделей также возможно ограничение распределения в соответствии со скрытой переменной до начала оказания воздействия, которое может привести к упрощению выборки модели, как мы уже наблюдали выше в примере с ограниченной смешанной моделью.

РЕЗЮМЕ

Многие задачи можно описать как обобщенные совокупности, состоящие из отдельных подмножеств. Если известно, какому подмножеству принадлежит

каждый наблюдаемый элемент данных, то можно смоделировать каждое подмножество как отдельную группу. Но в большинстве случаев у нас нет прямого доступа к подобной информации, поэтому больше подходит методика моделирования «неизвестных» данных с помощью смешанных моделей. Смешанные модели можно применять для попытки выделения истинных подмножеств в совокупности данных или как обобщенный статистический прием для моделирования сложных распределений посредством объединения более простых распределений. Можно даже попробовать создать некоторое усредненное решение.

В этой главе смешанные модели были разделены на три класса – конечные смешанные модели, смешанные модели с бесконечной размерностью и непрерывные смешанные модели. Конечные смешанные модели – это конечномерное взвешенное объединение двух и более распределений, при этом каждое распределение (компонент) представляет отдельную подгруппу данных. Теоретически компоненты могут быть почти любыми объектами, которые мы считаем полезными: от простых распределений, таких как гауссово распределение или распределение Пуассона, до более сложных объектов, например иерархических моделей или нейронных сетей. Теоретически для решения смешанной модели достаточно правильно поставить в соответствие каждой точке данных один из компонентов – это можно сделать, введя скрытую переменную z . Для z используется категориальное распределение, являющееся наиболее обобщенным дискретным распределением с априорным распределением Дирихле, которое представляет собой n -мерное обобщение бета-распределения. Формирование выборки дискретной переменной z может оказаться проблематичным, поэтому, возможно, удобнее ограничить эту переменную. В библиотеку PyMC3 включено нормальное смешанное распределение и смешанное распределение, которое выполняет такое ограничение автоматически, упрощая создание смешанных моделей в тех случаях, когда использование смешанной модели может привести к проблеме (самопроизвольного) перемещения меток, то есть к одной из форм неидентифицируемости. Одним из способов устранения неидентифицируемости является принудительное упорядочение компонентов, которое можно выполнить с помощью функции библиотеки PyMC3 `pm.potential()` или с помощью упорядочивающего преобразования (см. соответствующий раздел Jupyter Notebook в репозитории книги).

Одна из трудностей использования конечных смешанных моделей заключается в правильном определении количества компонентов. Вариант решения – сравнение набора моделей с оцениваемым количеством компонентов, а оценка должна выводиться с учетом возможного доступного уровня наших знаний и конкретной задачи. Другой вариант – попытка автоматической оценки количества компонентов по имеющимся данным. По этой причине была введена концепция процесса Дирихле, используемая для рассуждений с точки зрения бесконечных смешанных моделей. Процесс Дирихле – это версия распределе-

ния Дирихле с бесконечной размерностью, которую можно использовать для создания непараметрических смешанных моделей.

В конце главы были кратко рассмотрены модели, которые можно интерпретировать как непрерывные смешанные модели: биномиальное бета-распределение (ранее использованное для решения задачи о подбрасывании монеты), отрицательное биномиальное распределение, t -распределение Стюдента и даже иерархические модели.

УПРАЖНЕНИЯ

1. Сгенерируйте синтетическое распределение из объединения трех гауссовых распределений. В соответствующем разделе Jupyter Notebook в репозитории к этой главе изучите пример, демонстрирующий решение этой задачи. Выполните подгонку конечной гауссовой смешанной модели с 2, 3 и 4 компонентами.
2. Используйте информационные критерии WAIC и LOO для сравнения результатов, полученных в упражнении 1.
3. Внимательно изучите и выполните следующие примеры из документации на библиотеку PyMC3 (<https://pymc-devs.github.io/pymc3/examples>):
 - Marginalized Gaussian Mixture Model (https://docs.pymc.io/notebooks/marginalized_gaussian_mixture_model.html);
 - Dependent density regression (https://docs.pymc.io/notebooks/dependent_density_regression.html);
 - Gaussian Mixture Model with ADVI (<https://docs.pymc.io/notebooks/gaussian-mixture-model-adv.html>) (более подробную информацию об ADVI см. в главе 8 «Механизмы статистического вывода»).
4. Выполните упражнение 1 с использованием процесса Дирихле.
5. Предположим, что нам неизвестны правильные виды/метки в наборе данных Iris. Используйте смешанную модель для кластеризации трех видов ирисов, выбрав один из признаков по вашему усмотрению (например, длину чашелистика).
6. Выполнить упражнение 5, но с использованием двух признаков.

Глава 7

.....

Гауссовы процессы

«Ты одинок? У тебя есть ты сам. Бесконечно много самих тебя».

– Рик Санчес

(по крайней мере, тот, что из измерения C-137),
главный герой мультипликационного сериала
«Рик и Морти»

В предыдущей главе мы рассмотрели процесс Дирихле, бесконечномерное обобщение распределения Дирихле, которое можно использовать для установления априорного распределения для неизвестных непрерывных распределений. В этой главе рассматривается гауссов процесс – бесконечномерное обобщение гауссова распределения, которое можно использовать для установления априорного распределения для неизвестных функций. Процесс Дирихле и гауссов процесс применяются в байесовской статистике для создания гибких моделей, в которых количество параметров разрешается увеличивать при росте объема данных.

В этой главе рассматриваются следующие темы:

- функции как вероятностные объекты;
- ядра;
- гауссовы процессы с гауссовыми правдоподобиями;
- гауссовы процессы с негауссовыми правдоподобиями.

ЛИНЕЙНЫЕ МОДЕЛИ И НЕЛИНЕЙНЫЕ ДАННЫЕ

В главах 3 и 4 мы рассматривали создание моделей следующей обобщенной формы:

$$\theta = \psi(\varphi(X)\beta). \quad (7.1)$$

Здесь θ – параметр для некоторого распределения вероятностей (например, для среднего значения гауссова распределения), параметр p биномиального распределения, параметр средней интенсивности событий распределения Пуассона и т. п. Мы называем ψ функцией обратной связи, а φ – это функция, вычисляющая квадратный корень, или полиномиальная функция. Для случая простой линейной регрессии ψ – функция тождественного отображения.

Подгонку (или обучение) байесовской модели можно рассматривать как поиск апостериорного распределения весовых коэффициентов β , и этот процесс называют взвешенным представлением аппроксимирующих функций. Как мы уже наблюдали ранее, в примере с применением полиномиальной регрессии при определении φ как нелинейной функции можно отобразить результаты (выходные данные) в пространство признаков. Затем мы устанавливаем связь (и регулируем ее) линейного отношения в пространстве признаков, которое нелинейно, с действительным пространством. Мы видели, что при использовании полинома наиболее подходящей степени можно точно подогнать (отрегулировать) любую функцию. Но если при этом не применяется некоторая форма регуляризации (например, использование априорных распределений), то можно прийти к моделям, которые «запоминают» обрабатываемые данные, или, другими словами, к моделям с очень слабыми обобщающими свойствами.

Гауссовы процессы обеспечивают принципиальное решение для моделирования произвольных функций, позволяя самим данным эффективно определять сложность этих функций, одновременно устраняя или, по крайней мере, минимизируя возможную переподгонку.

В следующих разделах подробно рассматриваются гауссовы процессы с исключительно практической точки зрения. Мы почти не будем касаться математических основ и подробностей гауссовых процессов. Для изучения более формального математического обоснования обращайтесь к изданиям и ресурсам, указанным в главе 9 «Что дальше?».

ФУНКЦИИ МОДЕЛИРОВАНИЯ

Изучение гауссовых процессов начнем с первоначального описания способа представления функций как вероятностных объектов. Можно мысленно представить себе функцию f как отображение набора входных данных x в набор выходных данных y . Таким образом, можно записать:

$$y = f(x). \quad (7.2)$$

Один из способов представления функций – последовательное перечисление (список, таблица) каждого значения x_i с соответствующим значением y_i . Вероятно, вы помните этот способ представления функций, изучаемый в средней школе.

Таблица 7.1

x	y
0.00	0.46
0.33	2.60
0.67	5.90
1.00	7.91

В общем случае значения x и y размещены на действительной числовой прямой, следовательно, можно считать функцию (потенциально) бесконечным и упорядоченным списком пар значений (x_i, y_i) . Порядок в списке важен, потому что если произвольно переставлять значения, то будут получаться совершенно другие функции.

Функцию также можно представить в виде (потенциально) бесконечного массива, проиндексированного по значениям x , с весьма важным свойством: значения x не ограничены целыми числами, они могут быть действительными числами.

Используя эти описания, можно представить в числовом виде практически любую необходимую функцию. Но что, если необходимо представить функции в вероятностной форме? Тогда можно сделать то же самое, но отображение должно иметь вероятностную сущность. То есть можно установить каждое значение y_i как случайной переменной с гауссовым распределением с заданным средним значением и дисперсией. Таким образом, мы получаем уже не описание единственной конкретной функции, а описание семейства распределений.

Чтобы придать приведенному выше объяснению большую наглядность, используем фрагмент кода на языке Python для создания и визуального отображения двух примеров таких функций:

```
np.random.seed(42)
x = np.linspace(0, 1, 10)

y = np.random.normal(0, 1, len(x))
plt.plot(x, y, 'o-', label='the first one')

y = np.zeros_like(x)
for i in range(len(x)):
    y[i] = np.random.normal(y[i-1], 1)
plt.plot(x, y, 'o-', label='the second one')

plt.legend()
```

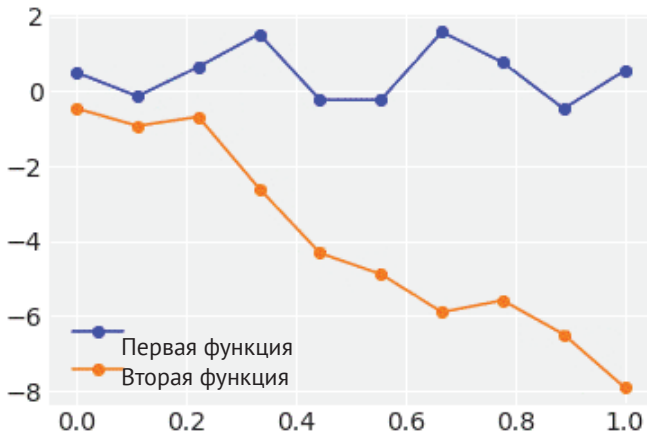


Рис. 7.1

На рис. 7.1 показано, что исследуемые в этом примере функции, использующие выборки из гауссовых распределений, не выглядят странно или нелепо, поэтому, возможно, мы выбрали правильный способ. Тем не менее подход, примененный для генерации диаграммы на рис. 7.1, ограничен и недостаточно гибок. Если ожидаются реальные функции для определения некоторой структуры или образца, то способ выражения первой функции не позволяет установить какие-либо отношения между точками данных. В действительности каждая точка совершенно независима от всех остальных точек, поскольку мы получили всего лишь 10 независимых элементов выборки из обобщенного одномерного гауссова распределения. Для второй функции была введена некоторая зависимость. Среднее значение точки y_{i+1} является значением y_i . Но в дальнейшем мы увидим, что существует более обобщенная методика захвата (определения) зависимостей (и не только между смежными точками).

Прежде чем продолжить, ответим на вопрос: почему мы используем именно гауссово распределение, а не какое-либо другое распределение вероятностей? Во-первых, ограничиваясь работой с гауссовыми распределениями, мы не теряем гибкости при определении функций разнообразных форм, так как каждая точка потенциально обладает собственным средним значением и дисперсией. Во-вторых, потому что работа с гауссовыми распределениями очень удобна с математической точки зрения.

Многомерные гауссовы распределения и функции

На рис. 7.1 была представлена функция, использующая 1-мерное гауссово распределение для получения n элементов выборки. Другим вариантом решения является использование многомерного, то есть n -мерного, гауссова распределения для получения выборки в виде вектора длиной n . На практике может потребоваться генерация диаграммы, подобной изображенной на рис. 7.1, но с заменой `np.random.normal(0, 1, len(x))` на `np.random.multivariate_normal(np.zeros_like(x), np.eye(len(x)))`.

Вообще говоря, первое выражение равнозначно второму, но после замены можно воспользоваться ковариационной матрицей как способом кодирования информации о том, как точки данных связаны друг с другом. Определяя ковариационную матрицу как `np.eye(len(x))`, мы изначально полагаем, что каждая из 10 наблюдаемых точек имеет дисперсию 1, а дисперсия между ними (то есть их ковариации) равна 0 (следовательно, они независимы). Если заменить эти нули на другие (положительные) числа, то можно было бы получить ковариации, дающие совершенно другую картину. Таким образом, для представления функций в вероятностном виде необходимо многомерное гауссово распределение с соответствующей ковариационной матрицей, как мы увидим в следующем разделе.

Ковариационные функции и ядра

На практике ковариационные матрицы определяются с использованием функций, называемых ядрами (kernels). В статистической литературе встречаются

различные определения ядер с немного отличающимися математическими свойствами. Для наших целей определим, что ядро – это в основном симметричная функция, которая принимает два входных значения и возвращает ноль, если входные значения одинаковы, или положительное число в противном случае. Если эти условия выполнены, то можно интерпретировать выходные данные функции ядра как меру схожести между двумя входными значениями.

Среди множества доступных ядер наиболее часто используемым является экспоненцированное квадратическое ядро:

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right). \quad (7.3)$$

Здесь $\|x - x'\|^2$ – среднеквадратическое евклидово расстояние:

$$\|x - x'\|^2 = (x_1 - x'_1)^2 + (x_2 - x'_2)^2 + \dots + (x_n - x'_n)^2. \quad (7.4)$$

На первый взгляд может показаться неочевидным, но экспоненцированное квадратическое ядро описывается формулой, похожей на формулу гауссова распределения (см. формулу 1.3). Поэтому такое ядро иногда называют гауссовым ядром. Сомножитель ℓ в знаменателе известен как линейный масштаб (или ширина окна, или дисперсия), который управляет шириной ядра.

Чтобы лучше понять роль ядер, определим функцию Python для вычисления экспоненцированного квадратического ядра:

```
def exp_quad_kernel(x, knots, l=1):
    """exponentiated quadratic kernel"""
    return np.array([np.exp(-(x-k)**2 / (2*l**2)) for k in knots])
```

Следующий фрагмент кода и рис. 7.2 демонстрируют, как выглядит ковариационная матрица 4×4 для различных входных данных. Набор входных данных I относительно прост и состоит из значений $[-1, 0, 1, 2]$. После тщательного изучения этого примера вы должны попробовать выполнить его с другими входными данными (см. упражнение 1).

```
data = np.array([-1, 0, 1, 2])
cov = exp_quad_kernel(data, data, 1)

_, ax = plt.subplots(1, 2, figsize=(12, 5))
ax = np.ravel(ax)

ax[0].plot(data, np.zeros_like(data), 'ko')
ax[0].set_yticks([])
for idx, i in enumerate(data):
    ax[0].text(i, 0+0.005, idx)
ax[0].set_xticks(data)
ax[0].set_xticklabels(np.round(data, 2))
#ax[0].set_xticklabels(np.round(data, 2), rotation=70)

ax[1].grid(False)
im = ax[1].imshow(cov)
colors = ['w', 'k']
```

```

for i in range(len(cov)):
    for j in range(len(cov)):
        ax[1].text(j, i, round(cov[i, j], 2),
                    color=colors[int(im.norm(cov[i, j]) > 0.5)],
                    ha='center', va='center', fontdict={'size': 16})
ax[1].set_xticks(range(len(data)))
ax[1].set_yticks(range(len(data)))
ax[1].axis.tick_top()

```

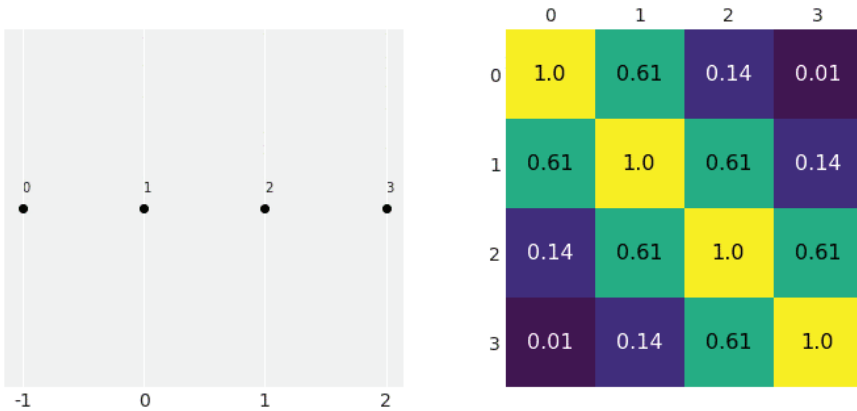


Рис. 7.2

В левой части рис. 7.2 показаны входные данные, значения по оси x , представляющие значения каждой точки данных с указанием порядкового номера около каждой точки (числа от 0 до 3). В правой части рис. 7.2 размещена тепловая карта, представляющая ковариационную матрицу, полученную с помощью экспоненцированного квадратического ядра. Более светлый цвет обозначает более высокую ковариацию. Здесь можно видеть, что тепловая карта симметрична, а на главной диагонали расположены наибольшие значения. Значение каждого элемента ковариационной матрицы обратно пропорционально расстоянию между точками, а значения на главной диагонали являются результатом сравнения каждой точки данных с самой собой. Мы получаем самое близкое расстояние 0 и самую высокую ковариацию 1 для этого ядра. Для других ядер возможны иные значения.



Ядро выполняет преобразование расстояния между точками данных по оси x в значения ковариаций для значений ожидаемой функции (по оси y). Таким образом, чем ближе две точки по оси x , тем более похожими ожидаются их значения по оси y .

Резюмируя все описанное выше, можно сказать, что мы убедились в возможности использования многомерных нормальных распределений с заданной ковариацией для моделирования функций. Кроме того, можно использовать функции ядер для создания ковариаций. В следующем примере функция `exp_quad_kernel` применяется для определения ковариационной матрицы мно-

гомерного нормального распределения, затем используются элементы выборки из этого распределения для представления функций:

```
np.random.seed(24)
test_points = np.linspace(0, 10, 200)
fig, ax = plt.subplots(2, 2, figsize=(12, 6), sharex=True, sharey=True, constrained_
layout=True)
ax = np.ravel(ax)
for idx, l in enumerate((0.2, 1, 2, 10)):
    cov = exp_quad_kernel(test_points, test_points, l)
    ax[idx].plot(test_points, stats.multivariate_normal.rvs(cov=cov, size=2).T)
    ax[idx].set_title(f'l={l}')
fig.text(0.51, -0.03, 'x', fontsize=16)
fig.text(-0.03, 0.5, 'f(x)', fontsize=16)
```

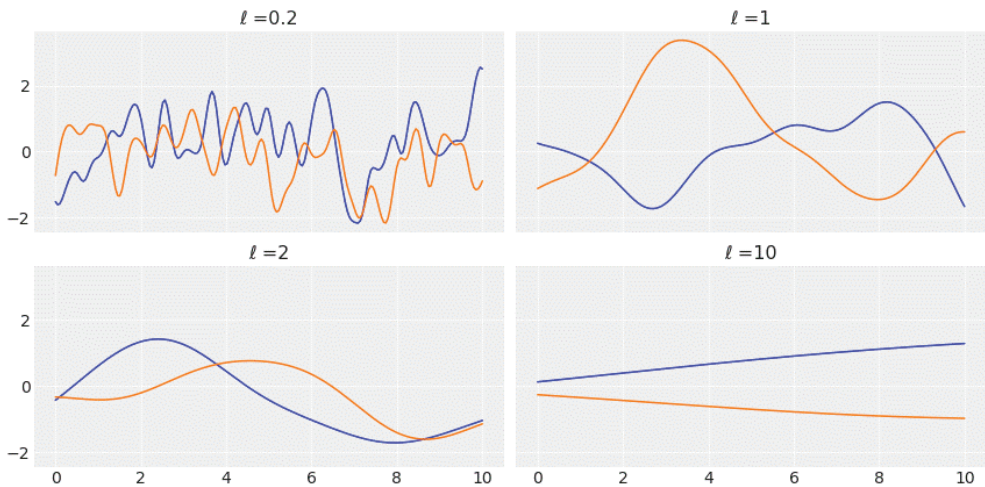


Рис. 7.3

На рис. 7.3 можно видеть, что гауссово ядро подразумевает большое разнообразие функций с параметром ℓ , управляющим степенью гладкости этих функций. Чем больше значение ℓ , тем более гладкой является функция.

Гауссовы процессы

Теперь мы готовы к полному пониманию сущности гауссовых процессов и методов их использования на практике.

Ниже приведено достаточно формальное определение гауссовых процессов, взятое из Википедии:

«В теории вероятностей и статистике гауссов процесс – это стохастический процесс (совокупность случайных величин, индексированных некоторым параметром, чаще всего временем или координатами), такой,

что любой конечный набор этих случайных величин имеет многомерное нормальное распределение, то есть любая конечная линейная комбинация из них нормально распределена».

Один из приемов для правильного понимания гауссовых процессов заключается в полном осознании того факта, что концепция гауссова процесса является мысленным (и математическим) вспомогательным средством, поэтому на практике нет никакой необходимости напрямую работать с этим бесконечным математическим объектом. Вместо этого мы только оцениваем гауссовы процессы в тех точках, где имеются данные. При этом бесконечномерный гауссов процесс сворачивается в конечное многомерное гауссово распределение с числом измерений, равным количеству точек данных. Математически это сворачивание выполняется с помощью маргинализации бесконечных ненаблюдаемых измерений. Теория гарантирует, что можно безболезненно отказать (в действительности маргинализировать) от всех точек, за исключением тех, которые реально наблюдаются. Теория также гарантирует, что мы всегда будем получать многомерное гауссово распределение. Таким образом, рис. 7.3 можно строго интерпретировать как настоящие выборки из гауссова процесса.

Отметим, что мы устанавливаем среднее значение многомерного гауссова распределения равным нулю и моделируем функции, используя только ковариационную матрицу, через экспоненцированное квадратическое ядро. Установка нулевого среднего значения многомерного гауссова распределения является общепринятой практикой при работе с гауссовыми процессами.



Гауссовы процессы удобны для создания байесовских непараметрических моделей, поскольку их можно использовать как априорные распределения для функций.

РЕГРЕССИЯ НА ОСНОВЕ ГАУССОВЫХ ПРОЦЕССОВ

Предположим, что можно смоделировать значение y как функцию f от x с добавлением некоторого шума:

$$y \sim N(\mu = f(x), \sigma = \varepsilon). \quad (7.5)$$

Здесь $\varepsilon \sim N(0, \sigma_\varepsilon)$.

Это похоже на предположение, сделанное в главе 3 для моделей линейной регрессии. Основное различие состоит в том, что теперь мы получаем априорное распределение с помощью функции f . Гауссовы процессы могут работать как такое априорное распределение, поэтому можно записать следующую формулу:

$$f(x) \sim GP(\mu_x, K(x, x')). \quad (7.6)$$

Здесь GP представляет распределение гауссова процесса с функцией среднего значения μ_x и ядром $K(x, x')$, или ковариационной функцией. Здесь мы воспользовались термином «функция» для того, чтобы подчеркнуть, что математически среднее значение и ковариация являются бесконечными объектами,

даже если на практике мы всегда работаем с конечными объектами.

Если априорное распределение является гауссовым процессом, а правдоподобие – нормальным распределением, то апостериорное распределение также является гауссовым процессом, и его можно вычислить аналитически:

$$p(f(X_*)|X_*, X, y) \sim N(\mu, \Sigma); \quad (7.7)$$

$$\mu = K_*^T K^{-1} y; \quad (7.8)$$

$$\Sigma = K_{**} - K_*^T K^{-1} K_*.$$

Здесь:

- $K = K(X, X)$;
- $K_* = K(X_*, X)$;
- $K_{**} = K(X_*, X_*)$.

X – это наблюдаемая точка данных, а X_* представляет тестовые (проверочные) точки, то есть новые точки, в которых мы хотим увидеть значение выводимой функции.

Как обычно, библиотека PyMC3 позволяет выполнить вывод, позаботившись обо всех математических подробностях, и при использовании гауссовых процессов. Создадим некоторые данные, затем выполним моделирование средствами библиотеки PyMC3:

```
np.random.seed(42)
x = np.random.uniform(0, 10, size=15)
y = np.random.normal(np.sin(x), 0.1)
plt.plot(x, y, 'o')
true_x = np.linspace(0, 10, 200)
plt.plot(true_x, np.sin(true_x), 'k--')
plt.xlabel('x')
plt.ylabel('f(x)', rotation=0)
```

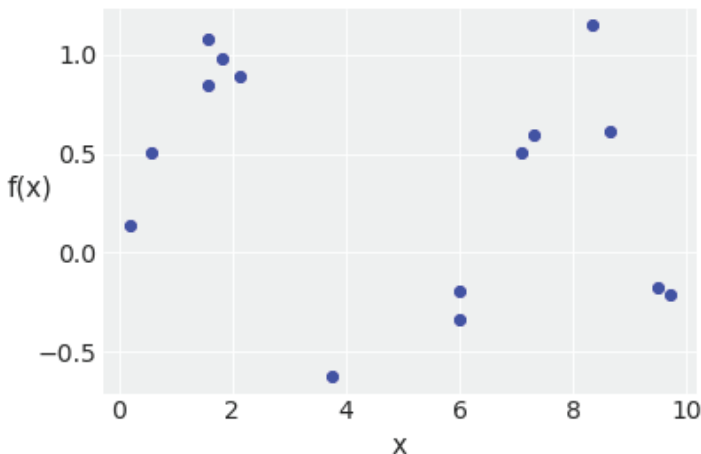


Рис. 7.4

На рис. 7.4 наблюдается действительно неизвестная функция, показанная в виде штриховой черной линии, а точки представляют элементы выборки (с шумом) из этой неизвестной функции.

Отметим, что для кодирования формул 7.7 и 7.8 в модель с использованием библиотеки PyMC3 необходимо найти только параметр ϵ , дисперсию нормального правдоподобия и параметр линейного масштаба ℓ экспоненцированно-квадратического ядра.

Гауссовы процессы реализованы в библиотеке PyMC3 как последовательность классов языка Python, которые слегка отличаются от того, что мы наблюдали в предыдущих моделях. Тем не менее код остается весьма PyMC3-низкорованным. Я добавил в следующий фрагмент кода несколько комментариев, которые помогут проследить основные этапы определения гауссова процесса средствами PyMC3:

```
# Одномерный вектор-столбец входных данных
X = x[:, None]
with pm.Model() as model_reg:
    # априорное гиперраспределение для параметра линейного масштаба ядра
    l = pm.Gamma('l', 2, 0.5)
    # создание экземпляра ковариационной функции
    cov = pm.gr.cov.ExprQuad(1, ls=l)
    # создание экземпляра априорного распределения на основе гауссова процесса
    gr = pm.gr.Marginal(cov_func=cov)
    # априорное распределение
    e = pm.HalfNormal('e', 25)
    # правдоподобие
    y_pred = gr.marginal_likelihood('y_pred', X=X, y=y, noise=e)
```

Отметим, что вместо нормального правдоподобия, предполагаемого по формуле 7.7, здесь использован метод `gr.marginal_likelihood`. Вероятно, вы помните из главы 1 (формула 1.1) и из главы 5 (формула 5.13), что предельное правдоподобие представляет собой интеграл от правдоподобия и априорного распределения:

$$p(y|X, \theta) \sim \int p(y|f, X, \theta)p(f|X, \theta)df. \quad (7.9)$$

Как обычно, θ представляет все неизвестные параметры, x – независимые переменные, а y – зависимые переменные. Отметим, что маргинализация выполняется в отношении значения функции f . Для априорного распределения гауссова процесса и нормального правдоподобия маргинализация может быть выполнена аналитически.

По мнению Билла Ингелса (Bill Engels), одного из основных разработчиков библиотеки PyMC3 и главного разработчика модуля гауссова процесса, чаще всего для параметров линейного масштаба априорные распределения, избегающие нулевых значений, работают лучше. Удобным априорным распределением, принимаемым по умолчанию для линейного масштаба ℓ , является `pm.Gamma(2, 0.5)`. Более подробные рекомендации по принимаемым по умол-

чанию удобным априорным распределениям от группы Stan можно найти по адресу <https://github.com/stan-dev/stan/wiki/Prior-Choice-Recommendations>.

```
az.plot_trace(trace_reg)
```

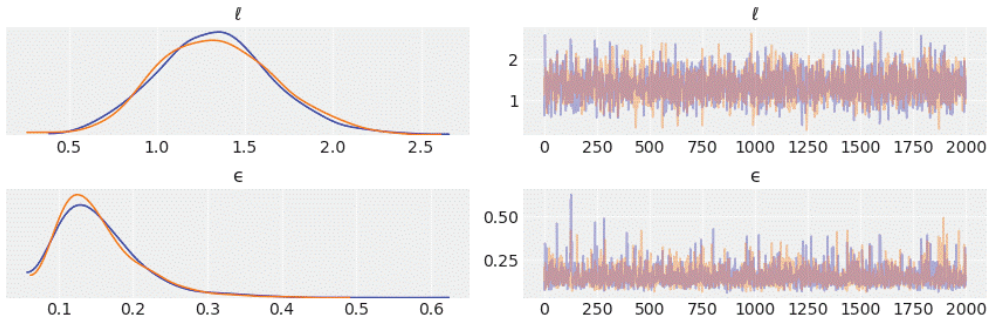


Рис. 7.5

После определения значений ℓ и ϵ могут потребоваться выборки из апостериорного распределения гауссова процесса, то есть выборки по функциям, подгоняемым к данным. Это можно сделать путем вычисления условного распределения, оцениваемого по новым локациям точек входных данных с использованием функции `gp.conditional`:

```
X_new = np.linspace(np.floor(x.min()), np.ceil(x.max()), 100)[: ,None]
with model_reg:
    f_pred = gp.conditional('f_pred', X_new)
```

В результате получаем новую случайную переменную в формате PyMC3 `f_pred`, которую можно использовать для получения элементов выборки из прогнозируемого апостериорного распределения (вычисленного по значениям `X_new`):

```
with model_reg:
    pred_samples = pm.sample_posterior_predictive(trace_reg, vars=[f_pred], samples=82)
```

Теперь можно сформировать диаграмму подогнанных функций по исходным данным для визуальной проверки того, насколько точно они соответствуют этим данным, а также для проверки неопределенности, соответствующей сделанным прогнозам:

```
_, ax = plt.subplots(figsize=(12,5))
ax.plot(X_new, pred_samples['f_pred'].T, 'C1-', alpha=0.3)
ax.plot(X, y, 'ko')
ax.set_xlabel('X')
```

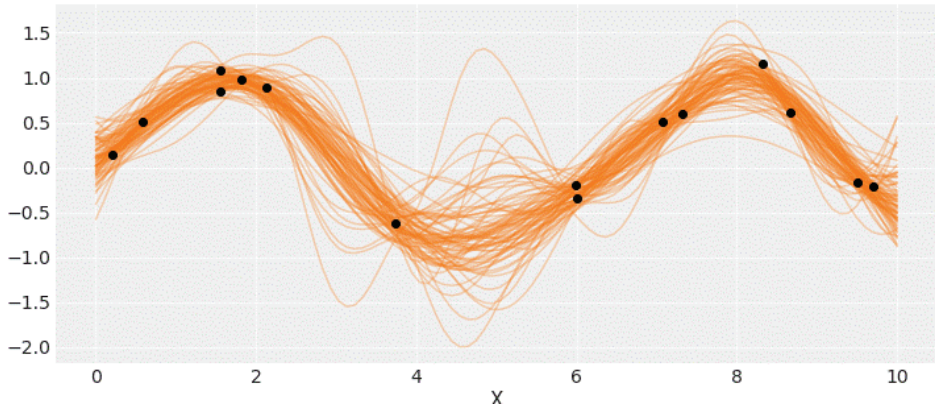


Рис. 7.6

В другом варианте решения можно воспользоваться функцией `rm.gr.util.plot_gp_dist` для получения более эффектного вида диаграммы. Каждый элемент диаграммы на рис. 7.7 представляет процентиль с рангом от 51 (более светлый цвет) до 99 (более темный цвет):

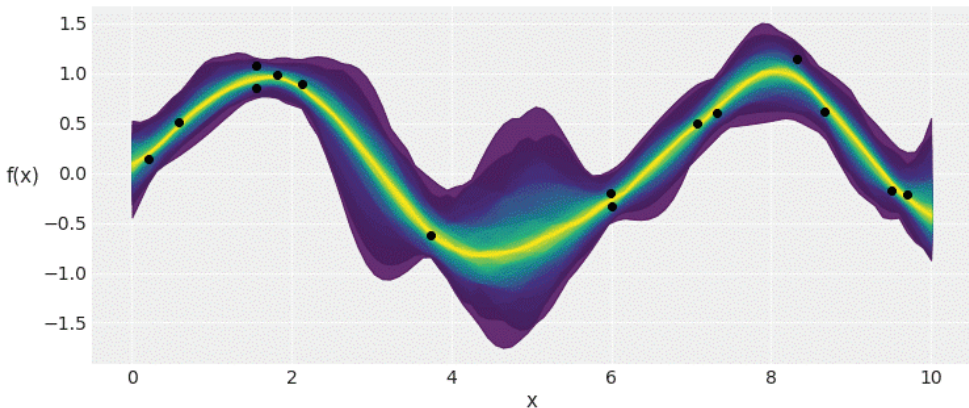


Рис. 7.7

Еще один вариант решения – вычисление вектора среднего значения и стандартного отклонения условного распределения, оцениваемого в конкретно взятой точке пространства параметров. В приведенном ниже примере используется среднее значение (по выборкам в трассировке `trace`) для ℓ и ε . Можно вычислить среднее значение и дисперсию с помощью функции `gp.predict`. Это возможно, потому что библиотека `PyMC3` вычисляет апостериорное распределение аналитически:

```
_, ax = plt.subplots(figsize=(12,5))
point = {'l': trace_reg['l'].mean(), 'e': trace_reg['e'].mean()}
mu, var = gp.predict(X_new, point=point, diag=True)
sd = var**0.5

ax.plot(X_new, mu, 'C1')
ax.fill_between(X_new.flatten(),
               mu - sd, mu + sd,
               color="C1",
               alpha=0.3)

ax.fill_between(X_new.flatten(),
               mu - 2*sd, mu + 2*sd,
               color="C1",
               alpha=0.3)

ax.plot(X, y, 'ko')
ax.set_xlabel('X')
```

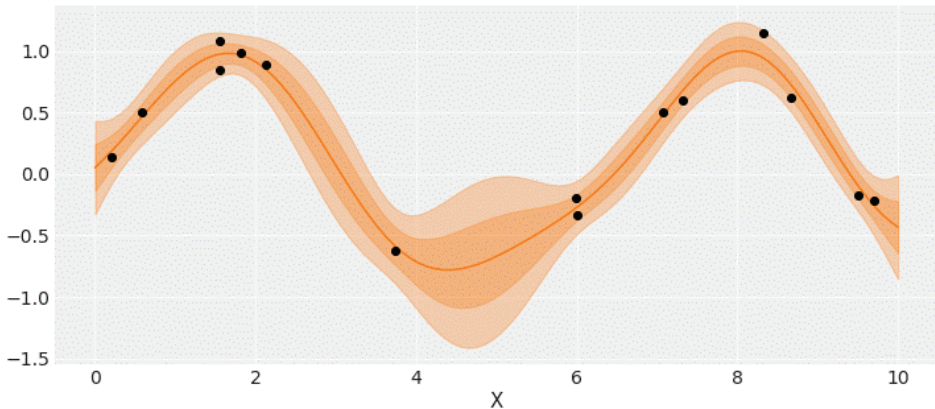


Рис. 7.8

Как мы уже наблюдали ранее в главе 4, можно воспользоваться линейной моделью с негауссовым правдоподобием и правильно выбранной функцией обратной связи для расширения ранга линейных моделей. То же самое можно сделать и для гауссова процесса. Например, можно использовать правдоподобие Пуассона с экспоненциальной функцией обратной связи. Для подобной модели апостериорное распределение уже не будет трактоваться аналитически, но все же можно применить численные методы для его аппроксимации. В следующих разделах мы более подробно рассмотрим этот тип модели.

РЕГРЕССИЯ С ПРОСТРАНСТВЕННОЙ АВТОКОРРЕЛЯЦИЕЙ

Следующий пример взят из книги Ричарда МакЭлрета (Richard McElreath) «Statistical Rethinking». Автор разрешил воспроизвести этот пример в данной кни-

ге. Я настоятельно рекомендую изучить книгу Ричарда МакЭлрета – в ней вы найдете множество качественных примеров, подобных приведенному здесь, и очень подробные и понятные описания. Единственным затруднением при чтении может стать использование в примерах этой книги R/Stan, но не стоит беспокоиться: можно найти версии Python/PyMC3 тех же примеров в репозитории <https://github.com/pymc-devs/resources>.

Однако вернемся к примеру. Имеется 10 различных островных сообществ. Для каждого из этих сообществ определено количество инструментов, которые используются обитателями островов. Некоторые теории дают следующий прогноз: более крупные группы населения разрабатывают и развивают больше инструментов, чем меньшие группы населения. Еще одним важным фактором является уровень контактов между группами населения.

Поскольку мы имеем количество инструментов как зависимую переменную, можно воспользоваться регрессией Пуассона для группы населения (совокупности) как независимой переменной. В действительности можно использовать логарифм от численности населения, потому что реальную значимость (в соответствии с предлагаемой теорией) имеет только порядок величины численности населения, а не абсолютный размер. Один из способов включения уровня контактов в создаваемую модель – сбор информации о том, как часто эти островные сообщества контактировали на протяжении некоторого исторического периода, и создание категориальной переменной типа высокий/низкий уровень контактов (см. столбец `contact` в структуре DataFrame `islands`). При другом подходе можно использовать расстояние между островами как вспомогательную характеристику (прокси-характеристику), связанную с уровнем контактов, поскольку вполне разумно предположить, что сообщества, ближе расположенные друг к другу, вступали в контакт чаще, чем сообщества, удаленные друг от друга.

Можно сформировать матрицу расстояний, составленную из значений, выраженных в тысячах километров, как результат считывания содержимого файла `islands_dist.csv`, прилагаемого к данной книге:

```
islands_dist = pd.read_csv('../data/islands_dist.csv', sep=',', index_col=0)
islands_dist.round(1)
```

Таблица 7.2

	MI	Ti	SC	Ya	Fi	Tr	Ch	Mn	To	Ha
Malekula	0.0	0.5	0.6	4.4	1.2	2.0	3.2	2.8	1.9	5.7
Tikopia	0.5	0.0	0.3	4.2	1.2	2.0	2.9	2.7	2.0	5.3
Santa Cruz	0.6	0.3	0.0	3.9	1.6	1.7	2.6	2.4	2.3	5.4
Yap	4.4	4.2	3.9	0.0	5.4	2.5	1.6	1.6	6.1	7.2
Lau Fiji	1.2	1.2	1.6	5.4	0.0	3.2	4.0	3.9	0.8	4.9
Trobriand	2.0	2.0	1.7	2.5	3.2	0.0	1.8	0.8	3.9	6.7
Chuuk	3.2	2.9	2.6	1.6	4.0	1.8	0.0	1.2	4.8	5.8

Таблица 7.2 (окончание)

	MI	Ti	SC	Ya	Fi	Tr	Ch	Mn	To	Ha
Manus	2.8	2.7	2.4	1.6	3.9	0.8	1.2	0.0	4.6	6.7
Tonga	1.9	2.0	2.3	6.1	0.8	3.9	4.8	4.6	0.0	5.0
Hawaii	5.7	5.3	5.4	7.2	4.9	6.7	5.8	6.7	5.0	0.0

В табл. 7.2 можно видеть, что главная диагональ матрицы заполнена нулями. Каждое островное сообщество удалено от себя самого на ноль километров. Кроме того, матрица симметрична – верхняя треугольная часть и нижняя треугольная часть (относительно главной диагонали) содержит одну и ту же информацию. Это прямое следствие того факта, что расстояние от точки А до точки В равно расстоянию от точки В до точки А.

Количество инструментов и численность населения сохранены в другом файле *islands.csv*, также размещенном в репозитории данной книги:

```
islands = pd.read_csv('../data/islands.csv', sep=',')
islands.head().round(1)
```

Таблица 7.3

	culture (сообщество)	population (числ. насел.)	contact (ур. кон- тактов)	total_tools (общ. колич. инстр.)	mean_TU	lat	lon	lon2	logpop (лога- рифм числ. насел.)
0	Malekula	1100	low	13	3.2	-16.3	167.5	-12.5	7.0
1	Tikopia	1500	low	22	4.7	-12.3	168.8	-11.2	7.3
2	Santa Cruz	3600	low	24	4.0	-10.7	166.0	-14.0	8.2
3	Yap	4791	high	43	5.0	9.5	138.1	-41.9	8.5
4	Lau Fiji	7400	high	33	5.0	-17.7	178.1	-1.9	8.9

Из приведенной в таблице структуры DataFrame мы будем использовать только столбцы *culture*, *total_tools*, *lat*, *lon2* и *logpop*:

```
islands_dist_sqr = islands_dist.values**2
culture_labels = islands.culture.values
index = islands.index.values
log_pop = islands.logpop
total_tools = islands.total_tools
x_data = [islands.lat.values[:, None], islands.lon.values[:, None]]
```

Модель, которую мы будем создавать, описывается следующими формулами:

$$f \sim \text{GP}([0, \dots, 0], K(x, x')); \quad (7.10)$$

$$\mu \sim \exp(\alpha + \beta x + f); \quad (7.11)$$

$$y \sim \text{Poisson}(\mu). \quad (7.12)$$

Здесь не указаны априорные распределения для параметров α и β , а также априорные гиперраспределения ядра. Переменная x – это логарифм от численности населения, а y – общее количество (типов) инструментов.

По существу, эта модель является регрессией Пуассона с одним нововведением по сравнению с моделями из главы 4: один из компонентов в линейной модели f формируется как результат гауссова процесса. Для вычисления ядра гауссова процесса будет использоваться матрица расстояний `islands_dist`. При таком подходе в модель эффективно внедряется мера схожести предъявленных уровней технологического развития (оцениваемая по матрице расстояний). Таким образом, вместо предположения о том, что общее количество типов инструментов зависит только от численности населения и не зависит от того или иного сообщества, мы моделируем количество типов инструментов в каждом сообществе как функцию их географической близости.

Эта модель, включая априорные распределения, реализована в следующем фрагменте кода с использованием библиотеки PyMC3:

```

ith pm.Model() as model_islands:
    η = pm.HalfCauchy('η', 1)
    ℓ = pm.HalfCauchy('ℓ', 1)
    cov = η * pm.gp.cov.ExpQuad(1, ls=ℓ)
    gp = pm.gp.Latent(cov_func=cov)
    f = gp.prior('f', X=islands_dist_sqr)

    α = pm.Normal('α', 0, 10)
    β = pm.Normal('β', 0, 1)
    μ = pm.math.exp(α + f[index] + β * log_pop)
    tt_pred = pm.Poisson('tt_pred', μ, observed=total_tools)
    trace_islands = pm.sample(1000, tune=1000)

```

Чтобы понять апостериорное распределение ковариационных функций с учетом расстояний, можно сформировать диаграммы нескольких выборок из апостериорного распределения:

```

trace_η = trace_islands['η']
trace_ℓ = trace_islands['ℓ']

_, ax = plt.subplots(1, 1, figsize=(8, 5))
xrange = np.linspace(0, islands_dist.values.max(), 100)

ax.plot(xrange, np.median(trace_η) * np.exp(-np.median(trace_ℓ) * xrange**2), lw=3)

ax.plot(xrange, (trace_η[::20][:, None] * np.exp(- trace_ℓ[::20][:, None] * xrange**2)).T,
        'C0', alpha=.1)

ax.set_ylim(0, 1)
ax.set_xlabel('distance (thousand kilometers)')
ax.set_ylabel('covariance')

```

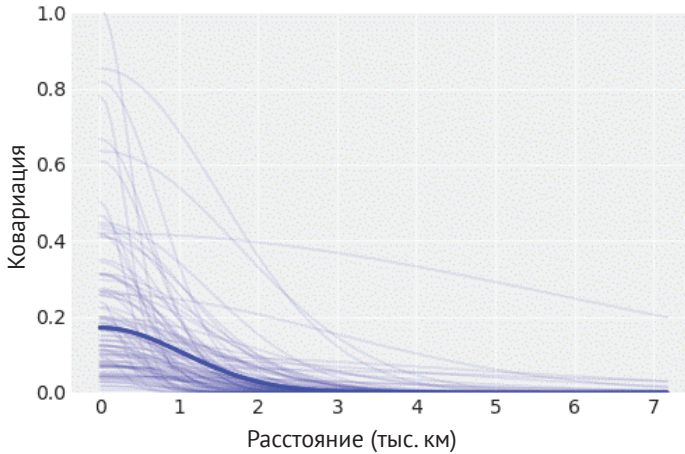



Рис. 7.9

Толстая линия на рис. 7.9 – это апостериорная медиана ковариации между парами сообществ как функции расстояния. Медиана используется потому, что распределение для ℓ и η очень сильно смещено. Здесь можно видеть, что в среднем ковариация не столь велика и снижается почти до нуля, начиная с расстояния, приблизительно равного 2000 км. Тонкие линии представляют неопределенность, и на диаграмме наблюдается высокая степень неопределенности.

Может оказаться интересным сравнение `model_islands` и апостериорное распределение, вычисленное по этой модели, с моделью `m_10_10` из репозитория <https://github.com/pymc-devs/resources>. Возможно, потребуется использование функций библиотеки ArviZ, таких как `az.summary` или `az.plot_forest`. Модель `m_10_10` похожа на модель `model_islands`, но не включает элемент гауссова процесса.

Теперь понаблюдаем, насколько сильна корреляция между островными сообществами в соответствии с созданной нами моделью. Для этого необходимо преобразовать ковариационную матрицу в корреляционную матрицу:

```
# вычисление апостериорного распределения медианы ковариации между островными сообществами
Σ = np.median(trace_η) * (np.exp(-np.median(trace_ℓ) * islands_dist_sqr))
# преобразование в корреляционную матрицу
Σ_post = np.diag(np.diag(Σ)**-0.5)
ρ = Σ_post @ Σ @ Σ_post
ρ = pd.DataFrame(ρ, index=islands_dist.columns,
columns=islands_dist.columns)
ρ.round(2)
```

Таблица 7.4

	MI	Ti	SC	Ya	Fi	Tr	Ch	Mn	To	Ha
MI	1.00	0.90	0.84	0.00	0.50	0.16	0.01	0.03	0.21	0.0
Ti	0.90	1.00	0.96	0.00	0.50	0.16	0.02	0.04	0.18	0.0
SC	0.84	0.96	1.00	0.00	0.34	0.27	0.05	0.08	0.10	0.0
Ya	0.00	0.00	0.00	1.00	0.00	0.07	0.34	0.31	0.00	0.0
Fi	0.50	0.50	0.34	0.00	1.00	0.01	0.00	0.00	0.77	0.0
Tr	0.16	0.16	0.27	0.07	0.01	1.00	0.23	0.72	0.00	0.0
Ch	0.01	0.02	0.05	0.34	0.00	0.23	1.00	0.52	0.00	0.0
Mn	0.03	0.04	0.08	0.31	0.00	0.72	0.52	1.00	0.00	0.0
To	0.21	0.18	0.10	0.00	0.77	0.00	0.00	0.00	1.00	0.0
Ha	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0

При выполнении нашего исследования заслуживают внимания два наблюдения: Гавайи (Hawaii) представляют собой чрезвычайно уединенное сообщество, поскольку расположено очень далеко от остальных островных сообществ. Также можно видеть, что Малекула (Malekula, MI), Тикопия (Tikopia, Ti) и Санта Крус (Santa Cruz, SC) сильно коррелируют друг с другом. Причина в том, что эти сообщества расположены очень близко друг к другу, а кроме того, количество типов инструментов в этих сообществах различается незначительно.

Теперь воспользуемся информацией о широте (lat) и долготе (lon2) для формирования диаграммы расположения островных сообществ в их относительных позициях:

```
# определение масштаба расположения точек для logpop
logpop = np.copv(log_pop)
logpop /= logpop.max()
psize = np.exp(logpop*5.5)
log_pop_seq = np.linspace(6, 14, 100)
lambda_post = np.exp(trace_islands['α'][:, None] + trace_islands['β'][:, None] * log_pop_seq)
_, ax = plt.subplots(1, 2, figsize=(12, 6))

ax[0].scatter(islands.lon2, islands.lat, psize, zorder=3)
ax[1].scatter(islands.logpop, islands.total_tools, psize, zorder=3)

for i, itext in enumerate(culture_labels):
    ax[0].text(islands.lon2[i]+1, islands.lat[i]+1, itext)
    ax[1].text(islands.logpop[i]+1, islands.total_tools[i]-2.5, itext)

ax[1].plot(log_pop_seq, np.median(lambda_post, axis=0), 'k--')
az.plot_hpd(log_pop_seq, lambda_post, fill_kwargs={'alpha':0},
            plot_kwargs={'color':'k', 'ls':'--', 'alpha':1})

for i in range(10):
    for j in np.arange(i+1, 10):
        ax[0].plot((islands.lon2[i], islands.lon2[j]),
                    (islands.lat[i], islands.lat[j]), 'C1-')
```

```

alpha=p.iloc[i, j]**2, lw=4)
ax[1].plot((islands.logpop[i], islands.logpop[j]),
           (islands.total_tools[i], islands.total_tools[j]), 'C1-',
           alpha=p.iloc[i, j]**2, lw=4)
ax[0].set_xlabel('longitude')
ax[0].set_ylabel('latitude')

ax[1].set_xlabel('log-population')
ax[1].set_ylabel('total tools')
ax[1].set_xlim(6.8, 12.8)
ax[1].set_ylim(10, 73)

```

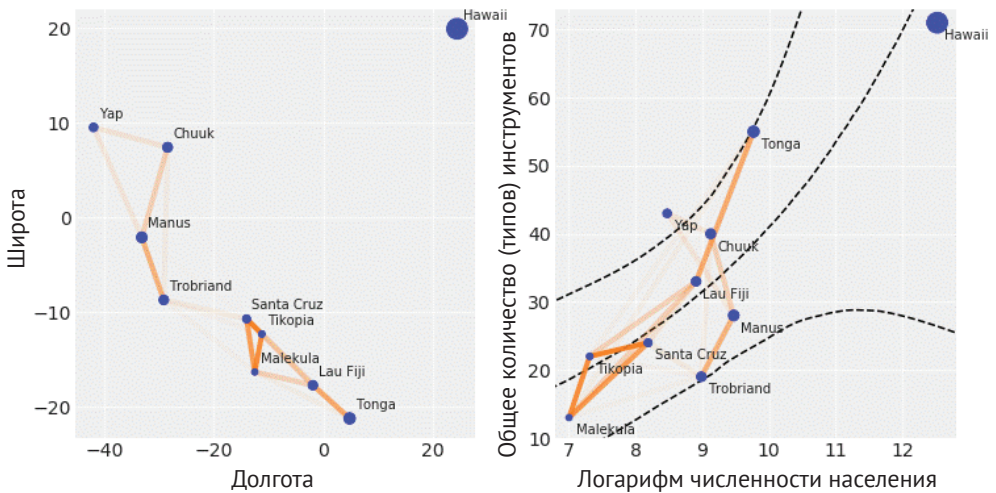


Рис. 7.10

В левой части рис. 7.10 показаны линии корреляций апостериорного распределения медианы, ранее вычисленные по островным сообществам на основе их относительного географического положения. Некоторые из линий не видны, поскольку уровень корреляции использовался для установки степени прозрачности линий (через параметр matplotlib `alpha`). В правой части рис. 7.10 изображены те же корреляции апостериорного распределения медианы, но отображаемые в соотношении количества типов инструментов и логарифма от численности населения. Штриховые линии представляют срединные значения (медианы) количества инструментов и интервал ПАР 94 % как функцию от логарифма численности населения. На обеих диаграммах размер точек пропорционален численности населения каждого островного сообщества.

Обратите внимание на то, как корреляции между островами Малекула, Тикопия и Санта Крус описывают тот факт, что эти сообщества обладают относительно малым количеством типов инструментов, близким к срединному значению или меньшим, чем ожидаемое количество типов инструментов для этих совокупностей. Похожая картина наблюдается и для островов Тробрианд

и Манус – они географически близки и имеют меньше типов инструментов, чем ожидалось по численности их населения. На Тонга больше типов инструментов, чем ожидалось по численности населения, и относительно высокая корреляция с Фиджи. Модель в определенном смысле сообщает, что Тонга оказывает положительное воздействие на Луа Фиджи, увеличивая общее количество типов инструментов, и противоположное воздействие на других близких соседей – острова Малекула, Тикопия и Санта Крус.

КЛАССИФИКАЦИЯ С ИСПОЛЬЗОВАНИЕМ ГАУССОВА ПРОЦЕССА

Применение гауссовых процессов не ограничивается регрессией. Их также можно использовать для классификации. В главе 4 мы наблюдали преобразование линейной модели в удобную модель классификации данных с применением правдоподобия Бернулли и логистической функции обратной связи (затем применялось правило границы решения для отдельных классов). Попробуем воспроизвести модель `model_0` из главы 4 для набора данных Iris, но на этот раз с использованием гауссова процесса вместо линейной модели.

Для этого сначала снова подготовим к работе набор данных Iris:

```
iris = pd.read_csv('../data/iris.csv')
iris.head()
```

Таблица 7.5

	sepal_length (длина чашелистика)	sepal_width (ширина чашелистика)	petal_length (длина лепестка)	petal_width (ширина лепестка)	species (виды)
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Начнем с простейшей возможной задачи классификации: два класса, *setosa* (ирис щетинистый) и *versicolor* (ирис разноцветный), и только одна независимая переменная, длина чашелистика. Как и ранее, закодируем категориальные переменные, *setosa* и *versicolor*, с числовыми значениями 0 и 1 соответственно:

```
df = iris.query("species == ('setosa', 'versicolor')")
y = pd.Categorical(df['species']).codes
x_1 = df['sepal_length'].values
X_1 = x_1[:, None]
```

Для этой модели вместо использования класса `pm.gp.Marginal` для создания экземпляра априорного распределения гауссова процесса воспользуемся классом `pm.gp.Latent`. Этот класс более обобщенный и может использоваться

с любой функцией правдоподобия, тогда как класс `Marginal` ограничен только гауссовыми функциями правдоподобия. Воспользуемся этим преимуществом для большей эффективности и математического обоснования объединения априорного распределения гауссова процесса с гауссовой функцией правдоподобия:

```
with pm.Model() as model_iris:
    l = pm.Gamma('l', 2, 0.5)
    cov = pm.gp.cov.ExpQuad(1, l)
    gp = pm.gp.Latent(cov_func=cov)
    f = gp.prior("f", X=X_1)
    # logistic inverse link function and Bernoulli likelihood
    y_ = pm.Bernoulli("y", p=pm.math.sigmoid(f), observed=y)
    trace_iris = pm.sample(1000, chains=1,
compute_convergence_checks=False)
```

После того как найдено значение ℓ , может потребоваться получение выборок из апостериорного распределения гауссова процесса. Раньше это делалось с помощью `marginal_gp_model`, но можно также вычислить условное распределение, оцениваемое по набору новых входных точек данных с помощью функции `gp.conditional`, как показано в следующем фрагменте кода:

```
X_new = np.linspace(np.floor(x_1.min()), np.ceil(x_1.max()), 200)[: , None]

with model_iris:
    f_pred = gp.conditional('f_pred', X_new)
    pred_samples = pm.sample_posterior_predictive(trace_iris, vars=[f_pred], samples=1000)
```

Для вывода результатов работы этой модели необходимо создать диаграмму, подобную показанной на рис. 4.4. Вместо аналитического вывода границы решения вычислим ее напрямую из `f_pred`, используя следующую удобную функцию:

```
def find_midpoint(array1, array2, value):
    """
    This should be a proper docstring :-)
    """
    array1 = np.asarray(array1)
    idx0 = np.argsort(np.abs(array1 - value))[0]
    idx1 = idx0 - 1 if array1[idx0] > value else idx0 + 1
    if idx1 == len(array1):
        idx1 -= 1
    return (array2[idx0] + array2[idx1]) / 2
```

Приведенный ниже фрагмент кода очень похож на код, используемый в главе 4 для генерации диаграммы на рис. 4.4:

```
_, ax = plt.subplots(figsize=(10, 6))

fp = logistic(pred_samples['f_pred'])
fp_mean = np.mean(fp, 0)

ax.plot(X_new[:, 0], fp_mean)
```

```
# генерация диаграммы данных (с некоторым джиттером) и истинной скрытой функцией
ax.scatter(x_1, np.random.normal(y, 0.02), marker='.', color=[f'C{x}' for x in y])

az.plot_hpd(X_new[:, 0], fp, color='C2')

db = np.array([find_midpoint(f, X_new[:, 0], 0.5) for f in fp])
db_mean = db.mean()
db_hpd = az.hpd(db)
ax.vlines(db_mean, 0, 1, color='k')
ax.fill_betweenx([0, 1], db_hpd[0], db_hpd[1], color='k', alpha=0.5)
ax.set_xlabel('sepal_length')
ax.set_ylabel('θ', rotation=0)
plt.savefig('B11197_07_11.png')
```

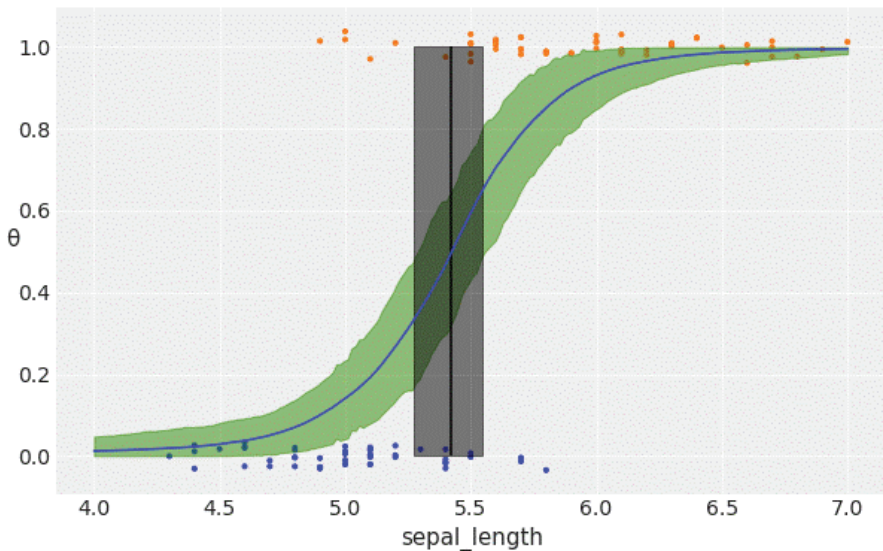


Рис. 7.11

Рисунок 7.11 очень похож на рис. 4.4. Распределение f_{pred} выглядит как сигмоидная кривая, за исключением хвостов, которые отклоняются вверх при меньших значениях x_1 и вниз при больших значениях x_1 . Это следствие того, что прогнозируемая функция смещается к априорному распределению при отсутствии данных (или при слишком малом количестве данных). Если некоторое беспокойство вызывает только граница решения, то это не слишком большая проблема, но если необходимо моделировать вероятности принадлежности к видам ириса щетинистого или ириса многоцветного для различных значений sepal_length , то потребуются улучшить эту модель и разработать более подходящее решение для хвостов. Один из способов достижения этой цели – добавление дополнительной структуры в гауссов процесс. Общеизвестным способом получения более качественных моделей гауссова процесса яв-

ляется объединение ковариационных функций для извлечения более точных подробностей о функции, которую мы пытаемся смоделировать.

Следующая модель `model_iris2` почти одинакова с моделью `model_iris`, за исключением ковариационной матрицы, которая моделируется как объединение трех ядер:

```
cov = K_{ExpQuad} + K_{Linear} + K_{whitenoise}(1E-5)
```

Добавляя линейное ядро, мы устраняем проблему отклонения хвостов, как можно видеть на рис. 7.12. Ядро «белого шума» (white noise) – это всего лишь математический прием для стабилизации вычисления ковариационной матрицы. Ядра для гауссовых процессов ограничены для обеспечения положительных значений элементов итоговой ковариационной матрицы. Но ошибки в вычислениях могут привести к нарушению этого условия. Одним из проявлений данной проблемы является получение нечисловых значений (NaN) при вычислении выборок прогнозируемого апостериорного распределения подготавливаемой функции. Для смягчения воздействия этой ошибки необходимо стабилизировать процесс вычисления, добавляя немного шума. В действительности в библиотеке PyMC3 уже предусмотрена такая стабилизация, но иногда необходим немного больший уровень шума, как показано в следующем фрагменте кода:

```
with pm.Model() as model_iris2:
    l = pm.Gamma('l', 2, 0.5)
    c = pm.Normal('c', x_1.min())
    tau = pm.HalfNormal('tau', 5)
    cov = (pm.gp.cov.ExpQuad(1, l) +
           tau * pm.gp.cov.Linear(1, c) +
           pm.gp.cov.WhiteNoise(1E-5))
    gp = pm.gp.Latent(cov_func=cov)
    f = gp.prior("f", X=X_1)
    # логистическая функция обратной связи и функция правдоподобия Бернулли
    y_ = pm.Bernoulli("y", p=pm.math.sigmoid(f), observed=y)
    trace_iris2 = pm.sample(1000, chains=1,
compute_convergence_checks=False)
```

Далее генерируются выборки из прогнозируемого апостериорного распределения в модели `model_iris2` для предварительно сгенерированных значений `X_new`:

```
with model_iris2:
    f_pred = gp.conditional('f_pred', X_new)
    pred_samples = pm.sample_posterior_predictive(trace_iris2, vars=[f_pred], samples=1000)
_, ax = plt.subplots(figsize=(10,6))

fp = logistic(pred_samples['f_pred'])
fp_mean = np.mean(fp, 0)

ax.scatter(x_1, np.random.normal(y, 0.02), marker='.', color=[f'C{ci}' for ci in y])

db = np.array([find_midpoint(f, X_new[:,0], 0.5) for f in fp])
```



```

db_mean = db.mean()
db_hpd = az.hpd(db)
ax.vlines(db_mean, 0, 1, color='k')
ax.fill_betweenx([0, 1], db_hpd[0], db_hpd[1], color='k', alpha=0.5)
ax.plot(X_new[:,0], fp_mean, 'C2', lw=3)
az.plot_hpd(X_new[:,0], fp, color='C2')

ax.set_xlabel('sepal_length')
ax.set_ylabel('θ', rotation=0)

```

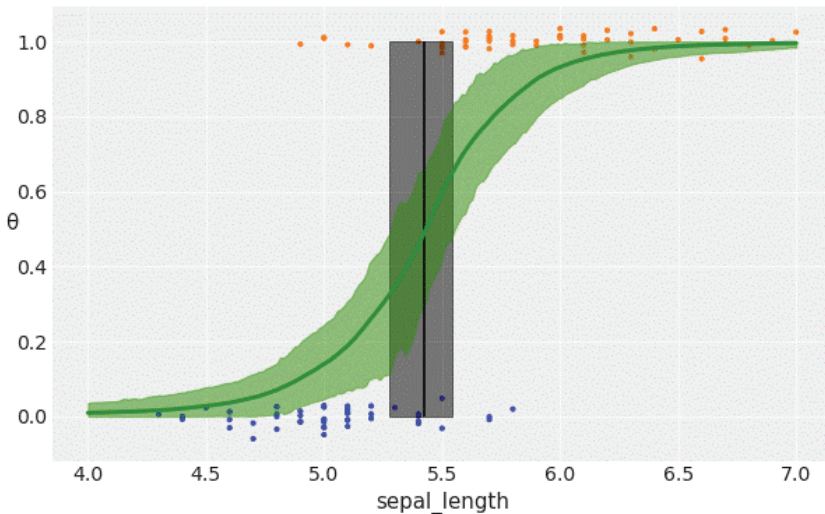


Рис. 7.12

Диаграмма на рис. 7.12 гораздо больше похожа на рис. 4.4, чем диаграмма на рис. 7.11. Этот пример выполнил две главные задачи:

- демонстрация возможности простого объединения ядер для получения более эффективных моделей;
- демонстрация возможности воспроизведения метода логистической регрессии с использованием гауссова процесса.

С учетом второго пункта логистическая регрессия действительно является особым случаем гауссовых процессов, потому что простая линейная регрессия – это всего лишь один особый случай гауссова процесса. В действительности многие известные модели можно рассматривать как особые случаи гауссовых процессов или, по крайней мере, они так или иначе связаны с гауссовыми процессами. Более подробно об этом можно прочитать в главе 15 книги Кевина Мерфи (Kevin Murphy) «Machine Learning: A Probabilistic Perspective».

На практике нет особого смысла использовать гауссов процесс для моделирования задачи, которую можно решить с помощью логистической регрессии. Гауссов процесс рекомендуется применять для моделирования данных более

сложной структуры, которые неэффективно захватываются менее гибкими моделями. Например, предположим, что необходимо смоделировать вероятность возникновения болезни как функцию от возраста. Оказывается, что для очень молодых и очень старых людей риск заболевания более высок, чем для людей среднего возраста. По этому описанию сформирован фиктивный набор данных *space_flu.csv*, который мы загружаем для обработки:

```
df_sf = pd.read_csv('../data/space_flu.csv')
age = df_sf.age.values[:, None]
space_flu = df_sf.space_flu

ax = df_sf.plot.scatter('age', 'space_flu', figsize=(8, 5))
ax.set_yticks([0, 1])
ax.set_yticklabels(['healthy', 'sick'])
```

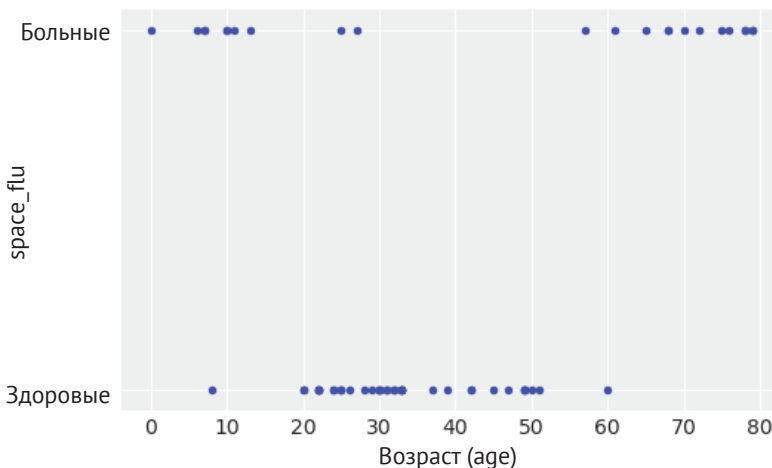


Рис. 7.13

Создаваемая ниже модель в основном та же самая, что и модель *model_iris*:

```
with pm.Model() as model_space_flu:
    l = pm.HalfCauchy('l', 1)
    cov = pm.gp.cov.ExpQuad(1, l) + pm.gp.cov.WhiteNoise(1E-5)
    gp = pm.gp.Latent(cov_func=cov)
    f = gp.prior('f', X=age)
    y_ = pm.Bernoulli('y', p=pm.math.sigmoid(f), observed=space_flu)
    trace_space_flu = pm.sample(1000, chains=1, compute_convergence_checks=False)
```

Теперь сгенерируем выборки из прогнозируемого апостериорного распределения для модели *model_space_flu*, затем сформируем диаграмму результатов:

```
X_new = np.linspace(0, 80, 200)[: , None]
```

```
with model_space_flu:
    f_pred = gp.conditional('f_pred', X_new)
    pred_samples = pm.sample_posterior_predictive(trace_space_flu, vars=[f_pred],
```

```

samples=1000)
_, ax = plt.subplots(figsize=(10, 6))
fp = logistic(pred_samples['f_pred'])
fp_mean = np.nanmean(fp, 0)
ax.scatter(age, np.random.normal(space_flu, 0.02), marker='.', color=[f'C{ci}' for ci in
space_flu])
ax.plot(X_new[:, 0], fp_mean, 'C2', lw=3)
az.plot_hpd(X_new[:, 0], fp, color='C2')
ax.set_yticks([0, 1])
ax.set_yticklabels(['healthy', 'sick'])
ax.set_xlabel('age')

```

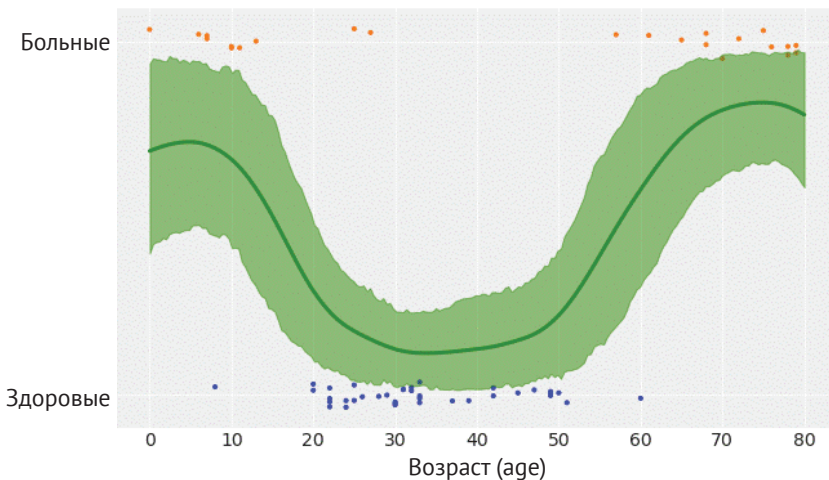


Рис. 7.14

Следует отметить, что на рис. 7.14 хорошо видно, что гауссов процесс способен весьма точно выполнить подгонку к этому набору данных, даже если для описания данных требуется более сложная функция, чем логистическая регрессия. Подгонка по этим данным была бы невозможной для простой логистической регрессии, если только не вводить некоторые специализированные для этого конкретного случая модификации, помогающие слегка улучшить ситуацию (см. упражнение 6 для подробного описания таких модификаций).

ПРОЦЕССЫ КОКСА

Теперь снова вернемся к примеру моделирования счетных (числовых) данных. Мы будем рассматривать два примера: норма (оценка), изменяющаяся во времени, и изменение нормы (оценки) в двумерном пространстве. Для этого воспользуемся функцией правдоподобия Пуассона, а оценка будет моделиро-

ваться с помощью гауссова процесса. Поскольку оценка распределения Пуассона ограничена положительными значениями, мы будем использовать экспоненциальную функцию в качестве функции обратной связи, как это было сделано в главе 4 для модели регрессии Пуассона с дополнением нулевыми значениями.

В статистической литературе переменная норма (оценка – rate) часто появляется под именем «интенсивность» (intencity), то есть этот тип задач называют оценкой интенсивности (intencity estimation). Кроме того, этот тип модели часто называют моделью Кокса (David Cox). Модель Кокса представляет собой тип процесса Пуассона, где сама норма (интенсивность) является стохастическим процессом. Как и в гауссовом процессе, это набор случайных переменных, при этом каждый конечный набор этих случайных переменных имеет многомерное нормальное распределение. Процесс Пуассона – это набор случайных переменных, при этом каждый конечный набор этих случайных переменных имеет распределение Пуассона. Можно мысленно представить процесс Пуассона как распределение поверх набора точек в заданном конкретном пространстве. Когда норма (интенсивность) процесса Пуассона сама является стохастическим процессом, например гауссовым, мы получаем то, что называется процессом Кокса.

Модель катастроф в угледобывающей промышленности

Первый пример известен как модель катастроф в угледобывающей промышленности. Этот пример составлен из записей об авариях на угольных шахтах в Великобритании с 1851 по 1962 год. Считается, что на количество аварий повлияли изменения, внесенные в правила техники безопасности за этот период. Нам нужно смоделировать интенсивность аварий как функцию времени. Исследуемый набор данных состоит из одного столбца, каждая запись в котором соответствует времени (дате и т. д.) аварии.

Загрузим эти данные и посмотрим некоторые из загруженных значений:

```
coal_df = pd.read_csv('../data/coal.csv', header=None)
coal_df.head()
```

Таблица 7.6

	0
0	1851.2026
1	1851.6324
2	1851.9692
3	1851.9747
4	1852.3142

Модель, которую мы будем использовать для подгонки по данным, загруженным в структуру DataFrame `coal_df`, записывается в виде следующих формул:

$$f(x) \sim \text{GP}(\mu_x, K(x, x')); \quad (7.13)$$

$$y \sim \text{Poisson}(f(x)). \quad (7.14)$$

Как вы видите, для решения этой задачи используется регрессия Пуассона. Возможно, сейчас вы удивлены и задумались о том, как будет выполняться регрессия, если в нашем распоряжении имеется только один столбец с датой аварии. Ответом будет дискретизация данных точно таким же способом, как если бы мы формировали гистограмму. Мы будем использовать центры элементов дискретизации как переменную x и счетчики по каждому элементу дискретизации как переменную y :

```
# дискретизация данных
years = int(coal_df.max().values - coal_df.min().values)
bins = years // 4
hist, x_edges = np.histogram(coal_df, bins=bins)
# вычисление положения центров дискретизованных данных
x_centers = x_edges[:-1] + (x_edges[1] - x_edges[0]) / 2
# преобразование xdata в форму, подходящую для гауссова процесса
x_data = x_centers[:, None]
# приведение данных к числовой характеристике интенсивности аварий в год
y_data = hist / 4
```

Теперь определим модель и сформируем ее решение с использованием библиотеки PyMC3:

```
with pm.Model() as model_coal:
    l = pm.HalfNormal('l', x_data.std())
    cov = pm.gp.cov.ExpQuad(1, ls=l) + pm.gp.cov.WhiteNoise(1E-5)
    gp = pm.gp.Latent(cov_func=cov)
    f = gp.prior('f', X=x_data)

    y_pred = pm.Poisson('y_pred', mu=pm.math.exp(f), observed=y_data)
    trace_coal = pm.sample(1000, chains=1)
```

Далее построим диаграмму полученных результатов:

```
_, ax = plt.subplots(figsize=(10, 6))
f_trace = np.exp(trace_coal['f'])
rate_median = np.median(f_trace, axis=0)
ax.plot(x_centers, rate_median, 'w', lw=3)
az.plot_hpd(x_centers, f_trace)
az.plot_hpd(x_centers, f_trace, credible_interval=0.5, plot_kwargs={'alpha': 0})
ax.plot(coal_df, np.zeros_like(coal_df)-0.5, 'k|')
ax.set_xlabel('years')
ax.set_ylabel('rate')
```

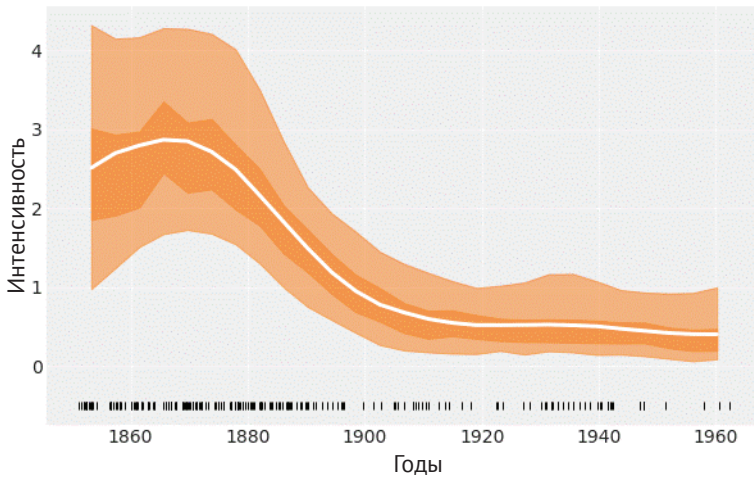


Рис. 7.15

На рис. 7.15 белой линией изображено срединное значение (медиана) интенсивности аварий как функция времени. Широкие полосы обозначают интервал ПАР 50 % (более темная полоса) и интервал ПАР 94 % (более светлая полоса). Внизу размещены черные маркеры для каждой аварии (иногда такой тип графика называют ленточной диаграммой – rug plot – напоминает край коврика). Здесь можно видеть, что интенсивность аварий уменьшается со временем, за исключением короткого начального интервала увеличения интенсивности. В документацию библиотеки PyMC3 включено описание модели катастроф в угледобывающей промышленности, но моделирование рассматривается с другой точки зрения. Настоятельно рекомендуется изучить пример из документации, поскольку он весьма полезен сам по себе, а кроме того, хорошо подходит для сравнения с методом, только что реализованным с помощью модели `model_coal`.

Отметим, что даже если данные дискретизируются, в результате мы получаем гладкую кривую. В этом смысле можно рассматривать модель `model_coal` (и в целом этот тип моделей) как построение гистограммы и последующее ее сглаживание.

Набор данных `redwood`

Теперь сосредоточим внимание на применении модели, использованной в предыдущем разделе, к задаче в двумерном пространстве с использованием данных о секвойях (`redwood`). Этот набор данных (распространяемый под лицензией GPL) взят из `GPstuff` – пакета гауссовых процессов для Matlab, Octave и R. Набор данных состоит из записей о местах расположения секвойевых деревьев в определенной географической области. Цель статистического вывода – определение интенсивности распределения секвой в этой области.

Как обычно, загружаем данные и формируем их диаграмму:

```
rw_df = pd.read_csv('../data/redwood.csv', header=None)
_, ax = plt.subplots(figsize=(8, 8))
ax.plot(rw_df[0], rw_df[1], 'C0.')
ax.set_xlabel('x1 coordinate')
ax.set_ylabel('x2 coordinate')
```

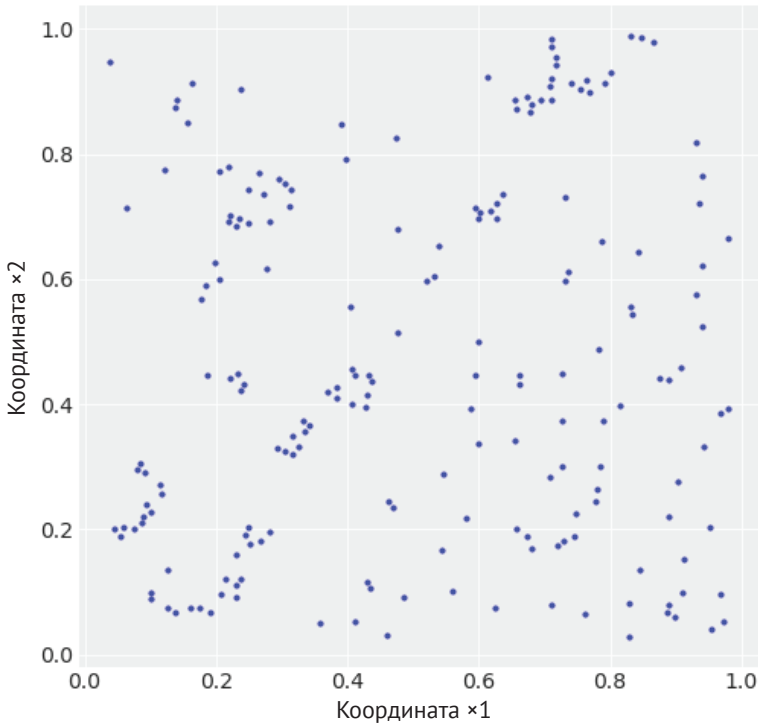


Рис. 7.16

Как и в примере с моделью катастроф в угледобывающей промышленности, необходимо дискретизировать исследуемые данные:

```
# дискретизация пространственных данных
bins = 20
hist, x1_edges, x2_edges = np.histogram2d(rw_df[1].values, rw_df[0].values, bins=bins)
# вычисление положения центров дискретизированных данных
x1_centers = x1_edges[:-1] + (x1_edges[1] - x1_edges[0]) / 2
x2_centers = x2_edges[:-1] + (x2_edges[1] - x2_edges[0]) / 2
# преобразование xdata в правильную форму для гауссова процесса
x_data = [x1_centers[:, None], x2_centers[:, None]]
# преобразование ydata в правильную форму для гауссова процесса
y_data = hist.flatten()
```

Отметим, что вместо выполнения вычислений по решетке данные x_1 и x_2 интерпретируются как отдельные. Это позволяет сформировать ковариационную матрицу для каждой координаты, эффективно уменьшая размер этой матрицы, необходимой для вычисления гауссова процесса. Нужна лишь осторожность и внимательность при использовании класса `LatentKron` для определения гауссова процесса. Важно отметить, что это не вычислительный прием, а математическое свойство структуры матриц этого типа, поэтому мы не вводим какую-либо аппроксимацию или ошибку в нашу модель. Мы просто формируем ее таким способом, чтобы создать возможность более быстрых вычислений:

```
with pm.Model() as model_rw:
    l = pm.HalfNormal('l', rw_df.std().values, shape=2)
    cov_func1 = pm.gp.cov.ExpQuad(1, ls=l[0])
    cov_func2 = pm.gp.cov.ExpQuad(1, ls=l[1])

    gp = pm.gp.LatentKron(cov_funcs=[cov_func1, cov_func2])
    f = gp.prior('f', Xs=x_data)

    y = pm.Poisson('y', mu=pm.math.exp(f), observed=y_data)
    trace_rw = pm.sample(1000)
```

Далее формируется диаграмма результатов:

```
rate = np.exp(np.mean(trace_rw['f'], axis=0).reshape((bins, -1)))
fig, ax = plt.subplots(figsize=(6, 6))
ims = ax.imshow(rate, origin='lower')
ax.grid(False)
ticks_loc = np.linspace(0, bins-1, 6)
ticks_lab = np.linspace(0, 1, 6).round(1)
ax.set_xticks(ticks_loc)
ax.set_yticks(ticks_loc)
ax.set_xticklabels(ticks_lab)
ax.set_yticklabels(ticks_lab)
cbar = fig.colorbar(ims, fraction=0.046, pad=0.04)
```

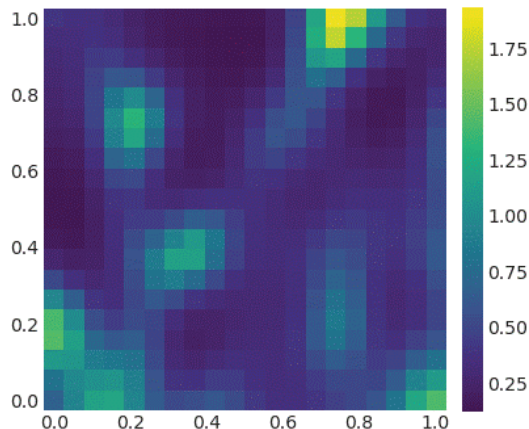


Рис. 7.17

На рис. 7.17 более светлый цвет обозначает более высокую интенсивность расположения деревьев по сравнению с зонами, обозначенными более темным цветом. Можно мысленно представить, что мы заинтересованы в поиске зон с высокой интенсивностью деревьев, возможно потому, что должны выяснить, как лес восстанавливается после пожара, или, скажем, мы изучаем свойства почвы, а сведения об интенсивности расположения деревьев используем как вспомогательные промежуточные данные.

РЕЗЮМЕ

Гауссов процесс – это обобщение многомерного гауссова распределения для бесконечного числа измерений, полностью определяемое функцией среднего значения (меаны) и ковариационной функцией. Поскольку теоретически можно представить функции как векторы бесконечной длины, появляется возможность использования гауссовых процессов как априорных распределений для функций. На практике мы не работаем с бесконечными объектами, а пользуемся многомерными гауссовыми распределениями с числом измерений, соответствующим количеству точек данных. Для определения соответствующей ковариационной функции используются правильно параметризованные ядра, и при исследовании этих гиперпараметров мы в конечном итоге определяем произвольно сложные функции.

В этой главе было приведено лишь краткое введение в гауссовы процессы. Мы рассмотрели регрессию, полупараметрические модели (в примере с островными сообществами), объединение двух и более ядер для улучшения описания неизвестной функции, а также возможности использования гауссовых процессов для решения задач классификации. Существует множество других тем, связанных с гауссовыми процессами. Но я надеюсь, что это краткое введение стимулирует читателей в достаточной степени для того, чтобы продолжать изучение и практическое использование гауссовых процессов и непараметрических моделей.

УПРАЖНЕНИЯ

1. Изучая пример в разделе «Ковариационные функции и ядра», убедитесь в том, что вы полностью понимаете отношение между входными данными `data` и сгенерированной ковариационной матрицей. Попробуйте использовать другие входные данные, например `data = np.random.normal(size=4)`.
2. Повторите выполнение кода, генерирующего диаграмму на рис. 7.3, с увеличением количества выборок из априорного распределения гауссова процесса приблизительно до 200. На первоначальной диаграмме количество выборок равно 2. Каков диапазон сгенерированных значений y ?

3. Для диаграммы, сгенерированной в упражнении 2, вычислите стандартное отклонение для значений y в каждой точке данных. Выполните это в следующих формах:
 - визуально, наблюдая диаграммы;
 - непосредственно по значениям, сгенерированным из `stats.multivariate_normal.rvs`;
 - исследуя ковариационную матрицу (если возникают какие-либо затруднения, вернитесь к упражнению 1).Согласованы ли значения, полученные перечисленными выше тремя методами?
4. Повторите выполнение модели `model_reg` и сформируйте новую диаграмму с использованием тестовых точек данных `test_points` `X_new` `np.linspace(np.floor(x.min()), 20, 100)[: ,None]`. Что вы наблюдаете на этой диаграмме? Как это связано с определением априорного распределения гауссова процесса?
5. Еще раз выполните упражнение 1, но на этот раз используйте линейное ядро (изучите соответствующий код для линейного ядра).
6. Внимательно изучите раздел документации библиотеки PyMC3 <https://docs.pymc.io/notebooks/GP-MeansAndCovs.html>.
7. Выполните модель логистической регрессии для данных `space_flu`. Что вы наблюдаете? Можете ли вы объяснить полученный результат?
8. Измените модель логистической регрессии для подгонки к данным. Подсказка: используйте полином второго порядка.
9. Сравните используемую в этой главе модель катастроф в угледобывающей промышленности с той же моделью, описанной в документации библиотеки PyMC3 (https://docs.pymc.io/notebooks/getting_started.html#Case-study-2:-Coal-mining-disasters). Опишите и объясните различия между этими моделями с точки зрения определения моделей и получаемых результатов.

Глава 8

.....

Механизмы статистического вывода

«Первый принцип: не следует обманывать самого себя, потому что нет другого человека, которого можно было бы обмануть так же легко».

– Ричард Фейнман

До настоящего момента наше внимание было сосредоточено на создании, интерпретации результатов и критике моделей. Мы полагались на «магию» функции `rm.sample` при автоматическом вычислении апостериорных распределений, скрытом от нас. Теперь необходимо сосредоточиться на изучении некоторых подробностей механизмов статистического вывода, используемых внутри этой функции. Общая цель инструментов вероятностного программирования, подобных библиотеке РумСЗ, – избавить пользователя от забот, связанных с методами формирования выборок. Но знание того, как получаются выборки из апостериорного распределения, весьма важно для полного понимания процесса статистического вывода. Кроме того, это знание может помочь глубже понять, когда и как эти методы дают критический сбой и что делать в подобных ситуациях. Если вам не нужно глубокое понимание того, как эти методы применяются для аппроксимации апостериорного распределения, то вы можете пропустить большую часть главы, но я настоятельно рекомендую, по крайней мере, прочесть раздел «Диагностирование выборок», так как в нем приводятся несколько правил, которые помогут при проверке надежности выборок из апостериорных распределений.

Существует множество методов вычисления апостериорного распределения. В этой главе рассматриваются некоторые общие идеи и принципы, но особое внимание будет уделено наиболее важным методам, реализованным в библиотеке РумСЗ.

В этой главе рассматриваются следующие темы:

- вариационные методы;
- алгоритм Метрополиса–Гастингса;
- метод Монте-Карло с использованием оператора Гамильтона;

- последовательный метод Монте-Карло;
- диагностирование выборок.

МЕХАНИЗМЫ СТАТИСТИЧЕСКОГО ВЫВОДА

Байесовские методы теоретически (концептуально) просты, но с математической и вычислительной точки зрения могут оказаться весьма сложными. Главная причина этой сложности заключается в том, что предельное правдоподобие, знаменатель в теореме Байеса (см. формулу 1.4), обычно принимает форму чрезвычайно трудоемкого для вычисления интеграла. Поэтому апостериорное распределение обычно оценивается в числовом виде с применением семейства методов Монте-Карло с использованием марковских цепей (Markov Chain Monte Carlo – MCMC) или недавно появившихся вариационных алгоритмов. Эти методы иногда называют механизмами статистического вывода (inference engines), потому что, по крайней мере теоретически, они способны аппроксимировать апостериорное распределение для любой вероятностной модели. Даже если на практике статистический вывод не всегда успешно работает, само существование таких методов стимулирует разработку языков и инструментальных средств вероятностного программирования, таких как библиотека PyMC3.

Цель языков и инструментов вероятностного программирования – отделить процесс создания модели от процесса статистического вывода для упрощения выполнения итеративных этапов создания, оценки и изменения/расширения модели (см. главы 1 и 2). Представляя процесс статистического вывода (но не процесс создания модели) как черный ящик, пользователи языков и инструментов вероятностного программирования (в частности, библиотеки PyMC3) могут полностью сосредоточиться на решении собственных конкретных задач, а всеми подробностями статистических вычислений занимается библиотека PyMC3. Именно так мы поступали до настоящего момента. Возможно, мы склонны считать, что это и есть очевидный или естественный подход. Но здесь важно отметить, что до появления языков и инструментов вероятностного программирования исследователи, создававшие вероятностные модели, также использовали их для разработки собственных методов выборки, обычно подгоняемых под эти модели, или исследователи вынуждены были упрощать свои модели, чтобы сделать их подходящими для конкретных математических аппроксимаций. В действительности эта практика сохраняется и поныне в некоторых научных кругах. Методика подгонки и упрощения может быть более ясной и простой и даже обеспечивать более эффективный способ вычисления апостериорного распределения, но при ее применении слишком высока вероятность ошибок, к тому же требуются значительные затраты времени даже для экспертов. Более того, методика подгонки и упрощения не подходит для применения большинством практиков-исследователей, заинтересованных в решении задач с помощью вероятностных моделей. Программное обеспе-

чение, подобное библиотеке PyMC3, стимулирует людей из самых разных областей деятельности начать применение вероятностных моделей, понижая математический и вычислительный входной барьер. Лично я считаю, что это уникальная возможность, а также стимул к более подробному изучению эффективных практических методов статистического моделирования, поэтому мы не должны обманывать самих себя. Предыдущие главы в основном были посвящены изучению основ байесовского моделирования, но сейчас мы переходим к изучению на теоретическом уровне способов реализации автоматического статистического вывода. Мы узнаем, когда и почему они приводят к критическим ошибкам и что делать в подобных ситуациях.

Существует несколько методов вычисления апостериорного распределения. Я позволил себе разделить их на две крупные группы:

- немарковские методы:
 - грид-вычисления (grid computing);
 - квадратическая аппроксимация;
 - вариационные методы;
- марковские методы:
 - алгоритм Метрополиса–Гастингса;
 - метод Монте-Карло с использованием оператора Гамильтона;
 - последовательный метод Монте-Карло.

НЕМАРКОВСКИЕ МЕТОДЫ

Начнем с рассмотрения механизмов статистического вывода с использованием немарковских методов. При некоторых условиях эти методы могут обеспечить быструю и достаточно точную аппроксимацию апостериорного распределения.

Грид-вычисления

Грид-вычисления (grid computing) – это простой метод грубой силы. Даже если нет возможности вычислить полное апостериорное распределение, можно вычислить априорное распределение и правдоподобие для всех точек, это достаточно часто (и даже, возможно, наиболее часто) возникающая ситуация. Предположим, что необходимо вычислить апостериорное распределение для модели с одним параметром, тогда аппроксимация методом грид-вычислений выглядит следующим образом.

1. Определение разумно обоснованного интервала для параметра (априорное распределение должно дать подсказку).
2. Разместить решетку (сетку) по точкам (в общем случае – на равных расстояниях) в выбранном интервале.
3. Для каждой точки в решетке (сетке) перемножить правдоподобие и априорное распределение (вероятность).

Дополнительно (но не обязательно) можно нормализовать вычисляемые значения, то есть разделить результат в каждой точке на сумму значений всех точек.

В следующем фрагменте кода реализован метод грид-вычислений для получения апостериорного распределения для модели подбрасывания монеты:

```
def posterior_grid(grid_points=50, heads=6, tails=9):
    """
    Реализация метода грид-вычислений для задачи подбрасывания монеты
    """
    grid = np.linspace(0, 1, grid_points)
    prior = np.repeat(1/grid_points, grid_points) # равномерное априорное распределение
    likelihood = stats.binom.pmf(heads, heads+tails, grid)
    posterior = likelihood * prior
    posterior /= posterior.sum()
    return grid, posterior
```

Предположим, что монета подброшена 13 раз и наблюдается выпадение трех орлов:

```
data = np.repeat([0, 1], (10, 3))
points = 10
h = data.sum()
t = len(data) - h
grid, posterior = posterior_grid(points, h, t)
plt.plot(grid, posterior, 'o-')

plt.title(f'heads = {h}, tails = {t}')
plt.yticks([])
plt.xlabel('θ');
```

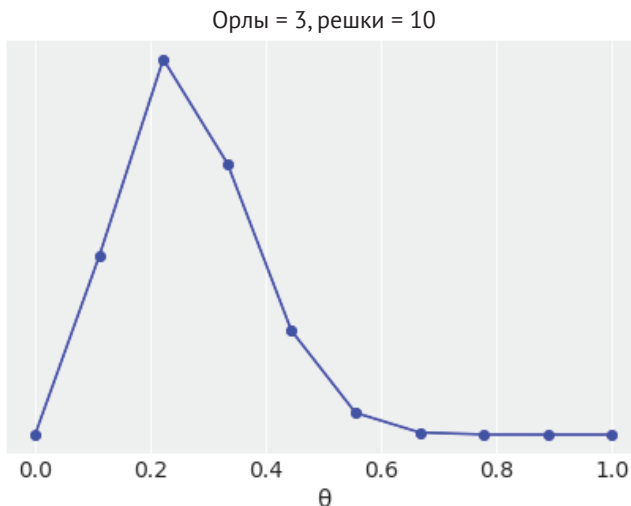


Рис. 8.1

Здесь легко заметить, что большее количество точек (или, что то же самое, уменьшение размера шага решетки) дает более точную аппроксимацию. Фактически в предельном случае бесконечного количества точек мы получим точное апостериорное распределение за счет увеличения вычислительных ресурсов.

Самое значительное затруднение при использовании метода грид-вычислений заключается в том, что этот метод плохо масштабируется при увеличении числа параметров, которые также называют измерениями. Это можно наблюдать на простом примере. Предположим, что необходимо сделать выборку в единичном отрезке (см. рис. 8.2), как в задаче подбрасывания монеты, и мы используем четыре равноудаленные точки. Это соответствует разрешению (шагу решетки) 0.25 единицы. Теперь предположим, что мы решаем двумерную задачу (квадрат на рис. 8.2) и используем решетку с тем же разрешением (шагом), тогда потребуется 16 точек. Для решения трехмерной задачи потребуется уже 64 точки (куб на рис. 8.2). В этом примере требуется в 16 раз больше ресурсов для выборки из куба со стороной 1, чем для отрезка длиной 1 с разрешением 0.25. Если будет принято решение о необходимости разрешения 0.1 единицы, то мы получим выборку из 10 точек для отрезка и 1000 точек для куба.

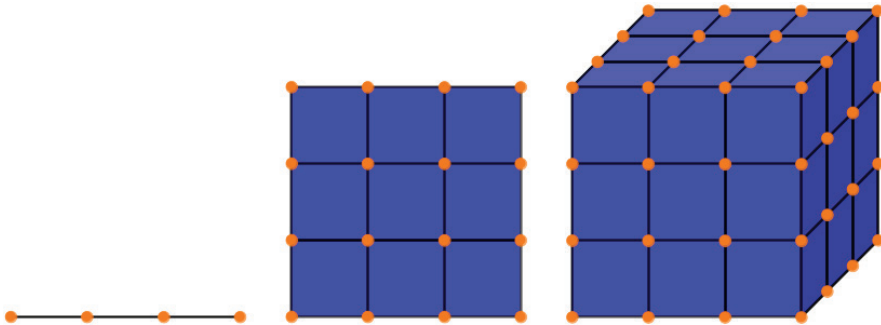


Рис. 8.2

Кроме экспоненциального увеличения количества точек наблюдается и другое явление, которое не является исключительным свойством метода грид-вычислений или какого-либо другого метода, – это свойство многомерных пространств. При увеличении числа параметров область пространства параметров, где сконцентрирована большая часть апостериорного распределения, становится все меньше и меньше по сравнению с объемом выборок данных. Это весьма распространенное явление в статистике и машинном обучении, известное как проклятие размерности (*curse of dimensionality*), но математики предпочитают называть это явление концентрацией меры (*concentration of measure*).

Обычно о проклятии размерности говорят, когда наблюдаются разнообразные подобные явления, отсутствующие в пространствах с низкими размер-

ностями, но возникающие в пространствах с высокими размерностями. Ниже приведены примеры таких явлений:

- при увеличении числа измерений евклидово расстояние между любой парой элементов выборки постоянно уменьшается. То есть в пространствах с высокими размерностями большинство точек в основном находится на одинаковом расстоянии друг от друга;
- в гиперкубе большая часть объема (данных) сосредоточена в его углах, а не в середине. В гиперсфере большая часть объема (данных) сосредоточена на ее поверхности, а не в середине;
- при высоких размерностях основная масса многомерного гауссова распределения не находится близко к среднему значению (или к моде), а расположена в окружающей оболочке (shell), которая перемещается от среднего значения к хвостам при увеличении размерности. Эта оболочка получила название типового множества (typical set).

Примеры исходного кода, демонстрирующего перечисленные выше и другие проявления проклятия размерности, можно найти на сайте <https://github.com/aloctavodia/BAP>.

В контексте нашего обсуждения все приведенные выше факты означают, что если не сделать разумный выбор способа оценки апостериорного распределения, то большая часть времени будет потрачена на вычисление значений, дающих почти нулевой вклад в апостериорное распределение, следовательно, ценные ресурсы будут растрочены напрасно. Метод грид-вычислений представляет собой не самый эффективный и интеллектуальный метод при выборе оценки апостериорного распределения. Таким образом, грид-вычисления не слишком полезны в качестве общего метода решения задач с высокими размерностями.

Метод квадратической аппроксимации

Квадратическая аппроксимация, также известная как метод Лапласа, или нормальная аппроксимация, включает аппроксимацию апостериорного распределения $p(x)$ в сочетании с гауссовым распределением $q(x)$.

Метод квадратической аппроксимации состоит из двух этапов.

1. Поиск моды апостериорного распределения. Это должно быть средним значением $q(x)$.
2. Вычисление матрицы Гессе (гессиана). По этой матрице можно вычислить стандартное отклонение для $q(x)$.

Первый этап может быть выполнен в числовом виде с использованием методов оптимизации, то есть методов поиска максимума или минимума функции. Для этой цели существует множество разработанных и полностью готовых к практическому применению методов. Поскольку для гауссова распределения мода и среднее значение равны, можно воспользоваться модой как средним значением аппроксимируемого распределения $q(x)$. Второй этап не столь очевиден. Можно приблизительно вычислить стандартное отклонение для $q(x)$,

оценивая кривизну моды/среднего значения $q(x)$. Это можно сделать, вычислив значение, обратное квадратному корню из матрицы Гессе. Матрица Гессе – это матрица второй производной функции, а обратная ей матрица позволяет получить ковариационную матрицу. С помощью библиотеки PyMC3 можно написать следующий код:

```
with pm.Model() as normal_approximation:
    p = pm.Beta('p', 1., 1.)
    w = pm.Binomial('w', n=1, p=p, observed=data)
    mean_q = pm.find_MAP()
    std_q = ((1/pm.find_hessian(mean_q, vars=[p]))**0.5)[0]
    mean_q['p'], std_q
```



Если попытаться использовать функцию `pm.find_MAP` из библиотеки PyMC3, то будет выведено предупреждающее сообщение. Из-за проклятия размерности использование оценки апостериорного максимума для представления апостериорного распределения или даже для инициализации метода формирования выборки, вообще говоря, является не самым лучшим выбором.

Рассмотрим, как квадратическая аппроксимация выглядит для модели биномиального бета-распределения:

```
# аналитические вычисления
x = np.linspace(0, 1, 100)
plt.plot(x, stats.beta.pdf(x, h+1, t+1), label='True posterior')

# квадратическая аппроксимация
plt.plot(x, stats.norm.pdf(x, mean_q['p'], std_q), label='Quadratic approximation')
plt.legend(loc=0, fontsize=13)

plt.title(f'heads = {h}, tails = {t}')
plt.xlabel('θ', fontsize=14)
plt.yticks([]);
```

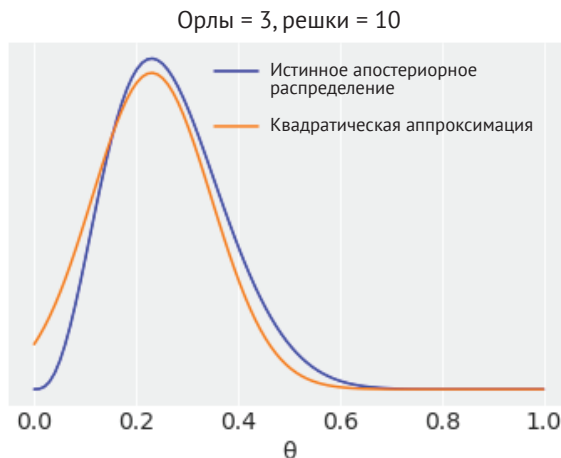


Рис. 8.3

На рис. 8.3 показано, что квадратическая аппроксимация дает неплохой результат, по крайней мере для этого конкретного примера. Строго говоря, можно применять метод Лапласа только к несвязанным переменным, то есть к переменным из пространства $\mathbb{R}^N$. Причина в том, что гауссово распределение является несвязанным распределением, так что если оно используется для моделирования связанного распределения (такого как бета-распределение), то в конечном итоге мы получим положительную оценку плотности, хотя в действительности плотность должна быть нулевой (за пределами интервала $[0, 1]$ для бета-распределения). Тем не менее метод Лапласа можно использовать, если сначала выполнить преобразование связанной переменной в несвязанную. Например, обычно мы используем полунормальное распределение для точного моделирования стандартного отклонения, потому что оно ограничено интервалом $[0, \infty)$, но можно сделать переменную с полунормальным распределением несвязанной, взяв логарифм от нее.

Метод Лапласа ограничен, но может успешно работать для некоторых моделей и применяться для получения аналитических выражений для аппроксимации апостериорного распределения. Кроме того, это один из конструктивных элементов более продвинутого метода, известного как INLA (Integrated Nested Laplace Approximation).

В следующем разделе мы рассмотрим вариационные методы, которые в некоторой степени похожи на аппроксимацию Лапласа, но являются более гибкими и мощными инструментами. Некоторые из вариационных методов можно автоматически применять к широкому спектру моделей.

Вариационные методы

В современной байесовской статистике в основном используются марковские методы (см. следующий раздел), но для некоторых задач эти методы могут оказаться слишком медленными. Вариационные методы являются альтернативным вариантом, который может оказаться более удачным выбором для больших наборов данных (big data) и/или для апостериорных распределений, требующих для вычисления чрезмерных затрат и ресурсов.

Общая идея вариационных методов заключается в аппроксимации апостериорного распределения более простым распределением по методике, похожей на метод Лапласа, но более конкретно проработанным способом. Такое упрощенное распределение можно найти, решив задачу оптимизации, состоящую из поиска самого близкого к апостериорному возможного распределения при некотором установленном способе измерения степени близости. Часто используемым способом измерения близости между распределениями является применение расхождения Кульбака–Лейблера (рассматриваемого в главе 5). Используя расхождение Кульбака–Лейблера, можно записать следующую формулу:

$$D_{\text{KL}}(q(\theta) \| p(\theta|y)) = \int q(\theta) \log \frac{q(\theta)}{p(\theta|y)} d(\theta). \quad (8.1)$$

Здесь q – упрощенное распределение, используемое для аппроксимации апостериорного распределения $p(\theta|y)$. q часто называют вариационным распределением, а при использовании метода оптимизации мы пытаемся определить параметры q (часто называемые вариационными параметрами), которые приближают q как можно ближе (в измерении расхождения Кульбака–Лейблера) к апостериорному распределению. Отметим, что в формуле 8.1 записано $D_{KL}(q(\theta)||p(\theta|y))$, а не $D_{KL}((\theta|y)||q(\theta))$, потому что это более удобный способ представления задачи и более эффективное решение. Но при этом необходимо уточнить, что запись расхождения Кульбака–Лейблера в другом направлении также может быть полезной и в действительности приводит к другому набору методов, которые здесь не рассматриваются.

Проблема с формулой 8.1 состоит в том, что нам неизвестно апостериорное распределение, поэтому напрямую его использовать невозможно. Необходимо найти другой способ выражения этой задачи. Приведенные ниже постепенные преобразования показывают, как это сделать. Если вам неинтересны промежуточные математические выкладки, то можно сразу перейти к формуле 8.7.

Сначала заменим условное распределение на его определение (см. главу 1, чтобы вспомнить определение условного распределения):

$$D_{KL}(q(\theta)||p(\theta|y)) = \int q(\theta) \log \frac{q(\theta)}{\frac{p(\theta|y)}{p(y)}} d(\theta). \quad (8.2)$$

Преобразуем формулу 8.2 в более удобный вид:

$$= \int q(\theta) \log \frac{q(\theta)}{p(\theta|y)} p(y) d(\theta). \quad (8.3)$$

Воспользовавшись свойствами логарифма, получим следующее выражение:

$$= \int q(\theta) \left(\log \frac{q(\theta)}{p(\theta|y)} + \log p(y) \right) d(\theta). \quad (8.4)$$

Переходим к сумме интегралов:

$$= \int q(\theta) \log \frac{q(\theta)}{p(\theta|y)} d(\theta) + \int q(\theta) \log p(y) d(\theta). \quad (8.5)$$

Так как интеграл от $q(\theta)$ равен 1, а выражение $\log p(y)$ можно вынести за пределы интеграла, получаем:

$$= \int q(\theta) \log \frac{q(\theta)}{p(\theta|y)} d(\theta) + \log p(y). \quad (8.6)$$

Еще раз воспользуемся свойствами логарифма, чтобы получить окончательную формулу:

$$D_{KL}(q(\theta) \parallel p(\theta|y)) = - \underbrace{\int q(\theta) \log \frac{p(\theta|y)}{q(\theta)} d(\theta)}_{\text{подтверждение нижней границы (evidence lower bound – ELBO)}} + \log p(y). \quad (8.7)$$

Поскольку $D_{KL} \geq 0$, то $\log p(y) \geq \text{ELBO}$, другими словами, подтверждение (или предельное правдоподобие) всегда больше или равно ELBO, отсюда и его название. Так как $\log p(y)$ является константой, можно сосредоточить все внимание на ELBO. Определение максимального значения ELBO равнозначно определению минимума дивергенции Кульбака–Лейблера. Таким образом, определение максимума ELBO является способом сделать $q(\theta)$ как можно более близким к апостериорному распределению $p(\theta|y)$.

Отметим, что до настоящего момента мы не применяли какую-либо аппроксимацию, а только выполняли некоторые алгебраические преобразования. Аппроксимация вводится во время выбора $q(\bullet)$. Теоретически $q(\bullet)$ может быть любым распределением на ваше усмотрение, но на практике следует выбирать распределения, с которыми проще работать. Одним из возможных вариантов решения является предположение о том, что многомерное апостериорное распределение можно описать независимыми одномерными распределениями. Математически это можно записать в виде следующей формулы:

$$q(\theta) = \prod_j q_j(\theta_j). \quad (8.8)$$

Этот вариант известен как аппроксимация методом самосогласованного поля. Аппроксимации методом самосогласованного поля широко применяются в физике для моделирования сложных систем со множеством взаимодействующих элементов как набора более простых подсистем, вообще не взаимодействующих друг с другом, или, в обобщенном случае, взаимодействия учитываются только в среднем.

Можно выбирать различные распределения q_j для каждого параметра θ_j . В общем случае распределения q_j выбираются из семейства экспоненциальных распределений, потому что с ними проще работать. К семейству экспоненциальных относятся многие распределения, которые мы использовали в этой книге, например нормальное и экспоненциальное распределения, бета-распределение, распределение Дирихле, гамма-распределение, распределение Пуассона, категориальное распределение и распределение Бернулли.

Правильно подобрав и разместив все описанные выше элементы, мы успешно преобразуем задачу статистического вывода в задачу оптимизации. Таким образом, по крайней мере теоретически, все, что необходимо для решения, – это использование готовых к практическому применению методов оптимизации и максимизация ELBO. На практике эти операции немного более сложны, но здесь мы подробно рассмотрели общую идею.

Вариационный статистический вывод с применением автоматического дифференцирования

Основным недостатком вариационного метода согласованного поля, описанного в предыдущем разделе, является необходимость разработки специализированного алгоритма для каждой модели. Не существует универсального метода реализации механизма статистического вывода, но вместо него можно предложить способ генерации методов, ориентированных на конкретные модели и требующих вмешательства пользователя. Однако многие специалисты уже обратили внимание на эту проблему и предложили решения, сосредоточенные на автоматизации вариационных методов. Недавно предложен метод вариационного статистического вывода с применением автоматического дифференцирования (Automatic Differentiation Variational Inference – ADVI) (см. <http://arxiv.org/abs/1603.00788>). С теоретической точки зрения этапы выполнения метода ADVI выглядят следующим образом:

- выполняется преобразование всех связанных распределений, чтобы расположить их на прямой действительных чисел, как это было сделано по методу Лапласа;
- выполняется аппроксимация несвязанных параметров с использованием гауссова распределения (это член q_j в формуле 8.8). Следует отметить, что гауссово распределение в преобразованном пространстве параметров не являлось гауссовым в исходном пространстве параметров;
- применение автоматического дифференцирования для максимизации ELBO.

В документации библиотеки PyMC3 (https://docs.pymc.io/nb_examples) приведено множество примеров практического использования вариационных методов статистического вывода с помощью этой библиотеки.

МАРКОВСКИЕ МЕТОДЫ

Существует семейство взаимосвязанных методов под общим названием метод Монте-Карло с использованием цепей Маркова (Markov chain Monte Carlo – MCMC). Эти стохастические методы позволяют получать выборки из истинного апостериорного распределения, пока мы способны вычислять правдоподобие и априорное распределение по точкам (данным). Это то же самое условие, которое необходимо для грид-вычислений, но методы MCMC превосходят аппроксимацию методом грид-вычислений. Причина в том, что методы MCMC способны получать больше выборок из областей с более высокой вероятностью, чем из областей с более низкой вероятностью. В действительности любой метод MCMC будет посещать (visit) каждую область пространства параметров в соответствии с их относительными вероятностями. Если вероятность области A в два раза выше, чем вероятность области B, то мы получим в два раза больше выборок из A, чем из B. Таким образом, даже если у нас нет возможно-

сти вычислить полное апостериорное распределение аналитически, мы можем воспользоваться методами МСМС для получения выборок из этого апостериорного распределения.

На теоретическом фундаментальном уровне математической статистики достаточно знать метод вычисления ожидаемых значений, например:

$$\mathbb{E}[f] = \int_{\theta} p(\theta) f(\theta) d\theta. \quad (8.9)$$

Ниже приведено несколько конкретных примеров этого обобщенного выражения:

- апостериорное распределение по формуле 1.14;
- прогнозируемое апостериорное распределение по формуле 1.17;
- предельное правдоподобие используемой конкретной модели по формуле 5.13.

С помощью методов МСМС выполняется аппроксимация формулы 8.9 с использованием конечного числа выборок:

$$\lim_{N \rightarrow \infty} \mathbb{E}_{\pi}[f] = \frac{1}{N} \sum_{n=1}^N f(\theta_n). \quad (8.10)$$

Главный вывод из формулы 8.10 – это равенство только лишь асимптотическое, то есть степень приближения к равенству увеличивается при бесконечном числе выборок. На практике всегда используется конечное число выборок, поэтому мы заинтересованы в том, чтобы методы МСМС обеспечивали сходимость к правильному ответу как можно быстрее – с минимально возможным числом выборок (также называемых *draws* – выпадениями жребия или выигрыша).

В общем случае уверенности в том, что какая-либо конкретная выборка по методам МСМС обеспечит сходимость, достичь не так-то просто, мягко говоря. Поэтому на практике необходимо полагаться на эмпирические проверки, чтобы быть уверенным в том, что получена надежная аппроксимация методами МСМС. Подобные проверки для выборок методами МСМС будут рассматриваться в разделе «Диагностирование выборок». Важно всегда помнить, что другие методики аппроксимации (в том числе и немарковские методы, рассматриваемые ранее в этой главе) также требуют эмпирических проверок, но мы этим не занимаемся, поскольку главное внимание в этой книге нацелено на методы Монте-Карло с использованием цепей Маркова.

Понимание на теоретическом (концептуальном) уровне методов МСМС может помочь в диагностировании выборок, получаемых этими методами. Но все же, рано или поздно, возникает вопрос: что означает это название? Впрочем, такой вопрос возникает не у всех и не всегда. Но для лучшего понимания, что такое методы Монте-Карло с использованием цепей Маркова, обозначаемые аббревиатурой МСМС, разделим это понятие на две части МС: метод Монте-Карло (МС – Monte Carlo) и цепь Маркова (МС – Markov Chain).

Метод Монте-Карло

Использование случайных чисел объясняет часть Монте-Карло в названии метода. Методы Монте-Карло – это весьма обширное семейство алгоритмов, использующих случайные выборки для вычисления или имитации заданного процесса. Монте-Карло – административный район княжества Монако, в котором расположены самые известные во всем мире казино. У одного из разработчиков метода Монте-Карло Станислава Улама (Stanislaw Ulam) дядя был заядлым игроком и много времени проводил в Монте-Карло. В основу вероятностного языка программирования и статистического пакета ПО Stan была заложена идея о том, что многие задачи, которые трудно решить или даже сформулировать точными аналитическими методами, можно эффективно исследовать и находить решения с помощью выборок из соответствующих совокупностей данных. По легенде (впрочем, исторически обоснованной), Улам, раскладывая пасьянс Solitaire (солитер) во время выздоровления после болезни, задался вопросом, какова вероятность того, что пасьянс сложится. Одним из способов решения этой задачи стало сведение ее к аналитической комбинаторной задаче. Другой способ, предложенный Stan, состоит в проведении ряда сеансов раскладки пасьянса Solitaire и подсчете сложившихся раскладок. Возможно, это кажется вам вполне очевидным или, по крайней мере, разумным, вероятно, вы даже использовали методы многократных выборок для решения статистических задач. Но следует напомнить, что мысленный эксперимент проводился около 70 лет назад, когда самые первые компьютеры для практического использования только еще начинали разрабатываться.

Первым практическим приложением метода Монте-Карло было решение трудноразрешимой задачи из области ядерной физики с использованием инструментальных средств того времени. В наши дни даже персональные компьютеры обладают мощностью, достаточной для решения многих интересных задач с применением метода Монте-Карло. Таким образом, эти методы стали доступными для решения самых разнообразных задач в науке, инженерной практике, промышленности и даже в искусстве. Классическим учебным примером применения метода Монте-Карло является получение числовой оценки точности знаков числа π . Существуют более эффективные методы определения точных десятичных знаков числа π , но в учебных целях мы все-таки рассмотрим метод Монте-Карло.

Оценку точности числа π можно получить, выполняя следующую процедуру.

1. Случайным образом выбрать («выбросить» как игральные кости) N точек в квадрате со стороной $2R$.
2. Начертить окружность радиусом R , вписанную в этот квадрат, и подсчитать количество точек, находящихся внутри построенной окружности.
3. Вычислить оценку приближенного значения числа π как отношение

$$4 \frac{\text{кол. внутри}}{N}.$$

Здесь необходимо сделать несколько замечаний:

- площади круга и квадрата пропорциональны количеству точек внутри окружности и общему количеству точек N соответственно;
- известно, что любая точка находится внутри окружности, если выполняется следующее неравенство: $\sqrt{(x^2 + y^2)} \leq R$;
- площадь квадрата равна $(2R)^2$, а площадь круга – πR^2 . Таким образом, нам известно, что отношение площади квадрата к площади круга равно числу π .

Написав несколько строк на языке Python, можно выполнить эту простую имитацию метода Монте-Карло и вычислить число π , а также относительную ошибку (погрешность) полученной оценки по сравнению с действительным значением π :

```
N = 10000

x, y = np.random.uniform(-1, 1, size=(2, N))
inside = (x**2 + y**2) <= 1
pi = inside.sum()*4/N
error = abs((pi - np.pi) / pi) * 100

outside = np.invert(inside)

plt.figure(figsize=(8, 8))
plt.plot(x[inside], y[inside], 'b.')
plt.plot(x[outside], y[outside], 'r.')
plt.plot(0, 0, label=f'π* = {pi:4.3f}\nerror = {error:4.3f}', alpha=0)
plt.axis('square')
plt.xticks([])
plt.yticks([])
plt.legend(loc=1, frameon=True, framealpha=0.9);
```

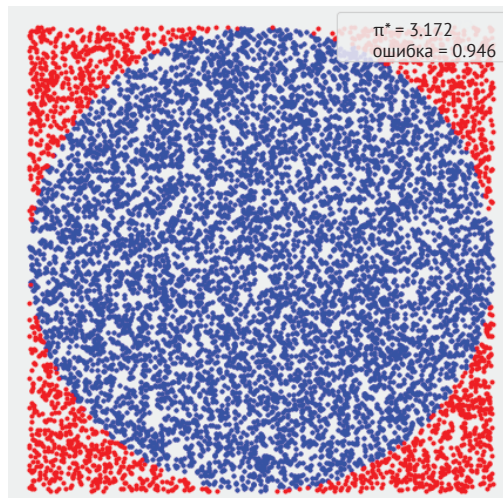


Рис. 8.4

В приведенном выше коде можно видеть, что переменная `outside` используется только для построения диаграммы. Она не нужна для вычисления приближенного значения $\hat{\pi}$. Также отметим, что поскольку наше вычисление ограничено единичной окружностью, можно пропустить вычисление квадратного корня при вычислении переменной `inside`.

Цепи Маркова

Цепь Маркова (Markov chain) – это математический объект, состоящий из последовательности состояний и набора вероятностей переходов, который описывает, как выполняются переходы между состояниями. Цепь является марковской, если вероятность перехода в любое другое состояние зависит только от текущего состояния. Имея такую цепь, можно выполнить случайное блуждание (random walk), выбрав начальную точку и переходя в другие состояния в соответствии с вероятностями перехода. Если каким-либо образом найти цепь Маркова с вероятностями переходов, пропорциональным распределению, которое требуется для выборки (в байесовском анализе это апостериорное распределение), то процедура выборки превращается в простые перемещения между состояниями в такой цепи.

Но как найти такую цепь, если апостериорное распределение заранее неизвестно? Существует такое понятие, как принцип детального равновесия. В соответствии с этим принципом (или условием) мы должны перемещаться по обратимому (реверсивному) пути (обратимый процесс – это широко применяемая в физике аппроксимация). Таким образом, вероятность пребывания в состоянии i и перехода в состояние j должна быть равной вероятности пребывания в состоянии j и перехода в состояние i . В действительности это условие не является необходимым, но оно достаточно, и в общем случае его проще подтвердить, поэтому оно почти всегда используется как исходная предпосылка в подавляющем большинстве широко распространенных методов MCMC.

В итоге, если удалось создать цепь Маркова, соответствующую принципу детального равновесия, можно сформировать выборку из этой цепи с гарантией того, что мы будем получать выборки из правильного распределения. Это чрезвычайно важный результат. Кроме того, это базовый механизм, используемый в программных инструментальных средствах, таких как библиотека PyMC3.

Вероятно, наиболее известным методом Монте-Карло с использованием цепей Маркова является алгоритм Метрополиса–Гастингса, который рассматривается в следующем разделе.

Алгоритм Метрополиса–Гастингса

Для некоторых распределений, таких как гауссово распределение, существуют весьма эффективные алгоритмы для получения из них выборок, но для других распределений это не всегда верно. Алгоритм Метрополиса–Гастингса (Metropolis-Hastings) позволяет получать выборки из любого распределения

вероятностей $p(x)$, предоставляя возможность вычислять, по крайней мере, значение, пропорциональное распределению, без учета фактора нормализации. Это очень полезное свойство, поскольку во многих задачах, не только из области байесовской статистики, вычисление фактора нормализации является трудоемким этапом.

Чтобы понять теоретическую концепцию алгоритма Метрополиса–Гастингса, воспользуемся следующей аналогией. Предположим, что необходимо узнать объем воды в некотором озере, а также определить, в какой части озера находится его самая глубокая точка. Вода непрозрачна, поэтому невозможно оценить глубину визуально, а площадь озера достаточно велика, так что сеточная (решеточная) аппроксимация не подходит. Для разработки стратегии формирования выборок мы обращаемся за помощью к двум своим лучшим друзьям: Марковскому и Монти. В результате плодотворного обсуждения принимается следующий алгоритм, для реализации которого требуется лодка, но не для развлечений. Можно даже воспользоваться простым деревянным плотом и очень длинным шестом. Это дешевле, чем сонар (звуковой гидролокатор), к тому же все деньги уже потрачены на приобретение лодки. Рассмотрим следующие этапы выполнения поставленной задачи.

1. Выбор случайного места на озере и перемещение на лодке к этому месту.
2. Использование шеста для измерения глубины в этой точке озера.
3. Перемещение лодки в другое произвольно выбранное место и выполнение нового измерения глубины.
4. Сравнение результатов двух измерений по следующему алгоритму:
 - если в новой точке глубина больше, чем в первой, то записать в блокнот эту глубину в новой точке и повторить процесс, начиная с шага 2;
 - если в новой точке глубина меньше, чем в первой, то возможны два варианта: принять или отказаться. Принятие означает, что мы записываем глубину в новой точке и повторяем процесс, начиная с шага 2. Отказ означает, что мы возвращаемся к первой точке и записываем (да, еще раз) значение измерения глубины в первой точке.

Правило для выбора решения о принятии или отказе называется критерием Метрополиса–Гастингса и в обобщенном смысле утверждает, что значение в новой точке измерений должно быть принято с вероятностью, пропорциональной отношению глубин в новой и старой точках.

Если выполнить приведенную выше процедуру, то можно получить не только общий объем воды в озере и его максимальную глубину, но и приближенную форму (и кривизну) дна озера. Нетрудно догадаться, что в этой аналогии кривизна дна озера – это апостериорное распределение, а самая глубокая точка – мода распределения. По утверждению нашего друга Марковского, чем больше число итераций, тем точнее приближение. Действительно, теория подтверждает, что при определенных общих условиях мы получим точный ответ, если извлечем бесконечное количество выборок. К счастью, на практике для подавляющего большинства задач можно достичь весьма точной аппроксимации, используя конечное и относительно небольшое количество выборок.

Приведенного выше описания достаточно для теоретического понимания работы алгоритма Метрополиса–Гастингса. Ниже алгоритм рассматривается более подробно и формально для тех, кто хочет узнать больше.

Выполнение алгоритма Метрополиса–Гастингса состоит из следующих этапов.

1. Выбор начального значения параметра x_i . Это можно сделать случайным образом или по обоснованному предположению.
2. Выбор значения нового параметра x_{i+1} с помощью выборки из распределения, обеспечивающего простое выполнение выборки, например из гауссова или равномерного распределения $Q(x_{i+1}|x_i)$. Можно считать этот шаг равнозначным некоторому изменению состояния x_i .
3. Вычисление вероятности принятия значения нового параметра с использованием критерия Метрополиса–Гастингса:

$$p_a(x_{i+1}|x_i) = \min\left(1, \frac{p(x_{i+1})q(x_i|x_{i+1})}{p(x_i)q(x_{i+1}|x_i)}\right). \quad (8.11)$$

4. Если вероятность, вычисленная на шаге 3, больше, чем значение, взятое из равномерного распределения в интервале $[0,1]$, то новое состояние принимается. В противном случае остается неизменным старое состояние.
5. Процедура итеративно повторяется, начиная с шага 2, пока имеется достаточное число выборок.

Необходимо сделать несколько замечаний:

- если прогнозируемое распределение $Q(x_{i+1}|x_i)$ симметрично, то получаем критерий Метрополиса (обратите внимание на отсутствие фамилии Гастингса):

$$p_a(x_{i+1}|x_i) = \min\left(1, \frac{p(x_{i+1})}{p(x_i)}\right); \quad (8.12)$$

- при выполнении шагов 3 и 4 предполагается, что мы всегда будем принимать переход в наиболее вероятное состояние. Менее вероятные значения параметра принимаются по вероятностному принципу, с учетом отношения между вероятностью значения нового параметра x_{i+1} и вероятностью значения старого параметра x_i . Этот критерий принятия предполагаемых шагов дает более эффективный способ, чем грид-аппроксимация, при этом обеспечивается корректность выборки;
- целевое распределение (апостериорное распределение в байесовской статистике) аппроксимируется списком значений параметров из выборки. Если значение принимается, то оно добавляется в список новых значений выборки x_{i+1} . Если значение не принято, то оно добавляется в список значений x_i (даже если значение повторяется).

После завершения процесса мы получаем список значений. Если все было сделано правильно, то полученные выборки являются аппроксимацией апос-

териорного распределения. Чаще всего прослеживаемые в описанном выше процессе значения представляют собой наиболее вероятные значения, соответствующие апостериорному распределению. Преимущество этой процедуры заключается в том, что анализ апостериорного распределения – это простая обработка массива значений, с которой мы уже достаточно поэкспериментировали во всех предыдущих главах.

В приведенном ниже фрагменте кода показана простейшая реализация алгоритма Метрополиса. Это не решение какой-либо реальной задачи, а всего лишь демонстрация возможности выборки из распределения вероятностей, если известно, как вычислить его плотность по всему диапазону точек. Также отметим, что в приведенной ниже реализации нет ничего байесовского – нет априорного распределения и даже нет данных. Напомним, что методы МСМС представляют собой весьма обобщенные алгоритмы, применимые к самым разнообразным задачам.

Первым аргументом функции `metropolis` является распределение из библиотеки `SciPy`. Предполагается, что неизвестно, как напрямую получать выборки из этого распределения:

```
def metropolis(func, draws=10000):
    """Простейшая реализация алгоритма Метрополиса"""
    trace = np.zeros(draws)
    old_x = 0.5 # func.mean()
    old_prob = func.pdf(old_x)

    delta = np.random.normal(0, 0.5, draws)
    for i in range(draws):
        new_x = old_x + delta[i]
        new_prob = func.pdf(new_x)
        acceptance = new_prob / old_prob
        if acceptance >= np.random.random():
            trace[i] = new_x
            old_x = new_x
            old_prob = new_prob
        else:
            trace[i] = old_x
    return trace
```

В следующем примере функция `func` определена как бета-функция, просто потому, что проще выбирать ее параметры и получать различные формы. Диаграммы выборок, получаемых по алгоритму Метрополиса, изображаются как гистограммы. Кроме того, истинное распределение представлено в виде непрерывной (оранжевой) линии:

```
np.random.seed(3)
func = stats.beta(2, 5)
trace = metropolis(func=func)
x = np.linspace(0.01, .99, 100)
y = func.pdf(x)
plt.xlim(0, 1)
```

```
plt.plot(x, y, 'C1-', lw=3, label='True distribution')
plt.hist(trace[trace > 0], bins=25, density=True, label='Estimated distribution')
plt.xlabel('x')
plt.ylabel('pdf(x)')
plt.yticks([])
plt.legend();
```

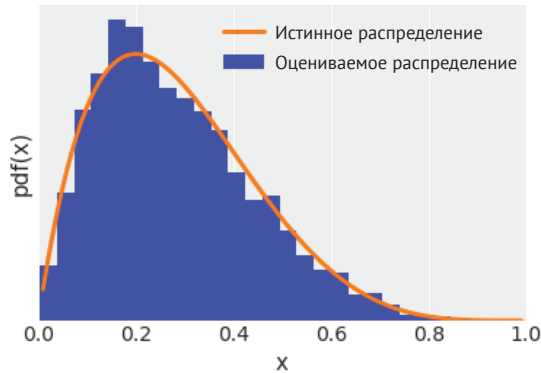


Рис. 8.5

Эффективность этого алгоритма сильно зависит от предлагаемого распределения: если предлагаемое состояние расположено очень далеко от текущего состояния, то чрезвычайно велик шанс отказа, а если предлагаемое состояние слишком близко, то исследование пространства параметров выполняется очень медленно. В обоих случаях потребуется намного больше выборок, чем в нормально отрегулированной ситуации. Обычно предлагается многомерное гауссово распределение, ковариационная матрица которого определяется на этапе настройки. Библиотека PyMC3 выполняет адаптационную настройку ковариации, следуя практическому правилу: идеальный диапазон критерия принятия состояния находится между 50 % для одномерного гауссова распределения и приблизительно 23 % для любого n -мерного гауссова целевого распределения.

Для методов MCMC часто требуется некоторое время, прежде чем начнется генерация выборок из целевого распределения. Поэтому на практике выполняется этап «разогрева», состоящий из удаления первой партии выборок. Выполнение «разогрева» – это чисто практический прием, не имеющий никакого отношения к теории процессов Маркова. В действительности такой прием не обязателен для бесконечного числа выборок. Таким образом, удаление первой партии выборок – это всего лишь специфический прием для получения более качественных результатов с учетом того, что мы можем обработать только конечное число выборок. Наличие теоретических гарантий или правил лучше, чем их отсутствие, но для любой практической задачи важно понимать различие между теорией и практикой. Напомним, что не следует ставить знак

равенства между объединением математических объектов и аппроксимацией этих объектов. Сферы, гауссовы распределения, цепи Маркова и все прочие математические объекты существуют только в идеальном воображаемом мире, но не в нашей несовершенной реальности.

Надеюсь, что предоставил достаточно теоретической информации для понимания алгоритма Метрополиса–Гастингса. Возможно, читателю потребуется неоднократное возвращение к изучению этого раздела, в этом нет ничего плохого. Основные идеи просты, но при этом требуют точного понимания.

Метод Монте-Карло с использованием механики Гамильтона

Методы МСМС, в том числе и алгоритм Метрополиса–Гастингса, дают теоретическую гарантию того, что при использовании достаточного числа выборок мы получим точную аппроксимацию истинного распределения. Но на практике для получения достаточного числа выборок может потребоваться значительное время, превышающее отведенный нам лимит. Поэтому были предложены альтернативы обобщенному алгоритму Метрополиса–Гастингса.

Многие из альтернативных методов, как и сам алгоритм Метрополиса–Гастингса, изначально были разработаны для решения задач статистической механики, области физики, которая изучает свойства атомных и молекулярных систем, следовательно, вполне естественным образом могут быть интерпретированы как аналоги физических систем. Одной из таких модификаций является метод Монте-Карло с использованием механики Гамильтона (Hamiltonian Monte Carlo – HMC), или гибридный метод Монте-Карло (Hybrid Monte Carlo – HMC). В упрощенном изложении механика Гамильтона (Hamiltonian) – это описание полной энергии физической системы. Также используется термин «гибридный» (Hybrid), потому что изначально этот метод разрабатывался как объединение (гибридизация) молекулярной механики, широко распространенной методики моделирования молекулярных систем, и алгоритма Метрополиса–Гастингса.

Метод Монте-Карло с использованием механики Гамильтона по существу представляет собой тот же алгоритм Метрополиса–Гастингса с одним весьма важным исключением – предложение новых позиций (точек) не является случайным. Для общего понимания теоретических концепций метода Монте-Карло с использованием механики Гамильтона без углубления в математические подробности снова вернемся к примеру с озером и лодкой. Вместо случайного перемещения лодки будем выбирать маршрут в соответствии с кривизной дна озера. Для выбора следующего пункта, в который должна быть направлена лодка, мысленно представим шар, катящийся по дну озера, начиная с нашей текущей позиции. Этот воображаемый шар обладает замечательными свойствами: он не только имеет идеальную сферическую поверхность, но у него также отсутствует трение, то есть он не замедляется водой или илом. Мы бросаем шар и позволяем ему свободно катиться некоторое время до тех пор, пока внезапно не остановим его. Затем мы принимаем это предложенное переме-

шение или отказываемся от него, используя критерий Метрополиса точно так же, как это делалось в чистом методе Метрополиса–Гастингса. Вышеописанная процедура повторяется многократно. При таком усовершенствованном алгоритме повышается шанс принятия новых позиций, даже если они расположены достаточно далеко от предыдущей позиции.

Перемещение в соответствии с кривизной пространства параметров превращается в более осмысленный способ перемещения, потому что позволяет устранить один из главных недостатков алгоритма Метрополиса–Гастингса: тщательное исследование пространства выборки требует отказа от большинства предлагаемых позиций. Но метод Монте-Карло с использованием механики Гамильтона позволяет получить более высокий коэффициент принятия даже для самых удаленных точек в пространстве параметров, следовательно, в результате мы получаем весьма эффективный метод выборки.

На этом завершим наш мысленный эксперимент и вернемся в реальный мир. Придется заплатить определенную цену за столь интеллектуальное предложение на основе механики Гамильтона. Необходимо вычислить градиенты нашей функции. Градиент – это обобщение концепции производной для двух и более измерений. Вычисление производной функции в одной точке показывает, в каком направлении функция возрастает, а в каком – уменьшается. Информацию о градиенте можно использовать для имитации движения шара по искривленной поверхности. В действительности мы применяем те же самые законы движения и математические правила, которые используются в классической физике для моделирования систем классической механики, таких как катящиеся шары, орбиты в планетарных системах и колебательные движения молекул.

Вычисление градиентов связано с определенным компромиссом: каждый очередной шаг выполнения метода Монте-Карло с использованием механики Гамильтона требует больших затрат, чем шаг выполнения алгоритма Метрополиса–Гастингса, но вероятность принятия этого шага намного выше по сравнению с алгоритмом Метрополиса–Гастингса. Чтобы найти баланс в пользу метода НМС, необходимо точно настроить несколько параметров модели НМС (это похоже на точную настройку ширины (размаха) предлагаемого распределения для эффективного сэмплера по методу Метрополиса–Гастингса). Настройка выполняется вручную, методом проб и ошибок, а также требует участия опытного пользователя, поэтому такая процедура является менее универсальным механизмом статистического вывода, чем нам хотелось бы. К счастью, в библиотеке PyMC3 имеется относительно новый механизм выборки No-U-Turn Sampler (NUTS). Этот метод подтвердил свою исключительную полезность, обеспечивая весьма высокую эффективность при решении байесовских моделей без необходимости вмешательства человека (или, по крайней мере, с минимальным вмешательством пользователя). Единственным недостатком NUTS является то, что он работает только для непрерывных распределений. Причина в том, что у нас нет возможности вычислять градиенты

для дискретных распределений. Библиотека PyMC3 решает эту проблему при помощи установления связи механизма NUTS с непрерывными параметрами, а алгоритм Метрополиса связывается с дискретными распределениями.

Для полноценного завершения изучения темы текущего раздела настоятельно рекомендуется внимательно понаблюдать за впечатляющей анимацией, созданной Чи Фэном (Chi Feng): <https://chi-feng.github.io/mcmc-demo/>.

Последовательный метод Монте-Карло

Одним из недостатков алгоритма Метрополиса–Гастингса и механизма NUTS (а также других вариантов метода Монте-Карло с использованием механики Гамильтона) является то, что если апостериорное распределение содержит несколько пиков, и эти пики разделены областями чрезвычайно низкой вероятности, то такие методы могут заикнуться на единственной моде и оставить без внимания все прочие моды.

Многие из методов, разработанных для устранения проблемы многочисленных минимумов, основаны на идее обобщенных функций (функций, описывающих смешанные распределения, – *tempered distribution*), которая также заимствована из статистической механики. Количество состояний, которые может принимать физическая система, зависит от температуры этой системы – при 0 градусов Кельвина (самая низкая возможная температура) любая система «застывает» в единственном состоянии. В другом предельном случае, при бесконечно высоких температурах все возможные состояния системы равновероятны. В общем, нас интересуют системы при некоторых средних температурах. Для байесовских моделей существует интуитивно понятный способ адаптации идеи обобщенных функций, если записать теорему Байеса немного по-другому:

$$p(\theta|y)_\beta = p(y|\theta)^\beta p(\theta). \quad (8.13)$$

Единственным различием в формулах 1.4 и 8.13 является появление параметра β , который называют инверсией температуры, или параметром темперинга (*tempering parameter*). Отметим, что при $\beta = 0$ мы получаем $p(y|\theta)^\beta = 1$, следовательно, обобщенное (смешанное – *tempered*) апостериорное распределение, а $p(\theta|y)_\beta$ становится обычным априорным распределением $p(\theta)$. При $\beta = 1$ обобщенное апостериорное распределение в действительности является полным апостериорным распределением. Поскольку выборка из априорного распределения в общем случае проще, чем выборка из апостериорного распределения (при увеличении значения β), мы начнем с выборки из более простого распределения и постепенно преобразуем ее в более сложное распределение, которое нас интересует на самом деле.

Существует множество методов, использующих эту идею. Одним из таких методов является последовательный метод Монте-Карло (*Sequential Monte Carlo* – SMC). Последовательный метод Монте-Карло в том виде, в котором он реализован в библиотеке PyMC3, можно кратко описать в следующем виде.

1. Инициализация параметра β нулевым значением.
2. Генерация N выборок S_β из обобщенного (смешанного) апостериорного распределения.
3. Увеличение значения β на небольшую величину.
4. Вычисление набора N весовых коэффициентов W . Весовые коэффициенты вычисляются в соответствии с новым обобщенным (смешанным) апостериорным распределением.
5. Получение S_w посредством повторной выборки S_β в соответствии с весовыми коэффициентами W .
6. Запуск N цепей Метрополиса, начиная каждую цепь с различных элементов выборки S_w .
7. Повторение процедуры, начиная с шага 3, до тех пор, когда $\beta \geq 1$.

Шаг повторной выборки (шаг 5) выполняется посредством удаления выборок с низкой вероятностью и замены их на выборки с более высокой вероятностью. На шаге запуска цепей Метрополиса (шаг 6) выполняется вспомогательное преобразование этих выборок для исследования пространства параметров.

Эффективность метода темперинга в высокой степени зависит от промежуточных значений β , которые обычно обозначают термином «планирование охлаждения» (из алгоритма имитации отжига). Чем меньше разность между двумя последовательными значениями β , тем ближе два последовательно вычисляемых смешанных апостериорных распределения, следовательно, тем проще будет переход из текущего состояния в следующее. Но если шаг последовательного увеличения значений слишком мал, то потребуются много промежуточных состояний, и в некоторый момент процесс превратится в расточительство огромных вычислительных ресурсов без реального улучшения точности результатов.

К счастью, последовательный метод Монте-Карло позволяет автоматически вычислять промежуточные значения β . Точное «планирование охлаждения» адаптируется к сложности задачи, то есть распределения, которые более сложны для извлечения выборок, потребуют больше промежуточных состояний, чем простые распределения.

Последовательный метод Монте-Карло в обобщенном виде показан на рис. 8.6, где на первой диаграмме изображено пять элементов выборки в виде (оранжевых) точек в некотором конкретном состоянии. На второй диаграмме показано, как изменяются весовые коэффициенты этих элементов выборки в соответствии с плотностью их объединенного (tempered) апостериорного распределения, обозначенного (синей) кривой линией. На третьей диаграмме показан результат выполнения определенного числа шагов алгоритма Метрополиса, начиная с элементов выборки с измененными весовыми коэффициентами из второй диаграммы. Обратите внимание на то, как два элемента выборки с более низкой плотностью апостериорного распределения (кружки меньшего размера в крайней правой и левой позициях) отбрасываются и уже не используются для формирования новой цепи Маркова.

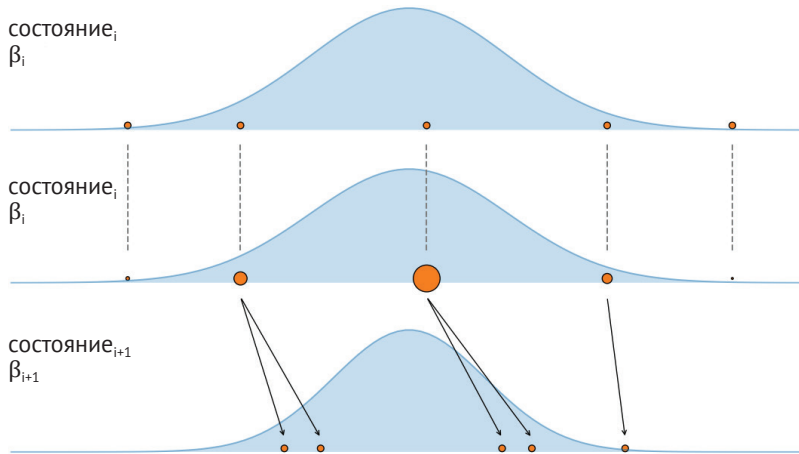


Рис. 8.6

Кроме промежуточных значений β , динамически вычисляются еще два параметра, основанных на интенсивности принятия предыдущего состояния: число шагов каждой цепи Маркова и ширина (размах) предлагаемого распределения.

Для реализации шага 6 последовательного метода Монте-Карло в библиотеке PyMC3 используется алгоритм Метрополиса. Это не единственный обязательный вариант выбора, но весьма разумный, к тому же подкрепленный как теоретическими, так и практическими обоснованиями. Важно отметить, что даже притом что последовательный метод Монте-Карло использует алгоритм Метрополиса, метод Монте-Карло обладает несколькими преимуществами:

- возможность получения выборок из мультимодальных распределений;
- отсутствие этапа «разогрева» благодаря коррективке весовых коэффициентов. Весовые коэффициенты вычисляются таким способом, что S_w аппроксимирует не $p(\theta|y)_{\beta_i^{****}}$, а $p(\theta|y)_{\beta_{i+1}^{****}}$, то есть приблизительно на каждом этапе запуска цепей MCMC из корректного объединенного апостериорного распределения;
- возможность генерации выборок с низкой автокорреляцией;
- возможность использования аппроксимации предельного правдоподобия (см. главу 5). Это всего лишь побочный эффект последовательного метода Монте-Карло, не требующий почти никаких дополнительных вычислений.

ДИАГНОСТИРОВАНИЕ ВЫБОРОК

Главная тема этого раздела – диагностирование выборок для алгоритма Метрополиса и механизма статистического вывода NUTS. Поскольку мы аппроксимируем апостериорное распределение с помощью конечного числа выборок,

важно проверить, получаем ли мы корректную выборку. В противном случае любой анализ по выборке будет бессмысленным. Можно выполнить несколько типов проверки – визуальные и числовые. Тесты спроектированы так, чтобы выявлять проблемы с получаемыми выборками, но они не предназначены для доказательства корректности распределения. Тесты могут только подтвердить, что конкретная выборка выглядит разумно. При обнаружении проблем в некоторой выборке возможны различные варианты их устранения:

- увеличение числа выборок;
- удаление некоторого количества выборок в самом начале трассировки. Этот прием уже упоминался как этап «разогрева». Процедура точной настройки библиотеки PyMC3 помогает снизить потребность в «разогреве»;
- изменение параметров механизма выборки, например увеличение продолжительности процедуры точной настройки или увеличение значения параметра `target_accept` для механизма выборки NUTS. При определенных условиях библиотека PyMC3 сама предлагает конкретные изменения;
- изменение параметров модели, то есть описание модели другим, но равнозначным способом;
- преобразование данных. Мы уже видели примеры преобразования данных в главах 4 и 5, где центрирование данных улучшало качество выборки из линейных моделей.

Чтобы сделать приведенные выше описания более понятными с практической точки зрения, воспользуемся минималистичной иерархической моделью с двумя параметрами: глобальным параметром a и локальным параметром b (параметр для групп). Для этой модели не нужны даже функция правдоподобия и данные. Здесь данные опущены, чтобы особо выделить тот факт, что некоторые из свойств, которые будут подробно описаны (особенно в разделе «Расхождения»), связаны со структурой модели, а не с данными. Мы будем рассматривать два альтернативных варианта параметризации одной и той же модели:

```
with pm.Model() as centered_model:
    a = pm.HalfNormal('a', 10)
    b = pm.Normal('b', 0, a, shape=10)
    trace_cm = pm.sample(2000, random_seed=7)

with pm.Model() as non_centered_model:
    a = pm.HalfNormal('a', 10)

    b_shift = pm.Normal('b_offset', mu=0, sd=1, shape=10)
    b = pm.Deterministic('b', 0 + b_shift * a)
    trace_ncm = pm.sample(2000, random_seed=7)
```

Различие между центрированной и нецентрированной моделями состоит в том, что первую мы подгоняем непосредственно по параметру уровня групп, а для второй параметр уровня групп моделируется как смещенное и промасштабированное гауссово распределение. Различия будут подробно рассматриваться с использованием нескольких диаграмм и числовых итоговых отчетов.

Сходимость

Механизм выборок для методов МСМС, например NUTS или алгоритм Метрополиса, может потребовать некоторого времени для обеспечения сходимости, то есть для начала извлечения выборок из корректного распределения. Ранее уже отмечалось, что методы МСМС обеспечивают теоретическую гарантию сходимости при весьма обобщенных условиях и бесконечном количестве выборок. К сожалению, на практике можно получить лишь конечную выборку, поэтому мы должны доверять только практическим проверкам, которые в лучшем случае дают лишь подсказки и предупреждения о том, что происходит или может произойти нечто неправильное, если проверка не пройдена. Но даже если проверка успешно выполнена, то это не дает никаких оснований считать, что все в порядке.

Один из способов визуальной проверки на сходимость – использование функции `plot_trace` из библиотеки ArviZ и тщательное исследование результатов ее выполнения. Чтобы лучше понять, что именно мы должны увидеть при исследовании полученных диаграмм, сравним результаты двух моделей, определенных выше (см. рис. 8.7 и 8.8):

```
az.plot_trace(trace_cm, var_names=['a'], divergences='top')
```

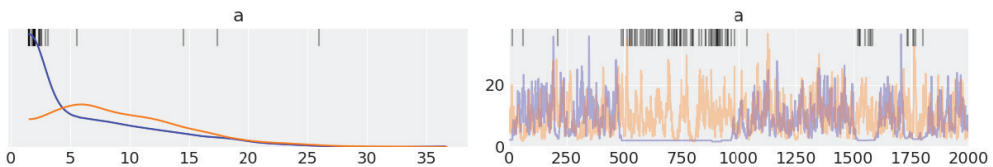


Рис. 8.7

```
az.plot_trace(trace_ncm, var_names=['a'])
```

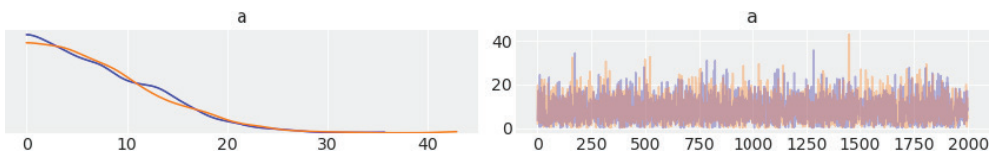


Рис. 8.8

Отметим, что ядерная оценка плотности (ЯОП) на рис. 8.8 более гладкая, чем на рис. 8.7. Гладкость ЯОП – это хороший признак, потому что негладкая кривая ЯОП может свидетельствовать о некоторой проблеме, например о необходимости увеличения числа элементов выборки или о более серьезной проблеме. Сама трассировка (trace) (диаграмма справа) должна выглядеть как белый шум, то есть мы не должны видеть каких-либо визуально распознаваемых шаблонов, нам нужна кривая с произвольными изгибами, похожая на трассировку на рис. 8.8.

Когда наблюдается именно такая картина, мы говорим, что получено качественное смешивание (good mixing). На рис. 8.7 показан пример патологического поведения – если внимательно сравнить его с рис. 8.8, то обнаружится, что перекрытие двух цепей больше на рис. 8.8, чем на рис. 8.7, а кроме того, можно заметить нечто подозрительное в нескольких областях трассировки на рис. 8.7. Особенно выделяется интервал от 500 до 1000 элементов: ясно видно, что одна из цепей (обозначенная синим цветом) зашла в тупик (почти горизонтальная линия).

Это действительно очень плохой признак. Механизм выборки отвергает все новые предложения, за исключением расположенных в самом близком соседстве, другими словами, выборка выполняется очень медленно, следовательно, чрезвычайно неэффективно. Для бесконечных выборок это не проблема, но при работе с конечными выборками это приводит к смещениям в результатах. Простой способ устранения такой проблемы – получение большего числа элементов выборки, но это помогает только при небольших смещениях, иначе число элементов выборки, требуемое для компенсации смещения, будет возрастать чрезвычайно быстро, и такой способ окажется бесполезным.

На рис. 8.9 показаны дополнительные примеры трассировок с качественным смешиванием (справа) и некачественным смешиванием (слева). Если существует более одной области, как для дискретных переменных или для мультимодальных распределений, то мы ожидаем, что трассировка не займет слишком много времени для одного значения или области с медленным перемещением в другие области, а с легкостью будет переходить из одной области в другую:

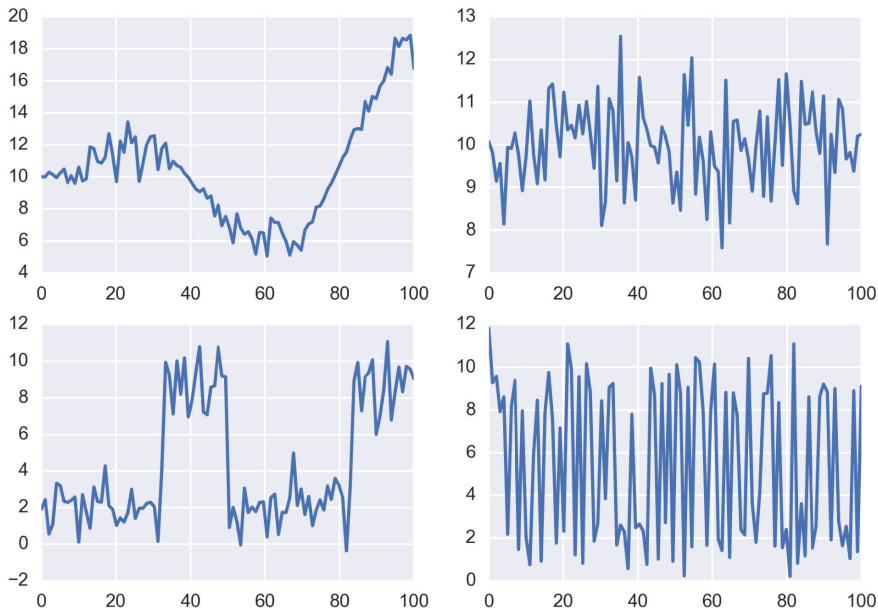


Рис. 8.9

Другим признаком правильной выборки методами МСМС является автоматически достигаемая похожесть трассировок, например первые 10 % (приблизительно) должны выглядеть похожими на остальные части трассировки, например на последние 50 % или 10 %. Еще раз подчеркнем: не следует пытаться обнаружить шаблоны, мы ожидаем нечто зашумленное. Это также можно наблюдать с помощью `az.plot_trace`. Если первая часть трассировки выглядит отличающейся от остальных частей, то это сигнал о необходимости этапа «разогрева» или увеличения числа выборок. Если наблюдается отсутствие автоматически достигаемой похожести в других частях трассировки или мы видим шаблон, то, возможно, требуется увеличение числа элементов выборки, но чаще это означает, что необходимо попробовать другой вариант параметризации. Для сложных моделей, возможно, даже потребуются применение сочетания всех перечисленных выше стратегий.

По умолчанию библиотека PyMC3 попытается выполнить независимые цепи в параллельном режиме (точное число параллельных процессов зависит от количества доступных процессоров). Это определяется с помощью аргумента `chains`, передаваемого в функцию `pm.sample`. Можно воспользоваться функциями `plot_trace` и `plot_forest` из библиотеки ArviZ для визуального исследования похожесть параллельных цепей друг на друга. Затем можно объединить параллельные цепи в одну для статистического вывода. Отметим, что параллельное выполнение цепей не приводит к чрезмерным затратам ресурсов.

Численный метод сравнения независимых цепей реализуется с использованием Rhat-статистики. Идея, лежащая в основе этого теста, заключается в вычислении дисперсии между цепями с учетом дисперсии внутри цепей. В идеальном случае следует ожидать значения 1. В качестве эмпирического правила будем считать хорошим результатом значение меньше 1.1, а большие значения сигнализируют о недостаточной сходимости. Это вычисление можно производить с помощью функции `az.r_hat`, нужно лишь передать в нее объект `trace`, созданный средствами библиотеки PyMC3. Rhat-диагностика также вычисляется по умолчанию функцией `az.summary` (возможно, вы помните ее использование в предыдущих главах) и опционально функцией `az.plot_forest` (при явном определении аргумента `r_hat=True`), как можно видеть в следующих примерах:

```
az.plot_forest(trace_cm, var_names=['a'], r_hat=True, eff_n=True)
```

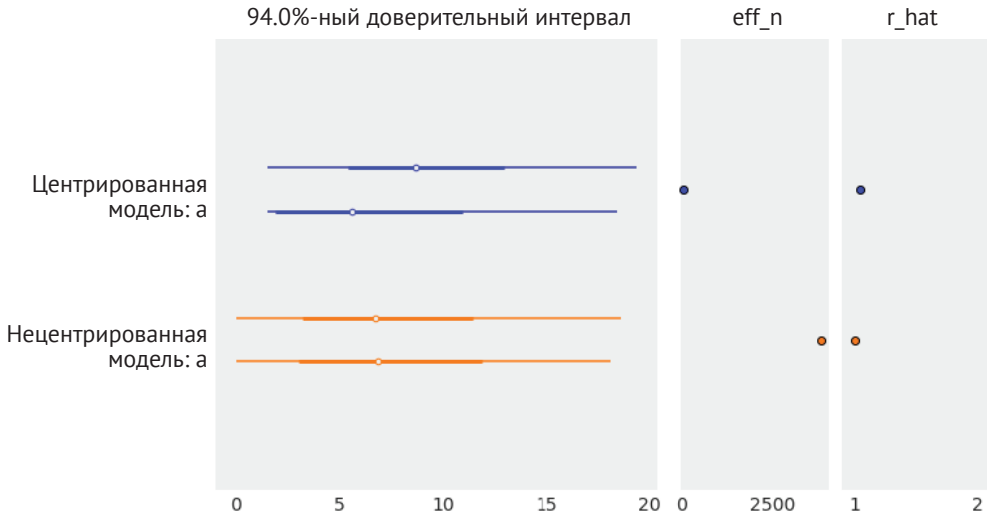


Рис. 8.10

Пример использования функции `az.summary`:

```
summaries = pd.concat([az.summary(trace_cm, var_names=['a']),
                        az.summary(trace_ncm, var_names=['a'])])
summaries.index = ['centered', 'non_centered']
summaries
```

Таблица 8.1

	Среднее значение (mean)	Стандартное распределение (sd)	mc_error	ПАР (hpd) 3 %	ПАР (hpd) 97 %	eff_n	r_hat
Центрированная модель centered	8.53	5.84	0.58	1.53	18.87	49.0	1.04
Нецентрированная модель non_centered	7.92	6.01	0.04	0.01	18.48	3817.0	1.00

Ошибка метода Монте-Карло

Одной из числовых характеристик, возвращаемых в итоговом отчете, является `mc_error`. Это оценка ошибки (погрешности), вводимая методом извлечения выборки. Оценка учитывает тот факт, что в действительности выборки не являются полностью независимыми друг от друга. Значение `mc_error` – это стандартная ошибка (погрешность) среднего значения x по n блокам, при этом каждый блок является частью трассировки `trace`:

$$mc_{\text{error}} = \frac{\sigma(x)}{\sqrt{n}}. \quad (8.14)$$

Эта ошибка (погрешность) должна быть меньше точности, которую мы требуем от получаемых результатов.

Автокорреляция

В идеальном случае выборка из распределения, в том числе и из апостериорного распределения, должна иметь автокорреляцию, равную нулю (если только корреляция не является ожидаемой, например во временных рядах). Выборка считается автокоррелированной, если значение на какой-либо произвольно выбранной итерации не является независимой от значений выборки на других итерациях. На практике выборки, сгенерированные методами MCMC, будут автокоррелированными, особенно при использовании алгоритма Метрополиса–Гастингса и в меньшей степени NUTS и SMC. В библиотеке ArviZ имеется удобная функция для построения диаграммы автокорреляции:

```
az.plot_autocorr(trace_cm, var_names=['a'])
```

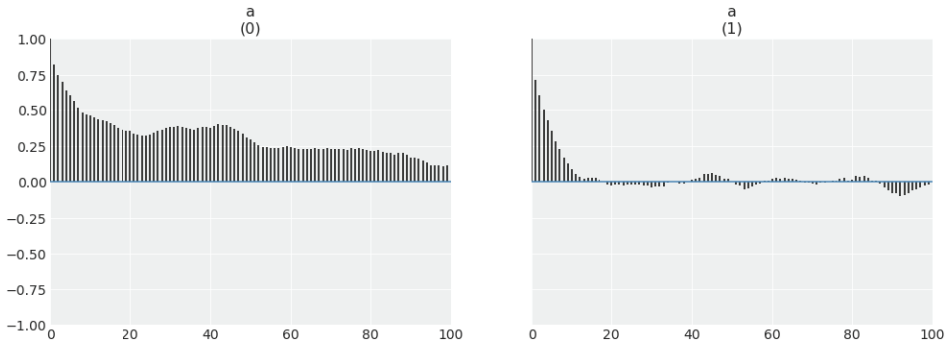


Рис. 8.11

Функция `az.plot_autocorr` показывает усредненную корреляцию значений выборки, сравниваемую с последовательностью точек (до 100 точек). В идеальном случае мы не должны наблюдать автокорреляцию. На практике нам нужны выборки, которые быстро приводятся к низким уровням автокорреляции. Сформируем и рассмотрим диаграмму автокорреляции для нецентрированной модели:

```
az.plot_autocorr(trace_ncm, var_names=['a'])
```

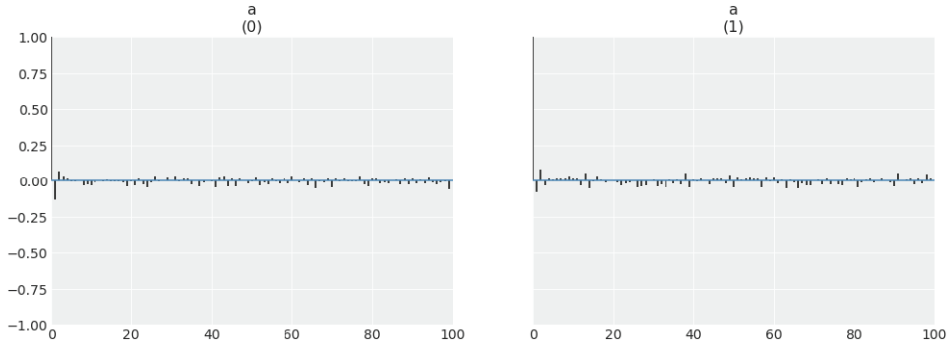


Рис. 8.12

При сравнении рис. 8.11 и 8.12 легко заметить, что в выборках из нецентрированной модели автокорреляция почти не наблюдается, в то время как выборки из центрированной модели демонстрируют более высокий уровень автокорреляции.

Эффективный размер выборки

Выборка с автокорреляцией содержит меньше информации, чем выборка того же размера без автокорреляции. В действительности можно использовать автокорреляцию для оценки размера некоторой произвольной выборки с равнозначной информацией, но без автокорреляции. Это называется эффективным размером выборки. Чем больше автокорреляция параметра, тем большее количество выборок потребуется для получения заданной точности, иначе говоря, автокорреляция оказывает негативное воздействие, снижая количество эффективных элементов выборки. Можно вычислить эффективный размер выборки средствами библиотеки ArviZ, а именно функцией `az.effective_n`. Кроме того, эффективный размер выборки вычисляется с помощью функций `az.summary` (как это было сделано в предыдущем разделе и в предыдущих главах) и `az.plot_forest`, если передать аргумент `eff_n=True` (см. рис. 8.9).

В идеальном случае эффективный размер выборки должен быть как можно более близким к реальному размеру выборки. Одним из преимуществ механизма NUTS над алгоритмом Метрополиса является то, что эффективный размер выборки NUTS обычно намного больше, чем эффективный размер выборки по алгоритму Метрополиса, таким образом, если вы используете NUTS, то в общем случае вам потребуется меньше выборок, чем при работе с алгоритмом Метрополиса.

Библиотека PyMC3 выдает предупреждающее сообщение, если эффективный размер выборки меньше 200 для любого параметра. По обобщенному правилу 100 эффективных элементов выборки должны обеспечить правильную оценку средних значений распределения, но наличие большего числа элементов выборки дает оценку, которая изменяется все меньше при каждом повторном выполнении модели. Это одна из причин, по которой установлено пороговое значение 200 для эффективного размера выборки. В большинстве задач от 1000 до 2000 эффективных значений более чем достаточно. Если необходима высокая точность количественных характеристик, которые зависят от хвостов распределения или чрезвычайно редких событий, то потребуется увеличение эффективного размера выборки.

Расхождения

В этом разделе будут рассматриваться проверки (тесты), предназначенные исключительно для механизма NUTS, поскольку они основаны на внутренней работе метода, а не являются свойством генерируемых выборок. Эти тесты основаны на так называемых расхождениях (divergences) и представляют собой мощный и достаточно точный метод диагностирования выборок.

При предыдущих попытках настройки моделей в этой книге, возможно, вы видели сообщения библиотеки PyMC3 о возникновении расхождения. Расхождения могут свидетельствовать о том, что механизм NUTS обнаружил в апостериорном распределении области с сильной кривизной, которые невозможно обработать корректно. Это говорит о том, что механизм выборки мог пропустить область пространства параметров, поэтому результаты будут искажены. В общем случае расхождения намного более чувствительны, чем проверки, обсуждаемые выше, поэтому они могут сообщать о проблемах, даже если все остальные тесты прошли успешно. Расхождения обладают замечательным свойством: они обычно появляются близко к проблематичным областям пространства параметров, следовательно, их можно использовать для идентификации возможной локации проблемы. Одним из способов визуального наблюдения расхождений является применение функции `az.plot_pair` с аргументом `divergences=True`:

```
_, ax = plt.subplots(1, 2, sharey=True, figsize=(10, 5), constrained_layout=True)
for idx, tr in enumerate([trace_cm, trace_ncm]):
    az.plot_pair(tr, var_names=['b', 'a'], coords={'b_dim_0':[0]}, kind='scatter',
                divergences=True, contour=False, divergences_kwargs={'color':'C1'},
                ax=ax[idx])
    ax[idx].set_title(['centered', 'non-centered'][idx])
```

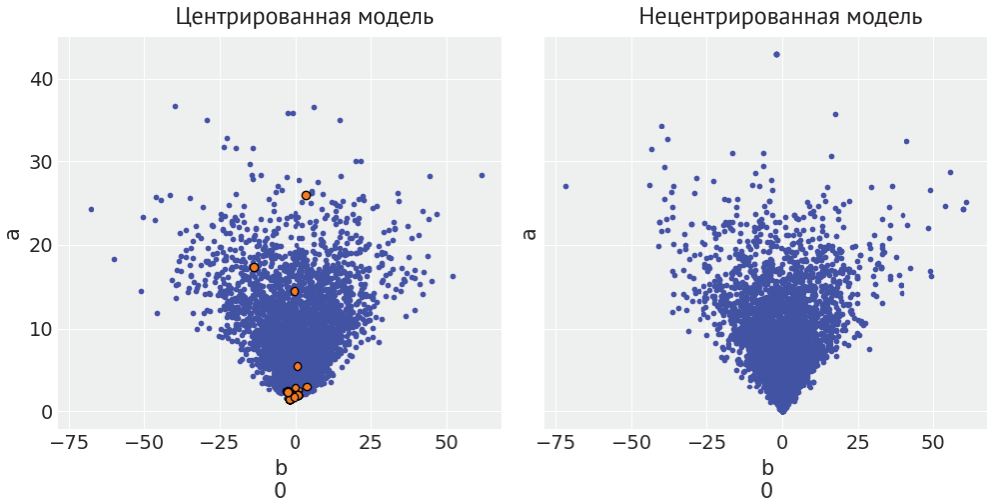


Рис. 8.13

На рис. 8.13 маленькие (голубые) точки – это обычные элементы выборки, а крупные (черные и оранжевые) точки представляют расхождения. Можно видеть, что в центрированной модели имеются расхождения, в основном сконцентрированные в узком нижнем конце воронки. Также можно видеть, что в том же месте нецентрированной модели расхождений нет. Предъявляя расхождения, механизм выборки сообщает, что испытывает серьезные затруднения при выборке из области около узкого нижнего конца воронки. Но мы можем убедиться, проверив рис. 8.13, что центрированная модель не содержит элементов выборки в самом нижнем конце воронки, то есть там, где сконцентрированы расхождения. Это явный признак возникновения проблемы.

Расхождения также отображаются на диаграммах трассировки ArviZ в виде черных маркеров «|», например на рис. 8.7. Обратите внимание на концентрацию расхождений в неправдоподобно плоских частях трассировки.

Еще одним полезным способом визуального наблюдения расхождений является параллельная диаграмма:

```
az.plot_parallel(trace_cm)
```

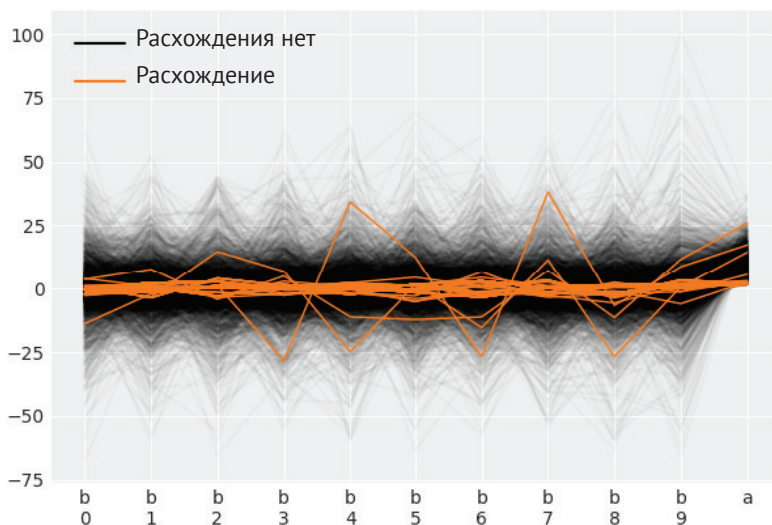


Рис. 8.14

Здесь можно видеть, что расхождения сконцентрированы около 0 для обоих параметров a и b . Диаграммы, подобные показанным на рис. 8.13 и 8.14, чрезвычайно важны, потому что позволяют узнать, в какой части пространства параметров могут возникать проблемы, а также потому, что помогают выявлять ложноположительные случаи. Последнее утверждение требует пояснения. Библиотека `PyMC3` использует определенную эвристику для пометки расхождений, но иногда эта эвристика может сообщать о наличии расхождений там, где их в действительности нет. Общее правило: если расхождения рассеяны по пространству параметров, то, вероятнее всего, это ложноположительные случаи, но если расхождения расположены компактно (сконцентрированы в некоторой области), то это с большой вероятностью признак проблемы. При появлении расхождений можно предложить три общих способа избавления от них или, по крайней мере, сокращения их количества:

- увеличение числа шагов точной настройки, например `pm.sample(tuning=1000)`;
- увеличение значения параметра `target_accept`, для которого по умолчанию установлено значение 0.8. Максимально возможное значение 1, поэтому можно попробовать значения 0.85 или 0.9;
- изменение параметризации модели. Как мы видели выше, нецентрированная модель является другим вариантом параметризации централизованной модели, и это привело к повышению качества выборок и отсутствию расхождений.

Нецентрированная параметризация

Выше нецентрированная модель была представлена как некий «волшебный фокус», позволяющий устранить проблему качества выборок. Рассмотрим этот «фокус» более подробно, заглянем глубже и развеем покров «волшебства».

На рис. 8.13 можно видеть, что между параметрами α и $\mathbf{b}$ существует корреляция. Так как $\mathbf{b}$ является вектором, состоящим из 10 элементов, выбрано построение диаграммы от $\mathbf{b}(0)$, хотя любой другой элемент вектора $\mathbf{b}$ должен формировать такой же шаблон. Это действительно абсолютно четко показано на рис. 8.14. Корреляция и характерная форма воронки являются следствием определения модели и способности этой модели частично концентрировать данные. При уменьшении значений α отдельные значения $\mathbf{b}$ располагаются ближе друг к другу и одновременно приближаются к глобальному среднему значению. Другими словами, уровень редуцирования (shrinkage) постоянно повышается, поэтому данные повышают концентрацию (приближаясь к точке полного объединения). Те же самые структуры, которые допускают частичное объединение (концентрацию), также вводят корреляцию, воздействующую на эффективность методов выборки.

В главе 3 мы видели, что линейные модели также приводят к возникновению корреляции (другой природы). Для таких моделей существует простое решение в виде центрирования данных. Возможно, и в рассматриваемом здесь примере возникает искушение сделать то же самое, но, к сожалению, это не поможет избавиться от проблемы выборки, связанной с формой воронки. Хитрым свойством формы воронки является варьирование корреляции при изменении позиции в пространстве параметров, таким образом, центрирование данных не поможет устранить или снизить степень корреляции этого типа. Как мы видели выше, методы МСМС, такие как алгоритм Метрополиса–Гастингса, испытывают затруднения при исследовании пространств с высокой корреляцией. Единственный способ получения этими методами корректных выборок – предложение нового шага в непосредственной близости от предыдущего шага. В результате процесс исследования становится сильно автокоррелированным и чрезвычайно медленным. Медленное объединение может стать настолько затянутым, что простое увеличение числа элементов выборки (draws) не будет иметь смысла и не обеспечит практически выполнимого решения. Механизмы выборки, такие как NUTS, лучше справляются с этой работой, если предлагают шаги, основанные на кривизне пространства параметров, но, как уже отмечалось ранее, эффективность процесса выборки в высшей степени зависит от этапа точной настройки. Для некоторых геометрических форм апостериорного распределения, например формируемых иерархическими моделями, на этапе точной настройки чрезмерно скрупулезно выполняется настройка области локального соседства, где начинают работу цепи. Это делает исследование других областей неэффективным, так как новые предложения становятся более случайными, напоминая поведение алгоритма Метрополиса–Гастингса.

РЕЗЮМЕ

Эта глава представляет собой теоретический обзор некоторых наиболее широко распространенных методов, используемых для вычисления апостериорного распределения, в том числе вариационных методов и методов Монте-Карло с использованием цепей Маркова. Особое внимание было уделено универсальным механизмам статистического вывода, методам, специально предназначенным для работы с любой произвольной моделью (или, по крайней мере, с широким диапазоном моделей). Эти методы являются основой любого вероятностного языка программирования или инструментального средства, поскольку обеспечивают автоматический статистический вывод, позволяя пользователям сосредоточиться на итеративном проектировании модели и интерпретации результатов. Также рассматривались методы численной и визуальной проверки для диагностирования выборок. Без правильной аппроксимации апостериорного распределения все преимущества и вся гибкость байесовского анализа исчезают, поэтому оценка качества процесса статистического вывода чрезвычайно важна для полной уверенности в действительной эффективности и качестве самого процесса статистического вывода.

УПРАЖНЕНИЯ

1. Используйте метод `grid-вычислений` с другими априорными распределениями, например попробуйте варианты `prior = (grid <= 0.5).astype(int)` или `prior = abs(grid - 0.5)` или сами придумайте и определите какие-нибудь экзотические априорные распределения. Поэкспериментируйте с другими данными, например с увеличением общего объема данных или с большей или меньшей четностью, относящейся к числу наблюдаемых выпадений орлов при подбрасывании монеты.
2. В коде вычисления оценки числа π сохраните значение N неизменным и выполните этот код еще несколько раз. Обратите внимание на различие результатов из-за использования случайных чисел, но при этом ошибки (погрешности) имеют приблизительно один и тот же порядок (проверьте и убедитесь в этом). Можете ли вы сформулировать интуитивно обоснованное предположение о том, как связаны друг с другом общее количество точек N и ошибка (погрешность) вычислений? Для более качественной оценки, возможно, потребуются изменить код, чтобы вычислить оценку как функцию от N . Новый код также можно выполнить несколько раз без изменения N и вычислить среднее значение ошибки (погрешности) и ее стандартное отклонение. Полученные результаты можно представить в виде диаграммы с помощью функции `plt.errorbar()` из библиотеки `matplotlib`. Попробуйте использовать множество значений N , например 100, 1000, 10000, то есть с изменением величины на порядок или почти на порядок.

3. Измените аргумент функции `func`, передаваемой в функцию `metropolis`. Попробуйте воспользоваться значениями априорного распределения из главы 1. Сравните этот код с методом грид-вычислений. Какая часть должна быть изменена, чтобы можно было использовать этот код для решения задачи байесовского статистического вывода?
4. Сравните свой ответ на вопрос из упражнения 3 с кодом Томаса Виецки (Thomas Wiecki): <http://twiecki.github.io/blog/2015/11/10/mcmc-sampling/>.
5. Используйте все диагностические инструментальные средства, рассмотренные в этой главе, для проверки нескольких моделей из предыдущих глав (не ограничивайтесь проверкой одной модели).
6. Повторно рассмотрите примеры исходного кода из всех предыдущих глав, найдите примеры кода с расхождениями и попытайтесь сократить их количество.

Глава 9

.....

Что дальше?

«Статистик – это технический термин для циничного ученого, специалиста по обработке данных».

– Джим Сэвэдж (*Jim Savage*)

Я написал эту книгу, чтобы дать представление об основных теоретических концепциях и практических методиках байесовской статистики для тех, кто уже знаком с языком программирования Python и соответствующим стеком данных, но не очень хорошо знает статистический анализ. После прочтения предыдущих восьми глав вы, как я надеюсь, в достаточной степени поняли практическое обоснование многих основных тем байесовской статистики. И хотя вы не превратитесь сразу в суперхакера-эксперта по байесовскому анализу, вы должны приобрести способность создания собственных вероятностных моделей для решения конкретных задач анализа данных. Если вы решили более глубоко освоить байесовскую статистику, то прочтения этой книги недостаточно – возможно, недостаточно прочесть и какую-то единственную книгу. Чтобы более свободно применять байесовскую статистику, потребуется многократное переосмысление ее идей и концепций с различных точек зрения.

В репозитории <https://github.com/aloctavodia/BAP> вы найдете примеры, дополняющие рассматриваемые в этой книге. Эти примеры не включены в текст книги из-за ограничений по ее объему или из-за недостатка времени. В действительности во время написания книги дополнительных примеров еще не было, но я пополняю комплект таких примеров время от времени. Для изучения дополнительного материала рекомендуется также обратиться к документации библиотеки PyMC3 <https://docs.pymc.io>, особенно к разделу Examples (Примеры), содержащему большое количество примеров моделей, среди которых есть и те, что рассматривались в этой книге, но многие окажутся новыми для вас. Как вам уже известно, ArviZ – это действительно новая библиотека, но мы уже создали образовательный ресурс по изучению практического использования анализа байесовских моделей. Надеемся, что это будет полезный ресурс, особенно для новичков в байесовском моделировании: https://github.com/arviz-devs/arviz_resources.

Если вы обнаружите ошибки и опечатки в тексте или в исходном коде примеров этой книги, то можете отправить сообщение об этом в репозиторий <https://github.com/aloctavodia/BAP>. Если у вас возникли общие вопросы по байесовской статистике, особенно если они связаны с использованием библиотек PyMC3 и ArviZ, можно задать их на сайте <https://discourse.pymc.io/>.

Ниже перечислены некоторые материалы и источники, которые определено повлияли на формирование моего байесовского образа мышления. Разумеется, этот список не следует считать исчерпывающим. Я уверен в том, что и вам эти материалы или хотя бы некоторые из них покажутся весьма полезными и стимулирующими.

Итак, если вы намерены продолжить изучение байесовской статистики, отнеситесь с вниманием к этому списку:

- настоятельно рекомендую книгу «Statistical Rethinking» Ричарда Мак-Элрета (Richard McElreath). Это великолепное введение в байесовский анализ. Небольшой недостаток заключается в том, что все примеры написаны на R/Stan. Но группа энтузиастов переписала примеры из этой книги на Python/PyMC3. Более подробную информацию можно найти в соответствующем репозитории GitHub: <https://github.com/pymc-devs/resources/tree/master/Rethinking>;
- еще одна книга, примеры которой были переписаны с использованием библиотеки PyMC3, – «Doing Bayesian Data Analysis» Джона К. Крушке (John K. Kruschke) (также известная как the puppy book – «книга для малышей»). Это еще одно превосходное введение в байесовский анализ. Большинство примеров из первого издания было портировано на Python/PyMC3 в репозитории GitHub: https://github.com/aloctavodia/Doing_bayesian_data_analysis. Второе издание можно найти здесь: <https://github.com/JWarmenhoven/DBDA-python>. В отличие от «Statistical Rethinking», «книга для малышей» гораздо больше внимания уделяет тому, как практически применять байесовский анализ по аналогии со многими общеизвестными методами частотного статистического анализа. В зависимости от того, что именно вам необходимо, в этой книге можно найти доводы как «за», так и «против»;
- Аллен Б. Дауни (Allen B. Downey) написал много хороших книг, и «Think Bayes» (<http://greenteapress.com/wp/think-bayes/>) не является исключением. В этой книге вы найдете несколько интересных примеров и сценариев, которые, несомненно, станут мощным стимулом и помогут освоить байесовский подход к решению задач. В этой книге PyMC3 не используется, но предлагается собственная библиотека на языке Python, фактически разработанная на основе «Think Bayes». Во втором издании, которое еще не было окончательно подготовлено на момент публикации моей книги, будет использоваться библиотека PyMC3 и, возможно, даже библиотека ArviZ. Дополнительную информацию можно найти в репозитории второго издания: <https://github.com/AllenDowney/ThinkBayes2>;

- книгу (в свободной и платной версиях) «Probabilistic Programming and Bayesian Methods for Hackers» написал Кэмерон Дэвидсон-Пайлон (Cameron Davidson-Pilon) с несколькими соавторами. Эта книга и ее «блокнотная» (notebook) версия изначально создавались с использованием библиотеки PyMC2, но сейчас адаптированы для библиотеки PyMC3: <https://github.com/quantopian/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers>;
- книга, на которую ссылаются как на «The Bayesian», то есть как на байесовскую книгу, – «Bayesian Data Analysis», автор Эндрю Гелман (Andrew Gelman) и др. Это великолепная книга, хотя ее не следует считать вводным курсом для начинающих, а лучше использовать как справочник, который не читают последовательно, страницу за страницей. Если вы не очень хорошо разбираетесь в статистике (не только в байесовской), то рекомендую сначала внимательно проштудировать «Statistical Rethinking» Ричарда МакЭлрета и только после этого перейти к «Bayesian Data Analysis». Также можно рекомендовать книгу «Data Analysis Using Regression and Multilevel/Hierarchical Models» Эндрю Гелмана и Дженифер Хилл (Jennifer Hill).

Если вы хотите продолжить изучение гауссовых процессов, то обратите внимание на следующую книгу:

- Карл Эдвард Расмуссен (Carl Edward Rasmussen) и Кристофер К. Уильямс (Christopher K. I. Williams) «Gaussian Processes for Machine Learning». В 2009 году книга получила премию ДеГроота (DeGroot Prize)¹ от международной организации International Society for Bayesian Analysis, но мы с нетерпением ждем новой редакции.

Следует отметить еще пару книг по машинному обучению, в которых достаточно серьезное внимание уделяется байесовскому анализу:

- «Machine Learning: A probabilistic Probabilistic Perspective», автор Кевин Мерфи (Kevin P. Murphy). Это великолепная книга, в которой автор пытается подробно объяснить, как работают многие методы и модели при использовании вероятностного подхода. Изложение материала может показаться немного суховатым и весьма лаконичным, но это зависит от вашей математической подготовки. Тем не менее в книге очень много примеров, поскольку она написана с исключительно практической целью. Кевин Мерфи заимствовал примеры и идеи из многих других источников, поэтому книга стала еще и подробным справочником-указателем на множество полезных ресурсов. Я впервые узнал о глубоком обучении именно из этой книги, когда эта технология была еще совсем юной;
- «Pattern Recognition and Machine Learning», автор Кристофер Бишоп (Christopher Bishop) – классическая книга по машинному обучению.

¹ Моррис Херман ДеГроот (1931–1989) – американский математик и статистик. – Прим. перев.

Содержимое в значительной мере пересекается с содержимым книги «Machine Learning: A probabilistic Probabilistic Perspective», хотя, возможно, здесь материал излагается с точки зрения байесовского анализа в большей степени. Вероятно, эту книгу читать немного проще, чем книгу Мерфи, которая, скорее, является справочником.

В детстве я мечтал о летающих автомобилях, о бесконечной энергии, о каблуках на Марсе или на Луне, о всемирном правительстве, обеспечивающем благосостояние всей человеческой расы. Да, я понимаю, я был мечтателем. По многим причинам мои мечты не сбылись. Вместо этого мы получили то, что невозможно было представить, по крайней мере мне, всего лишь пару десятилетий назад: общедоступность и простота использования чрезвычайно мощных компьютерных (вычислительных) методов. Одним из побочных эффектов этой компьютерной революции является то, что каждый человек с достаточно скромным знанием языка программирования, такого как Python, теперь получает доступ к невероятно огромному разнообразию вычислительных методов для анализа данных, имитации процессов и явлений и других сложных задач. Я считаю, что это огромные возможности, но в то же время необходимо весьма осторожно пользоваться этими методами. Мой способ изучения статистики как новичка и путь освоения практического использования стандартных, готовых к применению методов был нерациональным, слишком трудным и совершенно не связанным с упомянутыми выше изменениями. Лично для меня эта книга, возможно, является ответной реакцией на собственный нелегко приобретенный опыт.

Я пытался написать книгу по статистике, уделяя особое внимание методике моделирования и разумному анализу, зависящему от контекста. Я не уверен, удалось ли мне действительно достичь этой цели. Одна из причин неуверенности заключается в том, что мне необходимо продолжать учиться, чтобы узнать больше по этой теме (возможно, нам всем как сообществу необходимо продолжать изучать эту тему). Другая причина – правильный статистический анализ должен управляться конкретной областью знаний и контекстом, но представить конкретный контекст весьма трудно в книге, предназначенной для широкой аудитории. Однако я надеюсь, что все-таки представил читателям разумный и критический взгляд на статистические модели, привел полезные примеры и в достаточной степени стимулировал вас к дальнейшему обучению.

Предметный указатель

A

ArviZ, библиотека, 48, 60, 274, 323
сравнение моделей, 204
Automatic Differentiation Variational
Inference — ADVI, 301

B

Bias, 197

C

Concentration of measure, 295
Curse of dimensionality, 295

D

Deviance, 201
Discriminative classifier, 173
Divergence, 322
Draw, 302
d-мера Коэна, 81

E

Effect size, 79
ELBO — evidence lower bound, 300
Entropy, 222
Experimental design. См. Данные,
планирование эксперимента
Exploratory Data Analysis — EDA.
См. Статистика, метод, разведочный
анализ данных (РАД)

F

Fuzzy clustering, 245

G

Generative classifier, 173
Good mixing, 317
Grid computing, 293

H

Hamiltonian Monte Carlo — HMC, 310
Hard clustering, 245

Highest-Posterior Density — HPD, 47
Hybrid Monte Carlo — HMC, 310

I

Independently and identically distributed
variable — iid, 30
Inference engine, 292
Inferential statistics. См. Статистика,
метод, статистический вывод
INLA (Integrated Nested Laplace
Approximation), 298
Intensity, 284
estimation, 284
Interquartile range — IQR, 191
Inverse link function, 155
iris, набор данных, 157

K

Kernel, 261
Kullback-Leibler (KL) divergence, 224

L

Label-switching problem, 239
Lasso-перепеcсия, 220
Leave-one-out cross validation —
LOOCV, 199
Log pointwise predictive density —
lppd, 222

M

Markov chain, 305
Markov chain Monte Carlo — MCMC, 301
Maximum a posteriori — MAP, 105, 183,
202
Metropolis-Hastings, 305

N

NumPy, библиотека, 136

O

Occam razor, 194

Odds, 165
Overfitting, 133, 197

Р

pandas, библиотека, 159, 180
Parameter non-identifiability, 239
Patsy, библиотека, 184
Probabilistic programming languages — PPL, 55
Probability mass function — PMF, 176
PyMC3, библиотека, 122, 125, 131, 136, 164, 172, 173, 174, 190, 208, 273, 285
 механизм статистического вывода NUTS, 59
 модуль обобщенной линейной модели GLM (Generalized Linear Model), 183
 новый механизм выборки No-U-Turn Sampler (NUTS), 311
 основы использования и пример, 56
 пример решения задачи о подбрасывании монеты, 57
 реализация последовательного метода Монте-Карло, 312
 сообщение о возникновении расхождений, 322
 сравнение моделей, 204

R

Random walk, 305
Rate, 284
redwood, набор данных, 286
Region of Practical Equivalence — ROPE, 62
Rug plot, 286
Rvs — random variates, метод, 27

S

SciPy, библиотека, 225, 308
Sequential Monte Carlo — SMC, 216, 312
Shrinkage, 92, 325
Soft clustering, 245
Stacking, 208
Stick-breaking process, 248

T

Tempered distribution, 312
Tempering parameter, 312

Temporal series. См. Временной ряд
Theano, библиотека, 56, 136
tips, набор данных, 82
True distribution, 51
Typical set, 296

U

Underfitting, 197
Undersampling, 215

V

Variance, 197

W

Watanabe-Aikake information criterion — WAIC, 202
Widely applicable information criterion — WAIC, 202

Z

Zero-inflated Poisson — ZIP, 178

A

Автокорреляция, 320
 пространственная регрессия, пример, 270
Автоматическое
 дифференцирование, 57
Алгоритм Метрополиса,
 диагностирование выборок, 314
Алгоритм Метрополиса–Гастингса, 305

Б

Байесовский анализ, 46, 55
 обобщение апостериорного распределения, 47
 рабочий процесс (поток), 51
Байесовский вывод с одним параметром, 34
 задача о подбрасывании монеты, 35
Байесовское р-значение, 193
Байесовское моделирование, 22
Бета-распределение, 37, 235
 биномиальное, 254
Большой адронный коллайдер, 21

В

Вариационный метод, 298

Вероятностное программирование, 55
язык, 55

Вероятность, 23, 25
апостериорная, 33
априорная, 33
объяснение смысла, 23
описание неопределенности
модели, 24
плотность, 30
правило умножения, 25
превосходства, 82
по d-мере Коэна, 82
распределение, 22, 26
гауссово, 26, 27
нормальное, 26
параметр, 28
среднее значение, 28
событие, 24
совместная, 25
условная, 25

Весовой коэффициент Акаике, 205

Временной ряд, 30

Всемирная организация

здравоохранения (ВОЗ), 87, 147

Выпадение жребия или выигрыша, 302

Г

Гамма-распределение, 105

Гамма-функция, 38

Гауссов процесс, 264

классификация, 277

регрессия, 265

Гессиян, 296

Гибридный метод Монте-Карло, 310

Градиент, 311

Гребневая регрессия, 220

Д

Данные, 21

классификация

гауссов процесс, 277

набор данных, 26

нелинейные, 259

планирование эксперимента, 21

центрирование, 159

частичная выборка, 26

Диагностирование выборок, 314

автокорреляция, 320

качественное смешивание, 317

нецентрированная

параметризация, 325

ошибка (погрешность) метода

Монте-Карло, 319

расхождение (divergence), 322

сходимость, 316

эффективный размер выборки, 321

Диаграмма Крушке, 46, 105, 125, 236

Дисперсия, 197, 262

З

Задача о подбрасывании монеты, 35

выбор априорной вероятности, 37

выбор правдоподобия, 36

общая модель, 35

определение модели средствами

библиотеки PyMC3, 57

получение апостериорной

вероятности, 39

статистический вывод с одним

параметром, 35

статистический вывод средствами

библиотеки PyMC3, 58

И

Инверсия температуры, 312

Интенсивность, 284

Интервал плотности апостериорного

распределения (ПАР), 47

Интерквартильный размах, 191

Информационный критерий, 199, 200

bayes-o-meter™, 203

WAIC, 222

расхождение Кульбака-Лейблера

(РКЛ), 224

энтропия, 222

Акаике (AIC), 202

байесовский (Bayesian Information

Criterion — BIC), 203

Ватанабе-Аикаке, 202

логарифмическая функция

правдоподобия, 201

надежность вычислений WAIC

и LOO, 206

отклонение, 201
 отклонения (Deviance Information Criterion — DIC), 203
 парето-сглаженная выборка
 по значимости для перекрестной
 проверки LOOCV, 203
 связь с коэффициентами Байеса, 216
 усреднение моделей, 207
 часто применяемый
 информационный критерий
 (WAIC), 202

Испытание Бернулли, 254

Истинное усреднение байесовского
 моделирования, 208

К

Квадратическая аппроксимация, 296
 Квадратичный линейный
 дискриминантный анализ (КДА), 175
 Квартет Энскомба, 119, 130, 132
 Кластеризация, 245
 на основе моделей, 245
 нечеткая, 245
 строгая, 245
 Кластерный анализ, 245
 Ковариационная матрица, 261, 262, 280
 тепловая карта, 263
 Контрольная группа, 79
 Концентрация меры, 295
 Коэффициент Байеса, 210
 вычисление, 213
 общие проблемы, 215
 последовательный метод
 Монте-Карло, 216
 интерпретация, 211
 предельное правдоподобие
 свойства, 212
 связь с информационными
 критериями, 216
 Коэффициент детерминации, 115
 Коэффициент корреляции Пирсона, 114
 вычисляемый по многомерному
 гауссову распределению, 115
 Критерий фальсифицируемости
 (опровергаемости), введенный Карлом
 Поппером (Karl Popper), 194
 Кросс-валидация, 199

Л

Лезвие Оккама (правило), 194
 k-кратная перекрестная
 проверка, 199
 баланс между простотой
 и точностью, 197
 измерение прогнозируемой
 точности, 198
 лишние параметры приводят
 к переподгонке, 196
 метод LOOCV, 199
 недостаточное количество
 параметров приводит
 к недоподгонке, 197
 перекрестная проверка, 199
 Ленточная диаграмма, 286
 Линейная корреляция
 иерархическая, учет
 причинно-следственных
 связей, 128
 Линейная регрессия
 дисперсия переменной, 147
 иерархическая, 122
 корреляция, 128
 учет беспорядочности реальных
 данных, 128
 избыточная переменная, 137
 изменение данных для устранения
 корреляции, 109
 множественная, 102, 133
 добавление взаимодействий, 146
 маскировочный эффект
 переменных, 144
 мультиколлинеарность, 140
 слишком сильная корреляция, 140
 модель, 103
 недетерминированная, 143
 модель с сильной
 автокорреляцией, 108
 отсекаемый отрезок (свободный
 член), 103
 полиномиальная, 130
 интерпретация параметров, 131
 как конечная модель, 132
 простая, 102

робастная, 118
 связь с машинным обучением, 103
 со многими переменными, 133
 спутывающая переменная, 137
 угол наклона, 103
 центрирование данных, 110
 Линейный дискриминантный анализ (ЛДА), 174
 Линейный коэффициент корреляции, 114
 Линейный масштаб, 262
 Логарифмическая функция правдоподобия, 227
 Логарифм поточечно определяемой прогнозируемой плотности, 222
 Логистическая регрессия, 156
 множественная, 163
 граница решения, 163
 дискриминативная модель, 173
 интерпретация коэффициентов, 165
 использование функции softmax, 171
 обработка коррелирующих переменных, 167
 порождающая модель, 173
 работа с несбалансированными классами данных, 169
 реализация модели, 164
 тепловая карта, 168
 модель, 156
 применение к набору данных iris, 157
 робастная, 182

М

Марковский метод, 301
 Матрица Гессе, 296
 Машинное обучение, 20, 102
 признак (feature), 103
 с учителем (supervised learning), 103
 Мета-модель, 207, 208
 Метод
 максимального правдоподобия, 45
 частотной статистики, 45

Метод вариационного статистического вывода с применением автоматического дифференцирования, 301
 Метод главных компонент (PCA), 144
 Метод Лапласа, 296
 Метод Монте-Карло, 303
 ошибка (погрешность), 319
 Метод Монте-Карло с использованием механики Гамильтона, 310
 Метод Монте-Карло с использованием цепей Маркова, 301
 Метод наименьших квадратов, 103
 Метод темперинга, 313
 Механизм статистического вывода, 292
 NUTS
 диагностирование выборок, 314
 расхождение (divergence), проверка, 322
 диагностирование выборок
 сходимости, 316
 марковский метод, 301
 алгоритм
 Метрополиса–Гастингса, 306
 метод Монте-Карло, 303
 метод Монте-Карло с использованием механики Гамильтона, 310
 последовательный метод Монте-Карло, 312
 немарковский метод, 293
 вариационный метод, 298
 грид-вычисления, 293
 квадратическая аппроксимация, 296
 Механика Гамильтона (Hamiltonian), 310
 Модель, 22
 байесовская, 22, 46, 259
 нотация и визуализация, 46
 вероятностная, 22
 гауссова, 67
 статистический вывод, 68
 иерархическая, 86
 гиперпараметр, 87
 пример (химический сдвиг), 94
 редуцирование (shrinkage), 91

квадратическая, 189
 классификатор
 дискриминативный, 173
 порождающий, 173
 линейная, 189, 258
 обобщенная, 154
 логистическая, 157
 применяемая к набору данных
 iris, 159
 нецентрированная, 315
 нецентрированная
 параметризация, 325
 параболическая, 189
 полиномиальная второго
 порядка, 189
 смешанная, 231
 кластеризация, 245
 конечная, 232
 конечная, категориальное
 распределение, 234
 конечная, определение числа
 компонентов K , 241
 конечная, распределение
 Дирихле, 235
 неидентифицируемость, 238
 неидентифицируемость
 параметров, 239
 непараметрическая, 246
 непрерывная, 253
 непрерывная, t -распределение
 Стьюдента, 255
 непрерывная, биномиальное бета-
 распределение, 254
 непрерывная, отрицательное
 биномиальное распределение, 254
 проблема (самопроизвольного)
 перемещения меток, 239
 с бесконечной размерностью, 246
 с бесконечной размерностью,
 процесс Дирихле, 246
 создание, 45
 центрированная, 315
 Модель катастроф в угледобывающей
 промышленности, 284
 Мультиколлинеарность, 140

Н

Надежный статистический вывод, 73
 Недообучение, 197, 198
 Недоподгонка, 197, 198
 Нормальная аппроксимация, 296

О

Отклонение, 227
 Оценка, 284
 интенсивности, 284
 Оценка апостериорного
 максимума, 105, 183, 202

П

Параметр
 вариационный, 299
 концентрации, 247
 неидентифицируемость, 239
 темперинга, 312
 Перекрестная проверка, 199
 Переменная
 дисперсия, 147
 зависимая, 128
 избыточная, 137
 маскировочный эффект, 144
 независимая, 128
 скрытая (latent), 233
 случайная, 26
 дискретная, 29
 независимая одинаково
 распределенная случайная
 величина (нор), 30, 36
 непрерывная, 29
 совместно используемая
 (разделяемая), 148
 спутывающая, 137
 стандартное отклонение, 147
 Переменная норма, 284
 Переподгонка, 133, 197, 198
 Планирование охлаждения (из
 алгоритма имитации отжига), 313
 Плотность апостериорного
 распределения (ПАР), 60, 62, 112, 174
 Подтверждение нижней границы
 (ELBO), 300
 Последовательный метод

Монте-Карло, 216, 312
 Потенциал, 239
 Правдоподобие, 33
 предельное, 34
 Предельное правдоподобие, 211, 292
 Принцип детального равновесия, 305
 Проблема (самопроизвольного)
 перемещения меток, 239
 Проверка прогнозируемого
 апостериорного распределения, 188
 Проклятие размерности, 295
 Пространство практической
 равнозначности (ППР), 62
 Процесс Дирихле, 246
 Процесс Кокса, 283
 модель катастроф в угледобывающей
 промышленности, 284
 набор данных redwood, 286
 Процесс постепенного разделения
 стержня, 248
 Процесс Пуассона, 284
 Псевдоусреднение байесовского
 моделирования, 208

Р

Размер эффекта, 79
 Распределение
 апостериорное, 40, 259, 276, 293, 296
 визуализация, 111
 вычисление, 40
 графическое отображение, 40
 интерпретация, 111
 истинное, 301
 мода, 105
 обобщение, 47, 59
 проверка прогнозируемого
 апостериорного распределения, 49
 прогнозируемое, 49
 решения на основе, 61
 априорное, 168
 влияние на статистический
 вывод, 43
 гиперраспределение, 87, 123, 126
 информативное, 44
 методика выбора, 43
 неинформативное, 43

 регуляризация, 220
 регуляризирующее, 44, 144
 сопряженное, 39
 Бернулли, 156, 171
 бета-биномиальное, 35
 бимодальное, 238
 биномиальное, 37
 отрицательное, 254
 Больцмана, 171
 гауссово, 104, 118, 231, 233, 255,
 261, 296
 многомерное, 261
 Дирихле, 235
 дискретное, 36, 234, 254
 истинное, 51
 категориальное, 171, 234
 Коши, 74
 Лапласа, 220
 Лоренца, 74
 полунормальное, 69
 Пуассона, 176, 233, 284
 с дополнением нулевыми
 значениями (ZIP), 178
 функция вероятности, 176
 Стьюдента (t-распределение), 74, 119,
 255
 Расхождение, 322
 Расхождение Кульбака–Лейблера
 (РКЛ), 224, 298
 Регрессия
 на основе гауссова процесса, 265
 Пуассона, 271, 285
 с пространственной
 автокорреляцией, 270
 Регрессия Пуассона, 176
 модель Пуассона с дополнением
 нулевыми значениями (ZIP), 179
 Регуляризация Тихонова, 220
 Редуцирование, 92, 325

С

Случайная величина, 26
 Случайное блуждание, 305
 Смещение, 197
 Сравнение групп, 79
 набор данных tips, 82
 Среднеквадратическая ошибка, 201

Стандартное отклонение, 28, 47

Статистика, 19

метод

разведочный анализ данных
(РАД), 19

статистический вывод, 20

описательная, 20

форма моделирования, 20

Статистический вывод, 20

байесовский, 20

Стогование прогнозируемых

распределений, 208

Субдискретизация, 215

Сходимость, 316

Т

Теорема Байеса, 31, 210, 292

Теория вероятностей, 23

Типовое множество, 296

Точность

вне пределов выборки, 199

в пределах выборки, 198

Ф

Фиктивная контрольная группа, 80

Функция

softmax, 171

ковариационная, 261, 263

моделирования, 259

гауссов процесс, 264

ковариационная, 262

ядро, 262

обобщенная, 312

обратной связи, 155, 258

описывающая смешанные

распределения, 312

тождественного отображения, 155, 258

Функция потерь, 64

ожидаемых, 64

Х

Химический сдвиг, 68

Ц

Центральная предельная теорема, 67

Цепь Маркова, 305

Ш

Шанс, 165

Ширина окна, 262

Э

Экспериментальная группа, 79

Энтропия, 222

Я

Ядерная оценка плотности (ЯОП), 60,
68, 159, 231, 243, 316

Ядерный магнитный резонанс
(ЯМР), 68

Ядро, 261, 263

«белого шума» (white noise), 280

гауссово, 262

экспоненцированное

квадратическое, 262

Книги издательства «ДМК Пресс» можно заказать
в торгово-издательском холдинге «Планета Альянс» наложенным платежом,
выслав открытку или письмо по почтовому адресу:
115487, г. Москва, 2-й Нагатинский пр-д, д. 6А.
При оформлении заказа следует указать адрес (полностью),
по которому должны быть высланы книги;
фамилию, имя и отчество получателя.
Желательно также указать свой телефон и электронный адрес.
Эти книги вы можете заказать и в интернет-магазине: **www.a-planet.ru**.
Оптовые закупки: тел. **(499) 782-38-89**.
Электронный адрес: **books@aliants-kniga.ru**.

Освальдо Мартин

Байесовский анализ на Python

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com
Перевод *Снастин А. В.*
Корректор *Синяева Г. И.*
Верстка *Чаннова А. А.*
Дизайн обложки *Мовчан А. Г.*

Формат 70×100 1/16.

Гарнитура «PT Serif». Печать офсетная.

Усл. печ. л. 27,63. Тираж 200 экз.

Веб-сайт издательства: **www.dmkpress.com**
